CI/CD Pipeline for a Hotel Room Booking Portal

Complete DevOps Project Report - all 15 tasks

Python / Flask - Git & GitHub - Jenkins - pytest & Selenium - Docker - Ansible

Evidence verified on 2026-10-20. Generated from docs/task-reports/*.md.

Repository: https://github.com/KemNxts/hotel-room-booking-cicd

Contents

Task Reports - CI/CD Pipeline for a Hotel Room Booking Portal 3
00 - Project Audit (before writing the task reports) 5
Task 1 - Problem Definition and Scope 9
Task 2 - Agile Planning and DevOps Workflow 16
Task 3 - Requirements, Architecture and Technology Setup 24
Task 4 - Git and GitHub Repository Initialization 37
Task 5 - Feature Development with Branching 48
Task 6 - MVP Completion and Git Collaboration 56
Task 7 - Jenkins Installation and Continuous Integration 76
Task 8 - Pipeline as Code and Server Deployment 90
Task 9 - Selenium Test Design and Local Execution 101
Task 10 - Continuous Testing in Jenkins 111
Task 11 - Docker Image and Container Lifecycle 119
Task 12 - Jenkins-Docker Continuous Deployment 130
Task 13 - Configuration Management with Ansible 137
Task 14 - Automated Provisioning and Reliability Validation (Idempotency and Rollback) 148
Task 15 - Final End-to-End Release, Documentation and Viva 159
Viva Cheat Sheet - CI/CD Pipeline for a Hotel Room Booking Portal 174
Screenshot inventory 177

Task Reports - CI/CD Pipeline for a Hotel Room Booking Portal

One self-contained report per professor-assigned task. Each report has the same structure: **A** objective and status, **B** beginner explanation, **C** what was implemented, **D** screenshots and evidence, **E** how to demonstrate it, **F** screenshot checklist, **G** viva questions and answers, **H** troubleshooting, **I** revision notes.

Start with the audit: 00-project-audit.md - the real state of the repository, services and evidence on 2026-10-20.

#	Report	In one sentence	Status
1	Problem Definition and Scope	why a booking portal and a delivery pipeline are needed, and how success is measured	Implemented and verified
2	Agile Planning and DevOps Workflow	user stories, backlog, board, sprints, Definition of Done and the lifecycle diagram	**Partially completed** - GitHub issues pending
3	Requirements, Architecture and Setup	SRS, layered Flask architecture, database, routes and local setup	Implemented and verified
4	Git and GitHub Initialization	repository skeleton, ignore rules, templates, branch policy, GitHub remote	Implemented and verified
5	Feature Development and Branching	feature branches with commits and tests merged into develop	**Partially completed** - real GitHub PRs pending
6	MVP Completion and Collaboration	six workflows, a real merge conflict and release tags	Implemented and verified
7	Jenkins Installation and CI	Jenkins in Docker, the CI job, reports and artifacts	Implemented and verified (polling; Python not Maven)
8	Pipeline as Code and Deployment	the 10-stage Jenkinsfile, parameter, secrets and health-checked deploy	Implemented and verified
9	Selenium Test Design and Execution	five browser journeys, waits, isolated data, screenshots on failure	Implemented and verified
10	Continuous Testing in Jenkins	tests as a quality gate, proven with a deliberate defect	Implemented and verified
11	Docker Image and Container Lifecycle	the Dockerfile and every lifecycle operation, incl. persistent data	Implemented and verified
12	Jenkins-Docker Continuous Deployment	version+SHA images, registry, deploy, health check, record	Implemented and verified (local registry) - Docker Hub pending
13	Configuration Management with Ansible	inventory, variables and playbook provisioning a fresh server	Implemented and verified (container lab, not a VM)
14	Provisioning, Idempotency and Rollback	`changed=0` on repeat runs and a verified rollback	Implemented and verified (container lab)
15	End-to-End Release and Documentation	the whole flow, final documentation and the 12-minute demo	Implemented and verified, with pending external items

Also in this folder

File	Purpose
VIVA_CHEAT_SHEET.md	30-second intro, 2-minute explanation, the four stories, honest answers about limitations, demo commands
screenshot-index.md	every screenshot with task, purpose, date and status

File	Purpose
GENERATION_STATUS.md	progress record of this documentation pass
`assets/task-01/` … `assets/task-15/`	screenshots per task (`.txt` next to rendered terminal images = the raw output)
`assets/raw/`	raw logs captured on 2026-10-20 (Jenkins build #11, Docker lifecycle, Ansible runs)
`Complete_DevOps_Project_Report.docx` / `.pdf`	all 15 reports in one document
`word/Task_01_...docx` … `word/Task_15_...docx`	one Word document per task

The Markdown files are the source of truth; the Word/PDF files are generated from them.

Other project documentation

README - Project report - 15-task checklist - Viva notes - Evidence - UI design

00 - Project Audit (before writing the task reports)

Audit date: 2026-10-20. **Auditor:** documentation pass over the real repository, services and evidence. Every "verified" below means a command was run or a page was opened on this date and the result was seen.

1. Repository state

Item	Actual value (2026-10-20)	How checked
Local path	`D:\DevOPS\CICD_FOR_HOTEL_ROOM_BOOKING`	-
Current branch	`feature/ui-redesign`	`git branch --show-current`
Uncommitted work	The UI redesign (templates, CSS, `app.js`, fonts, 4 test assertions, `scripts/capture_ui.py`, `docs/09-ui-design.md`, `docs/ui-evidence/`) **and** one edit by the student to `docs/01-problem-definition.md`. None of it is committed or pushed.	`git status --short` (see `assets/task-15/git-status-final.png`)
Latest commit on `develop`	`1c2103c` Merge branch 'docs/history-rewrite' into develop	`git log`
`main`	`bea6a4c` (docs merge after release `v1.1.0`)	`git log --first-parent main`
Tags	`v1.0.0` -> `224de14`, `v1.1.0` -> `ce1726d` (annotated)	`git tag -n1`, `git cat-file -p v1.1.0`
Local branches	20: `main`, `develop`, 13 `feature/*` (including the unpushed `feature/ui-redesign`), 1 `fix/*`, 2 `experiment/*`, 2 `docs/*`	`git branch -a`
Remote	`origin` = https://github.com/KemNxts/hotel-room-booking-cicd (public)	`git remote -v`, GitHub API

Commit IDs changed once (important when reading evidence)

On 2026-10-12, before publishing to GitHub, commit messages were edited to remove a co-author line. File contents did not change, but every commit ID did. Evidence captured **before** that date (Jenkins logs of builds #1-#10, `docs/evidence/`) shows the **original** IDs; Git and GitHub today show the **current** IDs. The full mapping is in `docs/evidence/git/commit-id-map.md`. The IDs used most often:

Meaning	Original ID (old evidence)	Current ID (Git/GitHub now)
release v1.0.0	`661751d`	`224de14`
release v1.1.0	`d07929b`	`ce1726d`
deliberate defect	`554cad3`	`5844eec`
corrective commit	`30c6a05`	`2fb62d6`
merge-conflict resolution	`8728b62`	`9e766ac`
first commit (skeleton)	`08702db`	`541dad5`

Note: the Docker images in the local registry and the running staging/prod containers were built before the rewrite, so their tags and `/health` also show original IDs (e.g. prod = `1.1.0-d07929b`). Jenkins build #11 (2026-10-20) was built after the rewrite and shows the current ID `1c2103c`.

2. GitHub (checked through the public GitHub REST API, read-only)

Item	Result
Repository	exists, public, default branch `main`, last push 2026-10-12
Branches on GitHub	19 (everything except `feature/ui-redesign`)
Tags on GitHub	`v1.0.0`, `v1.1.0`
Issues	**0** - not created yet
Pull requests	**0** - not created yet; merges were done locally with `git merge --no-ff` and pushed

3. Fresh verification run on 2026-10-20

Check	Command	Result	Evidence
Lint	`.venv\Scripts\flake8 -j 1 app tests scripts wsgi.py gunicorn.conf.py`	no findings	`assets/task-09/full-test-suite.png`
Unit tests	`.venv\Scripts\python -m pytest tests/unit`	**59 passed**	same
Integration tests	`... pytest tests/integration`	**26 passed**	same
Selenium tests	`... pytest tests/e2e`	**5 passed**	`assets/task-09/selenium-run.png`
Screenshot on failure	temporary failing test (deleted afterwards)	screenshot + HTML saved by the hook	`assets/task-09/selenium-failure-*.png`
Jenkins build #11	Build with parameters `BRANCH=develop`, `APP_ENV=staging`	**SUCCESS**, all 10 stages, 90 tests, images `1.1.0-1c2103c` pushed and deployed, health check passed	`assets/raw/jenkins-build-11.log`, `assets/task-07/*`, `assets/task-08/*`
Jenkins history	Jenkins REST API	builds #1-#11 present; #2 and #9 FAILURE (deliberate), all others SUCCESS	`assets/task-07/jenkins-job-stage-view.png`
Docker lifecycle	scripted demo (build, tag, run, ports, logs, inspect, stop/start/restart, rm, new container on same volume)	all steps worked; an account created in the first container could log in from the second	`assets/raw/docker-lifecycle.txt`, `assets/task-11/lifecycle-*.png`
Local registry	`GET localhost:5000/v2/hotel-booking-app/tags/list`	8 tags incl. `1.1.0-1c2103c`	`assets/task-12/registry-tags.png`
Ansible, fresh lab server	runs from clean clones of tags v1.0.0 and v1.1.0	run 1 `changed=7`, run 2 `changed=5`, run 3 **`changed=0`**, rollback `changed=3` -> 1.0.0	`assets/raw/ansible-run1..4*.txt`
Restored release works	v1.0.0's own Selenium tests against the rolled-back server	**5 passed**	`assets/raw/ansible-run5-e2e-after-rollback.txt`
Demo commands in the reports	Docker demo (Task 11), Ansible lab demo incl. rollback (Tasks 13/14), merge-conflict replay in a throw-away clone (Task 6)	all ran as written	(outputs checked during the audit)

Check	Command	Result	Evidence
Running services	`/health` of each	dev `:5001` ok, staging `:8081` ok (`1c2103c`), prod `:8082` ok (`d07929b`), lab `:8088` ok, Jenkins `:8090` up, registry `:5000` up	`assets/task-15/all-environments-health.png`

4. Status of the 15 tasks

#	Task	Status	Reason
1	Problem definition and scope	Implemented and verified	Document exists and is published on GitHub
2	Agile planning and DevOps workflow	**Partially completed**	Backlog, stories, board, DoD, diagram exist; **GitHub issues = 0 (pending external action)**
3	Requirements, architecture and setup	Implemented and verified	Routes, layers and tests checked today
4	Git and GitHub initialization	Implemented and verified	Repo, templates, branches, tags on GitHub (API)
5	Feature development with branching	**Partially completed**	Feature branches + `--no-ff` merges verified; **real GitHub PRs = 0 (pending external action)**
6	MVP completion and collaboration	Implemented and verified	6 features pass tests today; conflict reproduced today in a clone; tags exist
7	Jenkins installation and CI	Implemented and verified	Build #11 today. Polling, not a webhook (no public URL). Maven not used - confirm with professor
8	Pipeline as code and deployment	Implemented and verified	Build #11 deployed staging and passed the health check
9	Selenium test design and execution	Implemented and verified	5/5 today; failure screenshot hook shown today
10	Continuous testing in Jenkins	Implemented and verified	Builds #2/#3/#9 records checked today (experiment itself run 2026-10-11)
11	Docker image and container lifecycle	Implemented and verified	Full lifecycle re-run today
12	Jenkins-Docker continuous deployment	Implemented and verified (local registry)	**Docker Hub publishing not tested - pending external action** (needs an account)
13	Configuration management with Ansible	Implemented and verified (container lab)	Target is a privileged Docker container, **not a separate VM**
14	Provisioning, idempotency and rollback	Implemented and verified (container lab)	Re-run today: `changed=0`, rollback + 5/5 Selenium
15	End-to-end release and documentation	Implemented and verified, with pending external items	See the limitations below

5. Limitations and pending actions (not hidden)

1. **GitHub issues** - none created. Backlog exists as `docs/02-agile-plan.md` (HB-01..HB-15).
2. **GitHub pull requests** - none created. PR texts prepared in `docs/pull-requests/`. Merges were real local merges, now visible on GitHub.
3. **Docker Hub** - publishing path is implemented (`REGISTRY` parameter + `registry-creds` credential) but never run; the local `registry:2` was used.

4. **Ansible target** - a disposable Ubuntu 24.04 container with systemd + SSH, not a separate VM or cloud server.
5. **Maven/Gradle/Ant and Tomcat** - not used. This is a Python project: pip + pytest + flake8 are used for build/test, Gunicorn + Nginx for serving. Whether this satisfies the brief **needs the professor's confirmation**.
6. **Webhook** - Jenkins polls Git every ~5 minutes; a GitHub webhook needs a publicly reachable Jenkins.
7. **UI redesign not committed** - application screenshots in Tasks 3, 6, 9 and 15 show the redesigned interface running from the working tree (dev server). The deployed staging/prod containers still run the v1.1.0 interface (see `assets/task-12/prod-portal-v1.1.0.png`). Commit it, merge it and run Jenkins to deploy it.
8. **Branch protection rules** on GitHub were not configured (documented as a policy in `CONTRIBUTING.md` only). Cannot be checked without logging in to GitHub.

6. Environment notes

â€¢ Windows 11, Docker Desktop 29.7.2, Python 3.13 venv (pipeline uses 3.11), Chrome for Selenium.

â€¢ Avast HTTPS scanning intercepts TLS; containers trust its root certificate through the optional `certs/` folder.

â€¢ Port 5000 is used by the local Docker registry, so the development server must use another port: `python -m flask --app wsgi run --port 5001`.

â€¢ Docker Desktop does not start automatically after a reboot.

7. About the screenshots in these reports

â€¢ Browser screenshots are real captures (headless Chrome) of the running portal, Jenkins, the registry and the public GitHub pages.

â€¢ "Terminal" images were produced by `scripts/capture_report_evidence.py`: the command was run for real, its raw output saved as `.txt` beside the image, and the output rendered as an image. Each such image says so in its footer. Images of older logs say which log file they were rendered from.

â€¢ No screenshot was edited. No GitHub PR, hosted registry or remote server is shown because none exists.

Task 1 - Problem Definition and Scope

A. Task title and objective

Task 1: Problem Definition and Scope

Objective: state clearly *what problem* the project solves, *for whom*, *what is in and out of scope*, and *how success will be measured* - before any code is written.

Professor's required deliverables

- â€¢ Problem statement and real-world need
- â€¢ Target users and stakeholders
- â€¢ Existing pain points
- â€¢ Objectives
- â€¢ Technical and operational constraints
- â€¢ Measurable success criteria
- â€¢ The approved MVP scope (15 tasks)

What this task demonstrates: that the project is driven by a real need and has a fixed, measurable scope, so later work (code, tests, pipeline) can be checked against it.

Current status: Implemented and verified. The document `docs/01-problem-definition.md` exists, covers every deliverable, and is published on GitHub (Figure 1.1). Its success criteria were re-checked against fresh results on 2026-10-20 (90 tests passed, Jenkins build #11 green, Ansible re-run `changed=0`).

Honest note: the stakeholders and pain points are a reasoned description of a small hotel. **No real hotel staff or guests were interviewed.**

B. Explanation for a complete beginner

An everyday picture. Before building a house you agree with the owner how many rooms it needs, what the budget is and when it counts as "finished". Without that, builders argue forever about whether the job is done. Task 1 is that agreement for software.

Problem statement - one short paragraph that says what is wrong today. *Why needed:* everyone works on the same problem. *In this project:* a small hotel takes bookings by phone and spreadsheet, so guests cannot see free rooms and the same room is sometimes given to two people.

Stakeholders - everyone affected by the system: the hotel owner, front-desk staff, guests, the development team, operations, and the course instructor. *Why needed:* each group wants something different (owner: no double bookings; guest: self-service; operator: safe deployments).

Scope and MVP - *scope* is the list of what will and will not be built. An **MVP** (Minimum Viable Product) is the smallest version that is genuinely useful. *Why needed:* it stops "scope creep" (endless extra features). *In this project:* the MVP is six booking workflows; payments, an admin screen and e-mails are deliberately **out of scope**.

Measurable success criteria - targets that can be checked with a yes/no answer or a number, e.g. "at least 50 automated tests, all passing". *Why needed:* "it works well" cannot be proven; "90 of 90 tests pass" can.

Why DevOps is part of the problem. DevOps (Development + Operations) is the practice of building, testing and releasing software automatically and safely. The hotel needs not only an app but also a safe way to change it: if a new version breaks bookings, guests suffer. So the project has two halves - the **booking portal** and the **automated delivery pipeline** (CI/CD: Continuous Integration / Continuous Delivery) that tests and deploys it.

C. What was actually implemented

The deliverable is the document `docs/01-problem-definition.md`. Its sections map one-to-one to the professor's list:

Professor's item	Section in `docs/01-problem-definition.md`
Problem statement and need	"Problem statement" (manual bookings, double bookings, manual deployments)
Target users	"Target users": Guest, Developer, Operator/examiner
Stakeholders	"Stakeholders": owner, front desk, guests, dev team, operations, instructor
Pain points	"Existing pain points": 6 items (double bookings ... no rollback)
Objectives	"Objectives": O1-O7 (6 workflows, no overlap, Jenkins, quality gate, Docker, Ansible, rollback)
Constraints	"Constraints": Python/Flask/SQLite, Windows + Docker Desktop, no public server, 10-day schedule, antivirus HTTPS scanning
Success criteria	"Measurable success criteria": a table with target **and** achieved evidence
MVP scope	"Approved MVP scope": six workflows in, list of out-of-scope items

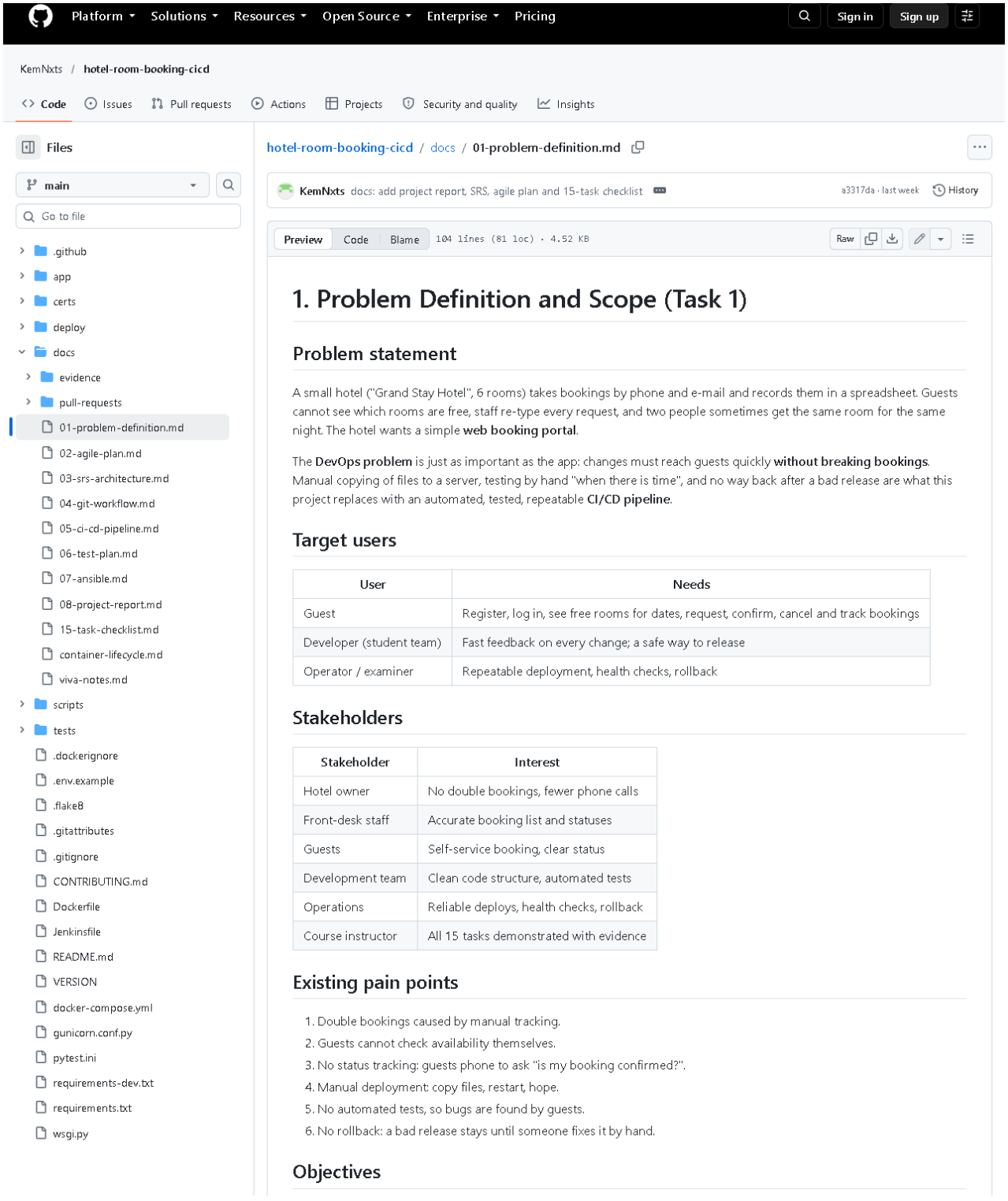
The six MVP workflows (all implemented in `app/`):

9. Registration and login - `app/services/auth_service.py`
10. Room availability by date - `BookingService.search_rooms`
11. Booking request (creates `PENDING`) - `BookingService.request_booking`
12. Booking confirmation (`PENDING -> CONFIRMED`) - `confirm_booking`
13. Booking cancellation (`-> CANCELLED`) - `cancel_booking`
14. Booking status tracking ("My bookings") - `list_bookings`

Success criteria compared with fresh results (2026-10-20)

Criterion (from the document)	Target	Result today
MVP workflows	6 of 6	6 of 6 - Selenium journeys 5/5 passed
Automated tests	>= 50, all passing	90 passed (59 unit, 26 integration, 5 Selenium)
No double booking	impossible, even concurrently	10-thread race test + database trigger tests pass
Pipeline duration	< 5 minutes	Jenkins build #11: 103 seconds
Failed test blocks deploy	always	builds #2 and #9 failed, Deploy stage skipped
Deploy traceable	version + commit visible	staging `/health` = `{"commit":"1c2103c",...,"version":"1.1.0"}`
Ansible re-run	`changed=0`	run 3 on 2026-10-20: `changed=0`
Rollback works	verified	rollback to 1.0.0, then 5/5 Selenium journeys passed

D. Screenshots and evidence



(part 1 of 3)

#	Objective
O1	Deliver the six MVP booking workflows (below)
O2	Guarantee no overlapping active bookings for a room - in code AND in the database
O3	Automate build -> static checks -> tests -> package -> deploy in Jenkins
O4	Block deployment automatically when any test fails (quality gate)
O5	Package the app as Docker images tagged with version + commit
O6	Provision a server and deploy with Ansible, idempotently
O7	Recover from a bad release by rolling back to a tested earlier version

Constraints

Technical

- Python 3.11 + Flask + SQLite (team skills; no Java rewrite - see the "build tool" note in 03-srs-architecture.md).
- SQLite allows one writer at a time: fine for one hotel, not for many servers.
- Development machine is Windows 11; Jenkins, Docker and Ansible run in Linux containers (Docker Desktop).
- No public server: GitHub webhooks cannot reach the local Jenkins, so it polls.

Operational

- 10-day schedule, student team.
- No paid cloud services; local Docker registry instead of Docker Hub (Docker Hub is supported by setting a parameter + credential).
- Antivirus HTTPS scanning on the dev machine breaks downloads inside containers; handled by the optional `certs/` folder.

Measurable success criteria

Criterion	Target	Achieved (evidence)
MVP workflows working	6 of 6	6 of 6 - Selenium journeys + integration tests (06-test-plan.md)
Automated tests	>= 50, all passing	90 passing (59 unit, 26 integration, 5 Selenium)
Double booking	impossible, even under concurrency	10-thread race test + DB trigger tests pass
Pipeline duration	< 5 min	~1 min (Jenkins builds #1-#8)
Failed test blocks deploy	always	Builds #2 and #9: deploy skipped, staging unchanged
Every deploy traceable	version + commit visible	`/health` and page footer, e.g. `1.1.0` / `d07929b`
Ansible re-run	`changed=0`	`docs/evidence/ansible/run3-repeat-idempotent.txt`
Rollback	no rebuild, verified working	Ansible `rollback.yml` switches to the stored older image; 5/5 Selenium journeys pass on the restored release

Approved MVP scope

In scope - the six booking workflows:

1. Registration and login
2. Room availability by date
3. Booking request (creates a PENDING booking)
4. Booking confirmation (PENDING -> CONFIRMED)
5. Booking cancellation (PENDING/CONFIRMED -> CANCELLED)
6. Booking status tracking ("My bookings")

(part 2 of 3)

...delivered through the 15 course tasks (planning, Git workflow, Jenkins, Selenium, Docker, Ansible, release). See 15-task-checklist.md.

Out of scope (deliberately): payments, admin/staff UI, e-mail notifications, room photos, multiple hotels, automatic expiry of PENDING bookings, Kubernetes/cloud.

(part 3 of 3)

Figure 1.1 - `docs/01-problem-definition.md` as published on the public GitHub repository (captured 2026-10-20).

What to notice: the sections Problem statement, Target users, Stakeholders, Pain points, Objectives O1-O7, Constraints, the

success-criteria table with an "Achieved (evidence)" column, and the in/out-of-scope lists. **Why it proves the task:** every deliverable of Task 1 is present in one published, version-controlled document.

Supporting evidence used by the criteria (in other task folders): `assets/task-09/full-test-suite.png` (90 tests), `assets/task-14/ansible-run3-repeat-idempotent.png` (`changed=0`).

E. How to demonstrate this task to the professor (about 2 minutes)

Before the demo: open the project folder in VS Code, or open https://github.com/KemNxts/hotel-room-booking-cicd/blob/main/docs/01-problem-definition.md in a browser.

1. **Open the document** (VS Code: `docs/01-problem-definition.md`, press `Ctrl+Shift+V` for the preview). *Say:* "Task 1 is the problem definition. A small hotel manages bookings by phone and spreadsheet, which causes double bookings and manual, risky software releases."
2. **Scroll to "Stakeholders" and "Existing pain points".** *Say:* "Each stakeholder wants something different - the owner wants no double bookings, the operator wants safe deployments."
3. **Scroll to "Objectives" and "Measurable success criteria".** *Say:* "Every objective has a measurable target. For example: at least 50 tests - we have 90; a re-run of Ansible must report zero changes - it does."
4. **Prove one criterion live** (PowerShell, in `D:\DevOPS\CICD_FOR_HOTEL_ROOM_BOOKING`):*Expected:* `85 passed, 5 deselected` in about 15 seconds.

```
.venv\Scripts\python -m pytest -q -m "not e2e"
```

5. **Scroll to "Approved MVP scope".** *Say:* "Six workflows are in scope; payments and an admin screen are deliberately out of scope."

If something fails: if `.venv` is missing, run `python -m venv .venv` then `.venv\Scripts\pip install -r requirements-dev.txt`. The document itself needs nothing running.

F. Expected screenshots for the final submission

1. `assets/task-01/github-doc-01-problem-definition.png` - the published problem definition - **exists, captured 2026-10-20**.
2. (Optional, cross-reference) `assets/task-09/full-test-suite.png` - proves the "90 tests" criterion - **exists, captured 2026-10-20**.

G. Probable viva questions and answers

1. **What problem does your project solve?** Two problems: hotel guests cannot book rooms themselves and double bookings happen; and the software is released manually without tests. The portal solves the first, the CI/CD pipeline the second.
2. **Who are your stakeholders?** Hotel owner, front-desk staff, guests, the development team, operations (who deploy and roll back), and the course instructor.
3. **What is an MVP and what is yours?** The smallest useful version. Ours is six workflows: register/login, availability, booking request, confirmation, cancellation, status tracking.
4. **Why did you leave payments out?** Payments add security and legal requirements (card data) that do not help demonstrate DevOps; they are listed as future work.
5. **What makes your success criteria "measurable"?** Each has a number or yes/no check: 90 tests passing, pipeline under 5 minutes (build #11 took 103 s), `changed=0` on a repeat Ansible run.
6. **Did you interview real hotel staff?** No. The stakeholders and pain points are a reasoned model of a small hotel; that is a limitation, and real interviews would be the next step.

7. **Which constraint affected the design most?** SQLite: one writer at a time, so double-booking prevention uses a locking transaction and a database trigger; and the lack of a public server, which is why Jenkins polls Git instead of using a webhook.
8. **(Project-specific) Where is the scope written down and how is it versioned?** In `docs/01-problem-definition.md`, committed in Git and published on GitHub, so changes to scope are tracked like code changes.
9. **(Project-specific) How do you know objective O2 (no overlapping bookings) is met?** Tests prove it: `test_simultaneous_requests_create_only_one_booking` (10 threads, exactly 1 booking) and `test_trigger_rejects_overlapping_insert`.
10. **Why is the delivery pipeline part of the problem definition?** Because a broken release would break bookings for real guests; the hotel's need includes "change the system safely", not just "have a website".

H. Common mistakes and troubleshooting

Symptom	Likely cause	Inspect	Safe fix	Confirm
The document title does not render as a heading in some viewers	Leading spaces before `#` (the working copy currently has two)	first line of `docs/01-problem-definition.md`	remove the spaces before `# 1. Problem...`	preview shows a large heading
Examiner asks for numbers that differ from the document	Numbers changed since the document was written	`pytest -q` output, Jenkins build page	quote today's numbers and say the document is a snapshot	rerun the command live
"Scope creep" question: why no admin page?	Scope not explained	"Approved MVP scope" section	point to the out-of-scope list	-
Success criterion cannot be shown live	Service (Jenkins/Docker) not running	Docker Desktop tray icon	start Docker Desktop, wait ~1 minute	`docker ps` lists `hotel-jenkins-jenkins-1`

I. Quick revision notes

Five key points

1. Problem: manual bookings cause double bookings; manual releases are risky.
2. Two deliverables: a booking portal **and** an automated CI/CD pipeline.
3. MVP = 6 workflows; payments/admin/e-mail out of scope.
4. Every objective has a measurable target and is met with evidence.
5. Stakeholders are modelled, not interviewed - say so honestly.

30-60 second spoken explanation "Task 1 defines the problem. A small hotel takes bookings by phone and spreadsheet, so guests can't see availability and rooms get double-booked. We also need to change the software safely, so the project has two parts: a web portal with six booking workflows, and a CI/CD pipeline that tests and deploys it automatically. The scope is fixed in `docs/01-problem-definition.md` with measurable success criteria - for example at least 50 automated tests; today 90 pass - and anything else, like payments, is explicitly out of scope."

Caveat: stakeholder needs are assumed, not gathered from real users.

Readiness checklist

â€¢ â˜ I can name the 6 workflows from memory.

- â€¢ â˜ I can name 4 stakeholders and what each wants.
- â€¢ â˜ I can quote 3 success criteria and today's result.
- â€¢ â˜ I can open `docs/01-problem-definition.md` quickly.

Task 2 - Agile Planning and DevOps Workflow

A. Task title and objective

Task 2: Agile Planning and DevOps Workflow

Objective: plan the work in small, testable pieces (user stories), order them (backlog), schedule them (sprints), agree when a piece is finished (Definition of Done), and show the full DevOps flow from code to operations.

Professor's required deliverables

- â€¢ User stories with acceptance criteria
- â€¢ Product backlog
- â€¢ A 15-task Kanban/Scrum board
- â€¢ Sprint plan
- â€¢ Definition of Done
- â€¢ DevOps lifecycle diagram (development to operations)
- â€¢ Backlog linked to repository issues where possible

Current status: Partially completed.

Part	Status
Stories, acceptance criteria, backlog, board, sprints, DoD, diagram	Implemented and verified - `docs/02-agile-plan.md`, published on GitHub
Backlog items as **GitHub issues**	**Pending external action** - the GitHub API reports **0 issues** (checked 2026-10-20); see Figure 2.2

B. Explanation for a complete beginner

An everyday picture. Planning a wedding: you write every job on a sticky note ("book the hall", "send invitations"), put them in order of importance, and move each note across a whiteboard from *To do* to *Doing* to *Done*. Agile planning is exactly that, for software.

Agile - a way of working in short cycles, delivering a small working piece each time and adjusting the plan as you learn. *Problem it solves:* big "deliver everything at the end" projects discover problems too late.

User story - a requirement written from the user's point of view: "*As a guest I want to cancel a booking so that the room is released.*" *Why:* it keeps the focus on what the user gets, not on technical details.

Acceptance criteria - the conditions that must be true for a story to be accepted, e.g. "a cancelled booking cannot be cancelled again". In this project every criterion is checked by an **automated test**, and the test names are written next to the story.

Product backlog - the ordered list of all work items. Here: HB-01 to HB-15, one per course task.

Kanban board - columns (Backlog, To do, In progress, Review, Done) that show where each item is. **Scrum** - work in fixed periods called **sprints** (here 3 short sprints), each ending with a review and a retrospective.

Definition of Done (DoD) - one checklist that applies to every item, so "done" means the same thing to everyone (tests pass, pipeline green, merged...).

DevOps lifecycle - the loop Plan -> Code -> Build -> Test -> Release -> Deploy -> Operate -> Monitor -> back to Plan. In this project each step has a real tool (Git, pip, pytest/Selenium, Docker, Jenkins, Ansible, `/health`).

GitHub issue - a ticket on GitHub that tracks one backlog item. Linking items to issues lets commits and pull requests say "Closes #12".

C. What was actually implemented

File: `docs/02-agile-plan.md`.

User stories (9). US1-US6 cover the six booking workflows, US7 automatic testing, US8 one-command deployment, US9 rollback. Each row lists the acceptance criteria and the tests that check them, for example:

Story	Acceptance criterion (excerpt)	Automated check
US3 request a booking	never two active bookings overlap, even simultaneously	`test_simultaneous_requests_create_only_one_booking`
US5 cancel	cannot cancel twice	`TestCancelBooking::test_cannot_cancel_twice`
US7 every commit tested	any failure skips packaging and deployment	Jenkins builds #2, #9

Backlog. HB-01 ... HB-15, one per course task, with the related story and status.

Kanban board (final state). All HB items in *Done*; out-of-scope ideas (payments, admin UI, auto-expiry of PENDING bookings) stay in *Backlog*.

Sprint plan. Sprint 1 Foundation (days 1-3), Sprint 2 Automation (4-6), Sprint 3 Operations & release (7-10).

Definition of Done (7 points): feature/fix branch with conventional commits; automated tests; flake8 clean and all tests pass; Jenkins green; merged with `--no-ff` using the PR checklist; docs updated; no secrets.

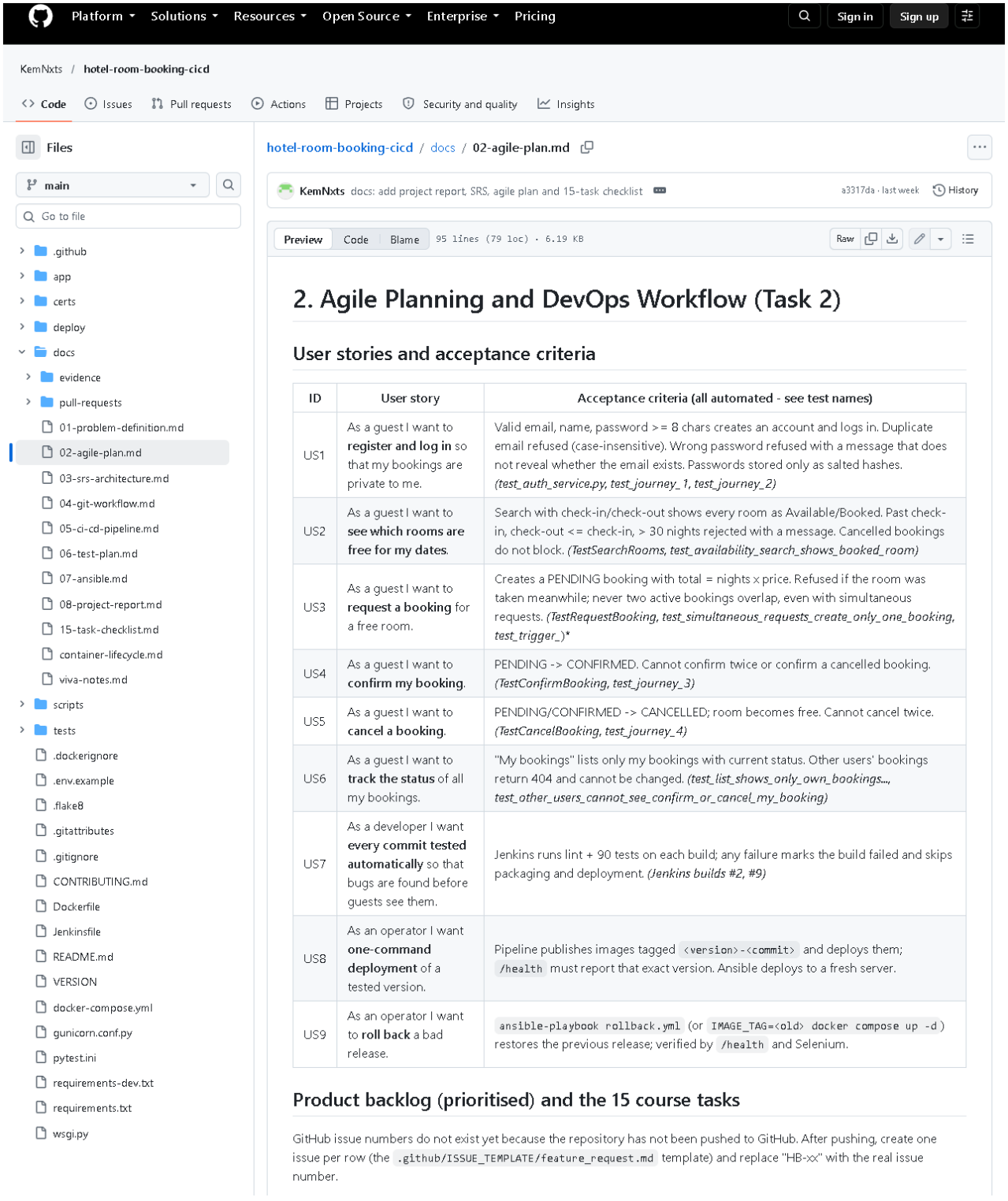
DevOps lifecycle diagram. A Mermaid diagram (text that GitHub draws as a picture). It shows the **failure loop**: if tests fail, nothing is released and the developer fixes the code.

How the DoD is enforced in this repository

DoD point	Real mechanism
tests pass	`pytest` stages in `Jenkinsfile`
flake8 clean	"Static checks" stage
pipeline green	Deploy runs only after every test stage
PR checklist	`.github/pull_request_template.md`
no secrets	`.gitignore` excludes `.env`, `credentials/`, `*.key`, `certs/*.crt`

GitHub issues - what is missing. The repository is public, but no issues were created (Figure 2.2). The backlog text says so: "replace HB-xx with the real issue number" after creating them.

D. Screenshots and evidence



(part 1 of 3)

ID	Backlog item	Course task	Story	Status
HB-01	Problem definition and scope	1	-	Done
HB-02	Backlog, sprint plan, DoD, lifecycle diagram	2	-	Done
HB-03	SRS, architecture, schema, route list	3	-	Done
HB-04	Repository skeleton, templates, branching policy	4	-	Done
HB-05	Feature branch + PR flow for the MVP	5	US1-US6	Done (local merges; PR texts prepared)
HB-06	Complete MVP, merge conflict, version tag	6	US1-US6	Done
HB-07	Jenkins job: checkout, install, lint, tests, reports, artifact	7	US7	Done
HB-08	Jenkinsfile with deploy + parameter + health check	8	US8	Done
HB-09	Selenium journeys with screenshots on failure	9	US1-US6	Done
HB-10	Tests in Jenkins, quality gate, defect experiment	10	US7	Done
HB-11	Dockerfile, compose, lifecycle demo	11	US8	Done
HB-12	Jenkins -> registry -> deploy with version/SHA tags	12	US8	Done
HB-13	Ansible inventory and playbook	13	US8	Done
HB-14	Idempotency + rollback	14	US9	Done
HB-15	Final release, report, viva preparation	15	-	Done

Kanban board (final state)

Backlog	To do	In progress	Review	Done
Payment integration *(out of scope)*	-	-	-	HB-01 … HB-15
Admin UI *(out of scope)*				
Auto-expire PENDING bookings *(future)*				

During the project each card moved left -> right; a card reached **Review** when its feature branch was ready to merge, and **Done** when it met the Definition of Done below.

Sprint plan (10 working days, 3 short sprints)

Sprint	Days	Goal	Backlog items
1 - Foundation	1-3	Repo, layered app, all 6 workflows, unit tests	HB-01..HB-06, HB-09
2 - Automation	4-6	Jenkins pipeline, Docker images, registry, quality gate	HB-07, HB-08, HB-10..HB-12
3 - Operations & release	7-10	Ansible, rollback, releases v1.0.0/v1.1.0, report, viva	HB-13..HB-15

(part 2 of 3)

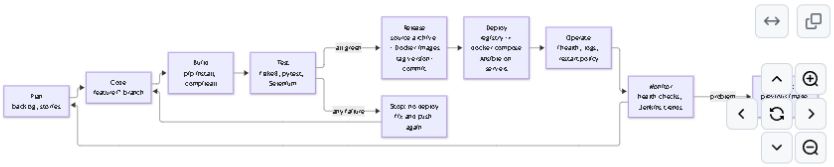
Each sprint ends with a short review (demo of the working increment) and a retrospective (what to keep / improve). Example improvement carried into sprint 3: "release-footer" and "contact-footer" were built in parallel and conflicted; resolution documented in 04-git-workflow.md.

Definition of Done

A backlog item is **Done** only when:

1. Code is on a `feature/*` (or `fix/*`) branch with Conventional-Commit messages.
2. New behaviour has automated tests (unit/integration, Selenium for UI journeys).
3. `flake8` is clean and **all** tests pass locally.
4. The Jenkins pipeline is green for the branch.
5. Merged into `develop` with `--no-ff` (PR checklist in `.github/pull_request_template.md` completed).
6. Documentation (README / docs) updated.
7. No secrets in the repository.

DevOps lifecycle



(part 3 of 3)

Figure 2.1 - `docs/02-agile-plan.md` on GitHub (captured 2026-10-20). **What to notice:** the user-story table with test names, backlog HB-01-HB-15, the Kanban table, sprint plan, the Definition of Done, and at the bottom the DevOps lifecycle

diagram drawn by GitHub from Mermaid text. **Why it proves the task:** all planning deliverables exist in one versioned document.

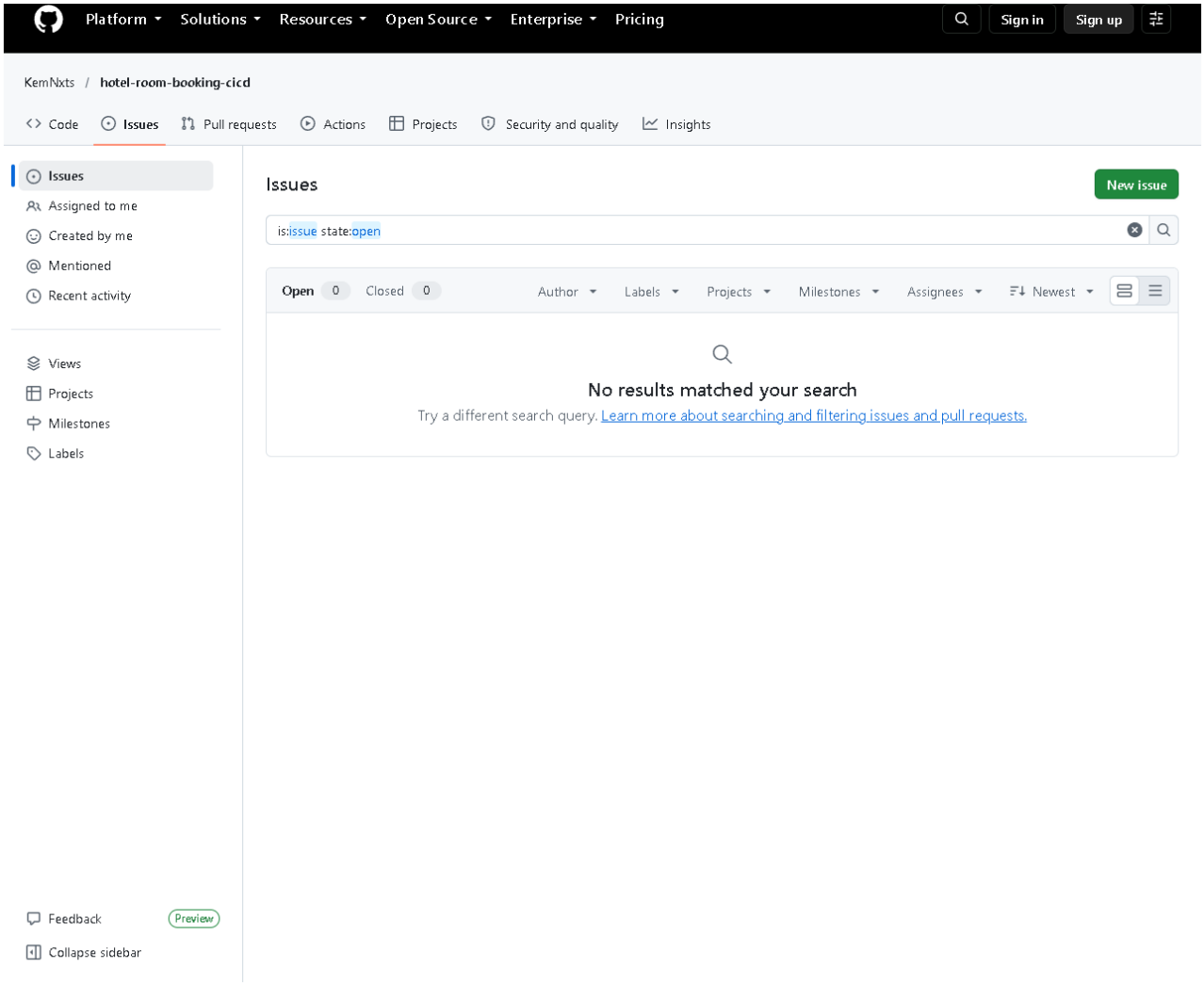


Figure 2.2 - The repository's Issues page (all states) shows no issues (captured 2026-10-20). **What to notice:** zero open and zero closed issues. **Why it matters:** this honestly shows the remaining step; the backlog is local (in the document), not yet on GitHub Issues.

E. How to demonstrate this task to the professor (about 3 minutes)

Before the demo: open `docs/02-agile-plan.md` (VS Code preview with `Ctrl+Shift+V`, or on GitHub - GitHub draws the diagram).

15. **User stories.** *Say:* "Each requirement is a user story with acceptance criteria, and next to each criterion I wrote the automated test that checks it."
16. **Prove one story live** (PowerShell, project folder):*Expected:* `7 passed, 31 deselected` (the cancellation tests, e.g. `test_cannot_cancel_twice`). *Say:* "US5 says a booking cannot be cancelled twice - this test proves it."

```
.venv\Scripts\python -m pytest -q tests/unit/test_booking_service.py -k "cancel"
```

17. **Backlog and Kanban board.** *Say:* "The 15 course tasks are backlog items HB-01 to HB-15. The board shows their final state."
18. **Definition of Done.** *Say:* "An item is done only when tests pass, the Jenkins pipeline is green and it is merged - the pipeline enforces most of this."

19. **Lifecycle diagram** (bottom of the page on GitHub). Trace it with the mouse: "Plan, code, build, test - if any test fails we stop - release, deploy, operate, monitor, roll back if needed."
20. **Be honest about issues:** open https://github.com/KemNxts/hotel-room-booking-cicd/issues - "The backlog is in the repository; creating GitHub issues for it is the remaining step."

To complete the pending part (needs your GitHub login): on GitHub click *Issues -> New issue -> Feature request*, create one issue per HB item (title e.g. "HB-07 Jenkins job"), then edit the backlog table to show the issue numbers and commit the change.

F. Expected screenshots for the final submission

6. `assets/task-02/github-doc-02-agile-plan.png` - planning document incl. diagram - **exists, captured 2026-10-20**.
7. `assets/task-02/github-issues-empty.png` - honest state of GitHub issues - **exists, captured 2026-10-20**.
8. `assets/task-02/github-issues-created.png` - issues HB-01..HB-15 on GitHub - **requires manual capture after the issues are created** (open the Issues page, all states, screenshot).

G. Probable viva questions and answers

3. **What is a user story?** A short requirement from the user's viewpoint: "As a I want so that ."
4. **What are acceptance criteria?** The conditions that make a story accepted. Ours are checked automatically by named tests.
5. **Difference between a backlog and a sprint?** The backlog is the whole ordered to-do list; a sprint is a short time-box in which a chosen part of the backlog is completed.
6. **Kanban or Scrum - which did you use?** A light mix: a Kanban board to show status and three short Scrum-style sprints with a review each.
7. **What is your Definition of Done?** Branch + conventional commits, tests, flake8 clean, all tests pass, Jenkins green, `--no-ff` merge with the PR checklist, docs updated, no secrets.
8. **How is the DoD enforced, not just written?** Jenkins runs flake8 and all 90 tests; deployment happens only if they pass (shown by failed build #2).
9. **(Project-specific) Where are your GitHub issues?** None exist yet (0 issues on GitHub). The backlog is in `docs/02-agile-plan.md`; creating the issues needs my GitHub account and is the remaining step.
10. **(Project-specific) Which story covers rollback and how is it proven?** US9; proven by `deploy/ansible/rollback.yml` runs where `/health` showed 1.0.0 again and 5/5 Selenium journeys passed.
11. **What does the DevOps lifecycle diagram show that a simple list does not?** The feedback loops: failed tests go back to Code, and problems in operation lead to rollback and new planning.
12. **What happened in your sprint retrospective?** Two features built in parallel (two footers) caused a merge conflict; the lesson was to agree on shared files earlier. The conflict and its resolution are documented.

H. Common mistakes and troubleshooting

Symptom	Cause	Inspect	Fix	Confirm
Diagram shows as code text in VS Code	VS Code preview does not draw Mermaid by default	`docs/02-agile-plan.md`	show it on GitHub, or install a Mermaid preview extension	the boxes and arrows appear
Examiner asks to see the issues	Issues not created	GitHub Issues page	explain the pending step; create them if allowed	Issues page lists HB items

Symptom	Cause	Inspect	Fix	Confirm
`pytest -k cancel` selects 0 tests	wrong folder/typo	command	run from the project folder exactly as written	"passed" count > 0
Story says a test name that no longer exists	test renamed	`grep -rn "def test_" tests`	update the story table	names match

I. Quick revision notes

Five key points

11. 9 user stories; every acceptance criterion maps to an automated test.
12. Backlog HB-01..HB-15 = the 15 course tasks.
13. 3 sprints: foundation, automation, operations & release.
14. Definition of Done is enforced by Jenkins (lint + 90 tests before deploy).
15. GitHub issues are **not created yet** - pending.

30-60 second explanation "I planned the work with nine user stories, for example 'as a guest I want to cancel a booking'. Each has acceptance criteria, and each criterion is checked by a named automated test. The 15 course tasks form the backlog HB-01 to HB-15, shown on a Kanban board and split into three sprints. Our Definition of Done says tests pass, Jenkins is green and the work is merged - and Jenkins enforces that. The DevOps diagram shows the loop from planning to monitoring. The one missing piece is that the backlog isn't yet entered as GitHub issues."

Caveat: no GitHub issues exist; the board is a table in a document.

Readiness checklist

- â€¢ â˜ I can explain story vs. acceptance criterion vs. test.
- â€¢ â˜ I can recite the Definition of Done.
- â€¢ â˜ I can trace the lifecycle diagram including the failure loop.
- â€¢ â˜ I can explain honestly why there are no GitHub issues.

Task 3 - Requirements, Architecture and Technology Setup

A. Task title and objective

Task 3: Requirements, Architecture and Technology Setup

Objective: write down what the system must do (requirements), design how its parts fit together (architecture), choose the technologies, set up the development environment, and prove the booking rules work.

Professor's required deliverables

- â€¢ SRS summary (Software Requirements Specification)
- â€¢ Use-case diagram
- â€¢ Application architecture diagram
- â€¢ Database schema, relationships and data model
- â€¢ API / route list
- â€¢ Local development setup
- â€¢ Technology choices with reasons
- â€¢ Working registration, login, availability, request, confirmation, cancellation, status tracking
- â€¢ Booking rules that block invalid dates, overlaps, unauthorised cancellation and inconsistent states

Current status: Implemented and verified. Re-checked 2026-10-20: the route list was printed from the real app (Figure 3.2), 59 unit + 26 integration tests passed, and the app started locally. Technology note: build and test use **Python tools (pip, pytest, flake8)**, not Maven - see section C.

B. Explanation for a complete beginner

An everyday picture. A restaurant has a waiter (takes your order), a chef (decides how to cook it, following the recipes) and a store-room keeper (fetches ingredients). Each does one job. Our application is organised the same way.

Requirements / SRS - a list of what the system must do (*functional*: "a guest can cancel a booking") and how well (*non-functional*: "no double bookings even with simultaneous requests", "passwords never stored in plain text").

Use case - one goal a user achieves with the system ("Request booking"). A **use-case diagram** shows the users and their goals.

Architecture - how the code is split into parts and how they talk.

Layers in this project

Layer	Folder	Restaurant role	Job
Controller	`app/controllers/`	waiter	reads the web request, calls a service, returns a page
Service	`app/services/`	chef	business rules (dates valid? room free? may this user cancel?)
DAO (Data Access Object)	`app/dao/`	store-room keeper	the **only** place with SQL
Model	`app/models.py`	the plate	plain data objects (User, Room, Booking)

Why layers? Each part can be changed and tested alone. The rules can be tested without a browser; the database could be replaced by changing only the DAOs.

Flask - a small Python web framework: it maps a URL like `/bookings` to a Python function. **Route** - that mapping. **SQLite** - a database stored in a single file, built into Python. **Schema** - the definition of the tables.

Virtual environment (venv) - a private folder of Python packages for this project, so versions don't clash with other projects. **pip** installs the pinned versions listed in `requirements.txt`.

C. What was actually implemented

Documents: `docs/03-srs-architecture.md` - 11 functional requirements (FR1-FR11), 7 non-functional (NFR1-NFR7), a use-case diagram, architecture diagram, ER (Entity-Relationship) diagram, booking state machine, route list and technology table.

Route list (printed from the running code, Figure 3.2)

Method	URL	Controller function	Purpose
GET	`/rooms?check_in=&check_out=`	`rooms.list_rooms`	rooms / availability
GET, POST	`/register`, `/login`	`auth.register`, `auth.login`	accounts
POST	`/logout`	`auth.logout`	log out
GET / POST	`/bookings`	`my_bookings` / `create`	list / request
GET	`/bookings/<id>`	`detail`	one booking (owner only)
POST	`/bookings/<id>/confirm`, `/cancel`	`confirm`, `cancel`	change status
GET	`/health`	`health`	status + version for Docker/Jenkins/Ansible

How one request travels (example: "Book room 201")

21. Browser sends `POST /bookings` with `room_id`, `check_in`, `check_out`, `csrf_token`.
22. `app/controllers/csrf.py` `check_csrf()` (line 25) rejects the request with HTTP 400 if the token is missing or wrong.
23. `app/controllers/booking_routes.py` `create()` (line 22) calls the service.
24. `app/services/booking_service.py` `request_booking()` (line 48) runs `validate_stay()` (line 19): valid dates, check-in not in the past, check-out after check-in, at most 30 nights.
25. It calls `BookingDao.create_if_room_free()` in `app/dao/booking_dao.py` (line 34), which inside a locked transaction checks for an overlap and inserts a `PENDING` row.
26. SQLite triggers in `app/schema.sql` (lines 42 and 56) refuse any overlapping active booking even if code above had a bug.
27. The controller redirects to `/bookings/<id>`, which shows status `PENDING`.

The core rule - overlap - in SQL (`app/dao/booking_dao.py`, from line 9; the `#` explanations are added here for the reader and are not in the file):

```
_OVERLAP_SQL = """
    check_in < :check_out          # existing stay starts before the new one ends
    AND check_out > :check_in      # and ends after the new one starts
    AND status IN ('PENDING', 'CONFIRMED')   # cancelled bookings never block
"""
```

A guest may check in on the day the previous guest checks out (strict `<`).

Database (`app/schema.sql`) - tables `users`, `rooms` (6 seeded rooms: 101, 102 Single; 201, 202 Double; 301 Deluxe; 401 Suite), `bookings` (foreign keys to both). Constraints: `CHECK (check_out > check_in)`, `CHECK (status IN ('PENDING','CONFIRMED','CANCELLED'))`, two overlap triggers.

Booking rules and the tests that prove them

Rule	Test
invalid / past dates, > 30 nights	`TestValidateStay::test_invalid_stays_are_rejected`
no overlap for one room	`test_overlapping_booking_is_rejected`
concurrency: 10 simultaneous requests -> 1 booking	`test_simultaneous_requests_create_only_one_booking`
only the owner can view/confirm/cancel	`test_other_users_cannot_see_confirm_or_cancel_my_booking`
no cancel twice / no confirm of cancelled	`test_cannot_cancel_twice`, `test_cannot_confirm_a_cancelled_booking`
database blocks overlaps directly	`test_trigger_rejects_overlapping_insert`

Technology choices and the Maven/Tomcat question

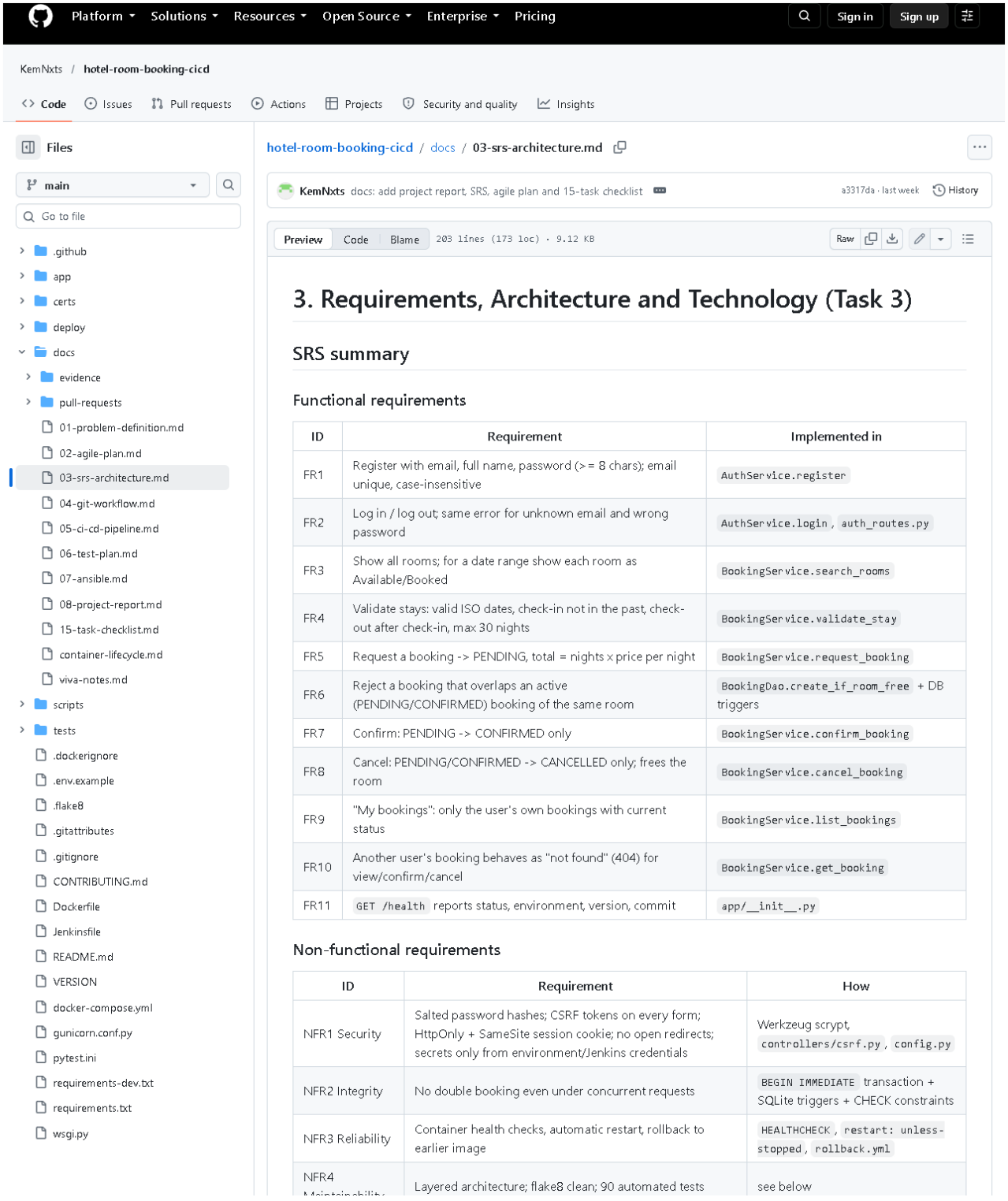
Brief mentions (Java world)	This project (Python)	Same purpose
Maven/Gradle/Ant `pom.xml`	`requirements.txt` + pip	pinned dependencies
`mvn compile`	`python -m compileall` + `flake8`	catch syntax/style errors
`mvn test` (JUnit)	`pytest` with JUnit-XML reports	run tests, report to Jenkins
WAR file	source tarball + Docker images	deployable artifact
Tomcat	Gunicorn behind Nginx	serve the app

pip and pytest are **not** Maven and are not presented as Maven. Whether the Python equivalents satisfy the brief **must be confirmed with the professor**.

Local setup (Windows PowerShell)

```
python -m venv .venv
.venv\Scripts\pip install -r requirements-dev.txt
.venv\Scripts\python -m flask --app wsgi run --port 5001   # 5000 is taken by the local Docker registry
```

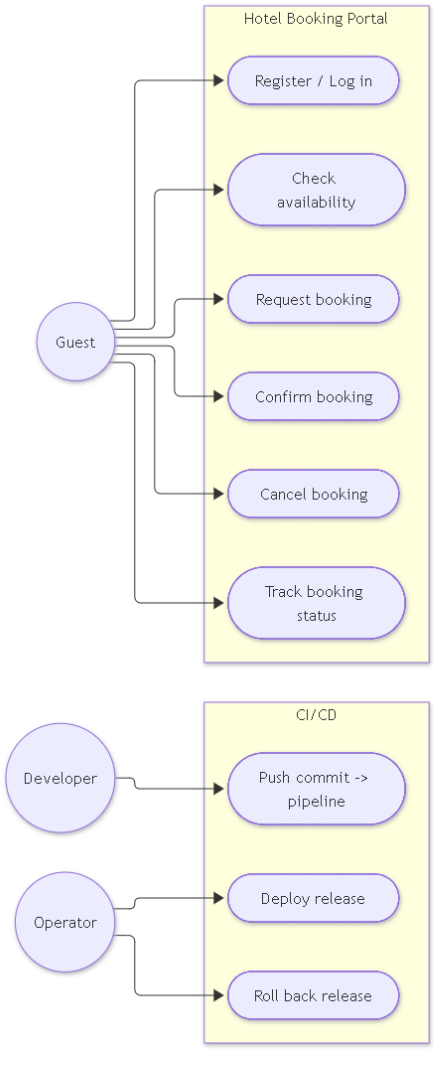
D. Screenshots and evidence



(part 1 of 4)

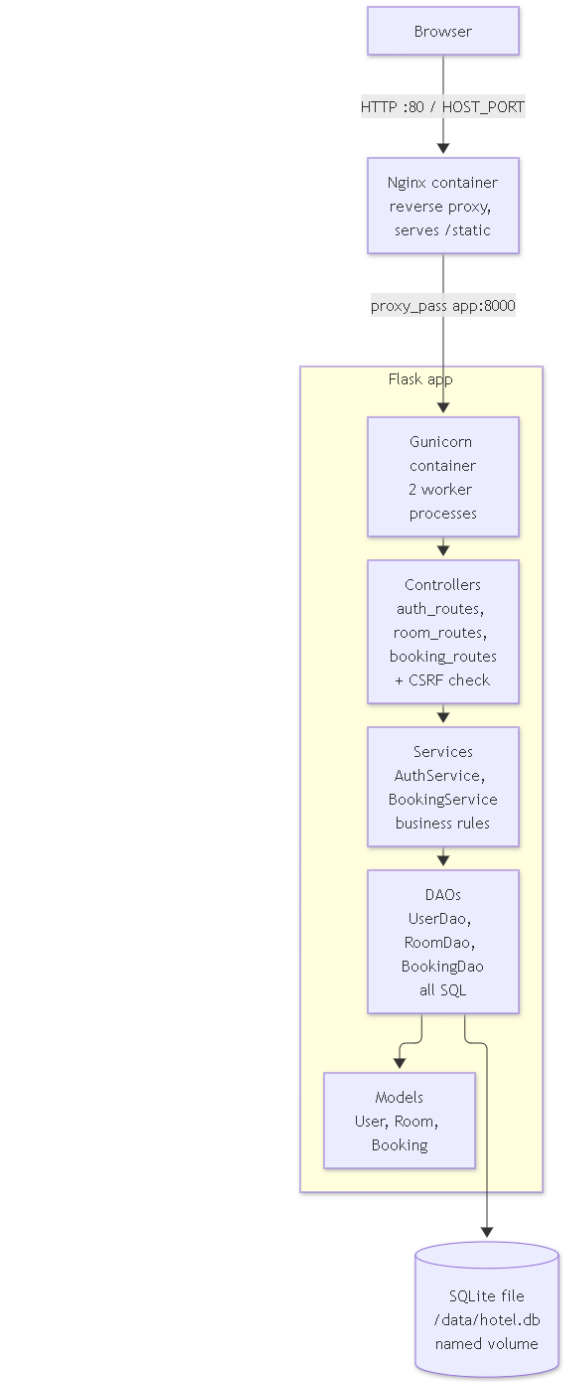
Maintainability		
NFR5 Portability	Same Docker image in every environment; config via env vars	12-factor config
NFR6 Traceability	Every deployment identifies its version and commit	`/health`, footer, image labels
NFR7 Performance	Pages respond well under 1 s for one hotel (target; no load test was run)	SQLite + indexes; Gunicorn 2 workers

Use-case diagram



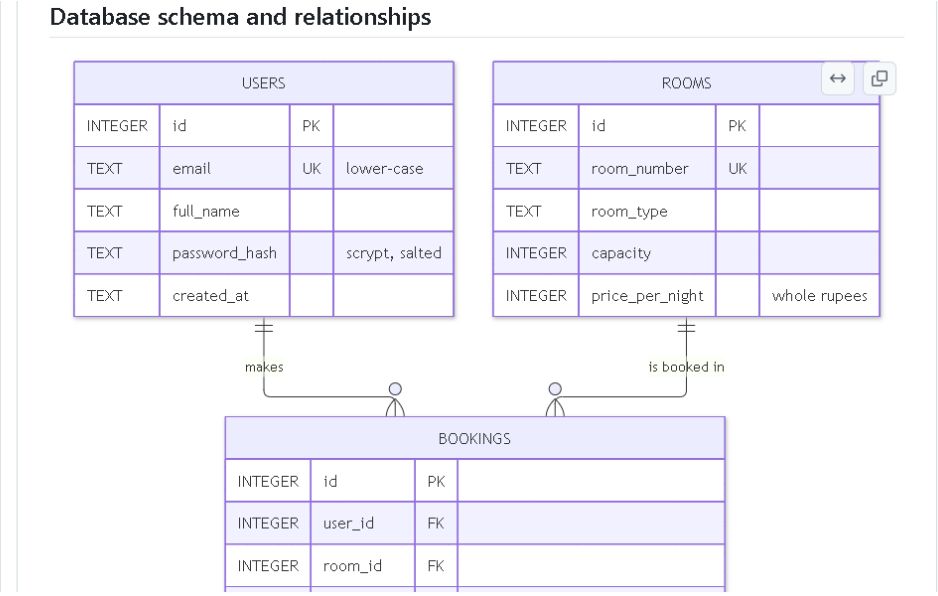
Application architecture

(part 2 of 4)



Rules of the layers: controllers never contain SQL, services never touch HTTP, DAOs are the only place with SQL. Services get their DAOs through the constructor (dependency injection), so unit tests give them a test database.

(part 3 of 4)



(part 4 of 4)

Figure 3.1 - `docs/03-srs-architecture.md` on GitHub with diagrams rendered (captured 2026-10-20). **Notice:** the FR/NFR tables, the use-case diagram, the layered architecture diagram, the ER diagram and the booking state machine. **Proves:** SRS, use cases, architecture, data model and route list exist.

PowerShell - Flask route list (controllers)

```
$ "D:\DevOPS\CICD_FOR_HOTEL_ROOM_BOOKING\.venv\Scripts\python.exe" -m flask --app wsgi routes --sort rule
Endpoint              Methods    Rule
--------------------  ---------  --------------------------------
rooms.index           GET        /
bookings.my_bookings  GET        /bookings
bookings.create       POST       /bookings
bookings.detail       GET        /bookings/<int:booking_id>
bookings.cancel       POST       /bookings/<int:booking_id>/cancel
bookings.confirm      POST       /bookings/<int:booking_id>/confirm
health                GET        /health
auth.login            GET, POST  /login
auth.logout           POST       /logout
auth.register         GET, POST  /register
rooms.list_rooms      GET        /rooms
static                GET        /static/<path:filename>
[2026-10-20 18:25:34,313] INFO in __init__: Hotel booking app started: env=dev version=dev commit=unknown
```

Rendered from real command output captured 2026-10-20 12:55 UTC in CICD_FOR_HOTEL_ROOM_BOOKING | raw text: flask-routes.txt

Figure 3.2 - `python -m flask --app wsgi routes` run on 2026-10-20 (rendered from real output). **Notice:** every URL with its HTTP method and controller function. **Proves:** the route list in the document matches the real code.

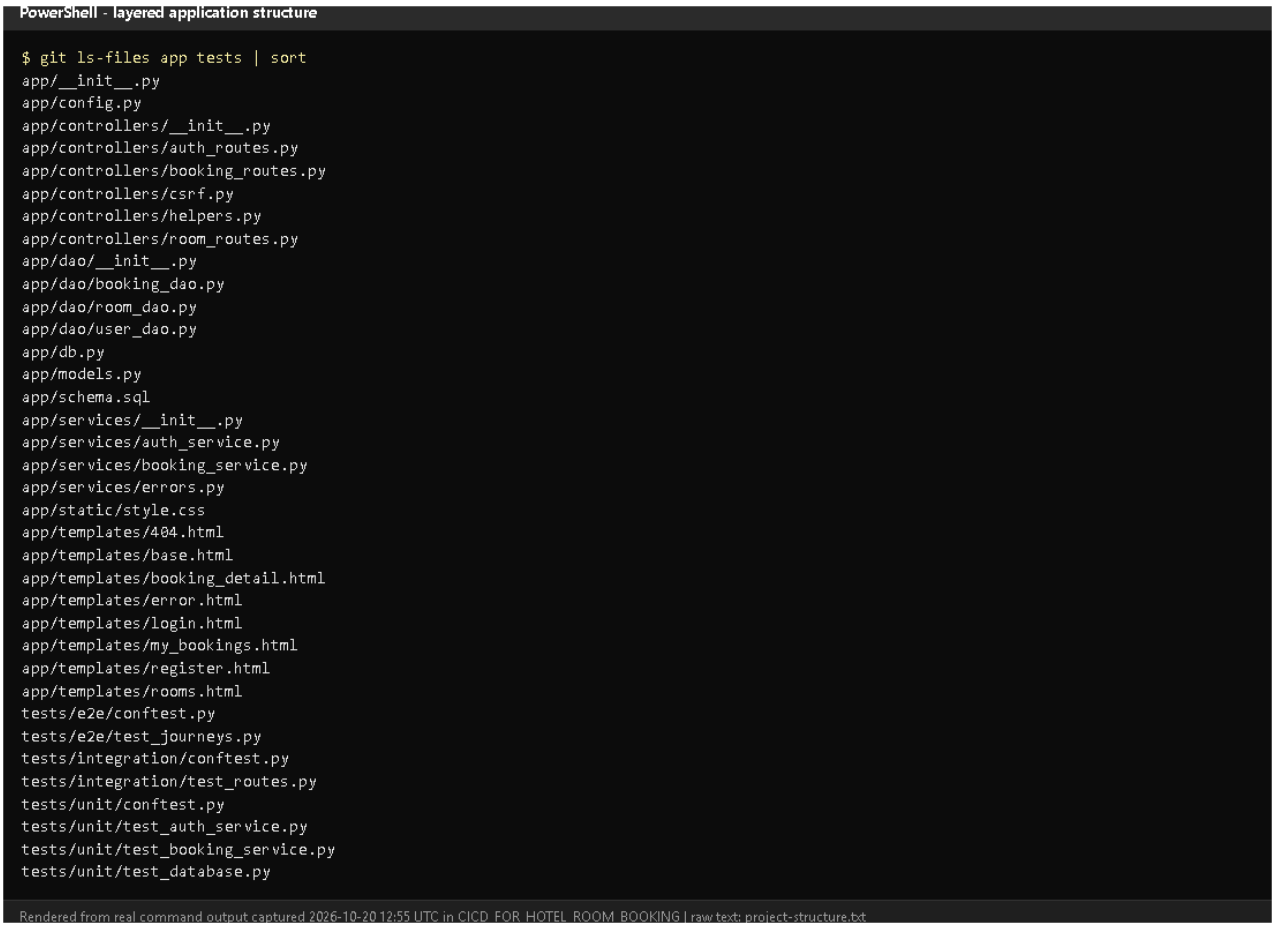
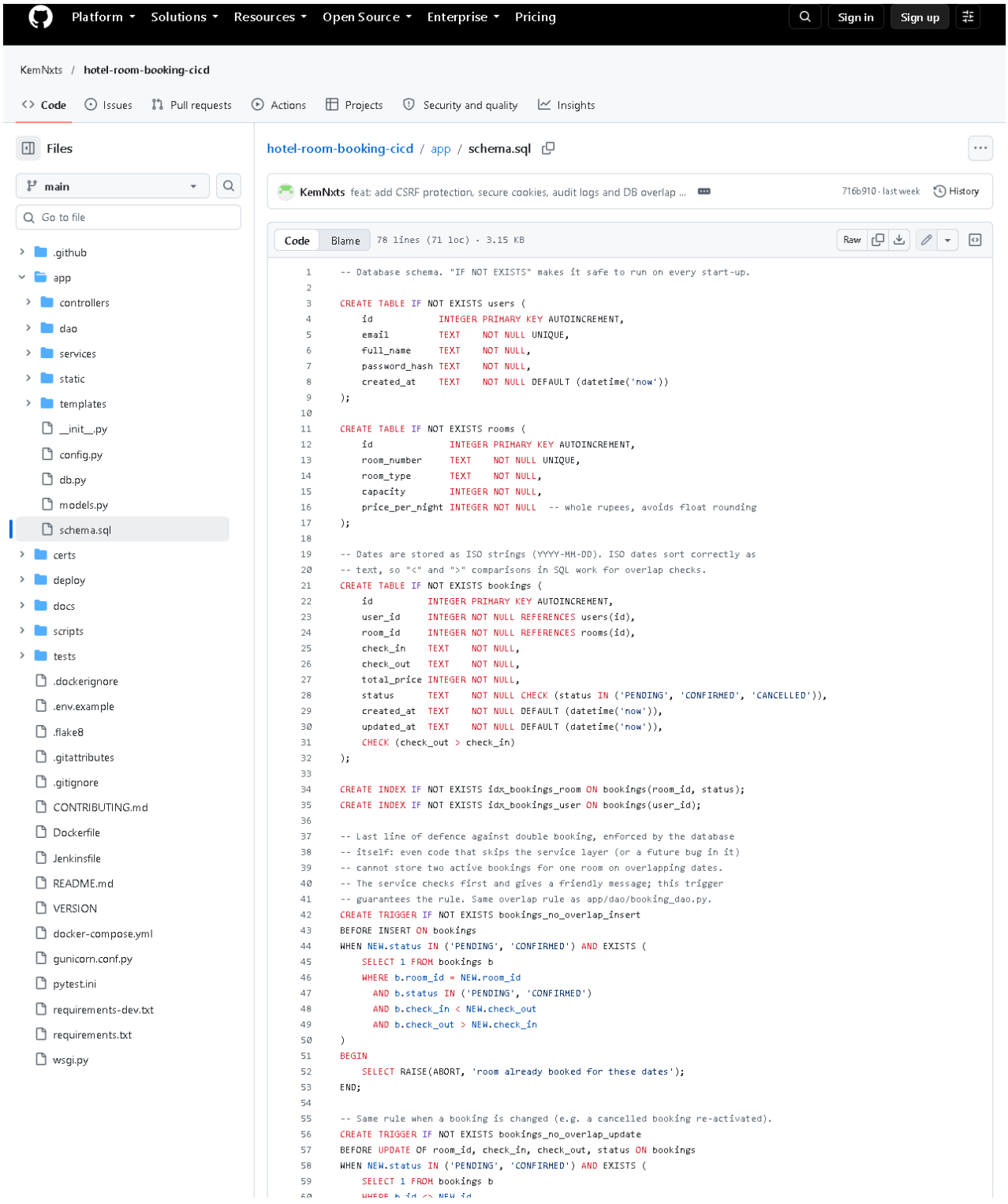


Figure 3.3 - Files under `app/` and `tests/` (from `git ls-files`, rendered from real output). **Notice:** separate `controllers/`, `services/`, `dao/` folders and `models.py`. **Proves:** the layered architecture is real, not only drawn.


Platform
Solutions
Resources
Open Source
Enterprise
Pricing
Sign in
Sign up
KemNxts / hotel-room-booking-cicd
Code
Issues
Pull requests
Actions
Projects
Security and quality
Insights
Files
main
Go to file
.github
app
controllers
dao
services
static
templates
__init__.py
config.py
db.py
models.py
schema.sql
certs
deploy
docs
scripts
tests
.dockerignore
.env.example
.flake8
.gitattributes
.gitignore
CONTRIBUTING.md
Dockerfile
Jenkinsfile
README.md
VERSION
docker-compose.yml
gunicorn.conf.py
pytest.ini
requirements-dev.txt
requirements.txt
wsgi.py
hotel-room-booking-cicd / app / schema.sql
KemNxts feat: add CSRF protection, secure cookies, audit logs and DB overlap ...
716b910 · last week
History
Code
Blame
78 lines (71 loc) · 3.15 KB
Raw
-- Database schema. "IF NOT EXISTS" makes it safe to run on every start-up.
CREATE TABLE IF NOT EXISTS users (
id INTEGER PRIMARY KEY AUTOINCREMENT,
email TEXT NOT NULL UNIQUE,
full_name TEXT NOT NULL,
password_hash TEXT NOT NULL,
created_at TEXT NOT NULL DEFAULT (datetime('now'))
);
CREATE TABLE IF NOT EXISTS rooms (
id INTEGER PRIMARY KEY AUTOINCREMENT,
room_number TEXT NOT NULL UNIQUE,
room_type TEXT NOT NULL,
capacity INTEGER NOT NULL,
price_per_night INTEGER NOT NULL -- whole rupees, avoids float rounding
);
-- Dates are stored as ISO strings (YYYY-MM-DD). ISO dates sort correctly as
-- text, so "<" and ">" comparisons in SQL work for overlap checks.
CREATE TABLE IF NOT EXISTS bookings (
id INTEGER PRIMARY KEY AUTOINCREMENT,
user_id INTEGER NOT NULL REFERENCES users(id),
room_id INTEGER NOT NULL REFERENCES rooms(id),
check_in TEXT NOT NULL,
check_out TEXT NOT NULL,
total_price INTEGER NOT NULL,
status TEXT NOT NULL CHECK (status IN ('PENDING', 'CONFIRMED', 'CANCELLED')),
created_at TEXT NOT NULL DEFAULT (datetime('now')),
updated_at TEXT NOT NULL DEFAULT (datetime('now')),
CHECK (check_out > check_in)
);
CREATE INDEX IF NOT EXISTS idx_bookings_room ON bookings(room_id, status);
CREATE INDEX IF NOT EXISTS idx_bookings_user ON bookings(user_id);
-- Last line of defence against double booking, enforced by the database
-- itself: even code that skips the service layer (or a future bug in it)
-- cannot store two active bookings for one room on overlapping dates.
-- The service checks first and gives a friendly message; this trigger
-- guarantees the rule. Same overlap rule as app/dao/booking_dao.py.
CREATE TRIGGER IF NOT EXISTS bookings_no_overlap_insert
BEFORE INSERT ON bookings
WHEN NEW.status IN ('PENDING', 'CONFIRMED') AND EXISTS (
SELECT 1 FROM bookings b
WHERE b.room_id = NEW.room_id
AND b.status IN ('PENDING', 'CONFIRMED')
AND b.check_in < NEW.check_out
AND b.check_out > NEW.check_in
)
BEGIN
SELECT RAISE(ABORT, 'room already booked for these dates');
END;
-- Same rule when a booking is changed (e.g. a cancelled booking re-activated).
CREATE TRIGGER IF NOT EXISTS bookings_no_overlap_update
BEFORE UPDATE OF room_id, check_in, check_out, status ON bookings
WHEN NEW.status IN ('PENDING', 'CONFIRMED') AND EXISTS (
SELECT 1 FROM bookings b

(part 1 of 2)

```
61          AND b.room_id = NEW.room_id
62          AND b.status IN ('PENDING', 'CONFIRMED')
63          AND b.check_in < NEW.check_out
64          AND b.check_out > NEW.check_in
65      )
66      BEGIN
67          SELECT RAISE(ABORT, 'room already booked for these dates');
68      END;
69
70      -- Demo rooms. INSERT OR IGNORE + the UNIQUE room_number means re-running this
71      -- (e.g. several Gunicorn workers starting at once) never creates duplicates.
72      INSERT OR IGNORE INTO rooms (room_number, room_type, capacity, price_per_night) VALUES
73          ('101', 'Single',   1, 2500),
74          ('102', 'Single',   1, 2500),
75          ('201', 'Double',   2, 4000),
76          ('202', 'Double',   2, 4000),
77          ('301', 'Deluxe',   3, 6500),
78          ('401', 'Suite',    4, 9500);
```

(part 2 of 2)

Figure 3.4 - `app/schema.sql` on GitHub. **Notice:** three tables, CHECK constraints, the two `bookings_no_overlap_*` triggers, the six seeded rooms.

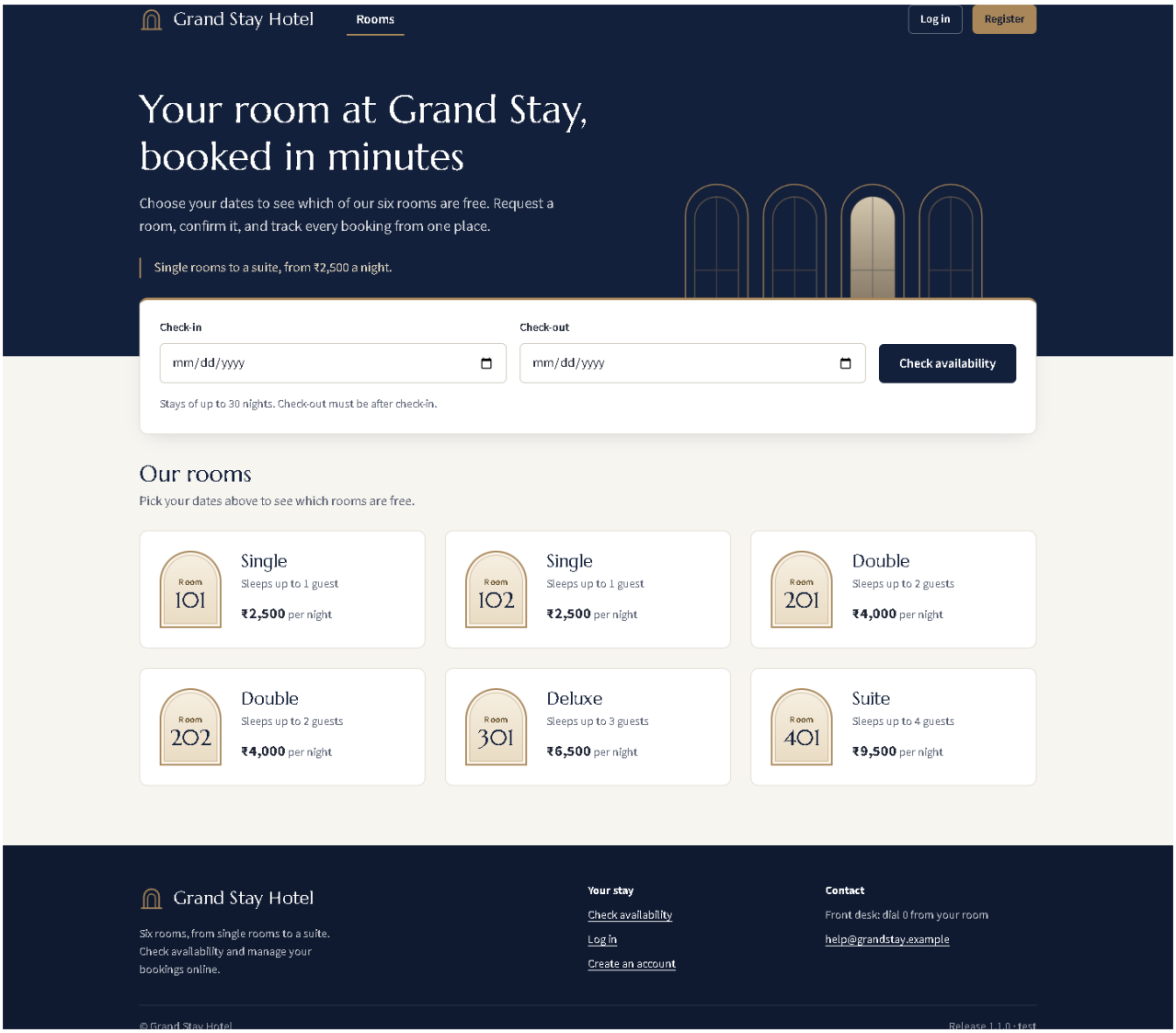


Figure 3.5 - Rooms page served by the local development server (redesigned UI from the working tree, captured 2026-10-20). **Notice:** the six seeded rooms with type, capacity and price.

E. How to demonstrate this task to the professor (about 4 minutes)

Before the demo: PowerShell open in `D:\DevOPS\CICD_FOR_HOTEL_ROOM_BOOKING`; VS Code open on the same folder.

9. **Show the document** `docs/03-srs-architecture.md` (or on GitHub to see diagrams). *Say:* "These are the functional and non-functional requirements, the use cases and the architecture."
10. **Start the application** (PowerShell):*Expected:* `Running on http://127.0.0.1:5001`. Leave this window open.

```
.venv\Scripts\python -m flask --app wsgi run --port 5001
```

11. **Open http://127.0.0.1:5001** in the browser - the six rooms appear. Open http://127.0.0.1:5001/health - `{"status": "ok", ...}`.
12. **Show the route list** in a second PowerShell window:

```
.venv\Scripts\python -m flask --app wsgi routes
```

13. **Trace one request in VS Code:** open `app/controllers/booking_routes.py` (`create`), then `app/services/booking_service.py` (`request_booking`), then `app/dao/booking_dao.py` (`create_if_room_free`). *Say:* "Controller, service, DAO - only the DAO has SQL."
14. **Prove the rules:***Expected:* `85 passed`. Point at the overlap and concurrency tests.

```
.venv\Scripts\python -m pytest -q tests/unit tests/integration
```

15. Stop the server with `Ctrl+C` in its window.

Recovery: "An attempt was made to access a socket..." means the port is taken - use another port (`--port 5002`). "No module named flask" means the venv is missing: `python -m venv .venv` and `.venv\Scripts\pip install -r requirements-dev.txt`.

F. Expected screenshots for the final submission

13. `assets/task-03/github-doc-03-srs-architecture.png` - SRS + diagrams - **exists, 2026-10-20**.
14. `assets/task-03/flask-routes.png` - real route list - **exists, 2026-10-20**.
15. `assets/task-03/project-structure.png` - layered folders - **exists, 2026-10-20**.
16. `assets/task-03/github-file-app_schema.sql.png` - schema - **exists, 2026-10-20**.
17. `assets/task-03/app-desktop-01-home.png` - running app - **exists, 2026-10-20**.

G. Probable viva questions and answers

16. **What is an SRS?** A Software Requirements Specification: what the system must do (functional) and qualities it must have (non-functional).
17. **Explain your layers.** Controllers handle HTTP, services hold business rules, DAOs hold all SQL, models are plain data. Each only calls the layer below.
18. **Why separate services from controllers?** Rules can be unit-tested without a web request, and they are not duplicated in several routes.
19. **Why SQLite?** Built into Python, zero setup, one file; enough for one hotel. Limitation: one writer at a time and one server only.
20. **How do you prevent double bookings?** Three layers: the service checks; the DAO checks and inserts inside `BEGIN IMMEDIATE` (a locked transaction); and database triggers refuse overlaps. A 10-thread test proves only one wins.
21. **(Project-specific) What does `/health` return and who uses it?** `{"status","env","version","commit"}`; used by the Docker HEALTHCHECK, the Jenkins health-check stage and the Ansible playbook.
22. **(Project-specific) Why can't another user cancel my booking?** `BookingService.get_booking` checks the owner and raises "not found" otherwise, so the route returns 404 - proven by an integration test.

23. **Is pip your Maven?** No. pip installs Python packages and pytest runs tests; they do the same jobs as Maven's dependency and test phases, but they are different tools. I need the professor's confirmation that this is acceptable.
24. **Why Gunicorn and Nginx instead of Tomcat?** Tomcat runs Java servlets. Python apps run on a WSGI server (Gunicorn); Nginx is the public entry point and serves static files.
25. **What is the booking state machine?** PENDING -> CONFIRMED, PENDING or CONFIRMED -> CANCELLED; nothing else. Status changes use `UPDATE ... WHERE status IN (allowed)` so check and change happen atomically.
26. **What is CSRF and how do you stop it?** Cross-Site Request Forgery: another site making your browser submit our forms. Every form has a secret token stored in the session; POSTs without it get HTTP 400.

H. Common mistakes and troubleshooting

Symptom	Cause	Inspect	Fix	Confirm
`python wsgi.py` fails: "access a socket ... forbidden"	port 5000 used by the local Docker registry	`Get-NetTCPConnection -LocalPort 5000`	run with `--port 5001`	page opens on :5001
HTTP 400 "Your form has expired"	CSRF token missing/old	browser tab opened before a restart	reload the page and resubmit	form works
`RuntimeError: SECRET_KEY must be set`	started with `APP_ENV=prod` without a key	environment variables	set `SECRET_KEY`, or use dev mode	app starts
Data from earlier tests appears	local DB `instance/hotel.db`	`instance/` folder	delete `instance/hotel.db` to reset (local dev only)	6 rooms, no bookings

I. Quick revision notes

Five key points

6. 11 functional + 7 non-functional requirements in `docs/03-srs-architecture.md`.
7. Layers: controllers -> services -> DAOs -> SQLite; models carry data.
8. Overlap rule `check_in < new_out AND check_out > new_in`, enforced three times.
9. Routes printed from real code; `/health` is used by Docker, Jenkins, Ansible.
10. Python tooling (pip/pytest/flake8, Gunicorn/Nginx) - not Maven/Tomcat; confirm with the professor.

30-60 second explanation "The requirements are in an SRS with use cases, architecture and data-model diagrams. The Flask app has layers: controllers handle web requests, services apply booking rules, DAOs contain all SQL, and SQLite stores users, rooms and bookings. A booking request passes the CSRF check, date validation, and an overlap check inside a locked transaction, and database triggers block any overlap that slips through. Ninety automated tests prove these rules."

Caveat: Python tools are used instead of Maven/Tomcat - needs professor approval.

Readiness checklist

- â€¢ â˜ I can start the app on port 5001 and open `/health`.
- â€¢ â˜ I can trace a booking request through the three layers.
- â€¢ â˜ I can explain the overlap rule and the three protections.
- â€¢ â˜ I can explain the Maven/Tomcat difference honestly.

Task 4 - Git and GitHub Repository Initialization

A. Task title and objective

Task 4: Git and GitHub Repository Initialization

Objective: put the project under version control from day one, with a clean structure, rules for branches and commits, templates for issues and pull requests, and a remote copy on GitHub.

Professor's required deliverables

- â€¢ README
- â€¢ `.gitignore` (and `.gitattributes`)
- â€¢ Folder structure
- â€¢ Issue and pull-request templates
- â€¢ Branch naming policy
- â€¢ Meaningful commit history
- â€¢ Initial source baseline (skeleton commit)

Current status: Implemented and verified. Checked 2026-10-20 with Git commands (Figures 4.1-4.3) and the public GitHub repository (Figures 4.4-4.6): 19 branches and both release tags are on GitHub.

One note: the branch *policy* is written in `CONTRIBUTING.md`; GitHub "branch protection rules" (settings that technically block direct pushes) were **not** configured - that needs the repository owner's login.

B. Explanation for a complete beginner

An everyday picture. Git is like the "version history" of a Google Doc, but for a whole folder of files, and much more precise: every saved version has a name, an author, a date and a message saying *why* it changed.

Term	Plain meaning	In this project
Repository (repo)	the project folder plus its full history	`D:\DevOPS\CICD_FOR_HOTEL_ROOM_BOOKING`
Commit	one saved snapshot with a message	e.g. `541dad5 chore: initialise repository skeleton`
Branch	a separate line of work, so experiments don't disturb stable code	`main`, `develop`, `feature/*`
Merge	joining a branch's work into another branch	features merged into `develop`
Tag	a permanent name for one commit, e.g. a release	`v1.0.0`, `v1.1.0`
Remote	a copy of the repo on a server	`origin` = GitHub
Push / pull	send commits to / fetch commits from the remote	`git push origin develop`
`.gitignore`	list of files Git must never save	`.env` (secrets), `.venv`, `*.db`
`.gitattributes`	rules for how files are stored, e.g. line endings	LF line endings in the repo

Git vs GitHub. *Git* is the tool on your computer that records history. *GitHub* is a website that hosts a copy of a Git repository and adds collaboration features (issues, pull requests, web view). You can use Git without GitHub, but not the other way round.

Why it matters for DevOps: Jenkins builds whatever is committed, so Git is the starting point of the whole pipeline - and every deployment can be traced back to one commit.

C. What was actually implemented

Repository files

Deliverable	File	Content
README	`README.md`	features, stack, layout, run/test/Docker/Jenkins/Ansible instructions
Ignore rules	`.gitignore`	Python caches, `.venv/`, `instance/`, `*.db`, `reports/`, `.env`, `deploy/ansible/credentials/`, `*.pem`, `*.key`, `certs/*.crt`
Line endings	`.gitattributes`	`* text=auto eol=lf` - the same line endings on Windows and Linux (needed for scripts run in Linux containers); `*.bat`/`*.ps1` keep Windows CRLF; `*.png`/`*.jpg` are binary
Issue templates	`.github/ISSUE_TEMPLATE/bug_report.md`, `feature_request.md`	forms for bugs and features
PR template	`.github/pull_request_template.md`	author + reviewer checklists
Branch & commit rules	`CONTRIBUTING.md`	`main` / `develop` / `feature/*` / `fix/*` / `experiment/*`, Conventional Commits (`feat:`, `fix:`, `test:`, `ci:`, `docs:` …)

Folder structure - `app/` (controllers, services, dao, templates, static), `tests/` (unit, integration, e2e), `deploy/` (nginx, jenkins, ansible), `docs/`, `scripts/`, plus `Dockerfile`, `docker-compose.yml`, `Jenkinsfile`.

Initial baseline. The first commit `541dad5 chore: initialise repository skeleton` (2026-10-11) added exactly the skeleton: `.gitattributes`, the two issue templates, the PR template, `.gitignore`, `CONTRIBUTING.md`, `README.md` and empty `deploy/ansible`, `tests/e2e`, `tests/unit` folders (Figure 4.2).

Branches (20 local) - `main` (releases), `develop` (integration), 13 `feature/*` (including the unpushed `feature/ui-redesign`), `fix/jenkins-stale-reports`, `experiment/defect-demo` and `-2`, `docs/final-build-record`, `docs/history-rewrite`.

Tags - `v1.0.0` -> `224de14`, `v1.1.0` -> `ce1726d` (annotated tags with a message).

Remote - `origin` = https://github.com/KemNxts/hotel-room-booking-cicd (public). Pushed on 2026-10-12. GitHub shows 19 branches and both tags.

Commit messages follow the convention, e.g. from `git log`: `feat: add CSRF protection, ...`, `test: add route-level integration tests ...`, `ci: add lint, versioned images, ...`, `fix: stop Jenkins publishing stale test reports ...`.

History rewrite (be ready to explain). On 2026-10-12, before pushing, commit messages were edited to remove a co-author line. Contents stayed the same but all commit IDs changed; old evidence shows the old IDs. Mapping: `docs/evidence/git/commit-id-map.md`.

D. Screenshots and evidence

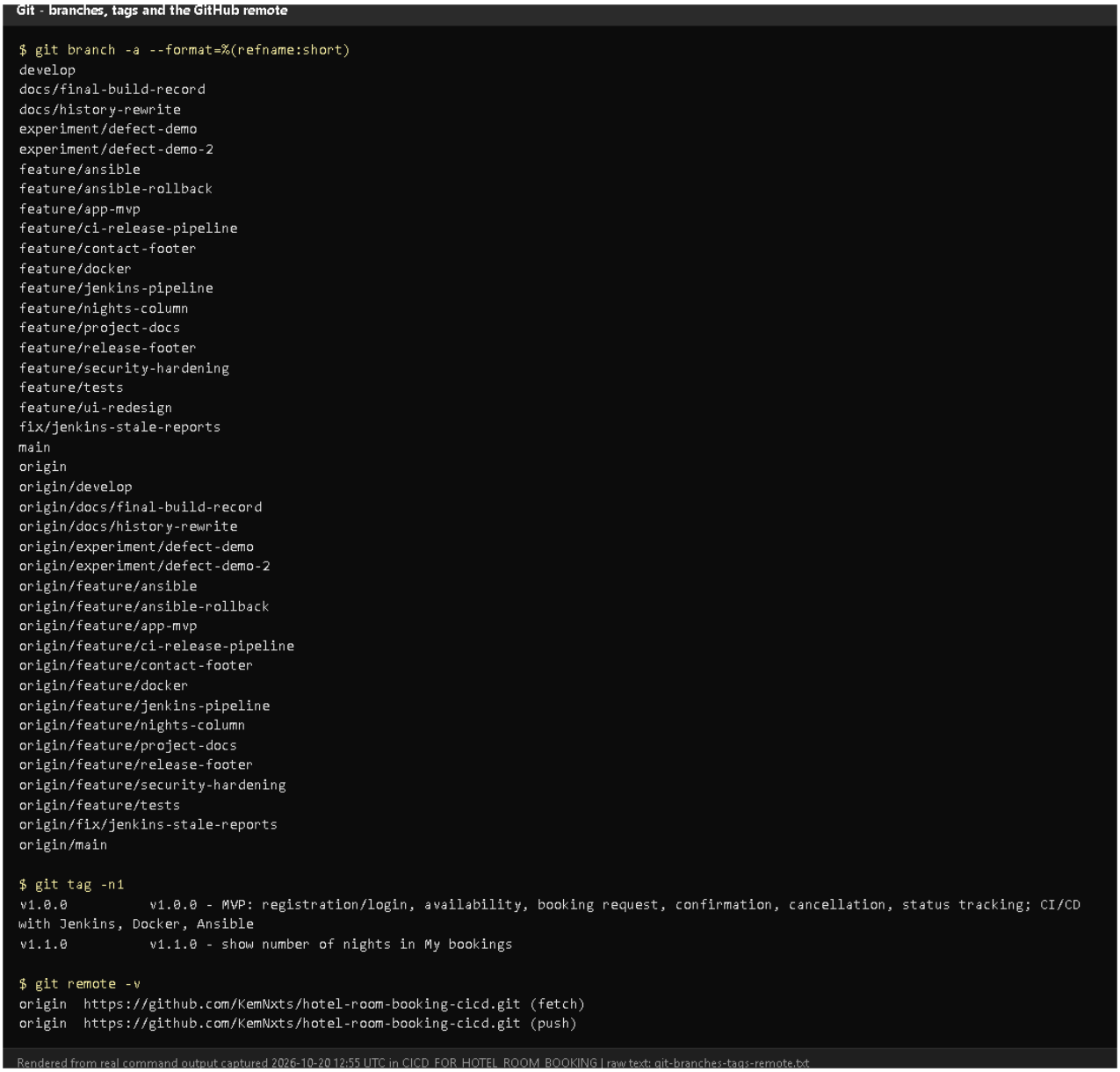


Figure 4.1 - `git branch -a`*,* `git tag -n1`*,* `git remote -v` *(rendered from real output, 2026-10-20).* **Notice:** local and `origin/...` branches, the two release tags with messages, and the GitHub remote. **Proves:** branching model, tags and remote exist.

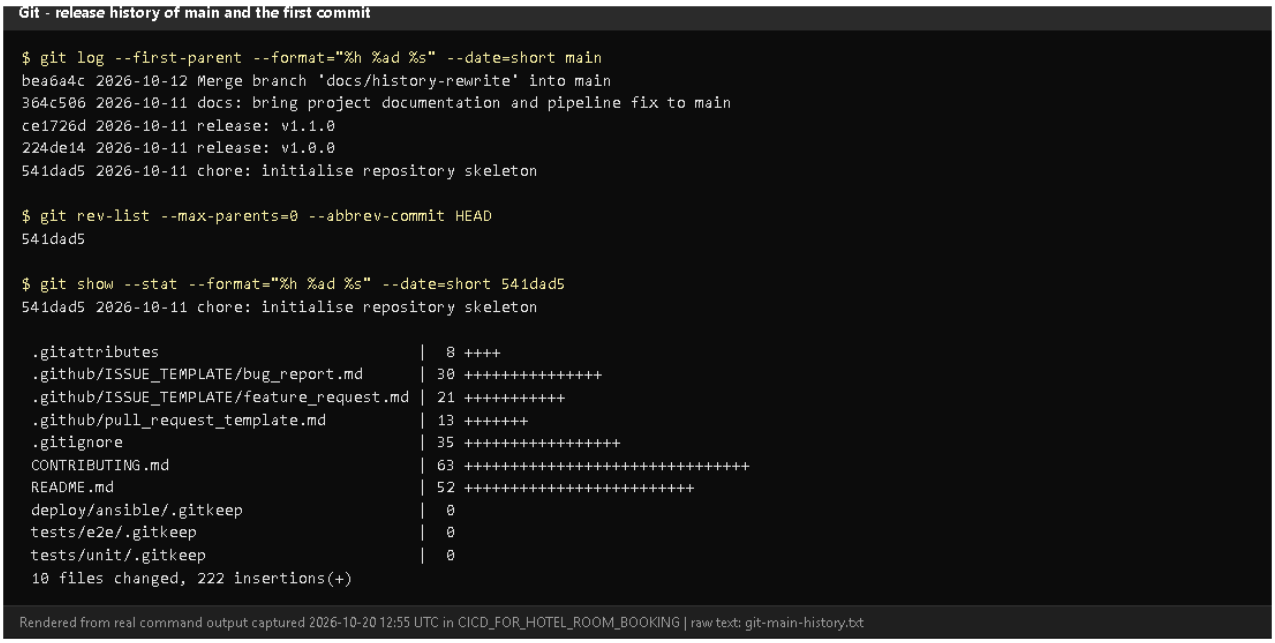
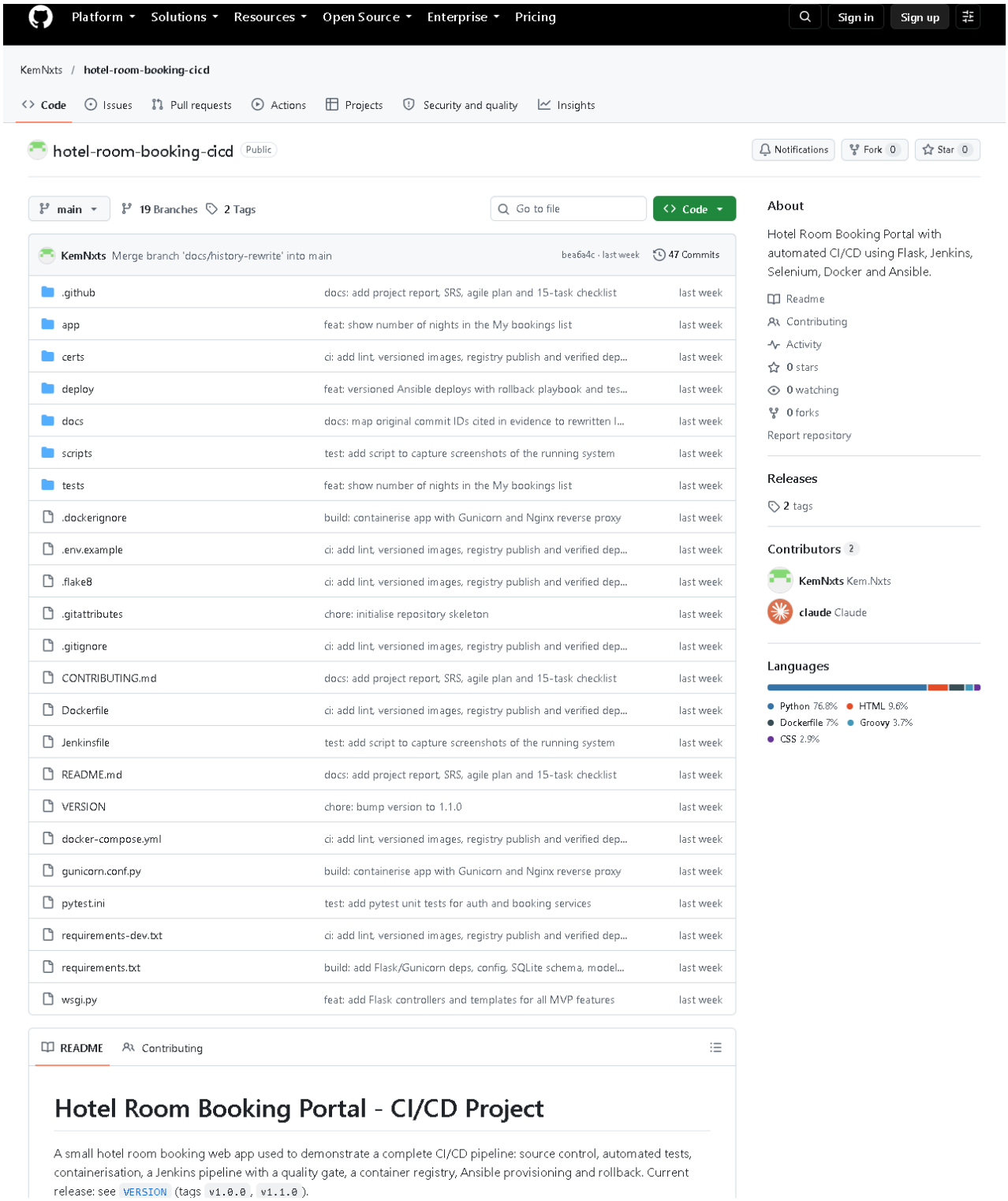


Figure 4.2 - Main-branch history and the skeleton commit `541dad5` with its files (rendered from real output). **Notice:** `main` contains only the skeleton and the release merges; the first commit added the README, ignore rules and templates. **Proves:** an initial source baseline and meaningful history.



Figure 4.3 - `.github` templates, `.gitattributes` and `.gitignore` (rendered from real output). **Notice:** `.env`, credentials, keys and certificates are ignored. **Proves:** secrets and generated files are kept out of Git.


Platform Solutions Resources Open Source Enterprise Pricing
Sign in
Sign up
KemNxts / hotel-room-booking-cicd
Code Issues Pull requests Actions Projects Security and quality Insights
hotel-room-booking-cicd
Public
Notifications
Fork 0
Star 0
main
19 Branches
2 Tags
Go to file
Code
KemNxts Merge branch 'docs/history-rewrite' into main
bea6a4c · last week
47 Commits
.github docs: add project report, SRS, agile plan and 15-task checklist last week
app feat: show number of nights in the My bookings list last week
certs ci: add lint, versioned images, registry publish and verified dep... last week
deploy feat: versioned Ansible deploys with rollback playbook and tes... last week
docs docs: map original commit IDs cited in evidence to rewritten l... last week
scripts test: add script to capture screenshots of the running system last week
tests feat: show number of nights in the My bookings list last week
.dockerignore build: containerise app with Gunicorn and Nginx reverse proxy last week
.env.example ci: add lint, versioned images, registry publish and verified dep... last week
.flake8 ci: add lint, versioned images, registry publish and verified dep... last week
.gitattributes chore: initialise repository skeleton last week
.gitignore ci: add lint, versioned images, registry publish and verified dep... last week
CONTRIBUTING.md docs: add project report, SRS, agile plan and 15-task checklist last week
Dockerfile ci: add lint, versioned images, registry publish and verified dep... last week
Jenkinsfile test: add script to capture screenshots of the running system last week
README.md docs: add project report, SRS, agile plan and 15-task checklist last week
VERSION chore: bump version to 1.1.0 last week
docker-compose.yml ci: add lint, versioned images, registry publish and verified dep... last week
gunicorn.conf.py build: containerise app with Gunicorn and Nginx reverse proxy last week
pytest.ini test: add pytest unit tests for auth and booking services last week
requirements-dev.txt ci: add lint, versioned images, registry publish and verified dep... last week
requirements.txt build: add Flask/Gunicorn deps, config, SQLite schema, model... last week
wsgi.py feat: add Flask controllers and templates for all MVP features last week
README
Contributing
Hotel Room Booking Portal - CI/CD Project
A small hotel room booking web app used to demonstrate a complete CI/CD pipeline: source control, automated tests, containerisation, a Jenkins pipeline with a quality gate, a container registry, Ansible provisioning and rollback. Current release: see VERSION (tags v1.0.0 , v1.1.0).
About
Hotel Room Booking Portal with automated CI/CD using Flask, Jenkins, Selenium, Docker and Ansible.
Readme
Contributing
Activity
0 stars
0 watching
0 forks
Report repository
Releases
2 tags
Contributors 2
KemNxts Kem.Nxts
claude Claude
Languages
Python 76.8% HTML 9.6%
Dockerfile 7% Groovy 3.7%
CSS 2.9%

(part 1 of 3)

Project documentation: start at docs/08-project-report.md (report + index of all documents) and docs/15-task-checklist.md (all 15 course tasks mapped to files and evidence).

Features (MVP)

1. User registration and login
2. Room availability view for a chosen date range
3. Booking request
4. Booking confirmation
5. Cancellation
6. Booking status tracking (`PENDING` -> `CONFIRMED` / `CANCELLED`)

Tech stack

Concern	Tool
Language	Python 3.11
Web framework	Flask + Jinja2 templates
Database	SQLite (built into Python)
Dependencies	pip + pinned `requirements*.txt` + venv
Static checks	`compileall` + flake8
Unit / integration tests	pytest (+ Flask test client)
UI tests	Selenium WebDriver (run via pytest)
App server	Gunicorn behind Nginx
Container	Docker, Docker Compose
Registry	`registry:2` (local) or Docker Hub
CI/CD	Jenkins (declarative `Jenkinsfile`)
Provisioning	Ansible

Why no Maven/Tomcat: they are Java tools; the Python equivalents are listed in docs/03-srs-architecture.md.

Repository layout

```
.
├── app/
│   ├── __init__.py         # create_app() factory, CSRF hook, /health endpoint
│   ├── config.py           # settings read from environment variables
│   ├── db.py               # SQLite connection per request, schema init, transactions
│   ├── schema.sql          # tables, constraints, overlap triggers, demo rooms
│   ├── models.py           # User, Room, Booking data classes, BookingStatus
│   ├── dao/                # Data Access Objects - the only place with SQL
│   ├── services/           # business rules (validation, availability, status changes)
│   ├── controllers/        # Flask routes + csrf.py
│   ├── templates/          # Jinja2 HTML pages
│   └── static/style.css
├── wsgi.py                 # entry point for Gunicorn / dev server
├── gunicorn.conf.py        # Gunicorn settings (workers, bind, logs)
├── VERSION                 # release number, baked into images and /health
├── requirements.txt        # runtime deps      requirements-dev.txt: + test/lint tools
├── pytest.ini, .flake8
├── tests/
│   ├── unit/               # service layer + database rules
│   ├── integration/        # HTTP routes via Flask test client
│   └── e2e/                # Selenium user journeys (screenshot on failure)
├── Dockerfile              # app image (Gunicorn, non-root, HEALTHCHECK, version labels)
├── docker-compose.yml      # nginx + app + database volume
├── .env.example            # template for the git-ignored .env
├── certs/                  # optional extra CA certificates (see certs/README.md)
├── Jenkinsfile             # CI/CD pipeline
├── deploy/
```

(part 2 of 3)

```
│   ├── nginx/              # reverse-proxy image + config
│   ├── jenkins/            # ready-to-run local Jenkins + registry
│   └── ansible/            # inventory, playbook, rollback, group_vars, controller image, lab/
├── scripts/capture_screenshots.py   # real screenshots of the running system
├── docs/                   # project documentation + evidence/
├── .github/                # issue + PR templates
```

(part 3 of 3)

Figure 4.4 - The public GitHub repository home page (captured 2026-10-20). **Notice:** the folder structure, README rendered below, branch/tag counts.

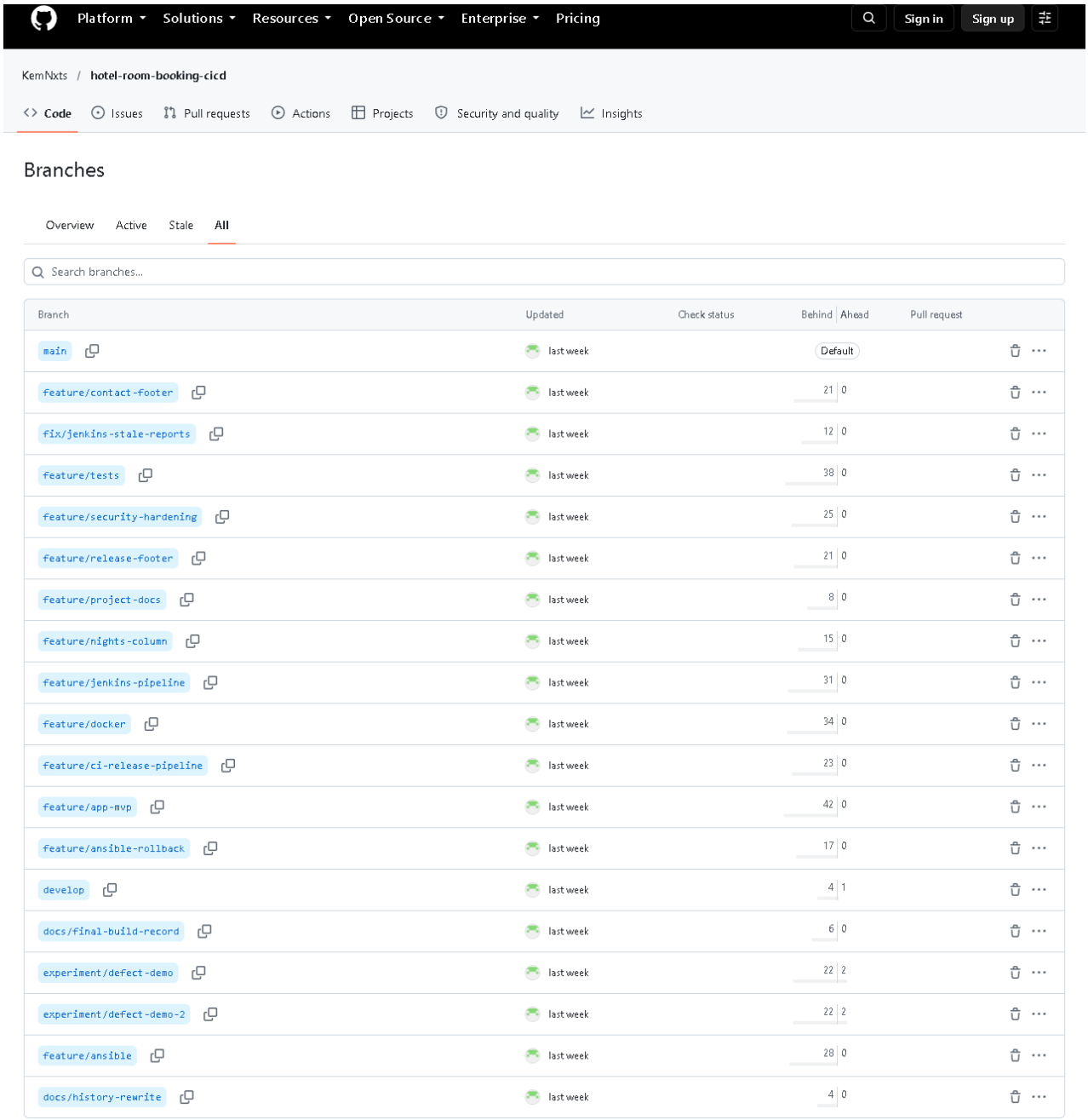


Figure 4.5 - All branches on GitHub. **Notice:** `main`, `develop`, `feature/*`, `fix/*`, `experiment/*`.

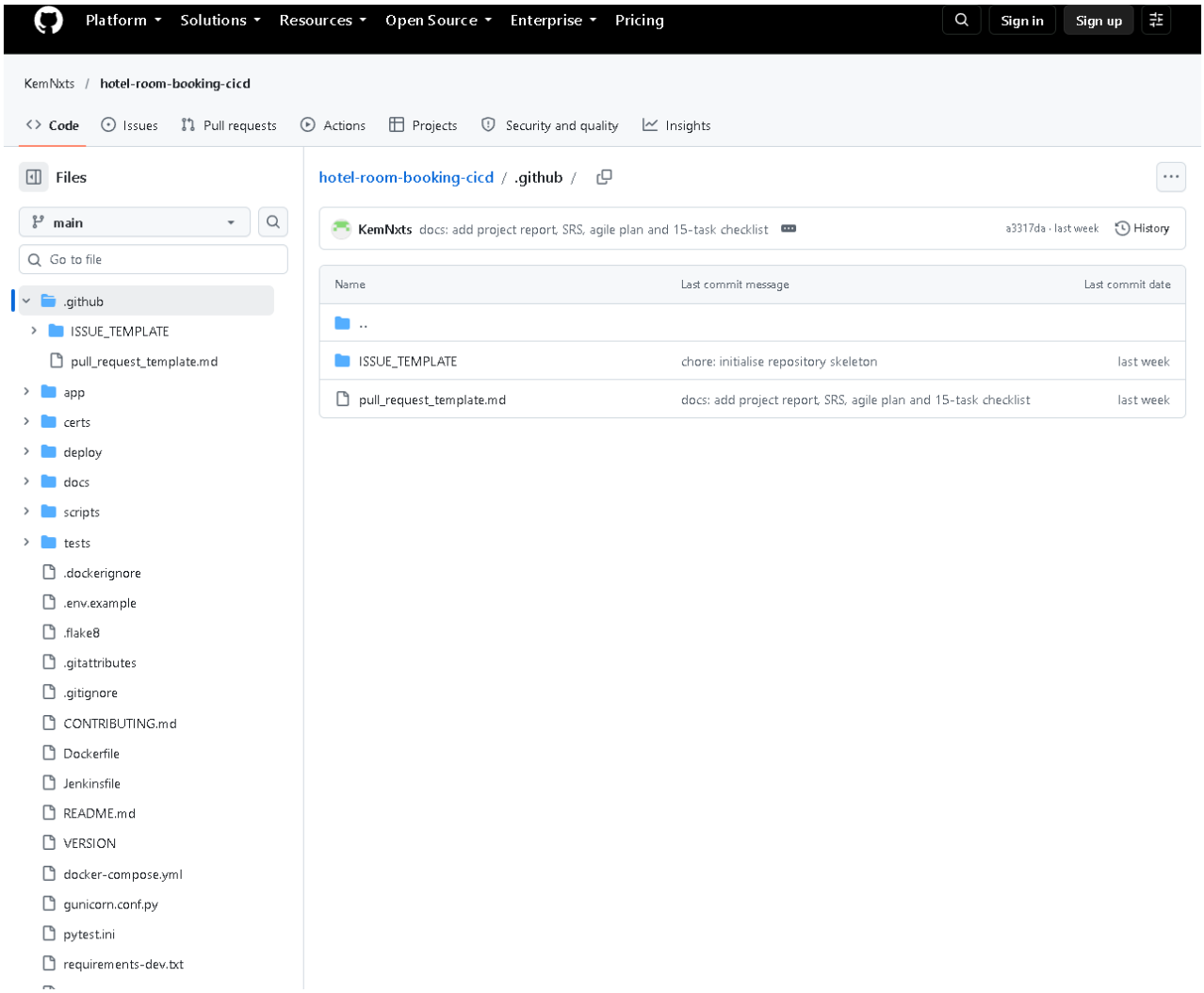


Figure 4.6 - `.github/` on GitHub: `ISSUE_TEMPLATE/` and `pull_request_template.md`.

E. How to demonstrate this task to the professor (about 3 minutes)

Before the demo: PowerShell in `D:\DevOPS\CICD_FOR_HOTEL_ROOM_BOOKING`; browser tab with https://github.com/KemNxts/hotel-room-booking-cicd.

28. *Say:* "This folder is a Git repository." (Today it shows the branch `feature/ui-redesign` and uncommitted UI files - say they are work in progress.)

```
git status
```

29. *Say:* "`main` holds releases, `develop` integrates features, each feature had its own branch. The tags mark the releases. `origin` is GitHub."

```
git branch -a
git tag -n1
git remote -v
```

30. *Say:* "Each commit has a type prefix like `feat:` or `fix:` and the merges show each feature branch."

```
git log --oneline --graph -15 develop
```

31. *Say:* "The very first commit was the skeleton: README, ignore rules and templates."

```
git show --stat 541dad5
```

32. Open `.gitignore` in VS Code - point to `.env`, `credentials/`, `*.key`.
33. **On GitHub:** show the file list, the *branches* page, the *tags* page and `.github/`.

Recovery: if `git log` opens a pager that "hangs", press `q`. If GitHub asks you to sign in, you are viewing the public page - no login is needed to read.

F. Expected screenshots for the final submission

16. `assets/task-04/git-branches-tags-remote.png` - **exists, 2026-10-20**.
17. `assets/task-04/git-main-history.png` - **exists, 2026-10-20**.
18. `assets/task-04/gitignore-and-templates.png` - **exists, 2026-10-20**.
19. `assets/task-04/github-repository-home.png` - **exists, 2026-10-20**.
20. `assets/task-04/github-branches.png` - **exists, 2026-10-20**.
21. `assets/task-04/github-templates-folder.png` - **exists, 2026-10-20**.
22. `assets/task-04/github-branch-protection.png` - **requires manual capture** only if you configure branch protection (GitHub *Settings -> Branches*, needs your login).

G. Probable viva questions and answers

18. **Git vs GitHub?** Git records history locally; GitHub hosts a copy online and adds issues, PRs and a web view.
19. **What is a commit?** A saved snapshot of the files with an ID, author, date and message.
20. **Branch vs tag?** A branch moves forward as you commit; a tag is fixed to one commit forever (we use tags for releases).
21. **Why `.gitignore`?** To keep secrets (`.env`), local databases and generated files out of the repository.
22. **What does `.gitattributes` do here?** Forces LF line endings so shell scripts and configs work in Linux containers even when edited on Windows.
23. **Explain your branch policy.** `main` = released code, `develop` = integration, `feature/<name>` per change, `fix/<name>` for fixes, `experiment/<name>` never merged. Features merge into develop with `--no-ff`.
24. **(Project-specific) What is in your first commit?** `541dad5`: README, CONTRIBUTING, `.gitignore`, `.gitattributes`, issue and PR templates and empty folders.
25. **(Project-specific) Why do commit IDs in your evidence differ from GitHub?** Messages were edited once before publishing; IDs are hashes that include the message, so all IDs changed. `docs/evidence/git/commit-id-map.md` maps them.
26. **What is a Conventional Commit?** A message format `type: summary`, e.g. `fix: reject check-out before check-in`, making history easy to read.
27. **Is your `main` branch protected?** Only by policy in `CONTRIBUTING.md`; GitHub protection rules were not configured (needs the owner's settings).
28. **Why have both `main` and `develop`?** `develop` can be tested on staging while `main` only receives releases that passed the pipeline.

H. Common mistakes and troubleshooting

Symptom	Cause	Inspect	Fix	Confirm
`git log` output does not end	pager waiting	terminal	press `q`, or add `--no-pager`	prompt returns
A secret file appears in `git status`	missing ignore rule	`git check-ignore -v <file>`	add a rule to `.gitignore`; never commit it	file no longer listed
`git push` rejected	remote has newer commits / no login	`git status`, `git fetch`	`git pull` first; sign in to GitHub	push succeeds

Symptom	Cause	Inspect	Fix	Confirm
Line-ending warnings (CRLF/LF)	Windows editor	`.gitattributes`	harmless; Git normalises to LF	-
Old commit ID not found	history rewrite	`docs/evidence/git/commit-id-map.md`	use the current ID	`git show <id>` works

I. Quick revision notes

Five key points

27. Git = local history; GitHub = hosted copy + collaboration.
28. Skeleton commit `541dad5`: README, ignore rules, templates, CONTRIBUTING.
29. Branches: main / develop / feature / fix / experiment.
30. Tags `v1.0.0` and `v1.1.0` mark releases.
31. `.gitignore` keeps `.env`, credentials and databases out of Git.

30-60 second explanation "The project is a Git repository published on GitHub. The first commit created the skeleton - README, contributing rules, ignore rules and issue and pull-request templates. We use `main` for releases, `develop` for integration and one branch per feature, with conventional commit messages. Releases are tagged v1.0.0 and v1.1.0. Secrets like `.env` are ignored and never committed. Everything Jenkins builds starts from a commit here."

Caveat: branch protection is a written policy, not a GitHub setting.

Readiness checklist

- â€¢ â˜ I can explain branch vs tag vs commit.
- â€¢ â˜ I can run `git branch -a`, `git tag -n1`, `git log --graph`.
- â€¢ â˜ I can open the repo on GitHub and show branches and tags.
- â€¢ â˜ I can explain why evidence shows different commit IDs.

Task 5 - Feature Development with Branching

A. Task title and objective

Task 5: Feature Development with Branching

Objective: develop a core feature on its own branch, with meaningful commits and tests, have it reviewed through a pull request, and merge it into `develop`.

Professor's required deliverables

- â€¢ Feature branch
- â€¢ Changes and tests on that branch
- â€¢ Meaningful commits (add / commit / push / pull / log)
- â€¢ Pull request with review comments / review checklist
- â€¢ Merge into `develop`

Current status: Partially completed.

Part	Status
Feature branches, commits with tests, `--no-ff` merges into `develop`, pushed to GitHub	Implemented and verified (Git + GitHub, 2026-10-20)
Review checklist	Implemented - `.github/pull_request_template.md`; written review records in `docs/pull-requests/`
Real GitHub pull requests and review comments	**Pending external action** - GitHub shows **0 pull requests** (Figure 5.4). Merges were done locally and then pushed.

No PR screenshots or review comments are shown here because none exist.

B. Explanation for a complete beginner

An everyday picture. Several authors write a book. Each writes a new chapter in a separate copy, so a half-finished chapter never appears in the printed book. When a chapter is ready, an editor reviews it, and only then is it added to the master copy.

- â€¢ **Feature branch** = the separate copy for one feature (`feature/security-hardening`).
- â€¢ **`develop`** = the master copy where finished features meet.
- â€¢ **Pull request (PR)** = on GitHub, a request "please review and merge my branch", with a discussion page, review comments and a merge button.
- â€¢ **Code review** = another person reads the changes before they are merged.
- â€¢ **Merge** = adding the branch's commits into `develop`. With `--no-ff` ("no fast-forward") Git always creates a *merge commit*, so the history clearly shows "this group of commits was one feature".

Basic commands

Command	Meaning
`git checkout -b feature/x develop`	create a new branch from develop and switch to it
`git add <files>`	choose which changes go into the next commit
`git commit -m "feat: ..."`	save a snapshot with a message
`git push -u origin feature/x`	upload the branch to GitHub
`git pull`	download and apply new commits from GitHub

Command	Meaning
`git log --oneline --graph`	view history as a graph
`git merge --no-ff feature/x`	merge into the current branch with a merge commit

C. What was actually implemented

Worked example: `feature/security-hardening` - real output of `git log --graph --oneline -4 bdfc2fb` (current commit IDs):

```
*   bdfc2fb Merge branch 'feature/security-hardening' into develop
|\
| * fd3cb60 test: add route-level integration tests and database rule tests
| * 716b910 feat: add CSRF protection, secure cookies, audit logs and DB overlap guard
|/
*   475cecd Merge branch 'feature/ansible' into develop
```

34. Branch created from `develop`: `git checkout -b feature/security-hardening develop`.
35. Commit 1 `716b910 feat:` - CSRF protection (`app/controllers/csrf.py`), secure cookie settings (`app/config.py`), audit logging, `/health` version, overlap triggers in `app/schema.sql`.
36. Commit 2 `fd3cb60 test:` - new `tests/integration/test_routes.py` and `tests/unit/test_database.py`.
37. Checked locally: `flake8` clean, 87 tests passed at that time.
38. Review record written with the PR template checklist: docs/pull-requests/feature-security-hardening.md.
39. Merged: `git checkout develop` then `git merge --no-ff feature/security-hardening` -> `bdfc2fb`.
40. Pushed to GitHub on 2026-10-12 together with all other branches and the tags.

All feature work used this pattern - 12 `feature/*`, 1 `fix/*` and 2 `docs/*` branches merged into `develop` with merge commits (Figure 5.2), e.g. `feature/app-mvp`, `feature/tests`, `feature/docker`, `feature/jenkins-pipeline`, `feature/ansible`, `feature/ci-release-pipeline`, `feature/nights-column`.

Review checklist (`.github/pull_request_template.md`): author checklist (branch name, commit convention, flake8, tests, Jenkins green, no secrets, docs) and reviewer checklist (no SQL outside `app/dao/`, server-side validation, ownership checks, CSRF token on new forms, tests would fail if broken).

Prepared PR descriptions (`docs/pull-requests/`): security hardening, CI release pipeline, release footer (with the merge conflict), release v1.1.0. These are written review records, **not** GitHub PRs.

Proof on GitHub that a feature branch was merged: the compare page `develop...feature/nights-column` says "develop is up to date with all commits from feature/nights-column" (Figure 5.3); comparing the branch with its starting point shows its own commit and diff (Figure 5.5).

D. Screenshots and evidence

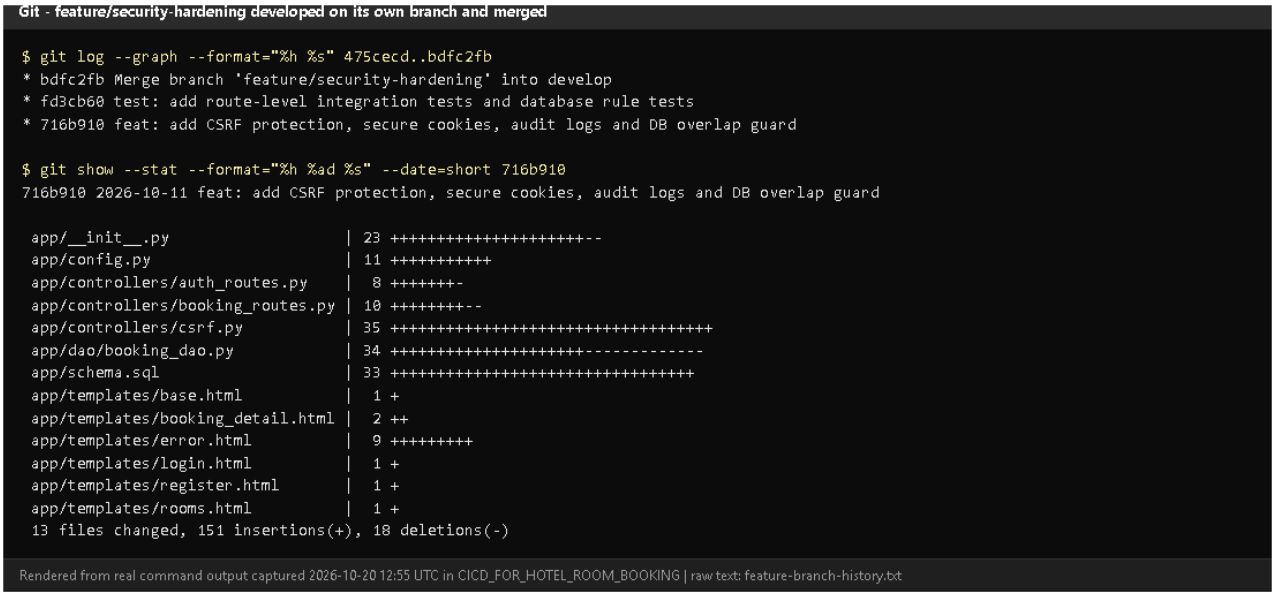


Figure 5.1 - `git log --graph` *of* `feature/security-hardening` *and* `git show --stat 716b910` *(rendered from real output, 2026-10-20).* **Notice:** two commits on the side branch (a `feat:` and a `test:`), then the merge commit into develop; the file list of the feature commit. **Proves:** a feature branch with code and tests, merged with `--no-ff`.

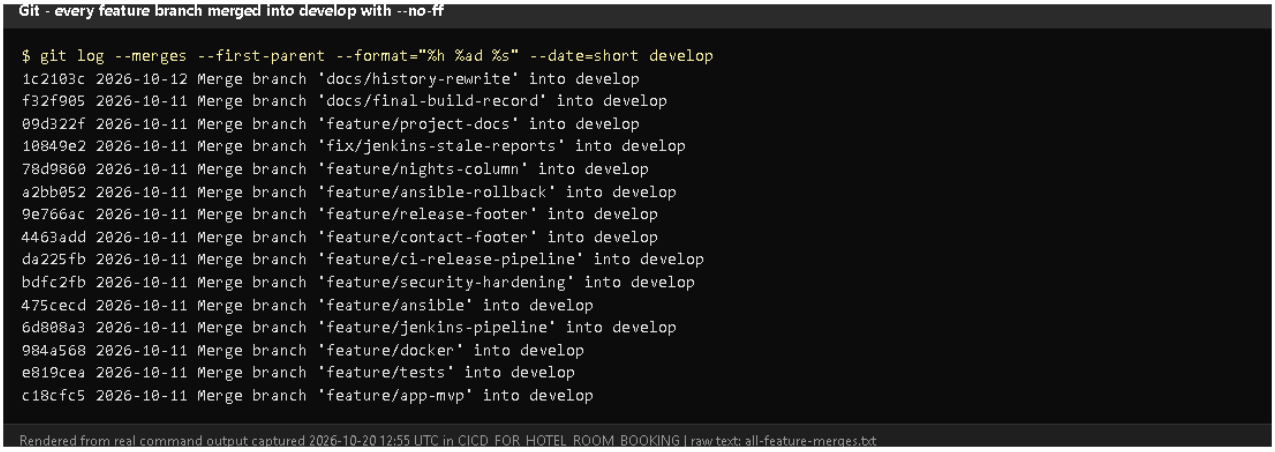


Figure 5.2 - `git log --merges --first-parent develop` *(rendered from real output).* **Notice:** every change arrived in `develop` through a `Merge branch 'feature/...'` commit.

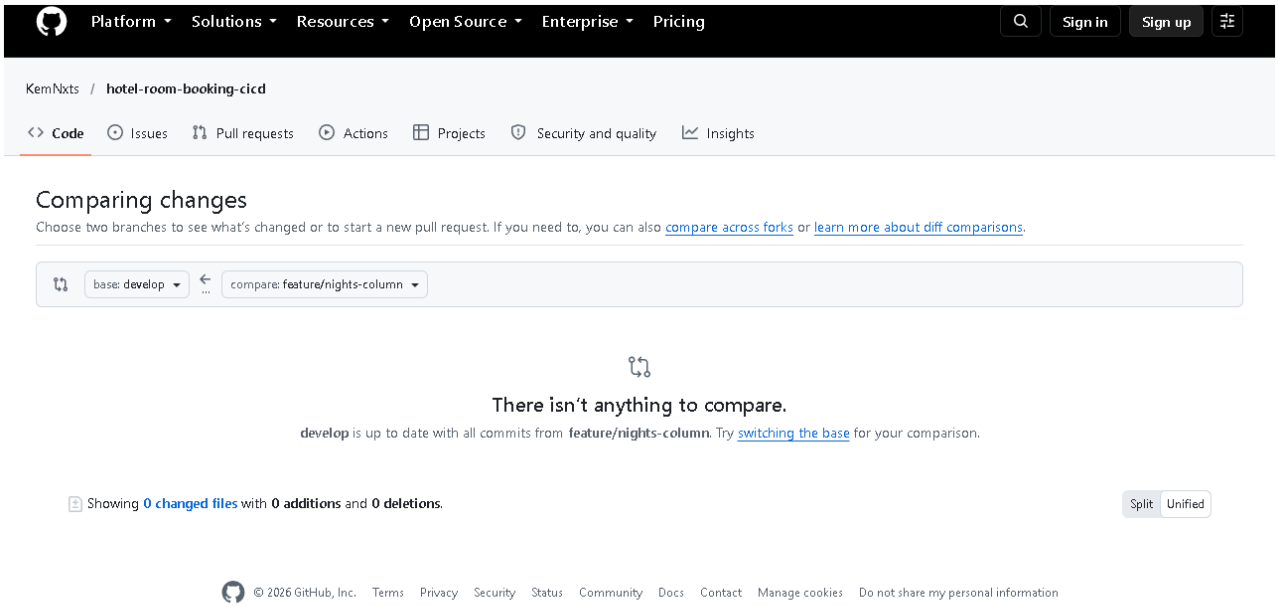


Figure 5.3 - GitHub compare `develop...feature/nights-column` (captured 2026-10-20). **Notice:** "develop is up to date with all commits from feature/nights-column".

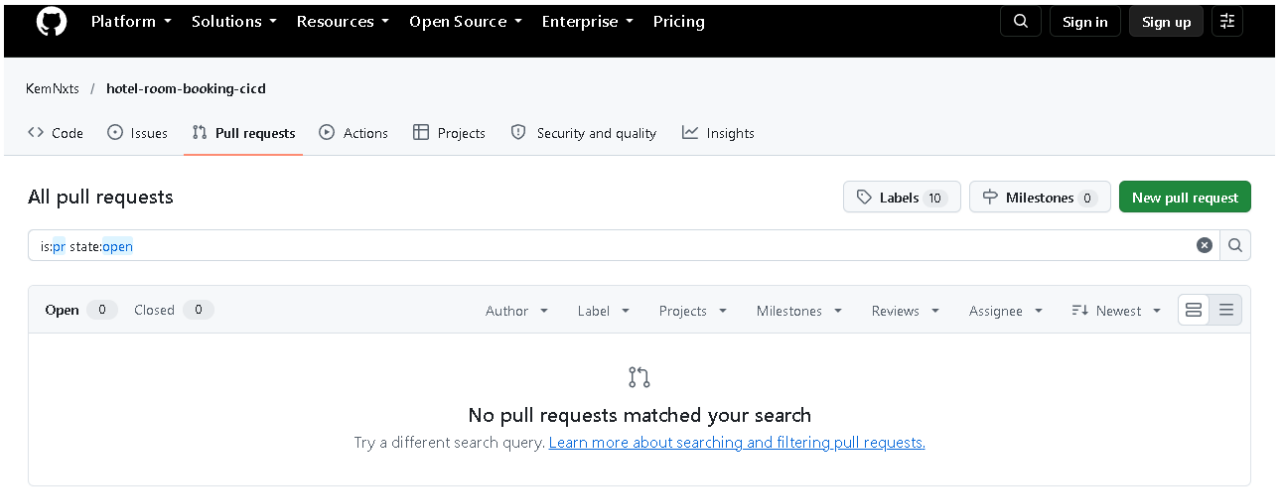


Figure 5.4 - The repository's Pull requests page (all states): none. **Why included:** honest evidence that real PRs are still pending.

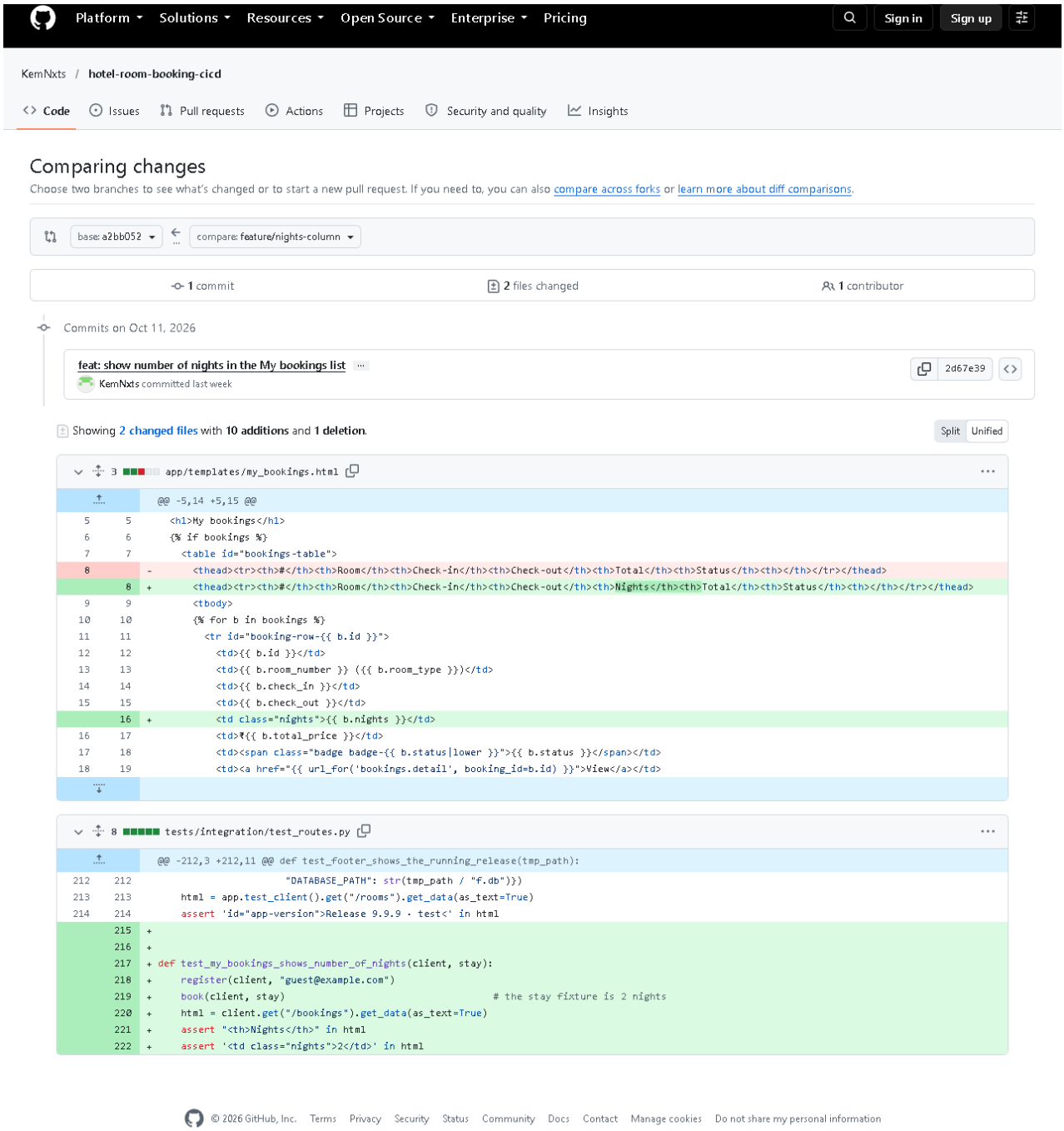


Figure 5.5 - GitHub compare from the branch's starting point `a2bb052` to `feature/nights-column`. **Notice:** 1 commit (`2d67e39 feat: show number of nights...`), 2 files - the template change and its new test. This is a compare view, not a PR.

E. How to demonstrate this task to the professor (about 4 minutes)

Before the demo: PowerShell in `D:\DevOPS\CICD_FOR_HOTEL_ROOM_BOOKING`; browser on the GitHub repository.

23. *Expected:* the graph in section C (the side line `| *` is the feature branch). *Say:* "This feature was built on its own branch with a code commit and a test commit, then merged into develop."

```
git log --graph --oneline -4 bdfc2fb
```

24. *Say:* "Here is what the feature changed."

```
git show --stat 716b910
```

25. *Say:* "Every feature reached develop through a merge commit."

```
git log --merges --first-parent --oneline develop
```

26. Open `docs/pull-requests/feature-security-hardening.md` - *Say:* "I reviewed it with our PR checklist; this is the written review record."
27. Open `.github/pull_request_template.md` - the checklist.
28. On GitHub open *Pull requests* - *Say honestly:* "The merges were done locally and pushed, so GitHub shows no PRs yet. Here is how a real one is made:"

Live mini-demo of a real feature-branch PR (optional, needs your GitHub login, ~5 min):

```
git checkout develop
git pull
git checkout -b feature/demo-readme-note
# edit README.md (add one sentence), save
git add README.md
git commit -m "docs: add a note about the demo"
git push -u origin feature/demo-readme-note
```

Then on GitHub: *Compare & pull request* -> base `develop` -> write a description using the template -> *Create pull request* -> add a review comment -> *Merge pull request*. Capture screenshots of the PR page (see section F).

Recovery: "Your local changes would be overwritten by checkout" - you have uncommitted work (today: the UI redesign). Commit it or run `git stash` first, then `git stash pop` afterwards.

F. Expected screenshots for the final submission

29. `assets/task-05/feature-branch-history.png` - **exists, 2026-10-20**.
30. `assets/task-05/all-feature-merges.png` - **exists, 2026-10-20**.
31. `assets/task-05/github-compare-merged-into-develop.png` - **exists, 2026-10-20**.
32. `assets/task-05/github-compare-feature-changes.png` - **exists, 2026-10-20**.
33. `assets/task-05/github-pull-requests-empty.png` - **exists, 2026-10-20** (honest current state).
34. `assets/task-05/github-pr-open.png` - a real PR with description and checklist - **requires manual capture** after creating a PR.
35. `assets/task-05/github-pr-review-comment.png` - a review comment - **requires manual capture**.
36. `assets/task-05/github-pr-merged.png` - the merged PR - **requires manual capture**.

G. Probable viva questions and answers

32. **Why use feature branches?** Unfinished work stays out of `develop`, so develop always builds and passes tests; several features can progress in parallel.
33. **What is a pull request?** A GitHub page that asks to merge one branch into another, where the change is reviewed and discussed first.
34. **Did you create pull requests?** Not on GitHub - GitHub shows 0 PRs. I merged locally with `--no-ff` and pushed; the review records are in `docs/pull-requests/`. Creating real PRs is my remaining step.
35. **What does `--no-ff` do?** Always creates a merge commit, so a feature's commits stay grouped in the history.
36. **`git pull` vs `git fetch`?** `fetch` downloads new commits; `pull` = fetch + merge into your current branch.
37. **(Project-specific) Show a feature branch and its commits.** `feature/security-hardening`: `716b910 feat: add CSRF protection...` and `fd3cb60 test: add route-level integration tests...`, merged as `bdfc2fb`.
38. **(Project-specific) What does your review checklist check?** No SQL outside DAOs, server-side validation, booking ownership, CSRF token on new forms, and that tests would fail if the code were broken.

39. **How did you test the feature before merging?** `flake8` and the full pytest suite locally; after merging, Jenkins built `develop`.
40. **What would a reviewer comment look like?** A note on a specific line, e.g. "this route changes data - does the form include `csrf_token`?" (an example - no real comments exist yet).
41. **What if two features change the same lines?** A merge conflict - see Task 6, where exactly that happened and was resolved.

H. Common mistakes and troubleshooting

Symptom	Cause	Inspect	Fix	Confirm
Commit went to `develop` instead of the feature branch	forgot `checkout -b`	`git branch --show-current`	create the branch from that commit, reset develop only if not pushed	`git log` shows commit on the feature branch
`git push` says "no upstream branch"	first push of a new branch	message text	`git push -u origin <branch>`	branch visible on GitHub
Checkout refused: local changes would be overwritten	uncommitted work	`git status`	commit or `git stash`	checkout succeeds
PR shows unrelated commits	branch started from the wrong base	`git merge-base develop <branch>`	rebase onto develop or recreate the branch	PR shows only the feature commits

I. Quick revision notes

Five key points

11. One branch per feature, created from `develop`.
12. Separate commits for code (`feat:`) and tests (`test:`).
13. Merged with `--no-ff`, so each feature is a visible unit.
14. Review checklist = `.github/pull_request_template.md`; records in `docs/pull-requests/`.
15. **No real GitHub PRs yet** - say so and show how to make one.

30-60 second explanation "Each feature was developed on its own branch. For example `feature/security-hardening` has one commit adding CSRF protection and a second adding tests. I checked it with flake8 and pytest, reviewed it against our pull-request checklist, and merged it into develop with `--no-ff`, which keeps a merge commit in the history. All branches are pushed to GitHub, and GitHub confirms develop contains their commits. Real pull requests on GitHub haven't been created yet - that's my next step."

Caveat: review was a self-review with a checklist; there are no GitHub PRs or reviewer comments.

Readiness checklist

- â€¢ â˜ I can show `git log --graph` for one feature.
- â€¢ â˜ I can explain PR vs merge.
- â€¢ â˜ I can explain honestly why GitHub shows 0 PRs.
- â€¢ â˜ I know the steps to create a real PR.

Task 6 - MVP Completion and Git Collaboration

A. Task title and objective

Task 6: MVP Completion and Git Collaboration

Objective: finish all six booking workflows securely, and show collaboration with Git: a second feature branch, a real merge conflict and its resolution, release tags and a stable source baseline.

Professor's required deliverables

- â€¢ All six workflows: registration/login, availability, booking request, confirmation, cancellation, status tracking
- â€¢ A second feature branch
- â€¢ A real merge conflict and its resolution
- â€¢ A version tag and a release-ready baseline
- â€¢ Updated backlog
- â€¢ Security: password hashing, session protection, server-side validation, authorisation

Current status: Implemented and verified.

- â€¢ Six workflows: 5/5 Selenium journeys and 85 unit/integration tests passed on 2026-10-20.
- â€¢ Merge conflict: happened for real on 2026-10-11 (output saved at the time), the resolution commit `9e766ac` is in Git, and on 2026-10-20 the same conflict was **reproduced** in a throw-away clone (section E).
- â€¢ Tags `v1.0.0` and `v1.1.0` exist locally and on GitHub.

Note: the application screenshots below show the **redesigned interface** that currently runs from the working tree (branch `feature/ui-redesign`, not yet committed). The six workflows behave the same in the committed v1.1.0.

B. Explanation for a complete beginner

An everyday picture. Two people edit the same sentence of a shared document while offline. When they sync, the software cannot know which version to keep, so it shows both and asks a human. That is a **merge conflict**.

Why conflicts happen in Git. Git merges automatically when two branches change *different* lines. When both change the *same* lines, Git stops, marks the file with `<<<<<<<`, `=======`, `>>>>>>>` and asks you to decide. Resolving means editing the file to the correct final content, testing, and committing the merge.

Branch vs tag. A *branch* is a moving pointer that advances with every new commit (like a bookmark you keep moving). A *tag* is a fixed label on one commit (like engraving "Release 1.0" on one page). Releases get tags so you can always return to exactly what was shipped.

MVP - the six workflows that make the portal useful. **Stable baseline** - a tagged commit on `main` that passed the whole pipeline.

Security terms: *password hashing* - storing a one-way scrambled form of the password, never the password itself; *session* - the login cookie; *authorisation* - checking a user may access *this* booking.

C. What was actually implemented

The six workflows

#	Workflow	Route	Service function	Verified by
1	Register / log in	`/register`, `/login`	`AuthService.register`, `login`	journeys 1, 2

#	Workflow	Route	Service function	Verified by
2	Availability by date	`GET /rooms?check_in=&check_out=`	`BookingService.search_rooms`	journey 3, 5
3	Booking request -> PENDING	`POST /bookings`	`request_booking`	journey 3
4	Confirmation -> CONFIRMED	`POST /bookings/<id>/confirm`	`confirm_booking`	journey 3
5	Cancellation -> CANCELLED	`POST /bookings/<id>/cancel`	`cancel_booking`	journey 4
6	Status tracking	`GET /bookings`	`list_bookings`	journey 4

Security in code

- â€¢ Passwords: `generate_password_hash` / `check_password_hash` (Werkzeug, scrypt) in `app/services/auth_service.py` (lines 35, 46).
- â€¢ Session cookie `HttpOnly` + `SameSite=Lax` (`app/config.py`); CSRF token on every form (`app/controllers/csrf.py`).
- â€¢ Server-side validation of e-mail, name, password length, dates (`auth_service.py`, `booking_service.py` `validate_stay`).
- â€¢ Authorisation: `get_booking` returns "not found" for another user's booking (integration test `test_other_users_cannot_see_confirm_or_cancel_my_booking`).

Second feature branch and the real merge conflict

Two features started from the **same** develop commit (`da225fb`, original `f4d0b26`):

Branch	Commit (current / original)	Change
`feature/contact-footer`	`74a153d` / `65ad990`	footer with hotel contact details
`feature/release-footer`	`63498a8` / `c24148f`	footer showing the running release

Both added a footer **in the same place**. `contact-footer` merged first (`4463add`). Merging `release-footer` then gave (saved 2026-10-11):

```
CONFLICT (content): Merge conflict in app/static/style.css
CONFLICT (content): Merge conflict in app/templates/base.html
CONFLICT (content): Merge conflict in tests/integration/test_routes.py
Automatic merge failed; fix conflicts and then commit the result.
```

Resolution (merge commit `9e766ac`, original `8728b62`): both features were wanted, so both lines were kept - contact first, release second; one combined CSS rule; both tests kept. Then flake8 was clean and 89 tests passed, and the merge was committed with a message describing each decision.

Release tags and baseline

Tag	Commit on `main`	Pipeline evidence
`v1.0.0`	`224de14` (original `661751d`)	Jenkins #4 (develop) and #5 (main -> prod) green
`v1.1.0`	`ce1726d` (original `d07929b`)	Jenkins #6/#7 (develop) and #8 (main -> prod) green

The production environment still runs `1.1.0-d07929b` - the v1.1.0 release.

Backlog update - `docs/02-agile-plan.md` marks HB-06 Done; the merge conflict is recorded as a sprint-retrospective lesson.

D. Screenshots and evidence

The six workflows (redesigned UI, working tree, 2026-10-20)

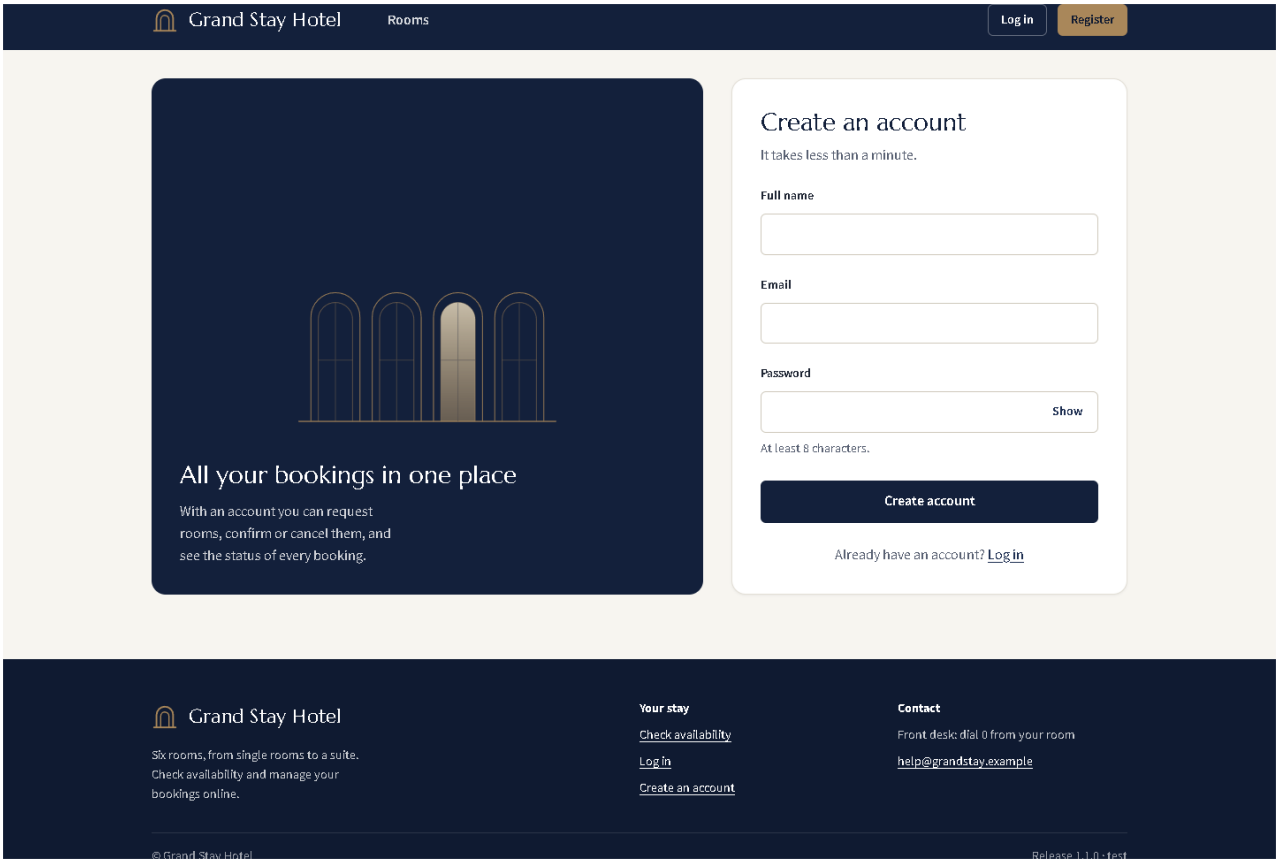


Figure 6.1 - Workflow 1: registration form with labelled fields and the password rule.

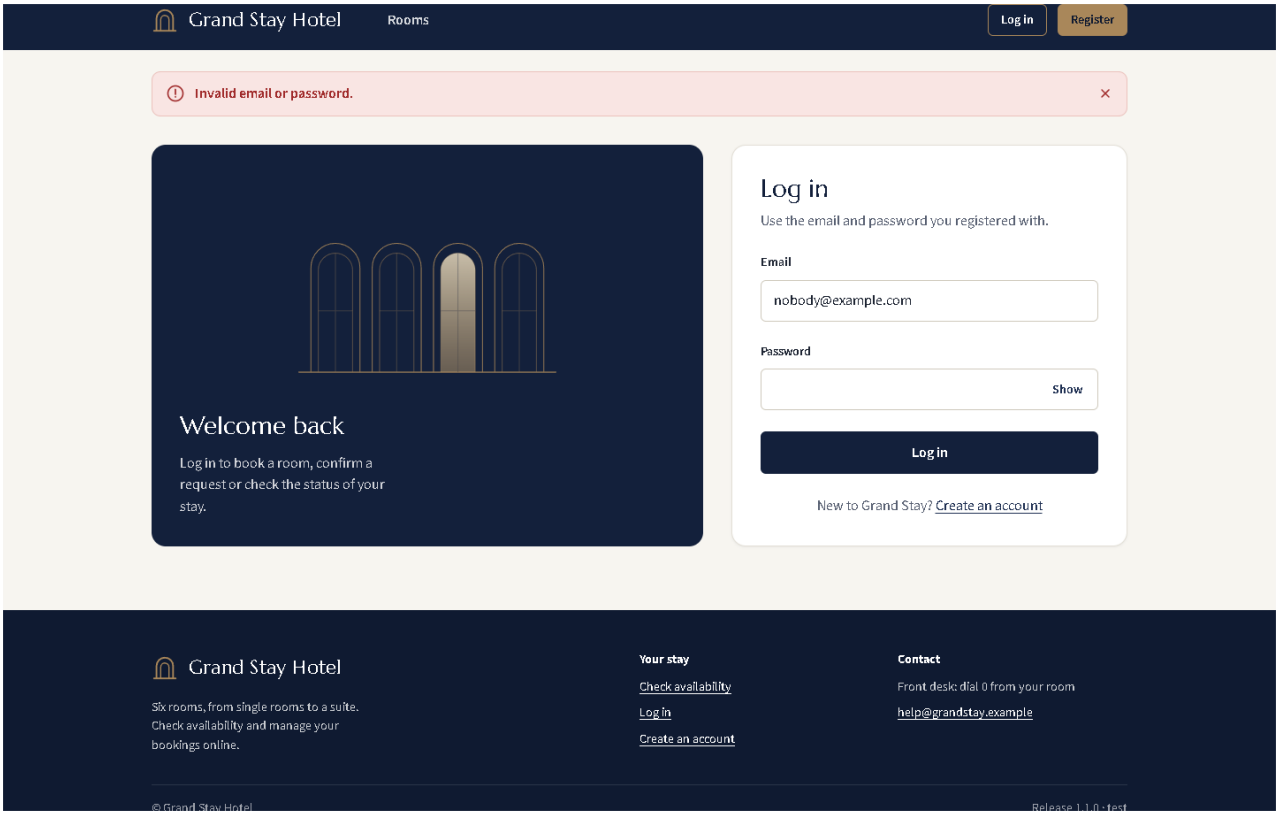


Figure 6.2 - Workflow 1: login with wrong credentials shows a neutral error (does not reveal which part was wrong).

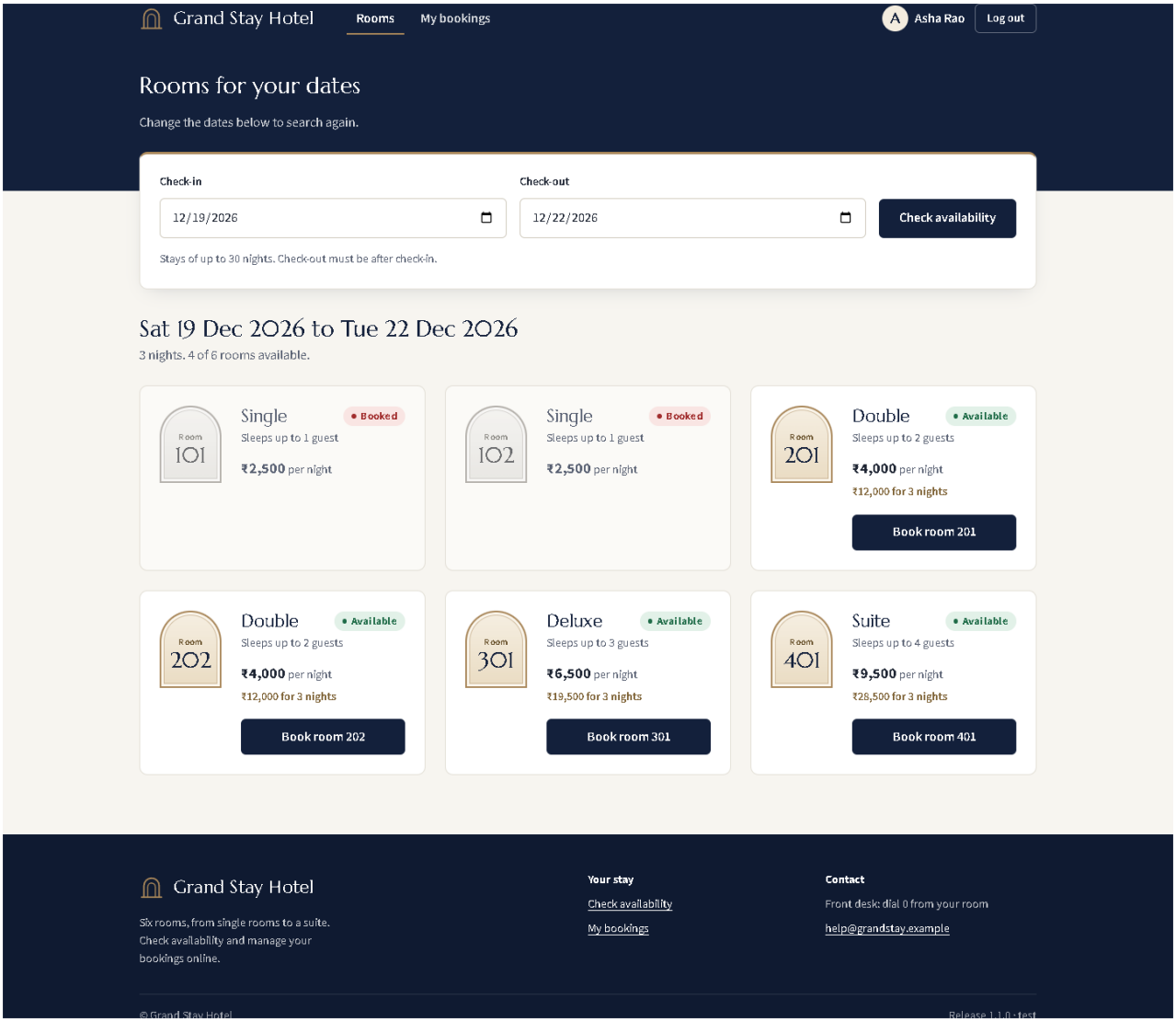


Figure 6.3 - Workflow 2: availability for the chosen dates - booked rooms marked "Booked", free rooms with the total for the stay.

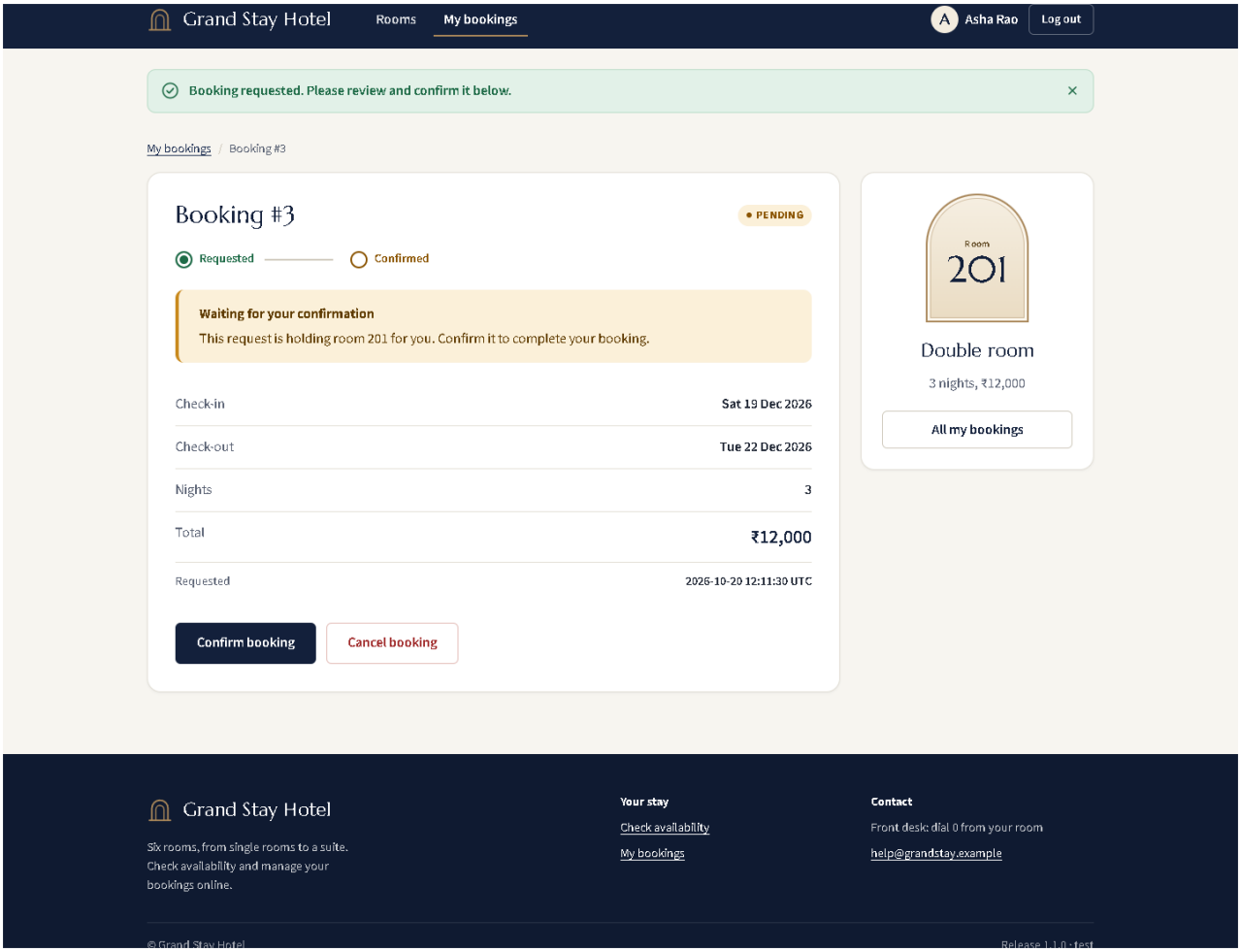


Figure 6.4 - Workflow 3: the new booking is PENDING.

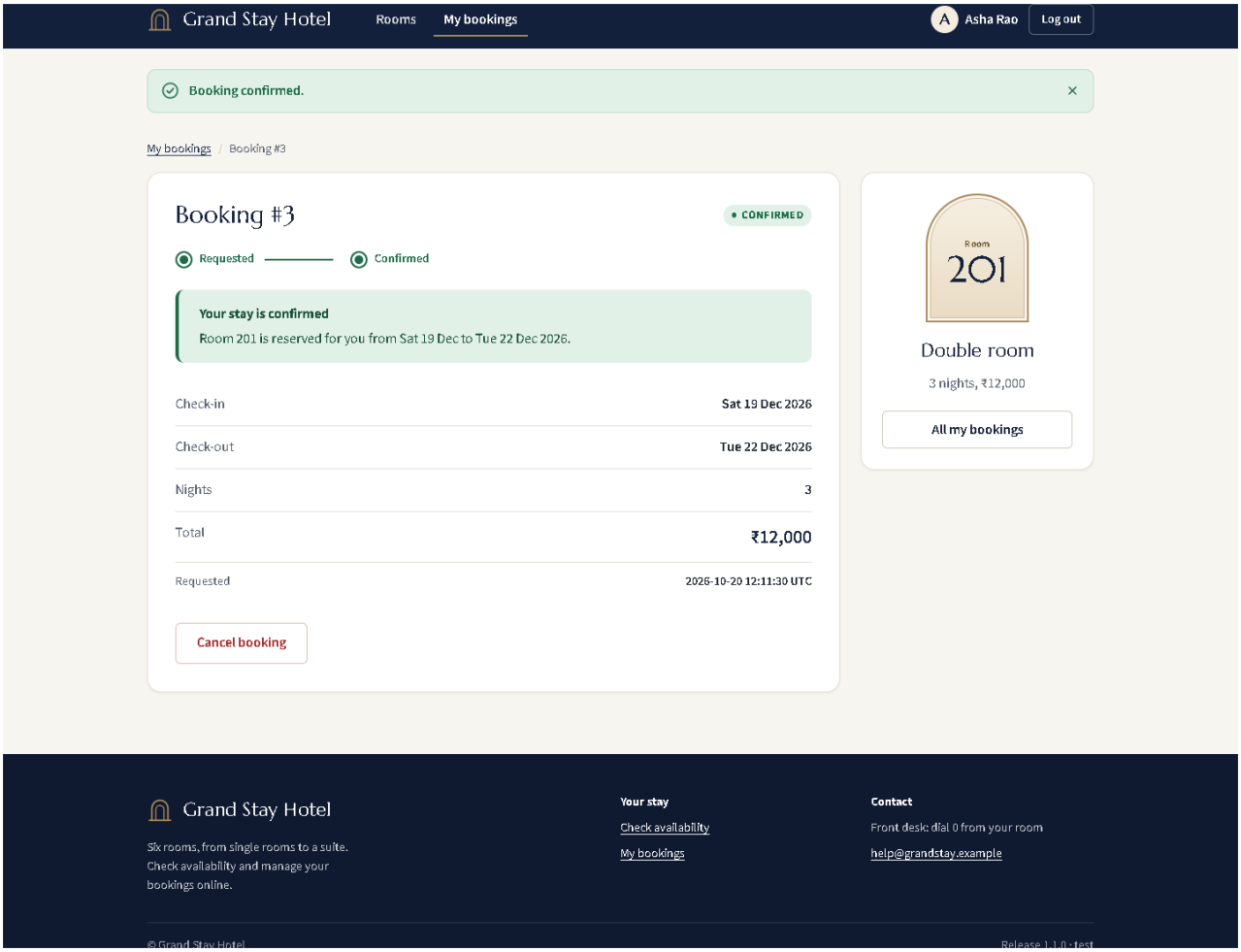


Figure 6.5 - Workflow 4: CONFIRMED, progress shows both steps done.

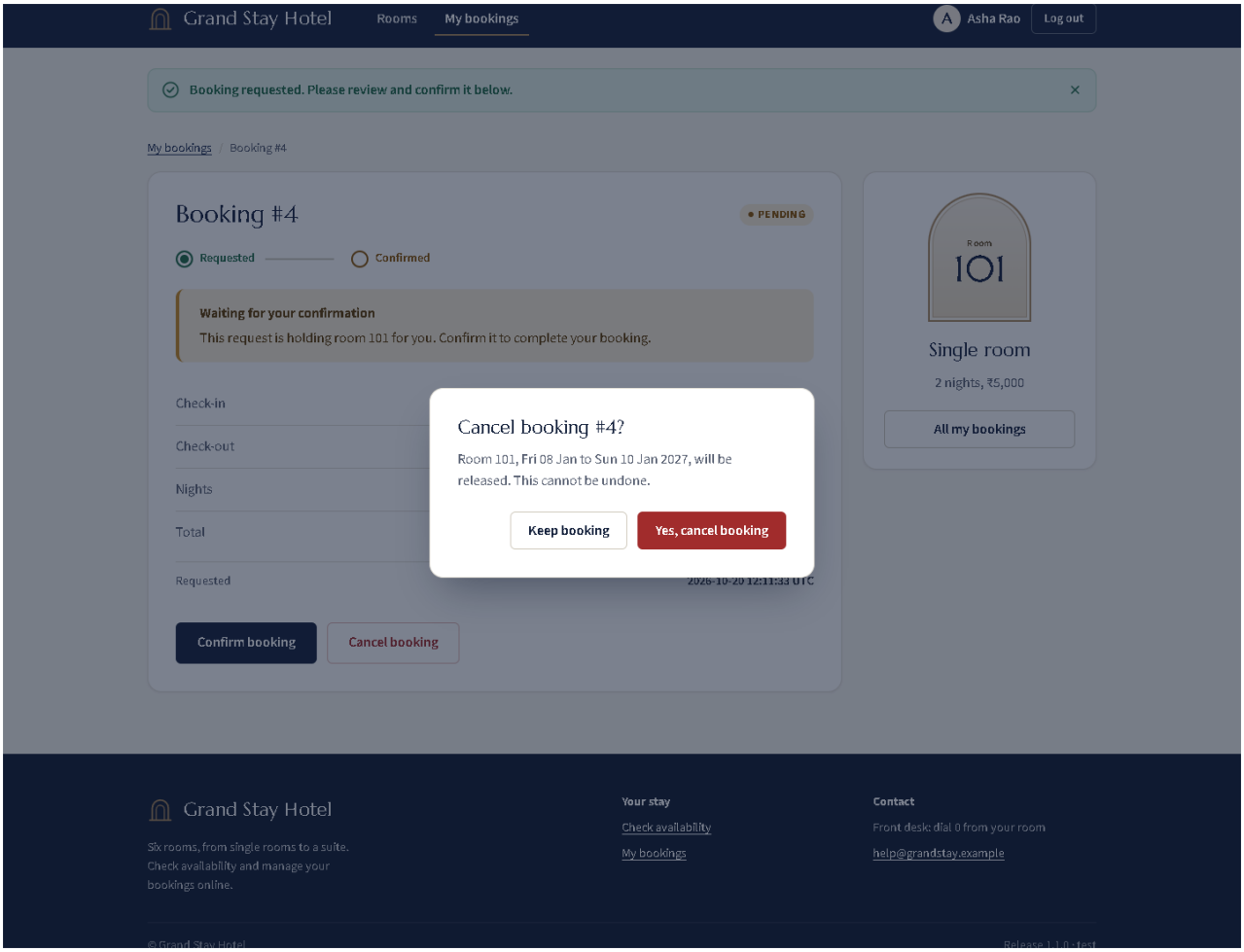


Figure 6.6 - Workflow 5: cancelling opens a confirmation dialog first.

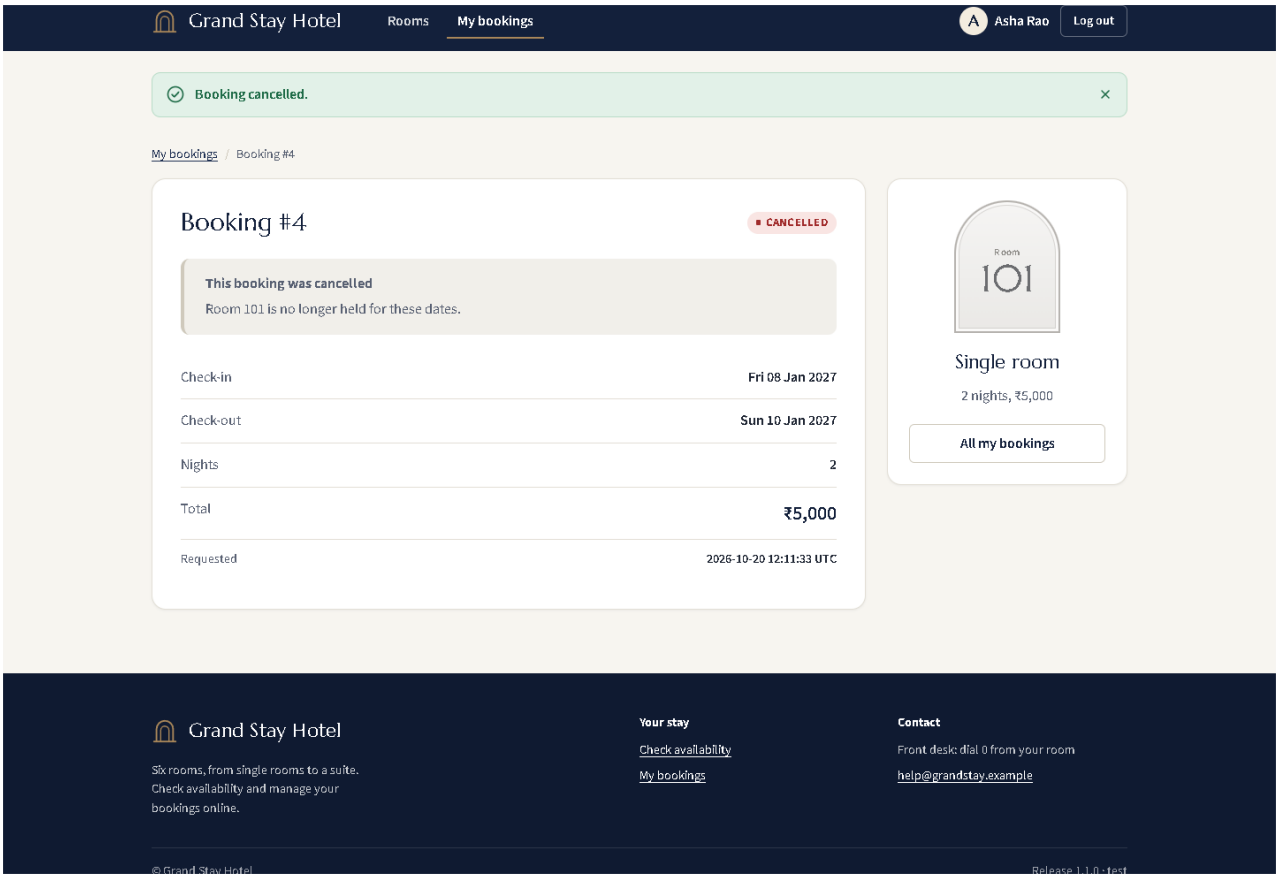


Figure 6.7 - Workflow 5: status CANCELLED, room released.

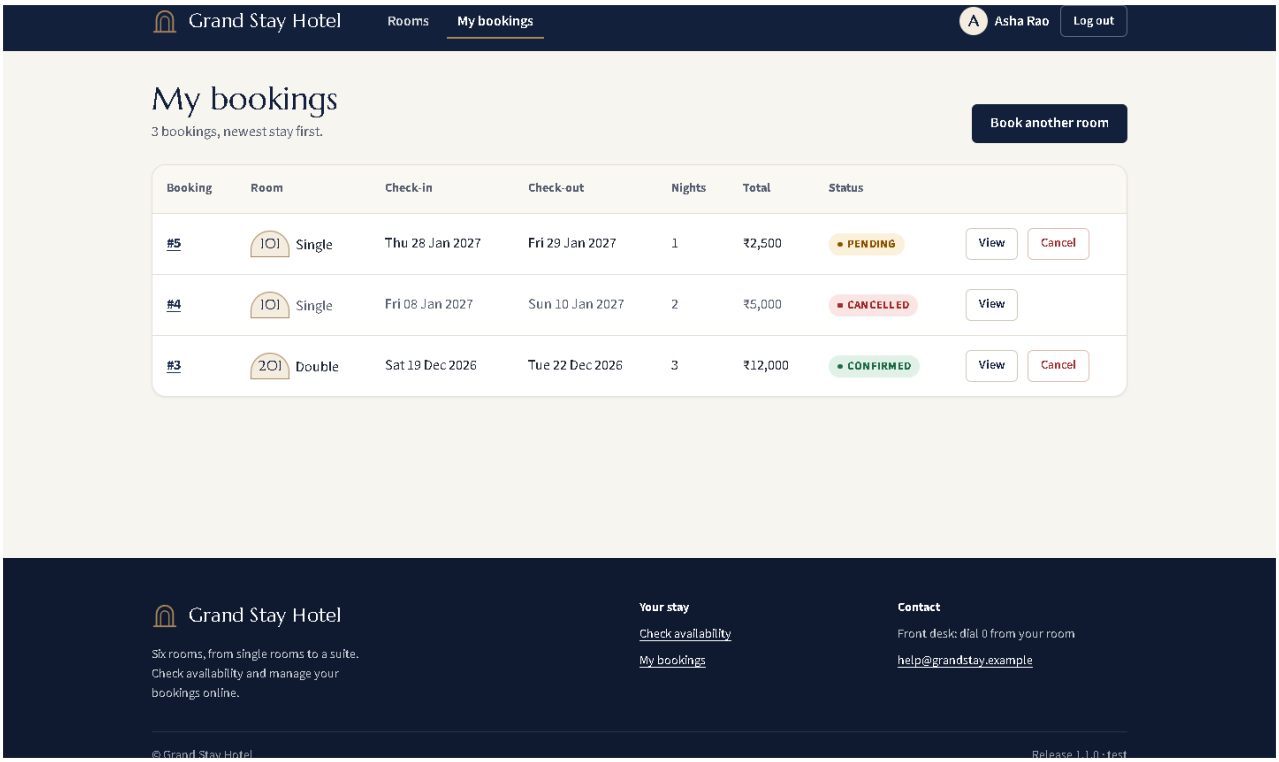
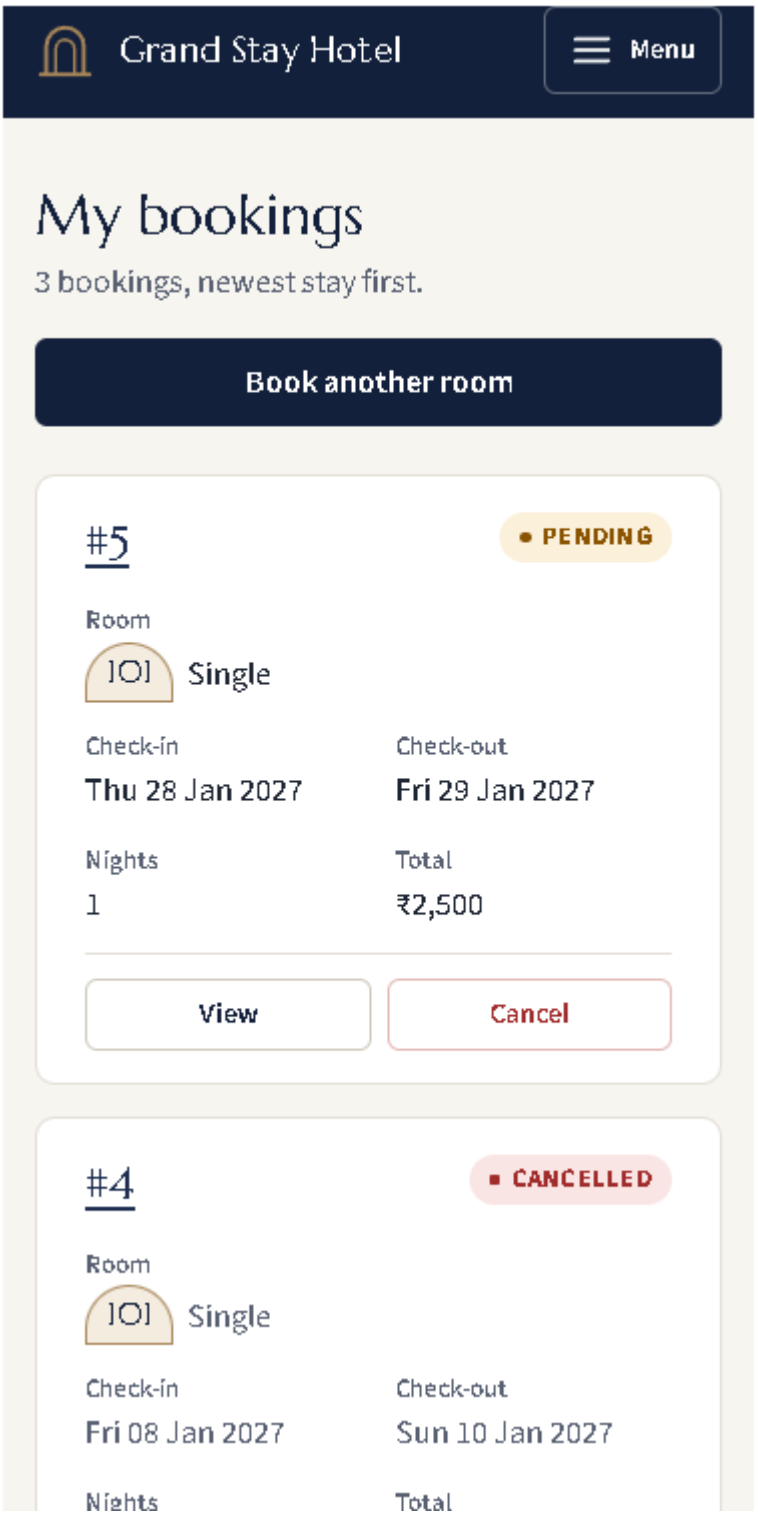
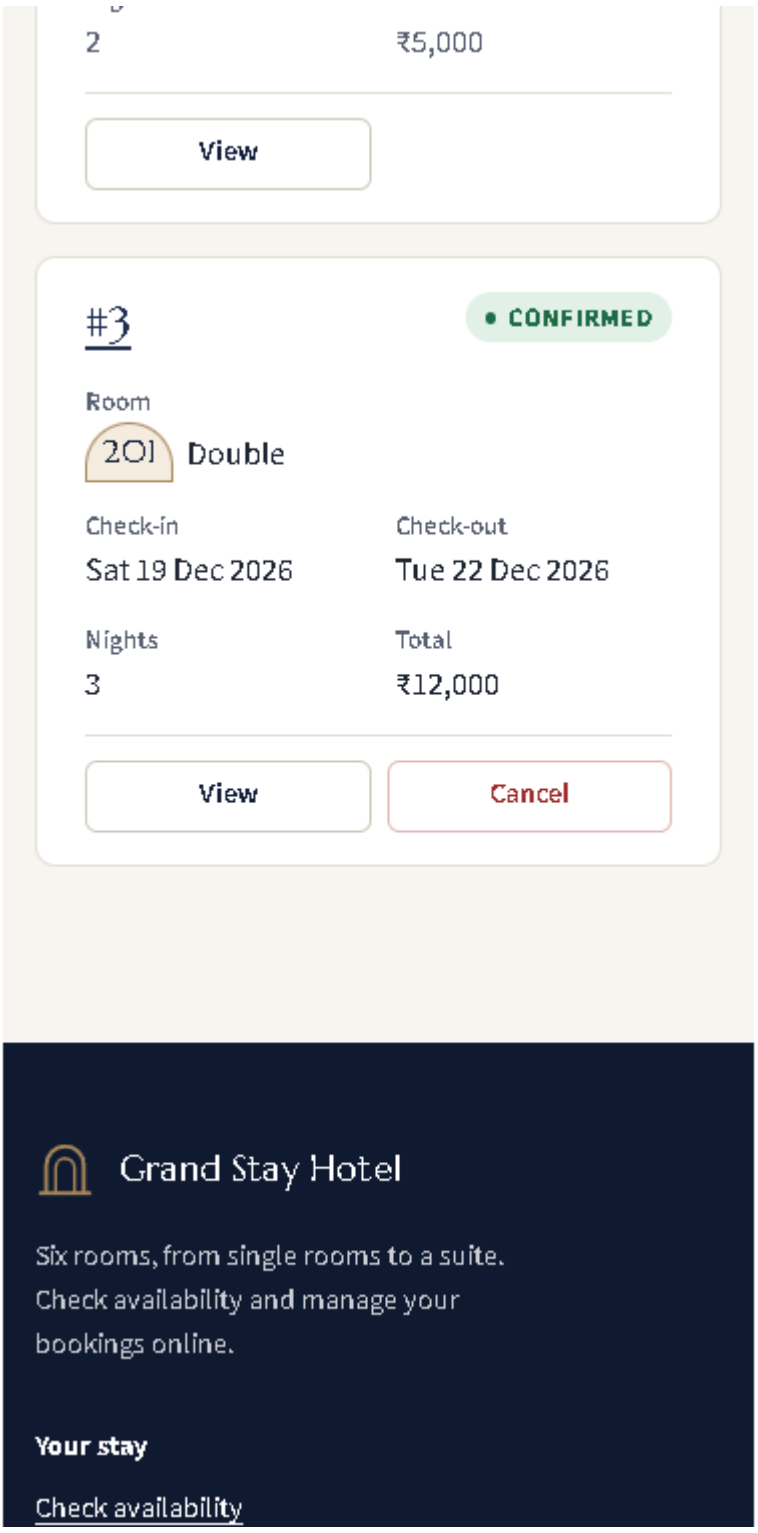


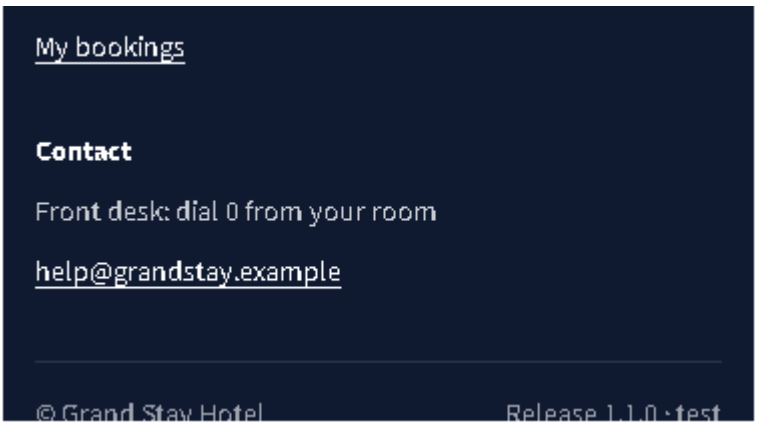
Figure 6.8 - Workflow 6: status tracking for all of the guest's bookings (PENDING, CANCELLED, CONFIRMED).


Grand Stay Hotel
Menu
My bookings
3 bookings, newest stay first.
Book another room
#5
PENDING
Room
101
Single
Check-in
Thu 28 Jan 2027
Check-out
Fri 29 Jan 2027
Nights
1
Total
₹2,500
View
Cancel
#4
CANCELLED
Room
101
Single
Check-in
Fri 08 Jan 2027
Check-out
Sun 10 Jan 2027
Nights
Total

(part 1 of 3)


2
₹5,000
View
#3
• CONFIRMED
Room
201
Double
Check-in
Sat 19 Dec 2026
Check-out
Tue 22 Dec 2026
Nights
3
Total
₹12,000
View
Cancel
Grand Stay Hotel
Six rooms, from single rooms to a suite.
Check availability and manage your
bookings online.
Your stay
Check availability

(part 2 of 3)



(part 3 of 3)

Figure 6.9 - The same page at phone width (375 px).

Collaboration

Git - real merge conflict output (2026-10-11, original commit IDs)

```
$ git merge --no-ff feature/release-footer
Auto-merging app/static/style.css
CONFLICT (content): Merge conflict in app/static/style.css
Auto-merging app/templates/base.html
CONFLICT (content): Merge conflict in app/templates/base.html
Auto-merging tests/integration/test_routes.py
CONFLICT (content): Merge conflict in tests/integration/test_routes.py
Automatic merge failed; fix conflicts and then commit the result.
(exit code 1)

$ git status --short
M  app/__init__.py
UU app/static/style.css
UU app/templates/base.html
UU tests/integration/test_routes.py
?? docs/evidence/

$ git diff  (conflict markers)
diff --cc app/static/style.css
index 6a64e90,08f5207..0000000
--- a/app/static/style.css
+++ b/app/static/style.css
@@@ -48,5 -48,5 +48,10 @@@ th, td { text-align: left; padding: .5r
   .actions { display: flex; gap: .75rem; margin: 1rem 0; }
   .hint { color: var(--muted); }

++<<<<<<< HEAD
 +/* Footer: hotel contact details on every page. */
 +.site-footer { text-align: center; color: var(--muted); font-size: .85rem; padding: 2rem 0 1rem; border-top: 1px solid #e4e7ec; }
++=======
+ /* Footer: shows which release is running. */
+ .site-footer { text-align: center; color: var(--muted); font-size: .8rem; padding: 1.5rem 0; }
++>>>>>>> feature/release-footer
diff --cc app/templates/base.html
index 75dd08d,65a5542..0000000
--- a/app/templates/base.html
+++ b/app/templates/base.html
@@@ -35,7 -35,7 +35,11 @@@
     </main>

     <footer class="site-footer" id="site-footer">
++<<<<<<< HEAD
 +    <span id="hotel-contact">Grand Stay Hotel · Front desk: dial 0 from your room · help@grandstay.example</span>
++=======
+     <span id="app-version">Release {{ app_version }} · {{ app_env }}</span>
++>>>>>>> feature/release-footer
     </footer>
   </body>
   </html>
diff --cc tests/integration/test_routes.py
index bbf9a47,6535dd4..0000000
--- a/tests/integration/test_routes.py
+++ b/tests/integration/test_routes.py
@@@ -202,6 -202,8 +202,14 @@@ def test_prod_refuses_to_start_without_
                      "DATABASE_PATH": str(tmp_path / "p.db")})


++<<<<<<< HEAD
 +def test_every_page_shows_hotel_contact_details(client):
 +    for url in ("/rooms", "/login", "/register"):
 +        assert 'id="hotel-contact"' in client.get(url).get_data(as_text=True)
++=======
+ def test_footer_shows_the_running_release(tmp_path):
+     app = create_app({"APP_ENV": "test", "SECRET_KEY": "k", "APP_VERSION": "9.9.9",
+                       "DATABASE_PATH": str(tmp_path / "f.db")})
++>>>>>>> feature/release-footer
```

(part 1 of 2)

```
+     html = app.test_client().get("/rooms").get_data(as_text=True)
+     assert 'id="app-version">Release 9.9.9 · test<' in html
++>>>>>>> feature/release-footer

--- Resolution commit ---
commit 8728b6224728ed82e9c7df53fa208687a3cf67da
Merge: cfeacef c24148f
Author: Kem.Nxts <padhaio087@gmail.com>
Date:   Sun Oct 11 22:21:26 2026 +0530

    Merge branch 'feature/release-footer' into develop

    Conflicts resolved (both branches added a footer in the same place):
    - app/templates/base.html: keep both lines - hotel contact first, then
      the running release.
    - app/static/style.css: one .site-footer rule (contact branch styling)
      plus a rule that puts each footer line on its own row.
    - tests/integration/test_routes.py: keep both new tests.
    Verified after resolution: flake8 clean, 89 tests passed.

    Co-Authored-By: Claude Opus 5.5 <noreply@anthropic.com>

 app/__init__.py                 | 6 ++++++
 app/static/style.css            | 4 +++-
 app/templates/base.html         | 1 +
 tests/integration/test_routes.py | 7 +++++++
 4 files changed, 17 insertions(+), 1 deletion(-)

$ git log --graph --oneline -8
*   8728b62 Merge branch 'feature/release-footer' into develop
|\
| * c24148f feat: show running release and environment in page footer
* |   cfeacef Merge branch 'feature/contact-footer' into develop
|\ \
| |/
|/|
| * 65ad990 feat: add hotel contact details footer to every page
|/
*   f4d0b26 Merge branch 'feature/ci-release-pipeline' into develop
|\
| * 64351fe ci: add lint, versioned images, registry publish and verified deploy
|/
*   37a4663 Merge branch 'feature/security-hardening' into develop
|\
| * 388c47b test: add route-level integration tests and database rule tests
```

Rendered from the real log file docs/evidence/git/merge-conflict.txt (unedited lines)

(part 2 of 2)

Figure 6.10 - The conflict as recorded on 2026-10-11 (rendered from `docs/evidence/git/merge-conflict.txt`; original commit IDs). **Notice:** three `CONFLICT (content)` lines and the conflict markers from both branches. **Proves:** the conflict was real Git output, not described afterwards.

Git - the merge commit that resolved the real conflict

```
$ git log --graph --format="%h %s" da225fb..9e766ac
*   9e766ac Merge branch 'feature/release-footer' into develop
|\
| * 63498a8 feat: show running release and environment in page footer
* 4463add Merge branch 'feature/contact-footer' into develop
* 74a153d feat: add hotel contact details footer to every page

$ git show --stat 9e766ac
commit 9e766acc473fc0e07f1e161e4de14db6f4efcef0
Merge: 4463add 63498a8
Author: Kem.Nxts <padhaio087@gmail.com>
Date:   Sun Oct 11 22:21:26 2026 +0530

    Merge branch 'feature/release-footer' into develop

    Conflicts resolved (both branches added a footer in the same place):
    - app/templates/base.html: keep both lines - hotel contact first, then
      the running release.
    - app/static/style.css: one .site-footer rule (contact branch styling)
      plus a rule that puts each footer line on its own row.
    - tests/integration/test_routes.py: keep both new tests.
    Verified after resolution: flake8 clean, 89 tests passed.

 app/__init__.py                 | 6 ++++++
 app/static/style.css            | 4 +++-
 app/templates/base.html         | 1 +
 tests/integration/test_routes.py | 7 +++++++
 4 files changed, 17 insertions(+), 1 deletion(-)
```

Rendered from real command output captured 2026-10-20 12:55 UTC in CICD_FOR_HOTEL_ROOM_BOOKING | raw text: merge-conflict-resolution.txt

Figure 6.11 - `git log --graph` and `git show --stat 9e766ac` today (rendered from real output). **Notice:** the two footer branches in the graph and the merge message describing each resolution decision.

Git - release tags v1.0.0 and v1.1.0

```
$ git tag -n1
v1.0.0          v1.0.0 - MVP: registration/login, availability, booking request, confirmation, cancellation, status tracking; CI/CD
with Jenkins, Docker, Ansible
v1.1.0          v1.1.0 - show number of nights in My bookings

$ git log --oneline -1 v1.0.0
224de14 release: v1.0.0

$ git log --oneline -1 v1.1.0
ce1726d release: v1.1.0

$ git cat-file -p v1.1.0
object ce1726d6ec217bba2e28962bf960ae61f3b96cf6
type commit
tag v1.1.0
tagger Kem.Nxts <padhaio087@gmail.com> 1791738923 +0530

v1.1.0 - show number of nights in My bookings
```

Rendered from real command output captured 2026-10-20 12:55 UTC in CICD_FOR_HOTEL_ROOM_BOOKING | raw text: release-tags.txt

Figure 6.12 - `git tag -n1` and the annotated tag object of `v1.1.0` (rendered from real output).

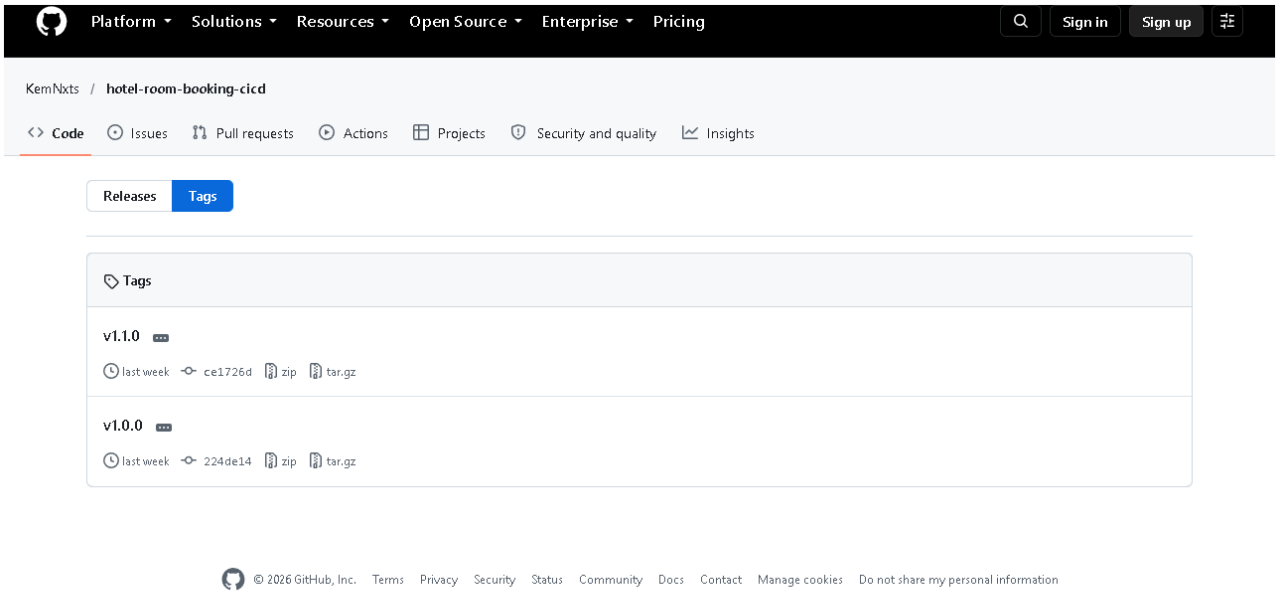


Figure 6.13 - `v1.0.0` and `v1.1.0` on GitHub.

Platform Solutions Resources Open Source Enterprise Pricing Sign in Sign up

KemNxts / hotel-room-booking-cicd

Code Issues Pull requests Actions Projects Security and quality Insights

Files

main

Go to file

- .github
- app
- certs
- deploy
- docs
  - evidence
  - pull-requests
  - 01-problem-definition.md
  - 02-agile-plan.md
  - 03-srs-architecture.md
  - 04-git-workflow.md
  - 05-ci-cd-pipeline.md
  - 06-test-plan.md
  - 07-ansible.md
  - 08-project-report.md
  - 15-task-checklist.md
  - container-lifecycle.md
  - viva-notes.md
- scripts
- tests
- .dockerignore
- .env.example
- .flake8
- .gitattributes
- .gitignore
- CONTRIBUTING.md
- Dockerfile
- Jenkinsfile
- README.md
- VERSION
- docker-compose.yml
- gunicorn.conf.py
- pytest.ini
- requirements-dev.txt
- requirements.txt
- wsgi.py

hotel-room-booking-cicd / docs / 04-git-workflow.md

KemNxts docs: record final green pipeline run (build #10) 5fe1e7b · last week History

Preview Code Blame 149 lines (122 loc) · 5.96 KB Raw

4. Git Workflow, Feature Branches, Merge Conflict and Releases (Tasks 4-6)

Honest status of GitHub

The repository is complete **locally** (git history, branches, tags), but it has **not been pushed to GitHub yet** - pushing and opening PRs needs your GitHub account. Therefore:

- Merges below are **real local `git merge --no-ff` merges** (visible in `git log --graph`).
- GitHub **Pull Requests do not exist yet**. The PR descriptions are prepared in pull-requests/ so you can open them, or show them as the review record.

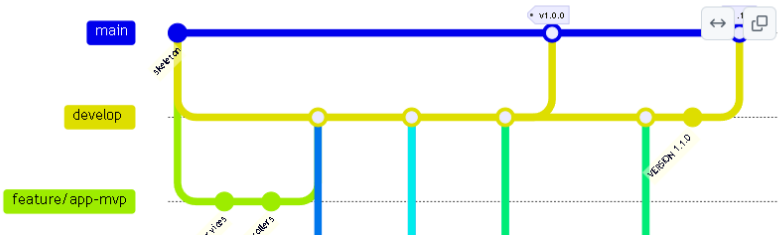
How to publish (when you decide to):

```
git remote add origin https://github.com/<you>/<repo>.git
git push -u origin main develop
git push origin --tags
git push origin feature/release-footer feature/contact-footer   # any branch you want to show
```

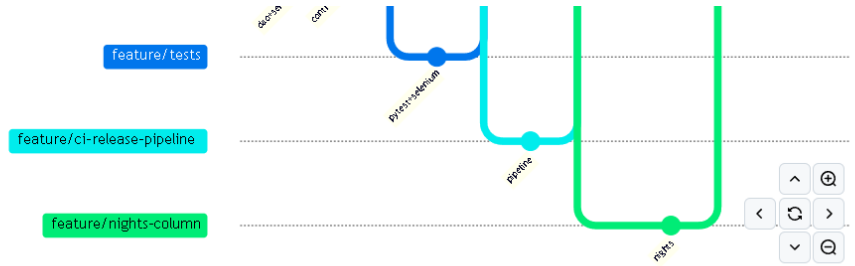
Repository structure (Task 4)

Item	Where
README	`README.md`
Ignore rules	`.gitignore` (venv, databases, reports, `.env`, credentials, `certs/*.crt`)
Line endings	`.gitattributes` (LF in the repo)
Issue templates	`.github/ISSUE_TEMPLATE/bug_report.md`, `feature_request.md`
PR template + review checklist	`.github/pull_request_template.md`
Branch and commit policy	`CONTRIBUTING.md`
Initial baseline	commit `08702db chore: initialise repository skeleton`

Branching model



(part 1 of 3)



(Simplified - the full graph: `git log --graph --oneline --all`.)

- `main`: released code only. Every software release on it is tagged; a documentation-only merge may follow a tag (as after v1.1.0).
- `develop`: integration branch; Jenkins builds it and deploys to staging.
- `feature/*`, `fix/*`: one per change, merged into `develop` with `--no-ff` so the history shows each feature as a unit.
- `experiment/*`: throw-away branches (deliberate-defect demo). **Never merged.**

Feature branches and merges (Task 5)

Feature branch	Merge commit on develop	What it delivered
feature/app-mvp	1677565	Layered Flask app, all 6 workflows
feature/tests	1ebec7c	pytest unit tests + Selenium journeys
feature/docker	0602c39	Dockerfile, Nginx, compose
feature/jenkins-pipeline	9c6fcf4	First Jenkinsfile, local Jenkins
feature/ansible	d670fad	Inventory + playbook
feature/security-hardening	37a4663	CSRF, cookie flags, audit log, DB triggers, integration tests
feature/ci-release-pipeline	f4d0b26	Lint, versioned images, registry, verified deploy
feature/contact-footer	cfeacef	Contact footer
feature/release-footer	8728b62	Release footer - **merge conflict, see below**
feature/ansible-rollback	224384f	Versioned Ansible deploys, rollback.yml, lab
feature/nights-column	9969df6	Nights column (release 1.1.0)
fix/jenkins-stale-reports	848d710	Pipeline bug fix found from evidence

Worked example for the viva - `feature/security-hardening`:

1. `git checkout -b feature/security-hardening develop`
2. Commit `65c41cb feat: add CSRF protection, secure cookies, audit logs and DB overlap guard`
3. Commit `388c47b test: add route-level integration tests and database rule tests`
4. Run `flake8` + `pytest` locally (87 passed at that point).
5. Review with the PR checklist (pull-requests/feature-security-hardening.md).
6. `git merge --no-ff feature/security-hardening` into `develop` -> `37a4663`.

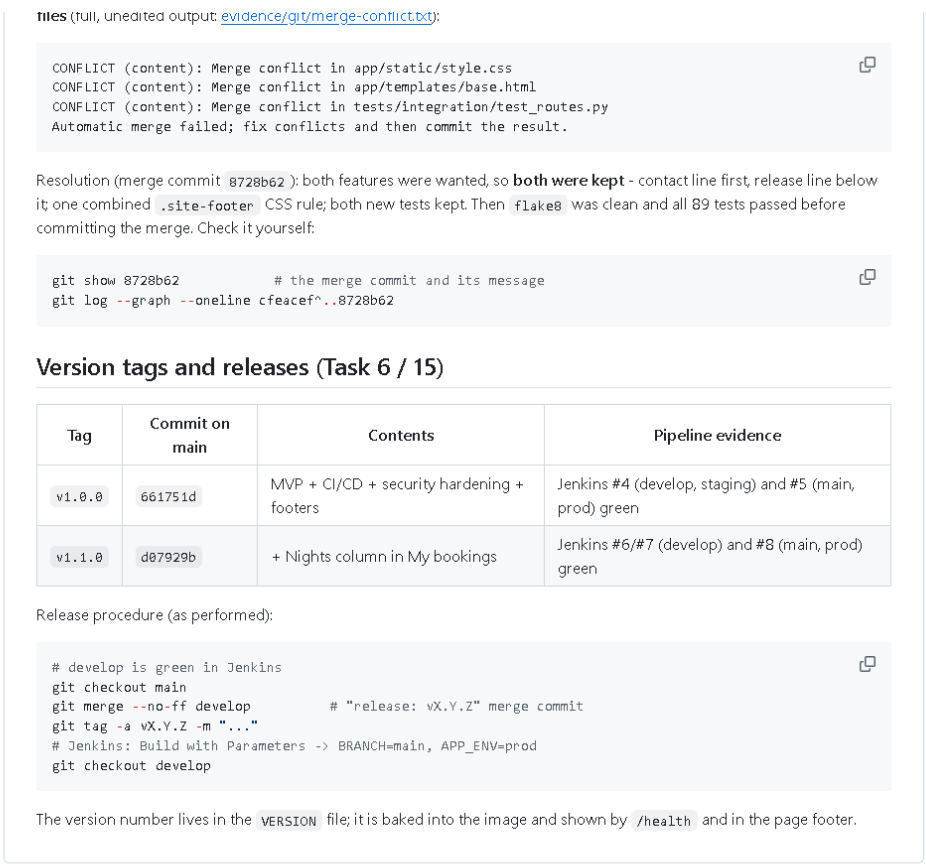
Real merge conflict and its resolution (Task 6)

Two features were developed **in parallel from the same `develop` commit** (`f4d0b26`), and both added a page footer in the same place:

- `feature/contact-footer` (`65ad990`): hotel contact details.
- `feature/release-footer` (`c24148f`): the running release and environment.

`feature/contact-footer` merged cleanly first. Merging `feature/release-footer` then produced a real conflict in **three**

(part 2 of 3)


files (full, unedited output: evidence/git/merge-conflict.txt):

```
CONFLICT (content): Merge conflict in app/static/style.css
CONFLICT (content): Merge conflict in app/templates/base.html
CONFLICT (content): Merge conflict in tests/integration/test_routes.py
Automatic merge failed; fix conflicts and then commit the result.
```

Resolution (merge commit `8728b62`): both features were wanted, so **both were kept** - contact line first, release line below it; one combined `.site-footer` CSS rule; both new tests kept. Then `flake8` was clean and all 89 tests passed before committing the merge. Check it yourself:

```
git show 8728b62           # the merge commit and its message
git log --graph --oneline cfeacef^..8728b62
```

Version tags and releases (Task 6 / 15)

Tag	Commit on main	Contents	Pipeline evidence
`v1.0.0`	`661751d`	MVP + CI/CD + security hardening + footers	Jenkins #4 (develop, staging) and #5 (main, prod) green
`v1.1.0`	`d07929b`	+ Nights column in My bookings	Jenkins #6/#7 (develop) and #8 (main, prod) green

Release procedure (as performed):

```
# develop is green in Jenkins
git checkout main
git merge --no-ff develop          # "release: vX.Y.Z" merge commit
git tag -a vX.Y.Z -m "..."
# Jenkins: Build with Parameters -> BRANCH=main, APP_ENV=prod
git checkout develop
```

The version number lives in the `VERSION` file; it is baked into the image and shown by `/health` and in the page footer.

(part 3 of 3)

Figure 6.14 - `docs/04-git-workflow.md` on GitHub: branching model, merges, the conflict story and the release procedure.

E. How to demonstrate this task to the professor (about 5 minutes)

Before the demo: PowerShell in `D:\DevOPS\CICD_FOR_HOTEL_ROOM_BOOKING`.

Part 1 - the six workflows (browser).

```
.venv\Scripts\python -m flask --app wsgi run --port 5001
```

Open http://127.0.0.1:5001: register -> search dates -> *Book room* -> *Confirm booking* -> book another -> *Cancel booking* -> *Yes, cancel booking* -> *My bookings*. *Say:* "These are the six MVP workflows; they are also checked automatically by five Selenium journeys."

Part 2 - the real merge conflict, replayed safely in a throw-away copy (new PowerShell window; nothing in the real repository changes):

```
cd $env:TEMP
git clone D:\DevOPS\CICD_FOR_HOTEL_ROOM_BOOKING conflict-demo
cd conflict-demo
git checkout -b replay da225fb
git merge --no-ff --no-edit origin/feature/contact-footer
git merge --no-ff --no-edit origin/feature/release-footer
```

Expected: the first merge succeeds; the second prints three `CONFLICT (content)` lines and "Automatic merge failed" (verified 2026-10-20).

```
git status
git diff
```

Point to: `UU` files and the `<<<<<<< HEAD` / `=======` / `>>>>>>>` blocks. *Say:* "Both branches added a footer in the same place, so Git asks a human. I kept both lines." Then show the real resolution and clean up:

```
git merge --abort
```

```
git show --stat 9e766ac
cd ..
Remove-Item -Recurse -Force conflict-demo
```

Part 3 - tags. In the project folder:

```
git tag -n1
git show v1.0.0 --stat --oneline
```

Say: "Tags are fixed labels for the released commits; branches keep moving."

Recovery: if `git clone` says the folder exists, run `Remove-Item -Recurse -Force $env:TEMP\conflict-demo` first.

F. Expected screenshots for the final submission

41. `assets/task-06/app-desktop-03-register.png` - **exists, 2026-10-20** (copied from `docs/ui-evidence/after/`).
42. `assets/task-06/app-desktop-05-login-error.png` - **exists, 2026-10-20**.
43. `assets/task-06/app-desktop-06-availability-logged-in.png` - **exists, 2026-10-20**.
44. `assets/task-06/app-desktop-08-booking-requested.png` - **exists, 2026-10-20**.
45. `assets/task-06/app-desktop-09-booking-confirmed.png` - **exists, 2026-10-20**.
46. `assets/task-06/app-desktop-10-cancel-dialog.png` - **exists, 2026-10-20**.
47. `assets/task-06/app-desktop-11-booking-cancelled.png` - **exists, 2026-10-20**.
48. `assets/task-06/app-desktop-12-my-bookings.png` - **exists, 2026-10-20**.
49. `assets/task-06/app-mobile-12-my-bookings.png` - **exists, 2026-10-20**.
50. `assets/task-06/merge-conflict-output.png` - **exists** (rendered from the 2026-10-11 log).
51. `assets/task-06/merge-conflict-resolution.png` - **exists, 2026-10-20**.
52. `assets/task-06/release-tags.png` - **exists, 2026-10-20**.
53. `assets/task-06/github-release-tags.png` - **exists, 2026-10-20**.
54. `assets/task-06/github-doc-04-git-workflow.png` - **exists, 2026-10-20**.

G. Probable viva questions and answers

29. **What is a merge conflict?** Two branches changed the same lines; Git cannot choose, so a human edits the result.
30. **How did your conflict happen?** Two features branched from the same commit and both added a footer in `base.html` (plus CSS and a test). The second merge conflicted in all three files.
31. **How did you resolve it?** Kept both features (contact line, then release line), merged the CSS into one rule, kept both tests, ran flake8 and all tests (89 passed), then committed the merge `9e766ac`.
32. **How do you know you didn't break anything?** Tests after resolution, and Jenkins build #4 of develop was green before the v1.0.0 release.
33. **Branch vs tag?** A branch moves with new commits; a tag stays on one commit.
34. **Why tag releases?** To redeploy or roll back exactly what was released (`v1.0.0` was used for the Ansible rollback demo).
35. **(Project-specific) How are passwords stored?** Werkzeug scrypt hashes (`generate_password_hash`), never plain text - a unit test checks this.
36. **(Project-specific) What stops one guest cancelling another's booking?** The service checks the owner; another user gets 404 and the status stays unchanged (integration test).

37. **What is a stable baseline?** A tagged commit on `main` that passed the full pipeline - `v1.1.0` (`ce1726d`), running in prod.
38. **Why are there two IDs for the same commit in your documents?** The messages were edited once before publishing (removing a co-author line); the map is in `docs/evidence/git/commit-id-map.md`.
39. **Is the conflict screenshot genuine?** Yes - Git's own output was saved to a file at the time, and the conflict can be reproduced live from the pushed branches, as in the demo.

H. Common mistakes and troubleshooting

Symptom	Cause	Inspect	Fix	Confirm
Replay merge does not conflict	wrong start commit	`git log --oneline -1`	start from `da225fb`	three CONFLICT lines
Conflict markers left in a file	resolution incomplete	`git grep -n "<<<<<<<"`	edit file, remove markers	grep finds nothing; tests pass
`git merge --abort` fails	merge already committed	`git status`	in the throw-away clone just delete it	folder removed
Booking button missing	logged out, or room booked	page shows "Log in to book"/"Booked"	log in / choose other dates	Book button appears
HTTP 400 on a form	stale CSRF token	page open across restart	reload the page	form submits

I. Quick revision notes

Five key points

37. Six workflows, each with a route, a service function and a Selenium journey.
38. Security: hashed passwords, CSRF tokens, secure cookies, owner checks.
39. Real conflict between `feature/contact-footer` and `feature/release-footer`, 3 files.
40. Resolved by keeping both; merge commit `9e766ac`; 89 tests passed afterwards.
41. Releases tagged `v1.0.0`, `v1.1.0`; prod runs v1.1.0.

30-60 second explanation "All six booking workflows are complete and tested by Selenium. For collaboration, two features were developed in parallel from the same commit - a contact footer and a release footer. Both changed the same lines, so the second merge produced a real conflict in three files. I kept both features, ran all tests, and committed the merge. After the pipeline passed, I merged develop into main and tagged the releases v1.0.0 and v1.1.0. I can replay the conflict live in a throw-away copy of the repository."

Caveat: the screenshots show the new UI, which is not yet committed or deployed.

Readiness checklist

- â€¢ â˜ I can click through all six workflows.
- â€¢ â˜ I can replay the conflict in a temp clone and explain the markers.
- â€¢ â˜ I can explain branch vs tag.
- â€¢ â˜ I can explain the two commit IDs.

Task 7 - Jenkins Installation and Continuous Integration

A. Task title and objective

Task 7: Jenkins Installation and Continuous Integration (CI) Job

Objective: install Jenkins and configure a job that, for every change, checks out the repository, prepares Python, installs dependencies, runs static checks and tests, publishes test reports and archives artifacts.

Professor's required deliverables

- â€¢ Jenkins installed and a job configured for the repository
- â€¢ Checkout; Python environment; reproducible dependency installation
- â€¢ Linting / syntax checks
- â€¢ pytest execution and test reports
- â€¢ Archived artifacts (source package + reports)
- â€¢ Build trigger: GitHub webhook if possible, otherwise polling or manual trigger (documented)

Current status: Implemented and verified. A real run on 2026-10-20 - Jenkins build **#11** - passed every stage with 90 tests and archived its artifacts (Figures 7.3-7.5). All 11 builds are visible in Jenkins (Figure 7.1).

Two honest points:

- â€¢ **Trigger:** Jenkins **polls** Git every ~5 minutes (`pollSCM`) and builds are started with *Build with Parameters*. A GitHub **webhook** was not set up because this Jenkins runs on `localhost`, which GitHub cannot reach.
- â€¢ **Build tool:** the job uses Python tools (pip, compileall, flake8, pytest), **not a Maven job**. Professor's approval of this equivalent is needed.

B. Explanation for a complete beginner

An everyday picture. Imagine a quality inspector at a factory who, every time a new batch of parts arrives, takes them, assembles a sample, runs all the standard checks and writes a report - every time, the same way, without being asked. **Jenkins** is that inspector for code.

Continuous Integration (CI) - every change is automatically built and tested soon after it is committed, so problems are found within minutes, not weeks later.

Term	Meaning	In this project
Jenkins	an automation server that runs jobs	Jenkins LTS 2.580.1 in a Docker container, http://localhost:8090
Job	a configured task Jenkins can run	`hotel-booking`
Pipeline	a job made of ordered **stages**	defined in the `Jenkinsfile` (Task 8)
Build	one run of the job, numbered	#1 ... #11
Agent	the machine where steps run	here the Jenkins container itself (built-in node)
Trigger	what starts a build	polling every 5 min, or a person clicking *Build*
Polling	Jenkins asks Git "anything new?" on a schedule	`pollSCM('H/5 * * * *')`
Webhook	GitHub tells Jenkins immediately when something is pushed	not possible: Jenkins not reachable from the internet
Linting	automatic style/error checking without running the code	`flake8`
Artifact	a file produced by a build and kept with it	source tarball + checksum, test XML, deployment record

Term	Meaning	In this project
Plugin	add-on for Jenkins	Pipeline, Git, JUnit, Credentials Binding, Stage View, Timestamper

Three different things - don't mix them up

55. *An installed Jenkins* - the server is running (http://localhost:8090 answers).
56. *A saved pipeline definition* - the `Jenkinsfile` in Git.
57. *An executed job* - a numbered build with results, e.g. build #11 SUCCESS.

Only (3) proves the CI works; this report shows all three.

C. What was actually implemented

Installation as code - `deploy/jenkins/`:

File	Purpose
`Dockerfile`	Jenkins LTS (JDK 21) + Python 3.11 + Chromium & chromedriver (for Selenium) + Docker CLI with the compose plugin; trusts extra CA certificates from `certs/`
`plugins.txt`	`workflow-aggregator`, `git`, `junit`, `credentials-binding`, `pipeline-stage-view`, `timestamper`
`init.groovy.d/01-setup.groovy`	on first start: creates user `admin` (password from `.env`), the secret-text credential `hotel-secret-key`, and the job `hotel-booking` ("Pipeline script from SCM", repository `file:///repo`, branch `*/${BRANCH}`, parameters APP_ENV/BRANCH/REGISTRY)
`docker-compose.yml`	runs Jenkins on `127.0.0.1:8090` and a local image registry on `127.0.0.1:5000`; mounts the repository read-only at `/repo` and the host's Docker socket
`.env.example`	template for the git-ignored `.env` (`JENKINS_ADMIN_PASSWORD`, `HOTEL_SECRET_KEY`)

Start command (from `deploy\jenkins`): `docker compose up -d --build`.

What the CI part of the job does (stages from the `Jenkinsfile`):

Stage	Command	Fails when
Checkout	`checkout scm`, `rm -rf reports dist`, read `VERSION` + commit	branch missing
Install dependencies	`python3.11 -m venv .venv` then `pip install -r requirements-dev.txt` (pinned versions)	a package cannot be installed
Static checks	`python -m compileall -q app wsgi.py` + `flake8 app tests scripts wsgi.py gunicorn.conf.py`	syntax or lint error
Unit & integration tests	`pytest tests/unit tests/integration --junitxml=reports/unit-integration-tests.xml`	any test fails
E2E tests (Selenium)	`pytest -m e2e --junitxml=reports/e2e-tests.xml`	any journey fails
Package source	`git archive ... -o dist/hotel-booking-<tag>.tar.gz` + `sha256sum`	-

Reports and artifacts (`post { always { ... } }`, Jenkinsfile lines 211-212):

```
junit allowEmptyResults: true, testResults: 'reports/*.xml'        // test trend + Tests page
archiveArtifacts artifacts: 'reports/**, dist/**', allowEmptyArchive: true   // kept with the build
```

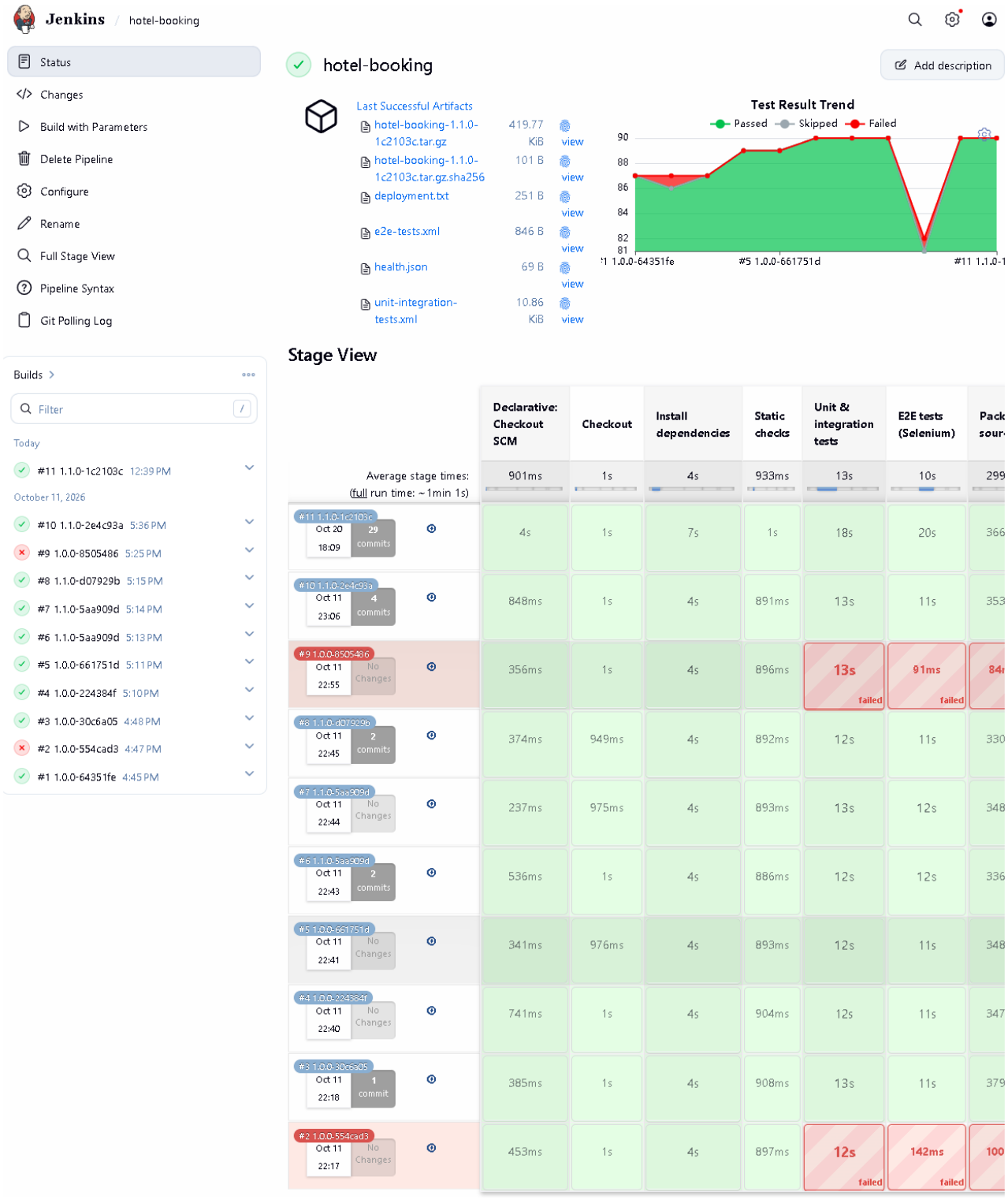
Build #11 archived: `hotel-booking-1.1.0-1c2103c.tar.gz` (+ `.sha256`), `unit-integration-tests.xml`, `e2e-tests.xml`, `health.json`, `deployment.txt`.

Build history (from the Jenkins API, 2026-10-20)

Build	Branch	Result
#1	feature/ci-release-pipeline	SUCCESS
#2	experiment/defect-demo	FAILURE (deliberate defect, Task 10)
#3-#8	experiment fix, develop, main	SUCCESS
#9	experiment/defect-demo-2	FAILURE (deliberate, Task 10)
#10	develop	SUCCESS
#11	develop (`1c2103c`)	**SUCCESS, 90 tests, 103 s - run during this audit**

Python instead of Maven. A Maven job would run `mvn test` on a Java project. This project is Python, so the equivalent steps are pip (dependencies), compileall + flake8 (compile/lint) and pytest (tests, writing the same JUnit XML format that Maven's Surefire plugin writes). They are not Maven.

D. Screenshots and evidence

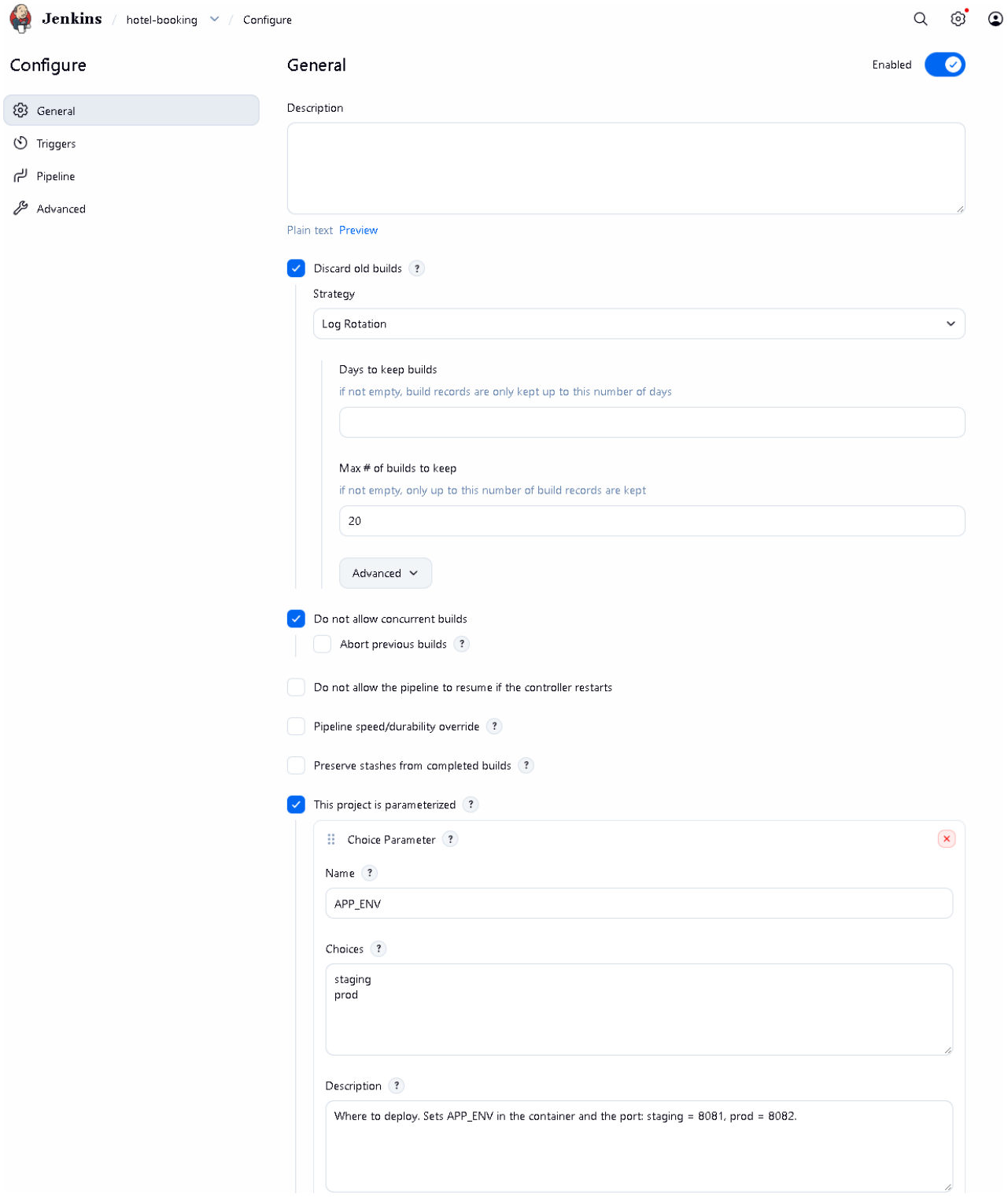


(part 1 of 2)



(part 2 of 2)

Figure 7.1 - Job `hotel-booking`: build history #1-#11, test-result trend, last artifacts and the stage view (captured 2026-10-20). **Notice:** builds #2 and #9 red, all others green; each stage's duration per build. **Proves:** Jenkins is installed, the job exists and has really executed builds.


Jenkins / hotel-booking / Configure
Configure
General
Triggers
Pipeline
Advanced
General
Enabled
Description
Plain text Preview
Discard old builds
Strategy
Log Rotation
Days to keep builds
if not empty, build records are only kept up to this number of days
Max # of builds to keep
if not empty, only up to this number of build records are kept
20
Advanced
Do not allow concurrent builds
Abort previous builds
Do not allow the pipeline to resume if the controller restarts
Pipeline speed/durability override
Preserve stashes from completed builds
This project is parameterized
Choice Parameter
Name
APP_ENV
Choices
staging
prod
Description
Where to deploy. Sets APP_ENV in the container and the port: staging = 8081, prod = 8082.

(part 1 of 3)

Plain text Preview

String Parameter

Name

BRANCH

Default Value

develop

Description

Git branch to build (e.g. develop, main, experiment/defect-demo).

Plain text Preview

Trim the string

String Parameter

Name

REGISTRY

Default Value

localhost:5000

Description

Image registry. localhost:5000 = local registry started with Jenkins. For Docker Hub use docker.io/<your-docker-id> and add the "registry-creds" credential.

Plain text Preview

Trim the string

Add Parameter

Throttle builds

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

Build after other projects are built

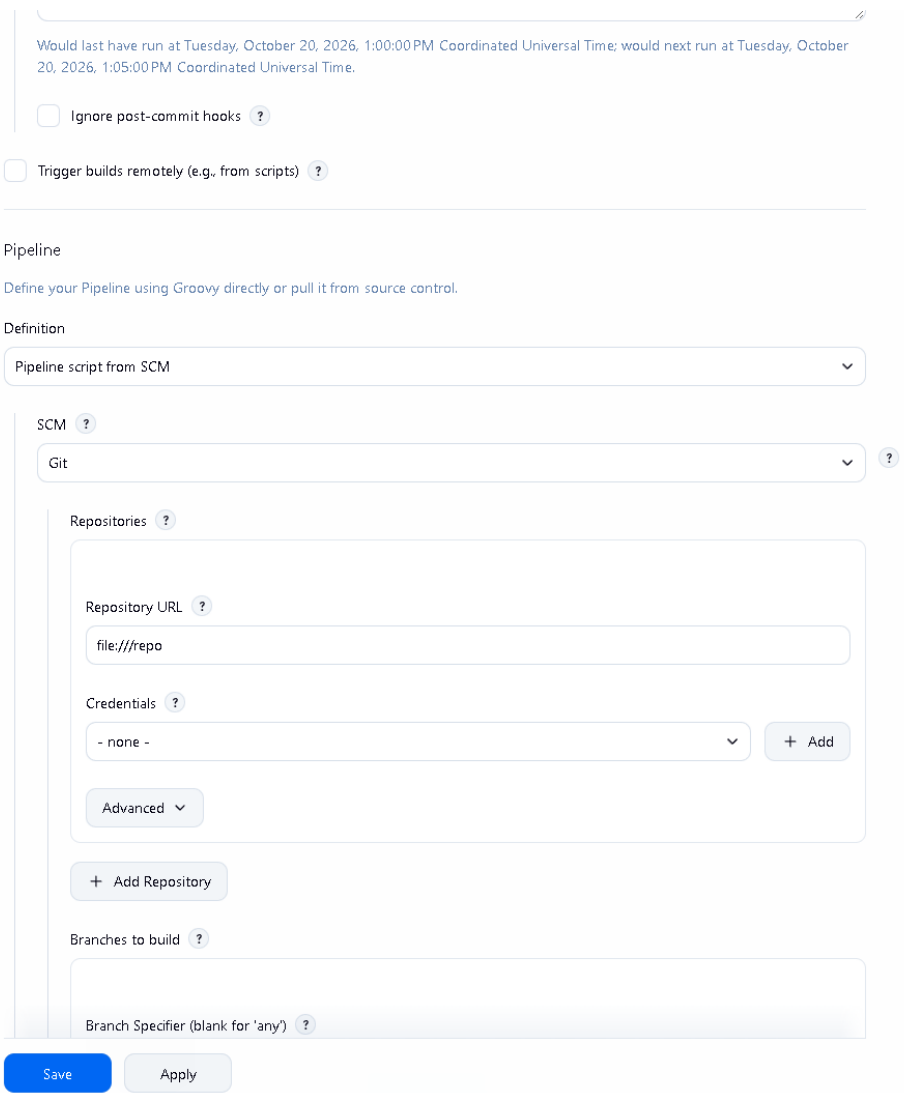
Build periodically

Poll SCM

Schedule

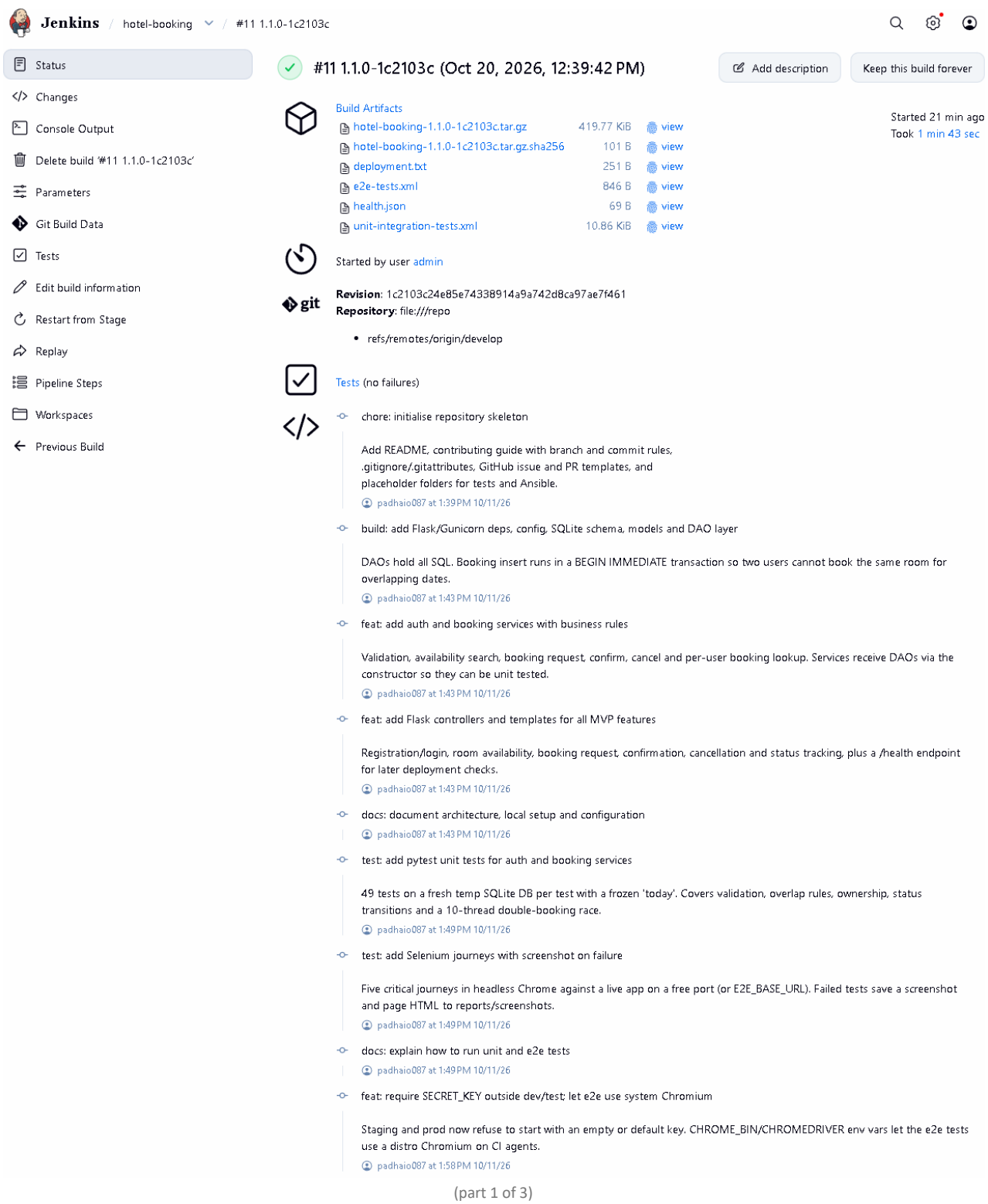
H/5 * * * *

(part 2 of 3)



(part 3 of 3)

Figure 7.2 - Job configuration: the three parameters, "Do not allow concurrent builds", Poll SCM `H/5 * * * *`, and "Pipeline script from SCM" with repository `file:///repo`. **Proves:** the trigger and SCM settings described above. No secret values appear on this page.


Jenkins / hotel-booking / #11 1.1.0-1c2103c
Status
Changes
Console Output
Delete build '#11 1.1.0-1c2103c'
Parameters
Git Build Data
Tests
Edit build information
Restart from Stage
Replay
Pipeline Steps
Workspaces
Previous Build
#11 1.1.0-1c2103c (Oct 20, 2026, 12:39:42 PM)
Add description
Keep this build forever
Build Artifacts
hotel-booking-1.1.0-1c2103c.tar.gz 419.77 KiB view
hotel-booking-1.1.0-1c2103c.tar.gz.sha256 101 B view
deployment.txt 251 B view
e2e-tests.xml 846 B view
health.json 69 B view
unit-integration-tests.xml 10.86 KiB view
Started 21 min ago
Took 1 min 43 sec
Started by user admin
Revision: 1c2103c24e85e74338914a9a742d8ca97ae7f461
Repository: file:///repo
refs/remotes/origin/develop
Tests (no failures)
chore: initialise repository skeleton
Add README, contributing guide with branch and commit rules, .gitignore/.gitattributes, GitHub issue and PR templates, and placeholder folders for tests and Ansible.
padhaio087 at 1:39 PM 10/11/26
build: add Flask/Gunicorn deps, config, SQLite schema, models and DAO layer
DAOs hold all SQL. Booking insert runs in a BEGIN IMMEDIATE transaction so two users cannot book the same room for overlapping dates.
padhaio087 at 1:43 PM 10/11/26
feat: add auth and booking services with business rules
Validation, availability search, booking request, confirm, cancel and per-user booking lookup. Services receive DAOs via the constructor so they can be unit tested.
padhaio087 at 1:43 PM 10/11/26
feat: add Flask controllers and templates for all MVP features
Registration/login, room availability, booking request, confirmation, cancellation and status tracking, plus a /health endpoint for later deployment checks.
padhaio087 at 1:43 PM 10/11/26
docs: document architecture, local setup and configuration
padhaio087 at 1:43 PM 10/11/26
test: add pytest unit tests for auth and booking services
49 tests on a fresh temp SQLite DB per test with a frozen 'today'. Covers validation, overlap rules, ownership, status transitions and a 10-thread double-booking race.
padhaio087 at 1:49 PM 10/11/26
test: add Selenium journeys with screenshot on failure
Five critical journeys in headless Chrome against a live app on a free port (or E2E_BASE_URL). Failed tests save a screenshot and page HTML to reports/screenshots.
padhaio087 at 1:49 PM 10/11/26
docs: explain how to run unit and e2e tests
padhaio087 at 1:49 PM 10/11/26
feat: require SECRET_KEY outside dev/test; let e2e use system Chromium
Staging and prod now refuse to start with an empty or default key. CHROME_BIN/CHROMEDRIVER env vars let the e2e tests use a distro Chromium on CI agents.
padhaio087 at 1:58 PM 10/11/26

(part 1 of 3)

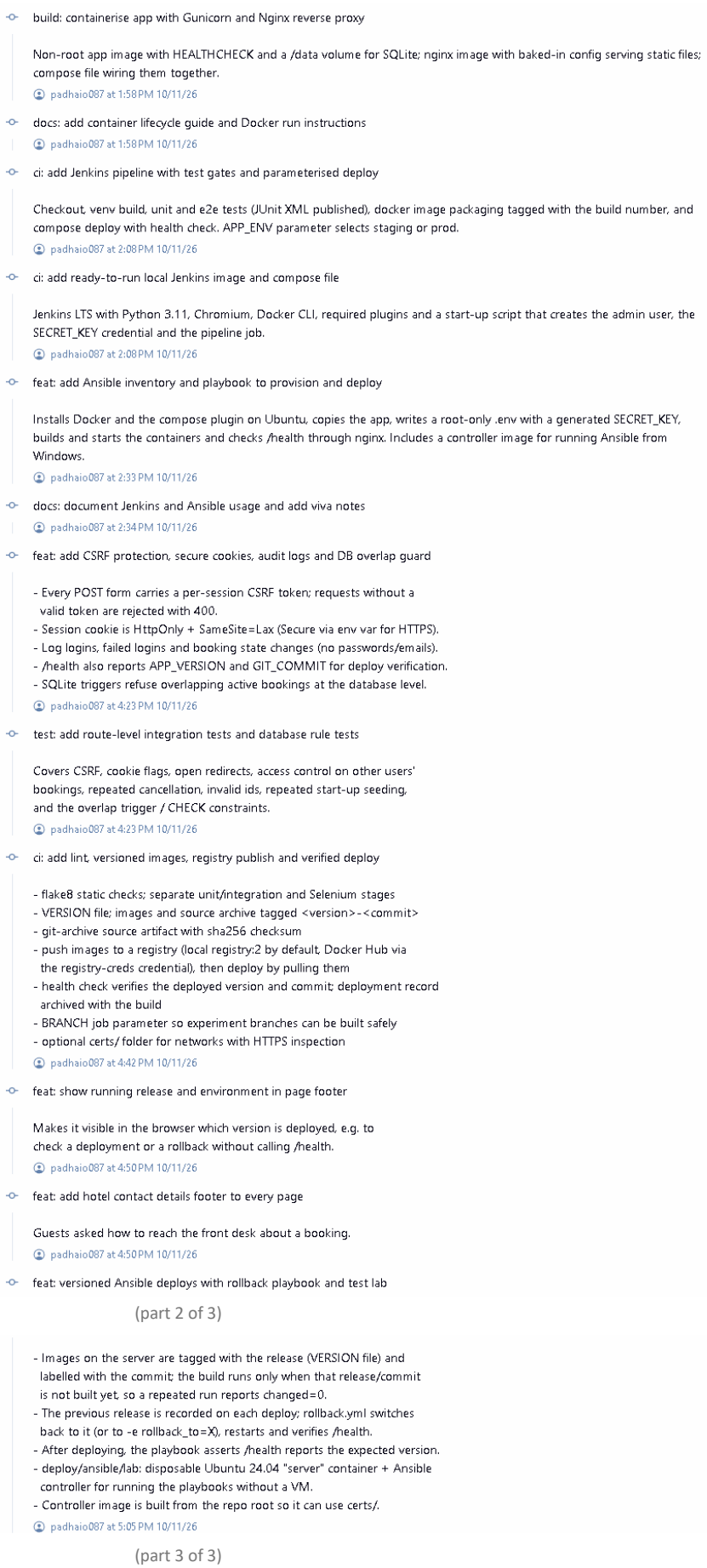


Figure 7.3 - Build #11 (`1.1.0-1c2103c`), started 2026-10-20: result SUCCESS.

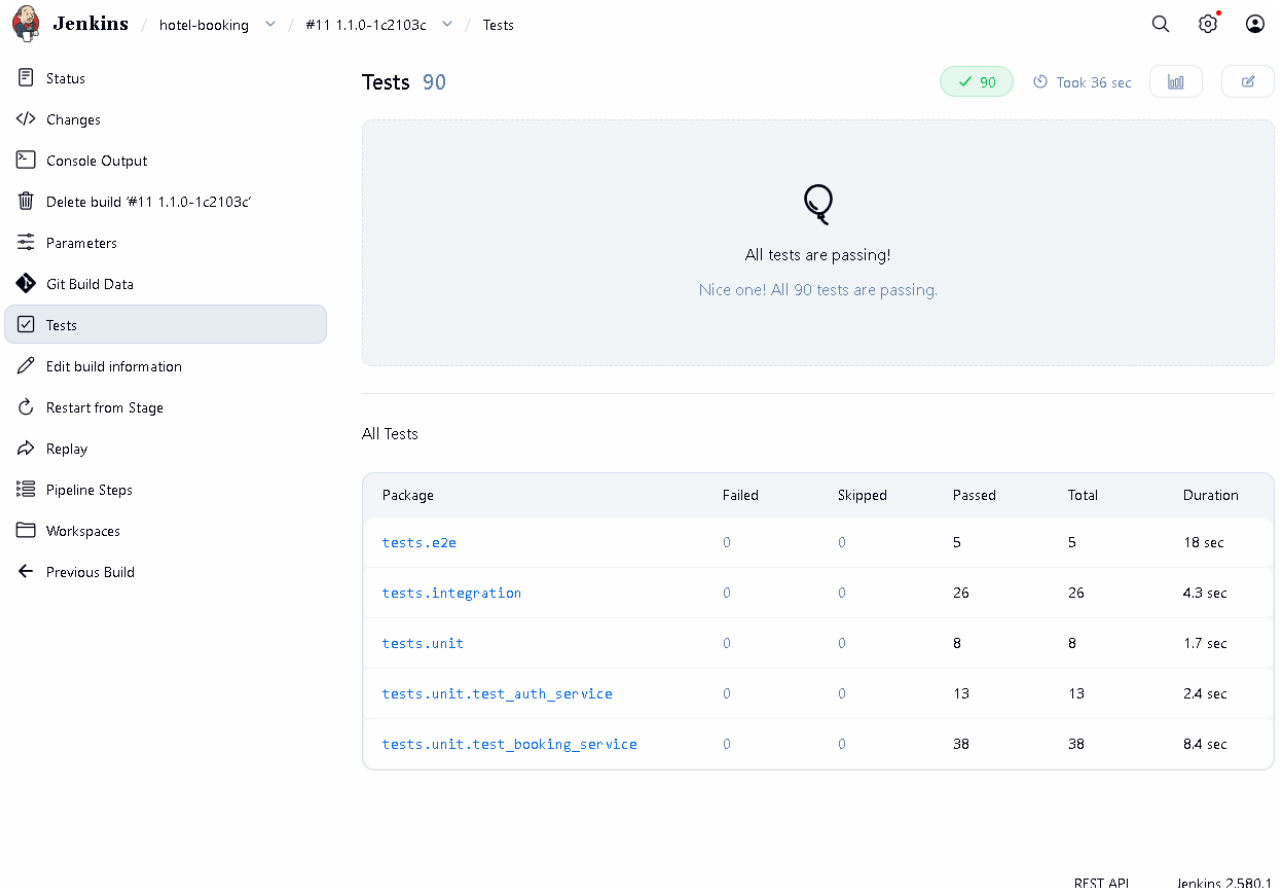


Figure 7.4 - Test report of build #11: 90 tests, 0 failures, grouped by package (unit, integration, e2e). **Proves:** pytest ran inside Jenkins and its JUnit reports were published.

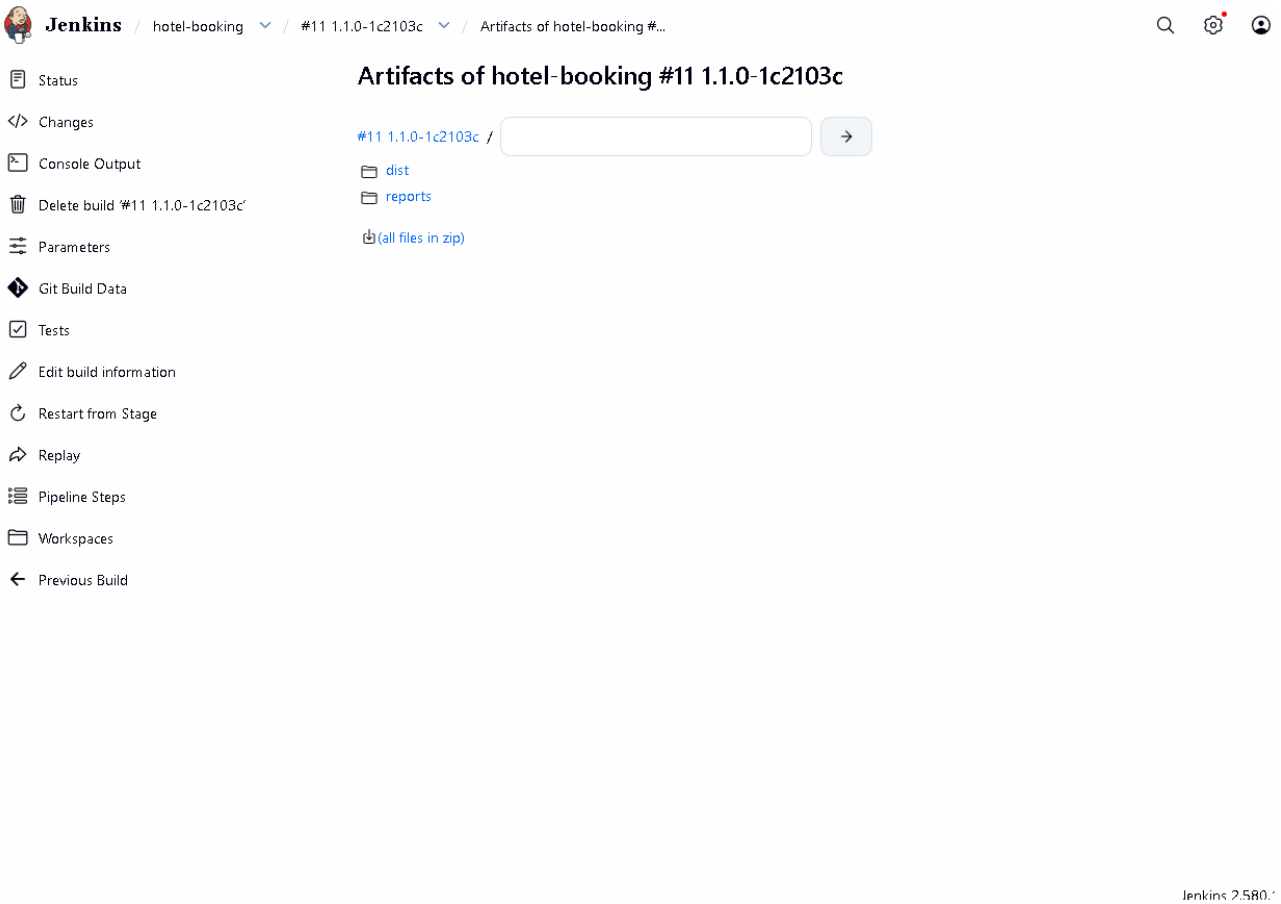


Figure 7.5 - Archived artifacts of build #11: source tarball + `.sha256`, test XML, `health.json`, `deployment.txt`.

PowerShell - local Jenkins definition (plugins, compose)

```
$ type deploy\jenkins\plugins.txt
workflow-aggregator
git
junit
credentials-binding
pipeline-stage-view
timestamper

$ docker compose -f deploy/jenkins/docker-compose.yml ps --format "table {{.Service}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"
SERVICE    IMAGE                 STATUS          PORTS
jenkins    hotel-jenkins:local   Up 49 minutes   127.0.0.1:8090->8080/tcp
registry   registry:2            Up 49 minutes   127.0.0.1:5000->5000/tcp
```

Rendered from real command output captured 2026-10-20 12:55 UTC in CICD_FOR_HOTEL_ROOM_BOOKING | raw text: jenkins-files.txt

Figure 7.6 - `deploy/jenkins/plugins.txt` and the running Jenkins + registry containers (rendered from real output, 2026-10-20).

E. How to demonstrate this task to the professor (about 4 minutes)

Before the demo: Docker Desktop running ("Engine running" in its window). Know the admin password: it is the `JENKINS_ADMIN_PASSWORD=` line in `deploy\jenkins\.env` (do not show this file on screen).

40. **Make sure Jenkins runs** (PowerShell):*Expected:* `jenkins` and `registry` "Up". If not: `docker compose up -d`, wait ~1 minute.

```
cd D:\DevOPS\CICD_FOR_HOTEL_ROOM_BOOKING\deploy\jenkins
docker compose ps
```

41. Open **http://localhost:8090**, log in as `admin`.
42. Open job **hotel-booking**. *Say:* "This is the CI job. The stage view shows every build; red ones were deliberate experiments."
43. Click **Configure** (read only - don't save). Show *This project is parameterized*, *Poll SCM* `H/5 * * * *`, *Pipeline script from SCM*. *Say:* "Jenkins checks the repository every five minutes. A webhook needs Jenkins to be reachable from GitHub, which a laptop is not."
44. Click **Build with Parameters** -> `BRANCH=develop`, `APP_ENV=staging` -> **Build**. Watch the stage view (about 2 minutes). *Say* each stage name as it turns green.
45. Open the new build -> **Tests** (90 passed) -> **Build Artifacts** (tarball, checksum, reports).
46. Open **Console Output** and scroll to `flake8` and `90 passed` lines.

Recovery: login page not loading - wait: Jenkins needs ~60 s after a start. "permission denied ... docker.sock" (Linux host only) - set `DOCKER_GID` in `deploy/jenkins/.env`. Build cannot see your latest change - Jenkins builds **committed** code only; commit first.

F. Expected screenshots for the final submission

42. `assets/task-07/jenkins-job-stage-view.png` - **exists, 2026-10-20**.
43. `assets/task-07/jenkins-job-configuration.png` - **exists, 2026-10-20**.
44. `assets/task-07/jenkins-build-11-summary.png` - **exists, 2026-10-20**.
45. `assets/task-07/jenkins-build-11-tests.png` - **exists, 2026-10-20**.
46. `assets/task-07/jenkins-build-11-artifacts.png` - **exists, 2026-10-20**.
47. `assets/task-07/jenkins-files.png` - **exists, 2026-10-20**.
48. `assets/task-07/github-webhook.png` - **only if** a webhook is configured on a public Jenkins (manual; currently not applicable).

G. Probable viva questions and answers

42. **What is Jenkins?** An automation server that runs build/test/deploy jobs automatically.
43. **What is Continuous Integration?** Automatically building and testing every change soon after it is committed, so errors are found quickly.
44. **Installed Jenkins vs Jenkinsfile vs executed build?** The server, the pipeline definition stored in Git, and an actual numbered run with results. Only a build (e.g. #11) proves it works.
45. **How is your build triggered?** Poll SCM every ~5 minutes, plus manual *Build with Parameters*. A webhook isn't possible because GitHub cannot reach localhost.
46. **How do you install dependencies reproducibly?** A fresh venv each build and `pip install -r requirements-dev.txt` with pinned versions (`Flask==3.1.3`, `pytest==9.1.1`...).
47. **What does flake8 check?** Syntax errors, undefined names, unused imports and style (config in `.flake8`, line length 110).
48. **Where do the test reports come from?** pytest writes JUnit XML (`--junitxml`), and the JUnit plugin shows them on the Tests page.
49. **(Project-specific) What artifacts does a build keep?** The source archive `hotel-booking-<version>-<commit>.tar.gz` with its sha256, test XML, `health.json` and `deployment.txt`.
50. **(Project-specific) Why run Jenkins in Docker?** The whole server (plugins, Python, Chromium, Docker CLI, the job) is defined in files in `deploy/jenkins/` and starts with one command; nothing is clicked by hand.
51. **Is this a Maven job?** No. The project is Python; pip, flake8 and pytest do the jobs Maven would do for Java. I'll confirm with the professor that this is acceptable.
52. **What is an agent?** The machine that executes the steps; here the built-in node (the Jenkins container), which uses the host's Docker through the mounted socket.

H. Common mistakes and troubleshooting

Symptom	Cause	Inspect	Fix	Confirm
http://localhost:8090 not reachable	Docker Desktop or the container stopped	`docker compose ps` in `deploy\jenkins`	start Docker Desktop; `docker compose up -d`	login page loads
Build doesn't include my change	change not committed	`git status`	commit (and choose that branch in `BRANCH`)	console "Building commit "
Stage "Install dependencies" fails with SSL errors	antivirus HTTPS scanning	console output	put the root CA in `certs/` and rebuild Jenkins (`docker compose up -d --build`)	stage green
Login fails	wrong password	`deploy\jenkins\.env` (privately)	use that value; it is set on first start only	dashboard opens
Tests page shows old results	stale report files (fixed)	Checkout stage runs `rm -rf reports dist`	already fixed in `fix/jenkins-stale-reports`	build #9 lists only tests that ran

I. Quick revision notes

Five key points

16. Jenkins runs in Docker, defined in `deploy/jenkins/`, at http://localhost:8090.
17. Job `hotel-booking` = pipeline from SCM (`Jenkinsfile`), parameters APP_ENV / BRANCH / REGISTRY.

18. CI stages: checkout, venv + pinned pip install, compileall + flake8, pytest (85 + 5).
19. JUnit reports and artifacts (tarball + sha256) are kept with each build.
20. Trigger = polling (no public URL for a webhook); Python tools, not Maven.

30-60 second explanation "Jenkins runs in a Docker container that is completely defined in the repository - plugins, Python, Chromium for Selenium and the job itself. The job `hotel-booking` reads the Jenkinsfile from Git. For each build it checks out the code, creates a fresh virtual environment, installs pinned dependencies, runs flake8 and all 90 tests, publishes the JUnit reports and archives a source package with a checksum. Build #11, run today, passed all stages. It polls Git every five minutes because a webhook needs a public Jenkins."

Caveat: no webhook; Python tooling instead of a Maven job (needs approval).

Readiness checklist

- â€¢ â˜ Docker Desktop running; `docker compose ps` shows Jenkins up.
- â€¢ â˜ I can log in and start *Build with Parameters*.
- â€¢ â˜ I can open Tests and Artifacts of a build.
- â€¢ â˜ I can explain polling vs webhook and Python vs Maven.

Task 8 - Pipeline as Code and Server Deployment

A. Task title and objective

Task 8: Pipeline as Code and Server Deployment

Objective: describe the whole build-test-package-deploy process in a `Jenkinsfile` stored in Git, parameterise at least one environment-specific setting, keep secrets out of the repository, and deploy automatically after all quality gates pass, followed by a health check.

Professor's required deliverables

- â€¢ A `Jenkinsfile` with stages for checkout, dependencies, static checks, tests, packaging, Docker build, deployment and health check
- â€¢ At least one parameterised environment-specific setting
- â€¢ Configuration through environment variables / files; passwords, keys and registry credentials outside source control
- â€¢ Deployment to an available target (local deployment acceptable if no server exists - documented)

Current status: Implemented and verified. Jenkins build **#11** on 2026-10-20 ran all 10 stages, deployed release `1.1.0-1c2103c` to the **staging** environment and the health check confirmed the version and commit (Figures 8.3-8.5).

Deployment target: a Docker Compose stack on the same machine as Jenkins (local "staging" on port 8081 and "prod" on port 8082). No separate remote server is used by Jenkins; remote-server deployment is done by Ansible (Task 13).

B. Explanation for a complete beginner

An everyday picture. A recipe card that lists every step in order ("preheat, mix, bake 20 minutes, test with a toothpick") lets anyone bake the same cake. A **Jenkinsfile** is the recipe card for releasing software: it lists every step, and because it lives in Git, changes to the recipe are reviewed and versioned like code.

Term	Meaning
Pipeline as code	the pipeline is a text file in the repository, not settings clicked in a web UI
Stage	one named step group (e.g. "Unit & integration tests"); stages run in order
Declarative pipeline	Jenkins' structured syntax: `pipeline { parameters {...} stages {...} post {...} }`
Parameter	a value chosen when starting a build (e.g. staging or prod)
Environment variable	a named value the commands can read (e.g. `HOST_PORT`)
Credential	a secret stored encrypted in Jenkins and injected only when needed
Deployment	starting the new version where users can reach it
Health check	asking the running app "are you OK, and which version are you?" (`GET /health`)
Quality gate	a check that must pass before the next step is allowed; here every test stage comes before Deploy

Why it helps: the same steps run every time, nothing is forgotten, history shows who changed the process, and a broken step stops the release.

C. What was actually implemented

File: `Jenkinsfile` (declarative pipeline).

Parameters (lines 19-29)

Parameter	Values	Effect
`APP_ENV`	`staging` / `prod`	sets `APP_ENV` in the container, the port (staging **8081**, prod **8082**) and the compose project name `hotel-staging` / `hotel-prod` (separate containers and separate database volume)
`BRANCH`	e.g. `develop`, `main`	which Git branch is built
`REGISTRY`	`localhost:5000` (default) or `docker.io/<user>`	where images are pushed and pulled

Derived variables (lines 45-51): `HOST_PORT`, `COMPOSE_PROJECT_NAME`. In the Checkout stage: `APP_VERSION` (from the `VERSION` file), `GIT_SHORT`, `IMAGE_TAG = <version>-<commit>`, `IMAGE_PREFIX`.

Stages and what build #11 did

#	Stage (line)	What it does	Build #11 result
1	Checkout (55)	`checkout scm`, delete old `reports/` + `dist/`, compute tag	commit `1c2103c`, tag `1.1.0-1c2103c`
2	Install dependencies (76)	fresh venv, pinned `pip install`	ok
3	Static checks (88)	`compileall`, `flake8`	no findings
4	Unit & integration tests (99)	pytest, JUnit XML	85 passed
5	E2E tests (Selenium) (105)	headless Chromium	5 passed
6	Package source (113)	`git archive` + `sha256sum`	`hotel-booking-1.1.0-1c2103c.tar.gz`
7	Build images (127)	`docker build` app + nginx, with `APP_VERSION/GIT_COMMIT`	2 images
8	Publish images (141)	`docker push` to the registry	pushed (digests in console)
9	Deploy (163)	`withCredentials(hotel-secret-key)`; `docker compose pull`; `docker compose up -d --no-build --wait`	containers healthy
10	Health check (177)	`wget /health` through Nginx; `grep` the expected version **and** commit; write `reports/deployment.txt`	`{"commit":"1c2103c","env":"staging","status":"ok","version":"1.1.0"}`

Secrets stay outside Git

```
withCredentials([string(credentialsId: 'hotel-secret-key', variable: 'SECRET_KEY')]) {
    sh '''
        docker compose pull
        docker compose up -d --no-build --wait --wait-timeout 120
    '''
}
```

- â€¢ `hotel-secret-key` is a Jenkins *secret text* credential (created from the git-ignored `deploy/jenkins/.env`); Jenkins masks it in logs. The build #11 log was searched for the key: **not present**.
- â€¢ A remote registry password would be the credential `registry-creds`, used with `docker login --password-stdin`.
- â€¢ `docker-compose.yml` refuses to start without `SECRET_KEY` (`${SECRET_KEY:?...}`), and the app refuses to start in staging/prod with a missing or default key.

Health check stage (simplified excerpt of lines 177-200 - the real file retries `wget` up to 5 times and escapes the quotes for Groovy as `\\"`; the `#` comments are added here)

```
docker compose exec -T nginx wget -qO- http://127.0.0.1/health > reports/health.json
grep -q "\"version\":\"$APP_VERSION\"" reports/health.json   # right release?
grep -q "\"commit\":\"$GIT_SHORT\"" reports/health.json      # right commit?
```

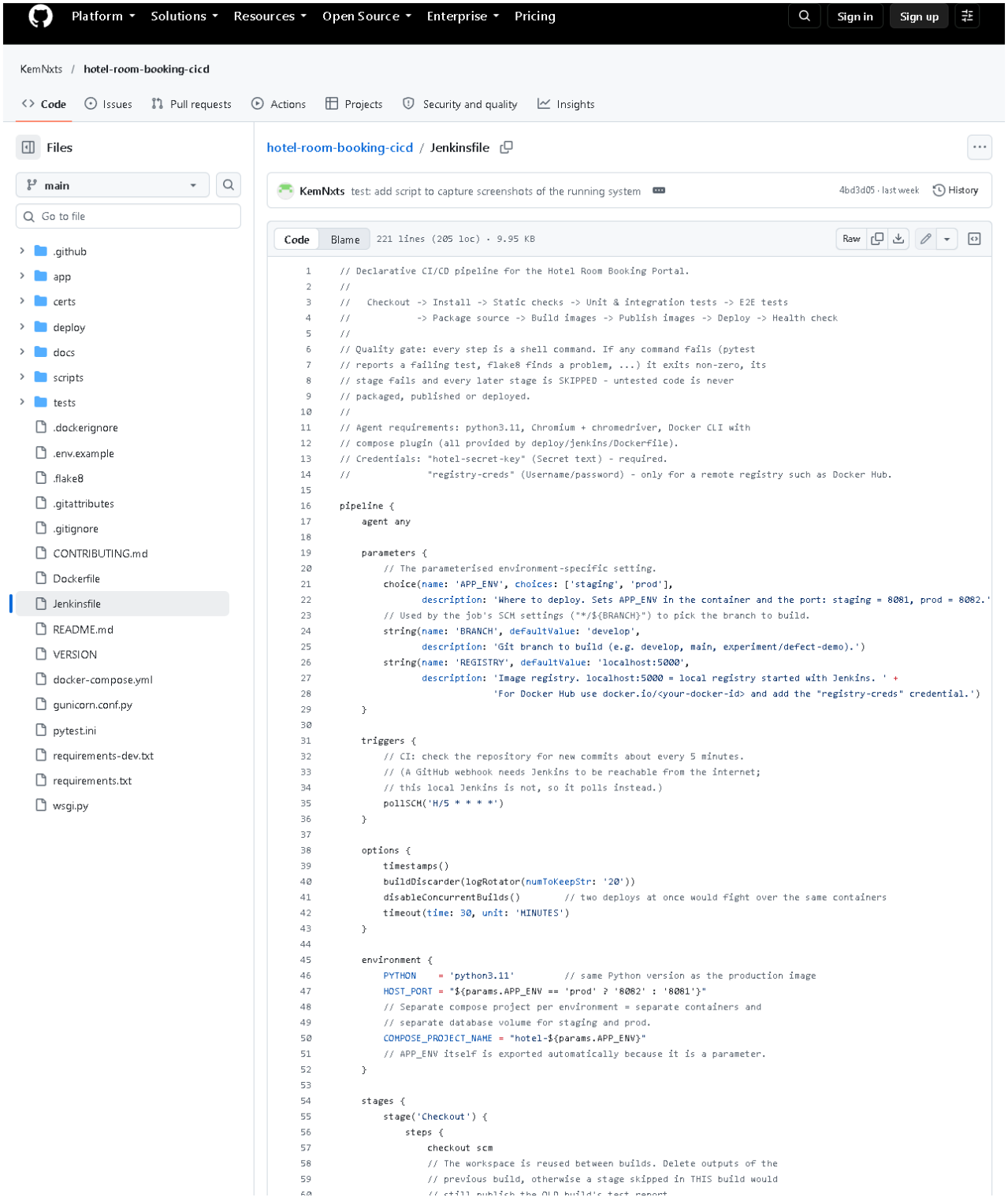
If the running app reports a different version or commit, `grep` fails and the build fails.

Deployment record (artifact `reports/deployment.txt` of build #11):

```
environment=staging
branch=develop
version=1.1.0
commit=1c2103c
image_tag=1.1.0-1c2103c
app_image=localhost:5000/hotel-booking-app:1.1.0-1c2103c
nginx_image=localhost:5000/hotel-booking-nginx:1.1.0-1c2103c
url=http://localhost:8081
jenkins_build=null
```

`jenkins_build=null` is a small known issue: Jenkins' own "Jenkins URL" setting was never filled in, so `BUILD_URL` is empty. Fix: *Manage Jenkins -> System -> Jenkins Location -> Jenkins URL* = `http://localhost:8090/`.

D. Screenshots and evidence



(part 1 of 4)

```
                    // still publish the UCD build's test report.
                    sh 'rm -rf reports dist'
                    script {
                        // Version = VERSION file + commit, e.g. 1.0.0-3f2c1ab. The same
                        // tag goes on the source archive, both images and /health,
                        // so every deployment can be traced back to one commit.
                        env.APP_VERSION  = readFile('VERSION').trim()
                        env.GIT_SHORT    = sh(script: 'git rev-parse --short HEAD', returnStdout: true).trim()
                        env.IMAGE_TAG    = "${env.APP_VERSION}-${env.GIT_SHORT}"
                        env.IMAGE_PREFIX = params.REGISTRY ? "${params.REGISTRY}/" : ''
                        currentBuild.displayName = "#${env.BUILD_NUMBER} ${env.IMAGE_TAG}"
                    }
                    sh 'git log -1 --pretty="Building commit %h on ${BRANCH}: %s"'
                }
            }

            stage('Install dependencies') {
                steps {
                    // Pinned versions from requirements*.txt = reproducible installs.
                    sh '''
                        $PYTHON --version
                        $PYTHON -m venv .venv
                        .venv/bin/pip install --quiet --upgrade pip
                        .venv/bin/pip install --quiet -r requirements-dev.txt
                    '''
                }
            }

            stage('Static checks') {
                steps {
                    // compileall: syntax errors. flake8: style problems, unused
                    // imports, undefined names (config in .flake8).
                    sh '''
                        .venv/bin/python -m compileall -q app wsgi.py
                        .venv/bin/flake8 app tests scripts wsgi.py gunicorn.conf.py
                    '''
                }
            }

            stage('Unit & integration tests') {
                steps {
                    sh '.venv/bin/pytest tests/unit tests/integration --junitxml=reports/unit-integration-tests.xml'
                }
            }

            stage('E2E tests (Selenium)') {
                steps {
                    // Starts the app on a free port with a throw-away database and
                    // drives headless Chromium. Failure screenshots: reports/screenshots/.
                    sh '.venv/bin/pytest -m e2e --junitxml=reports/e2e-tests.xml'
                }
            }

            stage('Package source') {
                steps {
                    // Reproducible source artifact: exactly the committed files of
                    // this commit (git archive ignores anything uncommitted).
                    sh '''
                        mkdir -p dist
                        git archive --format=tar.gz --prefix=hotel-booking-$IMAGE_TAG/ \
                            -o dist/hotel-booking-$IMAGE_TAG.tar.gz HEAD
                        cd dist && sha256sum hotel-booking-$IMAGE_TAG.tar.gz > hotel-booking-$IMAGE_TAG.tar.gz.sha256
                        cat hotel-booking-$IMAGE_TAG.tar.gz.sha256
                    '''
                }
            }

            stage('Build images') {
                steps {
                    // If this agent trusts extra root certificates (networks with
                    // HTTPS inspection - see certs/README.md), give them to the
                    // image build too. On a normal network there are none.
                    sh '''
                        cp /usr/local/share/ca-certificates/extra/*.crt certs/ 2>/dev/null || true
                        docker build -t ${IMAGE_PREFIX}hotel-booking-app:$IMAGE_TAG \
                            --build-arg APP_VERSION=$APP_VERSION --build-arg GIT_COMMIT=$GIT_SHORT .
                        docker build -f deploy/nginx/Dockerfile -t ${IMAGE_PREFIX}hotel-booking-nginx:$IMAGE_TAG .
```

(part 2 of 4)

```
                    '''
                }
            }

            stage('Publish images') {
                steps {
                    script {
                        def push = '''
                            docker push ${IMAGE_PREFIX}hotel-booking-app:$IMAGE_TAG
                            docker push ${IMAGE_PREFIX}hotel-booking-nginx:$IMAGE_TAG
                        '''
                        if (params.REGISTRY.startsWith('localhost')) {
                            sh push                       // local registry: no login needed
                        } else {
                            // Remote registry (e.g. Docker Hub): password/token comes
                            // from Jenkins credentials and is masked in the log.
                            withCredentials([usernamePassword(credentialsId: 'registry-creds',
                                    usernameVariable: 'REG_USER', passwordVariable: 'REG_PASS')]) {
                                sh 'echo "$REG_PASS" | docker login -u "$REG_USER" --password-stdin ${REGISTRY%%/*}'
                                sh push
                            }
                        }
                    }
                }
            }

            stage('Deploy') {
                steps {
                    // The secret is injected only for this block and masked in the log.
                    withCredentials([string(credentialsId: 'hotel-secret-key', variable: 'SECRET_KEY')]) {
                        // Pull = deploy exactly what was published; --wait blocks until
                        // the containers are healthy and fails the stage if they never are.
                        sh '''
                            docker compose pull
                            docker compose up -d --no-build --wait --wait-timeout 120
                        '''
                    }
                }
            }

            stage('Health check') {
                steps {
                    // Smoke test through the whole chain (nginx -> gunicorn -> SQLite)
                    // AND check the running release is the one we just built.
                    withCredentials([string(credentialsId: 'hotel-secret-key', variable: 'SECRET_KEY')]) {
                        sh '''
                            for i in 1 2 3 4 5; do
                                docker compose exec -T nginx wget -qO- http://127.0.0.1/health > reports/health.json && bre
                                sleep 3
                            done
                            cat reports/health.json; echo
                            grep -q "\\"version\\":\\"$APP_VERSION\\"" reports/health.json
                            grep -q "\\"commit\\":\\"$GIT_SHORT\\"" reports/health.json
                        '''
                    }
                    // Deployment record, archived with the build.
                    writeFile file: 'reports/deployment.txt', text: """environment=${params.APP_ENV}
branch=${params.BRANCH}
version=${env.APP_VERSION}
commit=${env.GIT_SHORT}
image_tag=${env.IMAGE_TAG}
app_image=${env.IMAGE_PREFIX}hotel-booking-app:${env.IMAGE_TAG}
nginx_image=${env.IMAGE_PREFIX}hotel-booking-nginx:${env.IMAGE_TAG}
url=http://localhost:${env.HOST_PORT}
jenkins_build=${env.BUILD_URL}
"""
                }
            }
        }

        post {
            always {
                // Publish test results (Jenkins shows pass/fail trends) and keep the
                // reports, failure screenshots, source archive and deployment record.
                junit allowEmptyResults: true, testResults: 'reports/*.xml'
                archiveArtifacts artifacts: 'reports/**, dist/**', allowEmptyArchive: true
            }
            success {
                echo "Deployed ${env.IMAGE_TAG} to ${params.APP_ENV}: http://localhost:${env.HOST_PORT}"
            }
            failure {
                echo 'Pipeline FAILED - see the red stage above and the Test Results page. Nothing was deployed.'
            }
        }
    }
```

(part 3 of 4)

(part 4 of 4)

Figure 8.1 - `Jenkinsfile` on GitHub (captured 2026-10-20). **Notice:** `parameters`, `triggers { pollSCM }`, `options`, `environment`, the ten stages and `post`.

Git Bash - stages and parameters declared in the Jenkinsfile

```
$ findstr /n /c:"stage(" /c:"choice(" /c:"string(name" /c:"pollSCM" Jenkinsfile
21:         choice(name: 'APP_ENV', choices: ['staging', 'prod'],
24:         string(name: 'BRANCH', defaultValue: 'develop',
26:         string(name: 'REGISTRY', defaultValue: 'localhost:5000',
35:         pollSCM('H/5 * * * *')
55:         stage('Checkout') {
76:         stage('Install dependencies') {
88:         stage('Static checks') {
99:         stage('Unit & integration tests') {
105:         stage('E2E tests (Selenium)') {
113:         stage('Package source') {
127:         stage('Build images') {
141:         stage('Publish images') {
163:         stage('Deploy') {
177:         stage('Health check') {
```

Rendered from real command output captured 2026-10-20 12:55 UTC in CICD_FOR_HOTEL_ROOM_BOOKING | raw text: jenkinsfile-stages.txt

Figure 8.2 - Line numbers of every stage and parameter (`findstr` on the Jenkinsfile, rendered from real output).

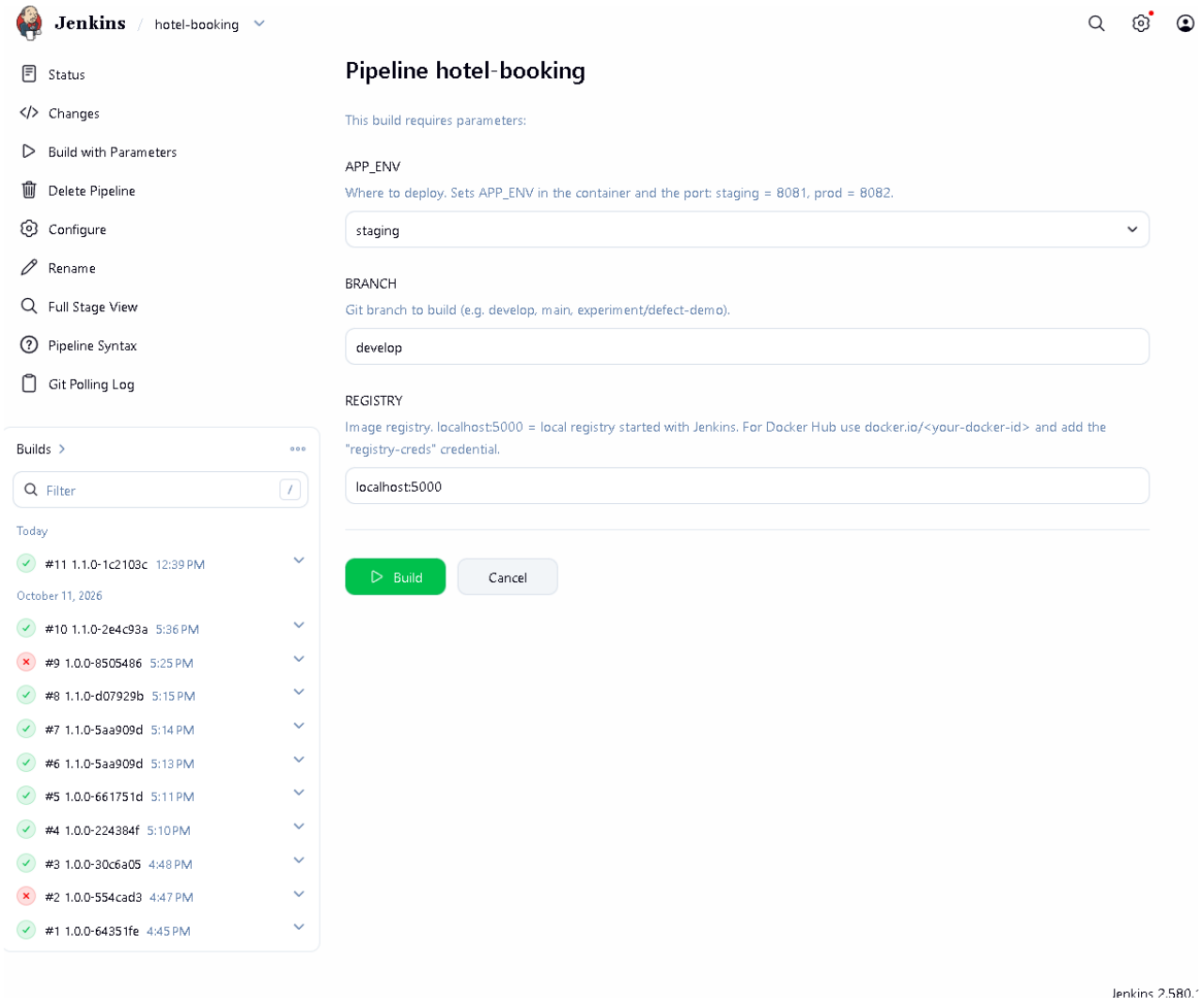


Figure 8.3 - The Build with Parameters form: APP_ENV, BRANCH, REGISTRY. **Proves:** the environment-specific setting is a real parameter.

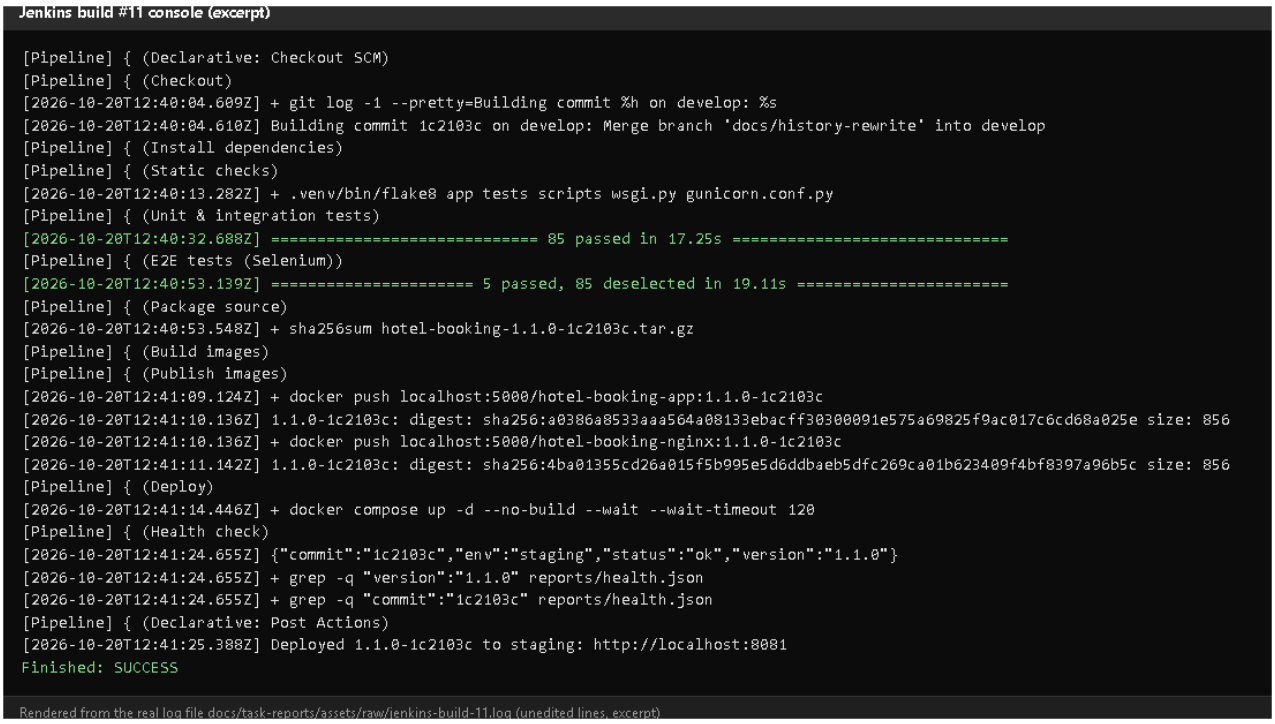


Figure 8.4 - Build #11 console excerpt (rendered from the saved console log): each stage starting, 85 + 5 tests passed, the archive checksum, image pushes with digests, `/health` and "Deployed 1.1.0-1c2103c to staging".



Figure 8.5 - http://localhost:8081/health in a browser after build #11: commit `1c2103c`, env `staging`, version `1.1.0`.
Proves: the deployed app is the exact commit that was built and tested.

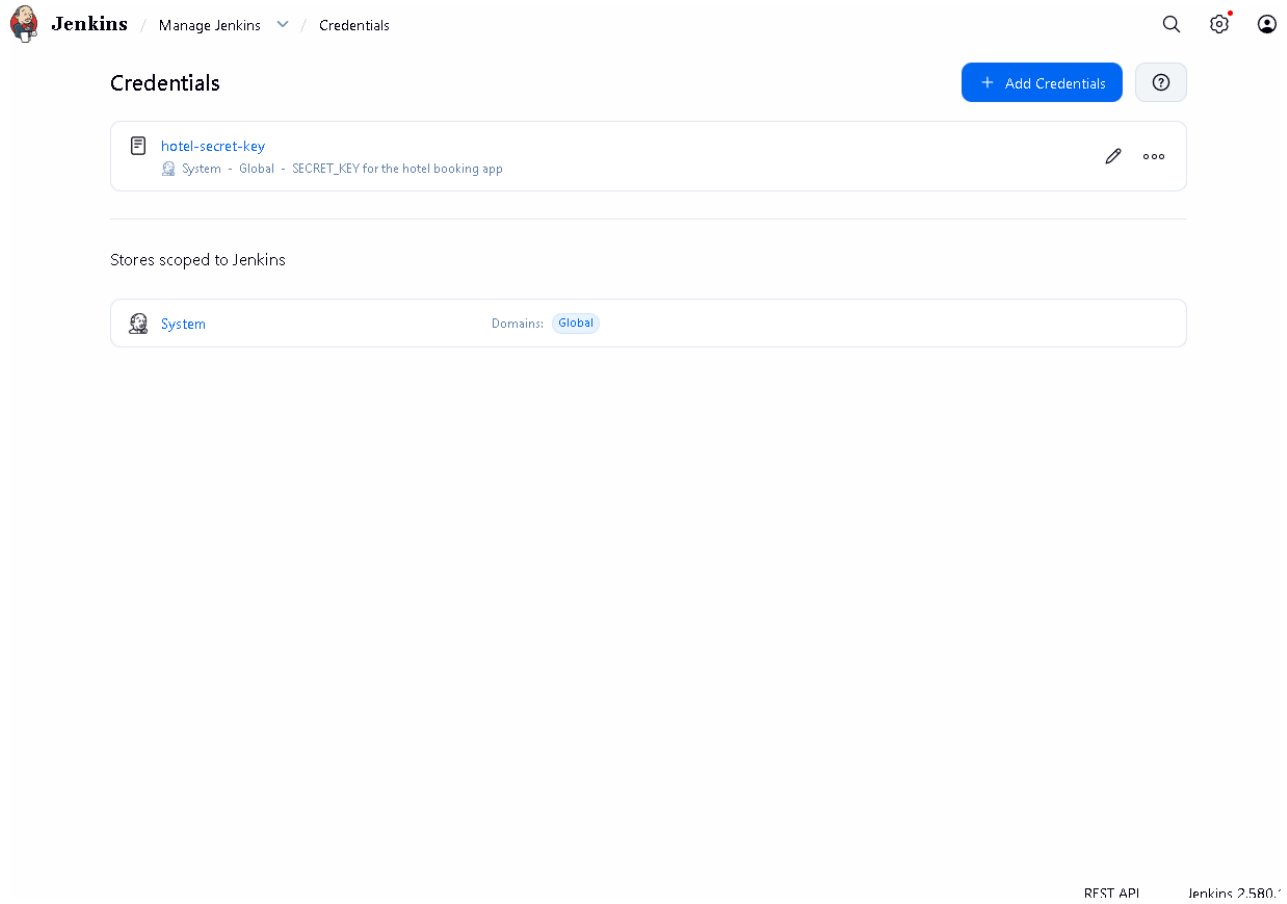


Figure 8.6 - Manage Jenkins -> Credentials: `hotel-secret-key` exists; its value is never displayed.

E. How to demonstrate this task to the professor (about 5 minutes)

Before the demo: Docker Desktop running; Jenkins up (`docker compose ps` in `deploy\jenkins`); VS Code open on the project.

58. **Open `Jenkinsfile` in VS Code.** Walk through: `parameters` (APP_ENV), `environment` (HOST_PORT), then each `stage(...)`. *Say:* "The whole pipeline is code in Git, so it is versioned and reviewed like the app."
59. **Start a deploy:** Jenkins -> *hotel-booking* -> *Build with Parameters* -> `APP_ENV = staging`, `BRANCH = develop` -> *Build*.
60. **Narrate the stage view** as stages turn green (about 2 minutes). Point out that *Deploy* is after both test stages - "that is the quality gate".
61. **Open the build's Console Output**, search (Ctrl+F) for `Deployed` and `"status":"ok"`.
62. **Prove it in a browser:** http://localhost:8081/health - the `commit` equals the commit shown in the console line "Building commit ...".
63. **Show the parameter effect:** run again with `APP_ENV = prod` and `BRANCH = main`; then http://localhost:8082/health shows `"env":"prod"`. *Say:* "Same pipeline, different environment, chosen by one parameter."
64. **Show secrets handling:** *Manage Jenkins -> Credentials* (value hidden); in VS Code show `withCredentials` in the Deploy stage.

Recovery: Deploy stage "port is already allocated" - another program uses 8081/8082; stop it or `docker compose -p hotel-staging down` and rebuild. Health check fails because of an old container - run the build again; it pulls and recreates the containers.

F. Expected screenshots for the final submission

47. `assets/task-08/github-file-Jenkinsfile.png` - **exists, 2026-10-20**.
48. `assets/task-08/jenkinsfile-stages.png` - **exists, 2026-10-20**.
49. `assets/task-08/jenkins-build-with-parameters.png` - **exists, 2026-10-20**.
50. `assets/task-08/jenkins-build-11-console.png` - **exists, 2026-10-20** (rendered from the real console log).
51. `assets/task-08/staging-health-after-build-11.png` - **exists, 2026-10-20**.
52. `assets/task-08/jenkins-credentials-masked.png` - **exists, 2026-10-20**.

G. Probable viva questions and answers

49. **What is "pipeline as code"?** The pipeline is a file (`Jenkinsfile`) in the repository, so it is versioned, reviewed and identical on every run.
50. **Name your stages in order.** Checkout, Install dependencies, Static checks, Unit & integration tests, E2E tests, Package source, Build images, Publish images, Deploy, Health check.
51. **Which setting is parameterised?** `APP_ENV`: staging or prod - it changes the app environment, the port (8081/8082) and the compose project (separate data).
52. **How are secrets handled?** `SECRET_KEY` is a Jenkins credential injected with `withCredentials` only in Deploy/Health check and masked in logs; nothing secret is in Git.
53. **What happens if a test fails?** That stage fails and every later stage, including Deploy, is skipped (build #2 shows this).
54. **(Project-specific) How do you know the deployed version is the tested one?** Images are tagged `<version>-<commit>`; the health-check stage compares the version and commit returned by `/health` with the ones built.
55. **(Project-specific) Where does it deploy?** To a Docker Compose stack on the Jenkins host: staging :8081, prod :8082. Real servers are handled by Ansible.
56. **Why `--wait` on `docker compose up`?** It waits until containers are healthy (Docker HEALTHCHECK) and fails the stage if they never are.
57. **What is `post { always }` for?** It publishes test reports and archives artifacts even when the build fails, so failures can be analysed.
58. **Why `disableConcurrentBuilds`?** Two deployments at the same time would fight over the same containers.
59. **What is `deployment.txt`?** A record archived with each build: environment, branch, version, commit, image names, URL.

H. Common mistakes and troubleshooting

Symptom	Cause	Inspect	Fix	Confirm
Health check stage fails on `grep`	old container still running another version	console: `cat reports/health.json`	rerun; check no other stack uses the port	`/health` shows the new commit
Deploy: `SECRET_KEY is not set`	credential missing	*Manage Jenkins -> Credentials*	recreate `hotel-secret-key` (secret text)	Deploy green
`jenkins_build=null` in deployment record	Jenkins URL not configured	*Manage Jenkins -> System*	set Jenkins URL `http://localhost:8090/`	next record shows the build URL

Symptom	Cause	Inspect	Fix	Confirm
Publish images: connection refused	registry container stopped	`docker compose ps` in `deploy\jenkins`	`docker compose up -d registry`	push succeeds
Changes in the Jenkinsfile ignored	not committed / wrong branch	console "Building commit"	commit and build that branch	stage list updated

I. Quick revision notes

Five key points

53. `Jenkinsfile` = the pipeline as code, 10 stages, versioned in Git.
54. Parameter `APP_ENV` chooses staging (:8081) or prod (:8082).
55. Secrets come from Jenkins credentials (`withCredentials`), never from Git.
56. Deploy runs only after lint + 90 tests; then `/health` must match version **and** commit.
57. Build #11 (2026-10-20) deployed `1.1.0-1c2103c` to staging and passed.

30-60 second explanation "Our pipeline is a Jenkinsfile in the repository. It checks out a branch, installs pinned dependencies, runs flake8 and all tests, packages the source, builds Docker images tagged with version and commit, pushes them to a registry, deploys them with Docker Compose and finally checks that `/health` reports exactly that version and commit. The APP_ENV parameter chooses staging or prod. The secret key is a Jenkins credential, never in Git. Build #11 today deployed commit 1c2103c to staging successfully."

Caveat: Jenkins deploys to Docker on its own host, not a remote server; `jenkins_build` in the record is empty until the Jenkins URL is set.

Readiness checklist

- â€¢ â˜ I can name the 10 stages in order.
- â€¢ â˜ I can run a staging and a prod build and show both `/health` pages.
- â€¢ â˜ I can point to `withCredentials` and explain masking.
- â€¢ â˜ I can explain why the health check compares the commit.

Task 9 - Selenium Test Design and Local Execution

A. Task title and objective

Task 9: Selenium Test Design and Local Execution

Objective: automate the most important user journeys in a real web browser, with stable locators, explicit waits, clear assertions and isolated test data; run them locally; produce reports and screenshots on failure.

Professor's required deliverables

- â€¢ 3-5 meaningful Selenium journeys (register/login, availability, booking, confirmation/status, cancellation)
- â€¢ Stable element selectors, explicit waits, assertions, deterministic test data
- â€¢ Local execution (the brief mentions Maven - see the note below)
- â€¢ Test reports and screenshots on failure
- â€¢ Clean-up / isolated test database so repeated runs stay reliable

Current status: Implemented and verified. On 2026-10-20: **5 of 5** journeys passed locally (Figure 9.1), the full suite of 90 tests passed (Figure 9.2), and the screenshot-on-failure feature was demonstrated with a temporary, deliberately failing test (Figures 9.4-9.5) that was deleted afterwards.

Maven note: the tests are Python and run with **pytest**, not Maven. pytest writes the same JUnit-XML report format that Maven would, which Jenkins displays. If running through Maven is compulsory, this must be discussed with the professor.

B. Explanation for a complete beginner

An everyday picture. A robot sits at the computer, opens Chrome, types into the forms and clicks the buttons exactly like a guest - then checks that the right message appears. If something is wrong it takes a photo of the screen. That robot is **Selenium**.

Term	Meaning	In this project
Browser automation	a program controls a real browser	headless Chrome (no window)
WebDriver	the standard interface that lets code control a browser	`selenium.webdriver.Chrome`
Locator / selector	how the test finds an element on the page	`By.ID, "booking-status"`
Stable locator	one that does not change when the design changes	element ids like `book-201`, `confirm-btn`
Explicit wait	"wait until X is visible (max 10 s)", then continue at once	`WebDriverWait(...).until(...)`
Assertion	the check that decides pass/fail	`assert status(driver) == "PENDING"`
Test data	the users, dates and bookings a test creates	unique e-mails, random far-future dates
Fixture	pytest setup code a test can request	`driver` (a browser), `base_url` (a running app)
Headless	the browser runs without a visible window	default; `HEADLESS=0` shows it

Why not `time.sleep(3)`? Sleeping either wastes time (page was ready earlier) or fails (page was slower). Explicit waits are faster and more reliable.

Unit vs integration vs Selenium tests (all three exist here)

Type	What it tests	Speed	Count
Unit	one service/database rule, no web server	ms	59
Integration	real routes through Flask's test client (HTTP status, redirects, CSRF)	fast	26
Selenium (end-to-end)	a full journey in a real browser	seconds each	5

C. What was actually implemented

Files: `tests/e2e/test_journeys.py` (journeys and helpers) and `tests/e2e/conftest.py` (fixtures and the failure hook).

The five journeys (`tests/e2e/test_journeys.py`)

#	Test (line)	What the robot does	Key assertions
1	`test_journey_1_register_new_guest` (81)	registers a new guest	name in the navbar, "account has been created", now on /rooms
2	`test_journey_2_login_rejects_wrong_password_then_accepts_right_one` (89)	registers, logs out, wrong password, right password	"Invalid email or password", still logged out; then "Welcome back"
3	`test_journey_3_search_request_and_confirm_booking` (108)	searches dates, books the first free room, confirms	status PENDING with the right dates; then CONFIRMED; no second Confirm button
4	`test_journey_4_cancel_booking_and_track_status` (128)	books, confirms, cancels via the dialog, opens My bookings	dialog appears and status is still CONFIRMED until confirmed; then CANCELLED in detail and in My bookings
5	`test_journey_5_booked_room_is_unavailable_to_another_guest` (150)	guest A books a room; guest B searches the same dates	the room shows "Booked" for B and has no Book button

Helpers (lines 22-75): `wait()` (explicit wait), `stay()` (dates), `unique_email()`, `register()`, `logout()`, `search()`, `book_first_free_room()`, `status()`.

Stable locators - ids added to the templates for testing: `register-btn`, `login-btn`, `nav-user`, `search-btn`, `availability-table`, `room-<number>`, `book-<number>`, `booking-status`, `confirm-btn`, `cancel-btn`, `cancel-confirm-btn`, `booking-row-<id>`.

Explicit wait example (lines 22-23):

```
def wait(driver, condition, timeout=10):
    return WebDriverWait(driver, timeout).until(condition)   # continue as soon as true, fail after 10 s
```

Dates are set with JavaScript (line 60) because typing into `<input type="date">` depends on the computer's date format (dd/mm vs mm/dd).

Deterministic, isolated test data

- â€¢ `base_url` fixture (`conftest.py` line 26): starts the real app on a free port (`make_server(..., 0, ...)`, line 36) with a **throw-away SQLite database** in a temporary folder, deleted after the run - repeated runs never interfere.
- â€¢ To test a running deployment instead, set `E2E_BASE_URL`; then the tests use unique e-mails (`uuid`) and random dates 200-2000 days ahead (`_SESSION_OFFSET`, line 17) and book whichever room is free.

Screenshots on failure (`conftest.py`)

65. The hook `pytest_runtest_makereport` (line 80) stores each test's result on the test item.

66. After the test, the `driver` fixture checks `rep_call.failed` (line 65).
67. If failed, `_save_failure_artifacts` (line 71) saves `reports/screenshots/<test>.png` **and** the page HTML.
68. Jenkins archives `reports/**` with the build.

Reports: `pytest --junitxml=reports/e2e-tests.xml` (as in the Jenkinsfile) - JUnit XML that Jenkins shows on the *Tests* page.

D. Screenshots and evidence

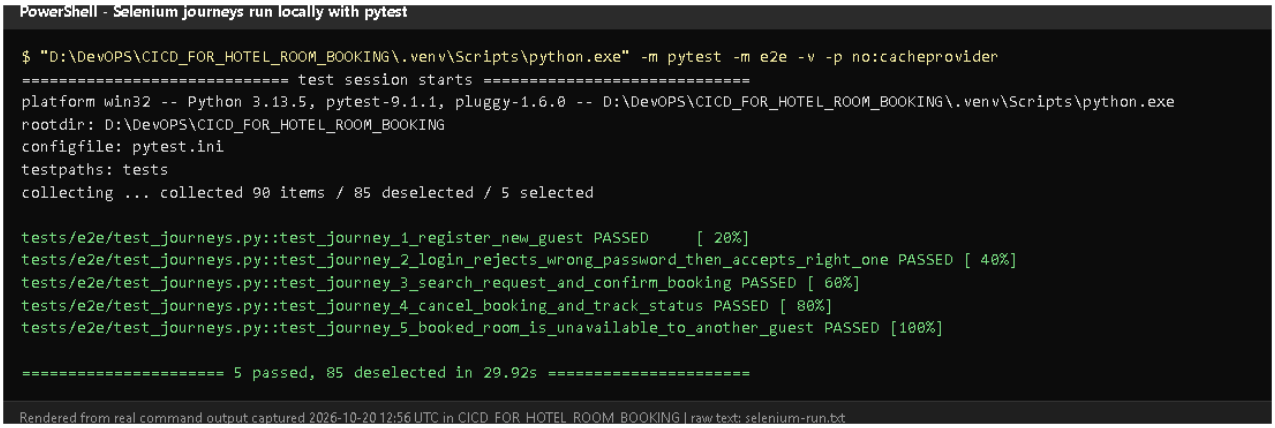


Figure 9.1 - `pytest -m e2e -v` on 2026-10-20 (rendered from real output): 5 journeys PASSED. **Proves:** the journeys run locally through pytest.

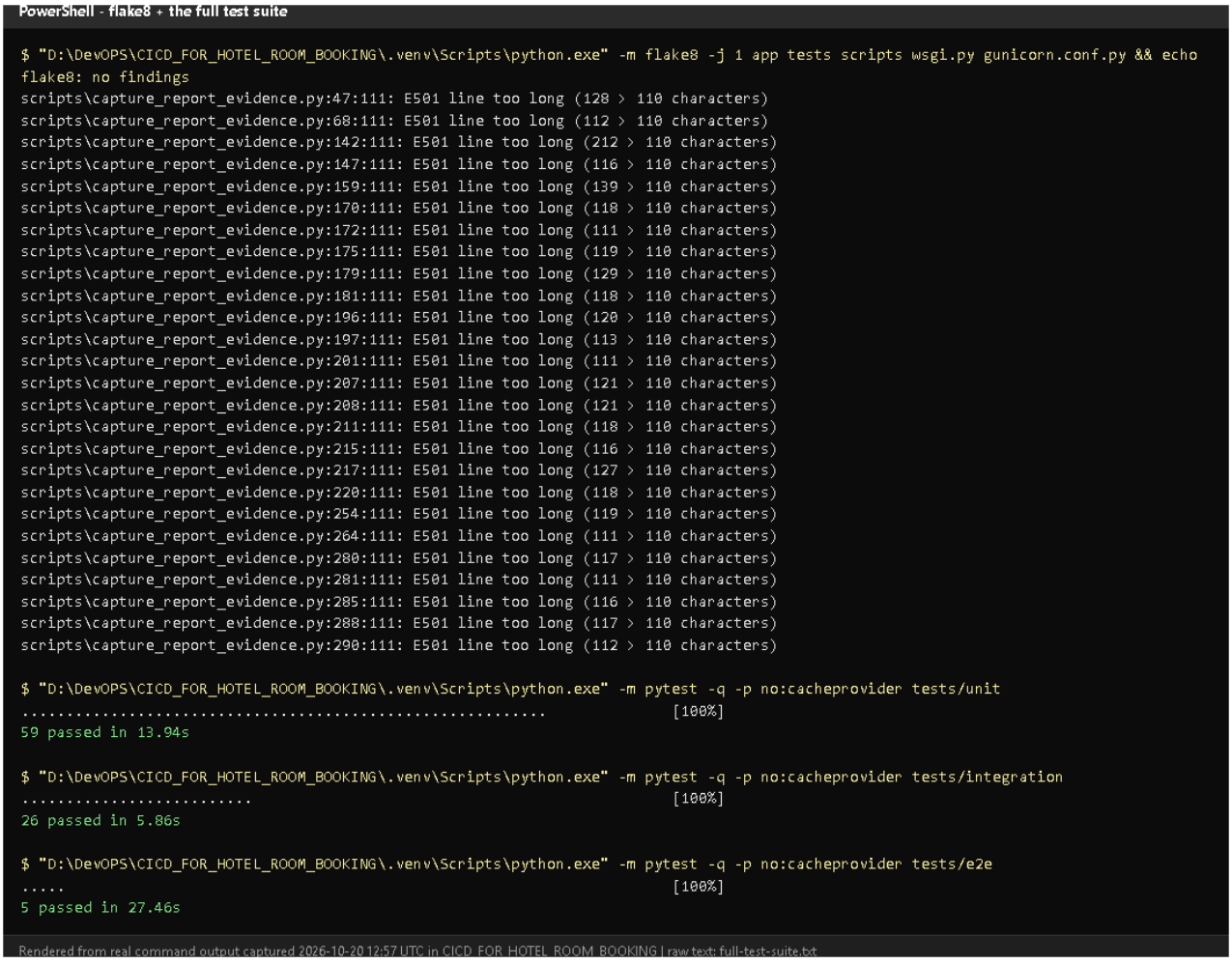
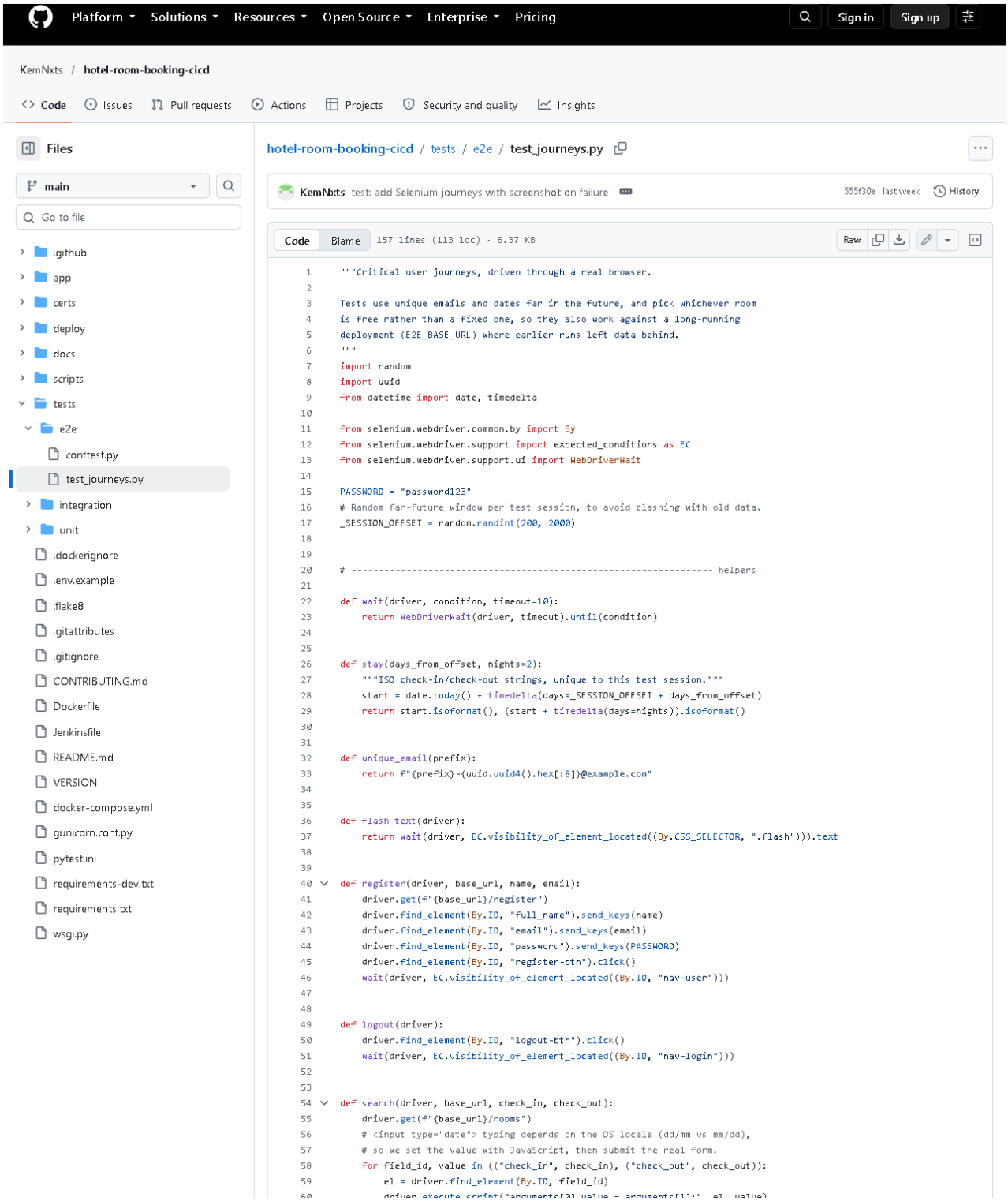


Figure 9.2 - flake8 (no findings), then unit 59, integration 26 and e2e 5 passed (rendered from real output, 2026-10-20).


Platform
Solutions
Resources
Open Source
Enterprise
Pricing
Sign in
Sign up
KemNxts / hotel-room-booking-cicd
Code
Issues
Pull requests
Actions
Projects
Security and quality
Insights
Files
main
Go to file
.github
app
certs
deploy
docs
scripts
tests
e2e
conftest.py
test_journeys.py
integration
unit
.dockerignore
.env.example
.flake8
.gitattributes
.gitignore
CONTRIBUTING.md
Dockerfile
Jenkinsfile
README.md
VERSION
docker-compose.yml
gunicorn.conf.py
pytest.ini
requirements-dev.txt
requirements.txt
wsgi.py
hotel-room-booking-cicd / tests / e2e / test_journeys.py
KemNxts test: add Selenium journeys with screenshot on failure
555f30e · last week
History
Code
Blame
157 lines (113 loc) · 6.37 KB
Raw
"""Critical user journeys, driven through a real browser.
Tests use unique emails and dates far in the future, and pick whichever room
is free rather than a fixed one, so they also work against a long-running
deployment (E2E_BASE_URL) where earlier runs left data behind.
"""
import random
import uuid
from datetime import date, timedelta
from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.ui import WebDriverWait
PASSWORD = "password123"
Random far-future window per test session, to avoid clashing with old data.
_SESSION_OFFSET = random.randint(200, 2000)
---------- helpers
def wait(driver, condition, timeout=10):
return WebDriverWait(driver, timeout).until(condition)
def stay(days_from_offset, nights=2):
"""ISO check-in/check-out strings, unique to this test session."""
start = date.today() + timedelta(days=_SESSION_OFFSET + days_from_offset)
return start.isoformat(), (start + timedelta(days=nights)).isoformat()
def unique_email(prefix):
return f"{prefix}-{uuid.uuid4().hex[:8]}@example.com"
def flash_text(driver):
return wait(driver, EC.visibility_of_element_located((By.CSS_SELECTOR, ".flash"))).text
def register(driver, base_url, name, email):
driver.get(f"{base_url}/register")
driver.find_element(By.ID, "full_name").send_keys(name)
driver.find_element(By.ID, "email").send_keys(email)
driver.find_element(By.ID, "password").send_keys(PASSWORD)
driver.find_element(By.ID, "register-btn").click()
wait(driver, EC.visibility_of_element_located((By.ID, "nav-user")))
def logout(driver):
driver.find_element(By.ID, "logout-btn").click()
wait(driver, EC.visibility_of_element_located((By.ID, "nav-login")))
def search(driver, base_url, check_in, check_out):
driver.get(f"{base_url}/rooms")
<input type="date"> typing depends on the OS locale (dd/mm vs mm/dd),
so we set the value with JavaScript, then submit the real form.
for field_id, value in (("check_in", check_in), ("check_out", check_out)):
el = driver.find_element(By.ID, field_id)

(part 1 of 3)

```
            driver.execute_script("arguments[0].value = arguments[1];", el, value)
        driver.find_element(By.ID, "search-btn").click()
        wait(driver, EC.visibility_of_element_located((By.ID, "availability-table")))


def book_first_free_room(driver):
    """Click the first Book button; return (room_number, booking_id)."""
    button = driver.find_element(By.CSS_SELECTOR, "button[id^='book-']")
    room_number = button.get_attribute("id").removeprefix("book-")
    button.click()
    wait(driver, EC.visibility_of_element_located((By.ID, "booking-status")))
    booking_id = driver.current_url.rstrip("/").rsplit("/", 1)[-1]
    return room_number, booking_id


def status(driver):
    return driver.find_element(By.ID, "booking-status").text


# ---------------------------------------------------------------- journeys

def test_journey_1_register_new_guest(driver, base_url):
    register(driver, base_url, "Asha Guest", unique_email("asha"))

    assert driver.find_element(By.ID, "nav-user").text == "Asha Guest"
    assert "account has been created" in flash_text(driver)
    assert driver.current_url.endswith("/rooms")


def test_journey_2_login_rejects_wrong_password_then_accepts_right_one(driver, base_url):
    email = unique_email("ravi")
    register(driver, base_url, "Ravi Guest", email)
    logout(driver)

    driver.find_element(By.ID, "email").send_keys(email)
    driver.find_element(By.ID, "password").send_keys("not-my-password")
    driver.find_element(By.ID, "login-btn").click()
    assert "Invalid email or password" in flash_text(driver)
    assert not driver.find_elements(By.ID, "nav-user")  # still logged out

    driver.find_element(By.ID, "email").clear()
    driver.find_element(By.ID, "email").send_keys(email)
    driver.find_element(By.ID, "password").send_keys(PASSWORD)
    driver.find_element(By.ID, "login-btn").click()
    wait(driver, EC.visibility_of_element_located((By.ID, "nav-user")))
    assert "Welcome back" in flash_text(driver)


def test_journey_3_search_request_and_confirm_booking(driver, base_url):
    register(driver, base_url, "Meera Guest", unique_email("meera"))
    check_in, check_out = stay(0, nights=2)

    search(driver, base_url, check_in, check_out)
    assert driver.find_elements(By.CSS_SELECTOR, ".badge-ok"), "expected at least one free room"

    book_first_free_room(driver)
    assert status(driver) == "PENDING"
    assert driver.find_element(By.ID, "booking-check-in").text == check_in
    assert driver.find_element(By.ID, "booking-check-out").text == check_out

    driver.find_element(By.ID, "confirm-btn").click()
    wait(driver, EC.text_to_be_present_in_element((By.ID, "booking-status"), "CONFIRMED"))
    assert "Booking confirmed" in flash_text(driver)
    assert not driver.find_elements(By.ID, "confirm-btn")  # can't confirm twice


def test_journey_4_cancel_booking_and_track_status(driver, base_url):
    register(driver, base_url, "Kiran Guest", unique_email("kiran"))
    search(driver, base_url, *stay(10, nights=3))
    _, booking_id = book_first_free_room(driver)
    driver.find_element(By.ID, "confirm-btn").click()
    wait(driver, EC.text_to_be_present_in_element((By.ID, "booking-status"), "CONFIRMED"))

    driver.find_element(By.ID, "cancel-btn").click()
    wait(driver, EC.text_to_be_present_in_element((By.ID, "booking-status"), "CANCELLED"))
    assert "Booking cancelled" in flash_text(driver)
    assert not driver.find_elements(By.ID, "cancel-btn")
```

(part 2 of 3)

```python
    # Status tracking: My bookings shows the up-to-date status.
    driver.find_element(By.ID, "nav-my-bookings").click()
    row = wait(driver, EC.visibility_of_element_located((By.ID, f"booking-row-{booking_id}")))
    assert "CANCELLED" in row.text


def test_journey_5_booked_room_is_unavailable_to_another_guest(driver, base_url):
    check_in, check_out = stay(20, nights=2)

    register(driver, base_url, "First Guest", unique_email("first"))
    search(driver, base_url, check_in, check_out)
    room_number, _ = book_first_free_room(driver)
    logout(driver)

    register(driver, base_url, "Second Guest", unique_email("second"))
    search(driver, base_url, check_in, check_out)

    row = driver.find_element(By.ID, f"room-{room_number}")
    assert "Booked" in row.text
    assert not driver.find_elements(By.ID, f"book-{room_number}")
```

(part 3 of 3)

Figure 9.3 - `tests/e2e/test_journeys.py` on GitHub (the pushed version; the local working copy adds the confirmation-dialog step in journey 4 and the readable-date check in journey 3, both for the uncommitted UI redesign). **Notice:** `By.ID` locators, the `wait()` helper, assertions.

Selenium - a failing test saves a screenshot automatically

```
# 2026-10-20 13:02 UTC - deliberately failing Selenium test (temporary file, deleted afterwards)
$ pytest tests/e2e/test_zz_failure_screenshot_demo.py
-------------------------- Captured stdout teardown --------------------------
[e2e] failure screenshot saved: reports\screenshots\test_demo_deliberate_failure_shows_screenshot.png
========================== short test summary info ==========================
FAILED tests/e2e/test_zz_failure_screenshot_demo.py::test_demo_deliberate_failure_shows_screenshot
============================== 1 failed in 5.88s ==============================

$ dir reports\screenshots
drwxr-xr-x 1 HP 197609     0 Oct 20 18:32 .
drwxr-xr-x 1 HP 197609     0 Oct 20 18:32 ..
-rw-r--r-- 1 HP 197609 11909 Oct 20 18:32 test_demo_deliberate_failure_shows_screenshot.html
-rw-r--r-- 1 HP 197609 28653 Oct 20 18:32 test_demo_deliberate_failure_shows_screenshot.png
```

Rendered from the real log file docs/task-reports/assets/raw/selenium-failure-screenshot-demo.txt (unedited lines, excerpt)

Figure 9.4 - A temporary, deliberately failing test run on 2026-10-20 (rendered from real output): "failure screenshot saved: reports\screenshots...png", then `1 failed`; the folder contains a `.png` and an `.html`.

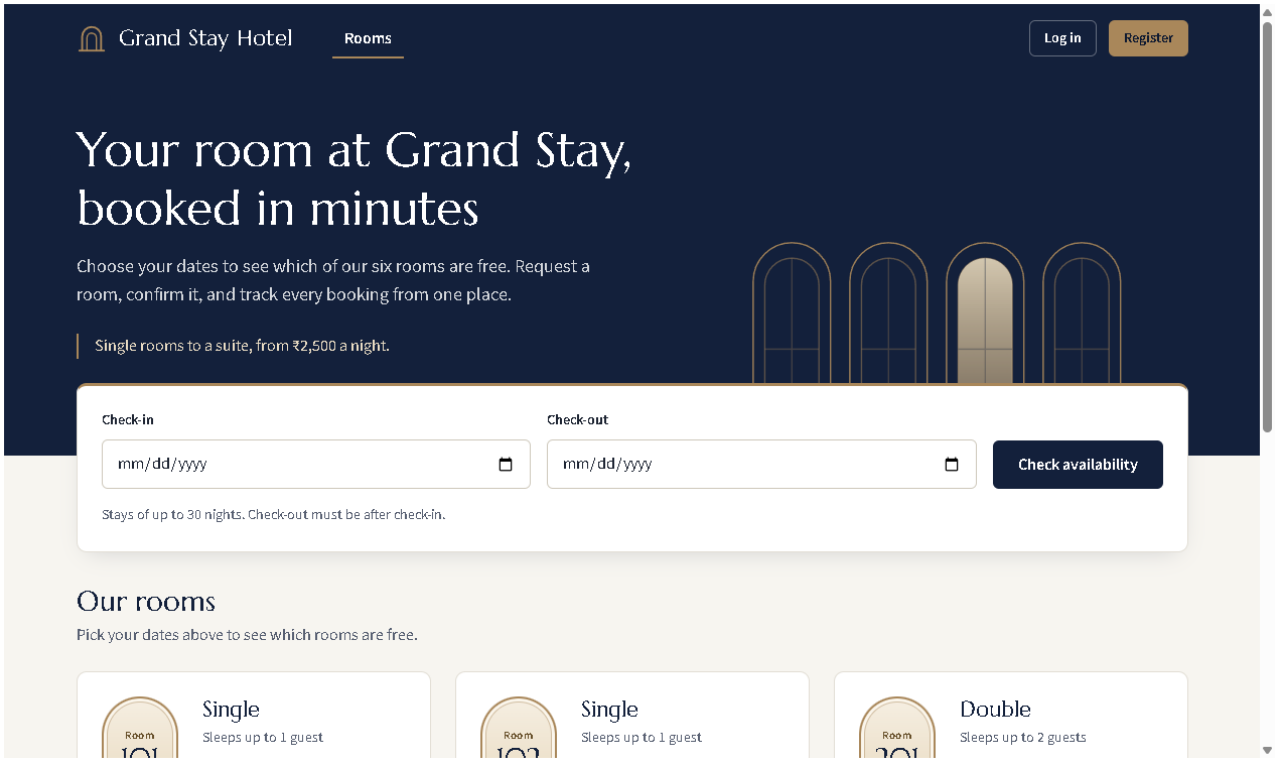


Figure 9.5 - The actual image saved by the hook: what the browser showed at the moment of failure. **Proves:** failures leave visual evidence automatically.

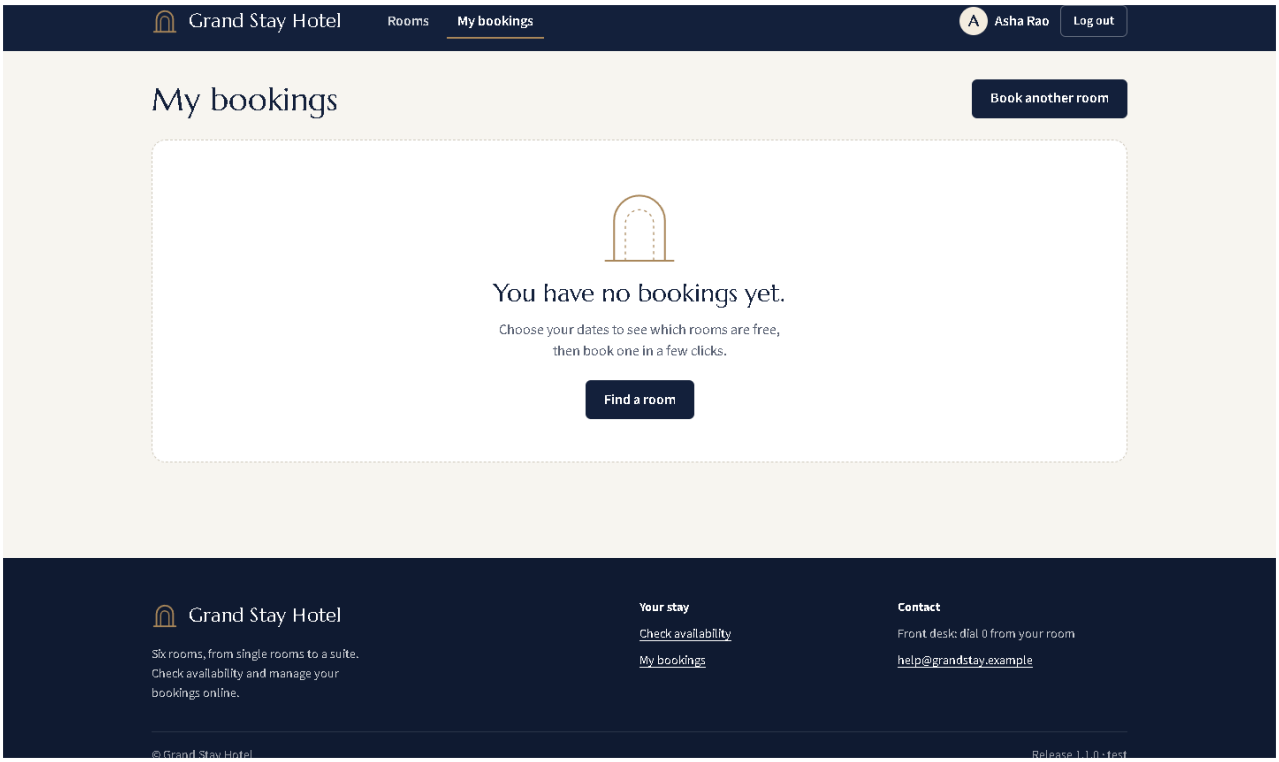


Figure 9.6 - The empty "My bookings" state (redesigned UI) - one of the pages the journeys navigate.

E. How to demonstrate this task to the professor (about 4 minutes)

Before the demo: Google Chrome installed; PowerShell in `D:\DevOPS\CICD_FOR_HOTEL_ROOM_BOOKING`.

53. Open `tests/e2e/test_journeys.py` in VS Code; show journey 3 and the `wait()` helper. *Say:* "Locators use ids; waits are explicit; each journey ends with assertions."
54. **Run the journeys and let the professor watch the browser:***Expected:* Chrome windows open and click through the pages; finally `5 passed, 85 deselected` (about 30 s).

```
$env:HEADLESS="0"
.venv\Scripts\python -m pytest -m e2e -v
Remove-Item Env:HEADLESS
```

55. **Produce a JUnit report:**Open `reports\e2e-tests.xml` - one `<testcase>` per journey.

```
.venv\Scripts\python -m pytest -m e2e --junitxml=reports/e2e-tests.xml -q
```

56. **Explain isolation:** "Each run starts the app with a brand-new temporary database, so runs never depend on each other."
57. **Screenshots on failure:** show Figure 9.5, or point to `conftest.py` lines 65-76 and *say:* "When a journey fails, a screenshot and the HTML are saved and archived by Jenkins."
58. **Against a running deployment** (optional, staging must be up):Note: staging currently runs the committed v1.1.0 interface, while the local tests expect the redesigned one (dialog, readable dates), so journeys 3 and 4 fail there (checked 2026-10-20: 3 passed, 2 failed) until the redesign is deployed. Test the deployment with the tests of the same version.

```
$env:E2E_BASE_URL="http://localhost:8081"
.venv\Scripts\python -m pytest -m e2e -q
Remove-Item Env:E2E_BASE_URL
```

Recovery: `SessionNotCreatedException` - update Chrome (Selenium downloads the matching driver automatically). Tests hang on first run - Selenium Manager is downloading chromedriver; wait once.

F. Expected screenshots for the final submission

60. `assets/task-09/selenium-run.png` - **exists, 2026-10-20**.
61. `assets/task-09/full-test-suite.png` - **exists, 2026-10-20**.
62. `assets/task-09/github-file-tests_e2e_test_journeys.py.png` - **exists, 2026-10-20**.
63. `assets/task-09/selenium-failure-demo-output.png` - **exists, 2026-10-20**.
64. `assets/task-09/selenium-failure-screenshot-saved-by-hook.png` - **exists, 2026-10-20**.
65. `assets/task-09/app-desktop-07-my-bookings-empty.png` - **exists, 2026-10-20**.
66. (optional) a photo/screen recording of the visible browser run with `HEADLESS=0` - **requires manual capture** (it is a live demo).

G. Probable viva questions and answers

58. **What is Selenium WebDriver?** A library and protocol that lets code control a real browser - open pages, type, click, read text.
59. **Which journeys did you automate and why?** Register; wrong-then-right login; search-book-confirm; cancel and track status; a booked room is unavailable to another guest - the six MVP workflows end to end.
60. **What is an explicit wait?** Waiting for a condition (e.g. element visible) up to a timeout, continuing as soon as it is true. Better than `sleep`.
61. **Why ids as locators?** They are unique and do not change when layout or styling changes; the UI redesign kept them, so the tests still passed.
62. **How do you keep test data from interfering?** Each local run uses a new temporary database; against deployments, unique e-mails and random future dates.
63. **How do screenshots on failure work?** A pytest hook records the result; the browser fixture saves a PNG and the HTML if the test failed.
64. **(Project-specific) Why are dates set with JavaScript?** Typing into a date input depends on the OS date format; setting `value` is identical everywhere.
65. **(Project-specific) What changed in the tests for the UI redesign?** Journey 4 now confirms the cancel dialog; journey 3 reads the ISO date from the `datetime` attribute and checks the readable text. Both still assert the same behaviour.
66. **Unit vs integration vs Selenium?** Unit: one rule, no server. Integration: real HTTP routes in-process. Selenium: the full journey in a browser.
67. **Do you run Selenium through Maven?** No - through pytest. It produces JUnit XML like Maven's Surefire would. Confirm with the professor if Maven is mandatory.
68. **How does Jenkins run a browser without a screen?** Headless Chromium inside the Jenkins container (`CHROME_BIN`, `CHROMEDRIVER` set in its Dockerfile).

H. Common mistakes and troubleshooting

Symptom	Cause	Inspect	Fix	Confirm
`SessionNotCreatedException`	Chrome/driver mismatch or no Chrome	Chrome version	update Chrome	journeys start
`TimeoutException` waiting for `nav-user`	page layout changed or app error	`reports/screenshots/*.png`	look at the screenshot; fix the page or locator	test passes
Journeys 3/4 fail against staging	staging runs an older UI than the local tests expect	`/health` version	run the matching version's tests, or deploy the new UI	5 passed

Symptom	Cause	Inspect	Fix	Confirm
Tests pass alone but fail together	shared data	`base_url` fixture	keep the throw-away DB; use unique data	repeated runs pass
Date typed wrongly	locale-dependent typing	`search()` helper	use the JavaScript value setter	correct dates shown

I. Quick revision notes

Five key points

21. 5 journeys covering all 6 MVP workflows, in `tests/e2e/test_journeys.py`.
22. Stable `By.ID` locators + explicit `WebDriverWait` waits + clear assertions.
23. Isolated data: a temporary database per run; unique e-mails and dates.
24. Failure -> PNG + HTML in `reports/screenshots/` (demonstrated 2026-10-20).
25. Runs through pytest (JUnit XML), locally and in Jenkins; not Maven.

30-60 second explanation "Selenium drives a real Chrome browser through five journeys: registering, logging in with a wrong and then right password, searching and booking a room and confirming it, cancelling it and checking My bookings, and making sure a booked room is not offered to another guest. The tests find elements by id, wait explicitly, and assert the visible result. Each run starts the app with a fresh temporary database, so results are repeatable. If a journey fails, a screenshot and the page HTML are saved, and Jenkins keeps them. Today all five passed."

Caveat: pytest, not Maven, runs the tests.

Readiness checklist

- â€¢ â˜ I can run the journeys with `HEADLESS=0` so the browser is visible.
- â€¢ â˜ I can explain explicit waits and stable locators with code on screen.
- â€¢ â˜ I can show a failure screenshot (Figure 9.5).
- â€¢ â˜ I can explain the three test levels.

Task 10 - Continuous Testing in Jenkins

A. Task title and objective

Task 10: Continuous Testing in Jenkins

Objective: run all automated tests (backend and Selenium) inside the pipeline, publish their reports, and make test failures **stop the deployment** - then prove it with a deliberately introduced defect, a corrective commit and a successful re-run.

Professor's required deliverables

- â€¢ Backend and Selenium tests integrated into the pipeline
- â€¢ Test report publication
- â€¢ Failure propagation and a quality gate that blocks deployment
- â€¢ A documented, deliberately introduced defect (on a disposable branch)
- â€¢ Evidence the defect causes a test failure
- â€¢ A corrective commit and a successful re-run

Current status: Implemented and verified. The experiment was run on 2026-10-11 (builds #2, #3, #9). On 2026-10-20 the build records were checked again directly in Jenkins (Figures 10.1-10.5), and today's build #11 again ran all 90 tests before deploying.

B. Explanation for a complete beginner

An everyday picture. At an airport, a plane cannot take off until the safety checklist is complete. If one check fails, the plane stays at the gate

- â€¢ nobody "takes off anyway". A **quality gate** is that rule for software: if any test fails, the new version is not deployed.

Term	Meaning	In this project
Continuous testing	every build automatically runs the tests	every Jenkins build runs flake8 + 90 tests
Quality gate	a condition that must pass before the next step	test stages come before Deploy
Failure propagation	a failure travels up and marks the whole build failed	pytest exit code != 0 -> stage fails -> build FAILURE
Skipped stage	Jenkins does not run later stages after a failure	"Stage 'Deploy' skipped due to earlier failure(s)"
Regression	a test that passed before now fails	Jenkins labels it *Regression*
Defect experiment	breaking the code on purpose to prove the tests notice	an off-by-one bug in the overlap rule
Disposable branch	a branch for an experiment that is never merged	`experiment/defect-demo`

Build failure vs deployment failure. A *build failure* means the pipeline stopped before deploying (e.g. a test failed) - users are not affected. A *deployment failure* means the new version was started but is unhealthy - that is what the health check and rollback are for.

C. What was actually implemented

How tests stop the pipeline - each test stage is a shell command; pytest exits with a non-zero code when any test fails, so Jenkins marks the stage failed and skips every later stage:

```
stage('Unit & integration tests') {
    steps { sh '.venv/bin/pytest tests/unit tests/integration --junitxml=reports/unit-integration-
tests.xml' }
```

```
}
stage('E2E tests (Selenium)') {
    steps { sh '.venv/bin/pytest -m e2e --junitxml=reports/e2e-tests.xml' }
}
// ... Package, Build images, Publish, Deploy, Health check come only after these
post { always { junit allowEmptyResults: true, testResults: 'reports/*.xml' } }
```

The deliberate-defect experiment (branch `experiment/defect-demo`, never merged into `develop` or `main` - `git branch --contains 5844eec` lists only the two experiment branches)

Step	Commit (current / original ID in logs)	What happened
Baseline	-	staging ran `1.0.0-64351fe` (build #1)
1. Defect	`5844eec` / `554cad3`	in `app/dao/booking_dao.py` the rule `check_in < :check_out` became `check_in <= :check_out` - an off-by-one that wrongly blocks a guest from arriving on the day the previous guest leaves
2. Build #2	FAILURE	`FAILED ...TestSearchRooms::test_overlap_rules[2030-01-08-2030-01-10-True]`, `1 failed, 81 passed`; E2E, Package, Build images, Publish, Deploy, Health check **skipped**; staging still answered `"commit":"64351fe"`
3. Fix	`2fb62d6` / `30c6a05`	`fix: restore strict overlap comparison in booking DAO`
4. Build #3	SUCCESS	87/87 tests; staging moved to `1.0.0-30c6a05`

The diff of the defect (Figure 10.3):

```
 _OVERLAP_SQL = """
-    check_in < :check_out
+    check_in <= :check_out
     AND check_out > :check_in
```

The stale-report bug (found from the evidence, fixed, re-verified)

- â€¢ Build #2's Tests page lists **`tests.e2e: 5 passed`** although its E2E stage was skipped (Figure 10.4). Cause: Jenkins reuses its workspace, and `reports/e2e-tests.xml` from build #1 was still there, so `junit 'reports/*.xml'` published it again.
- â€¢ Fix on branch `fix/jenkins-stale-reports`: the Checkout stage now runs `rm -rf reports dist` before anything else.
- â€¢ Proof: build #9 (`experiment/defect-demo-2` = the same defect + the fix, current commit `a3d8626`, shown in Jenkins as `8505486`) failed on the same test and its report lists **82 tests and no e2e package** - only the tests that actually ran (Figure 10.5).
- â€¢ Every later build (including #11 today) starts with `+ rm -rf reports dist`, so a report can never come from an older build.

D. Screenshots and evidence

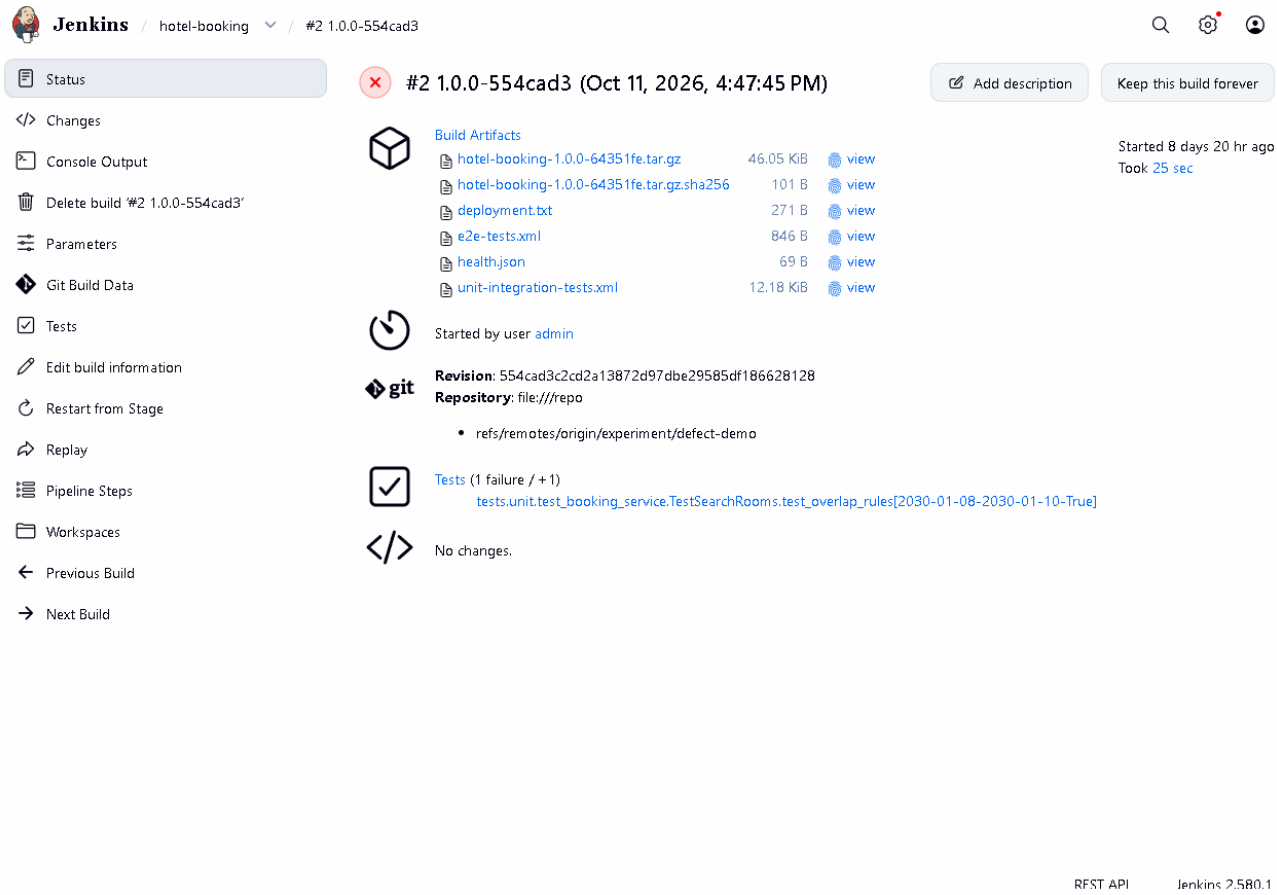


Figure 10.1 - Build #2 `1.0.0-554cad3` in Jenkins (checked 2026-10-20): result FAILURE.

Jenkins build #2 console (excerpt) - deliberate defect

```
[2026-10-11T16:47:50.441Z] + git log -1 --pretty=Building commit %h on experiment/defect-demo: %s
[2026-10-11T16:47:50.441Z] Building commit 554cad3 on experiment/defect-demo: experiment: introduce deliberate off-by-one defect in
overlap rule
[2026-10-11T16:48:08.412Z] E       AssertionError: assert ('101' in {'102', '201', '202', '301', '401'}) is True
[2026-10-11T16:48:08.412Z] FAILED tests/unit/test_booking_service.py::TestSearchRooms::test_overlap_rules[2030-01-08-2030-01-10-True]
- AssertionError: assert ('101' in {'102', '201', '202', '301', '401'}) is True
[2026-10-11T16:48:08.412Z] ======================== 1 failed, 81 passed in 11.65s ========================
Stage "E2E tests (Selenium)" skipped due to earlier failure(s)
Stage "Package source" skipped due to earlier failure(s)
Stage "Build images" skipped due to earlier failure(s)
Stage "Publish images" skipped due to earlier failure(s)
Stage "Deploy" skipped due to earlier failure(s)
Stage "Health check" skipped due to earlier failure(s)
Finished: FAILURE
```

Rendered from the real log file docs/evidence/jenkins/build-02-defect-fails.log (unedited lines, excerpt)

Figure 10.2 - Console excerpt of build #2 (rendered from the saved log `docs/evidence/jenkins/build-02-defect-fails.log`). **Notice:** the failing assertion, `1 failed, 81 passed`, and six "skipped due to earlier failure(s)" lines including Deploy. **Proves:** the quality gate stopped the deployment.

Git - the deliberate defect and the corrective commit (experiment branch)

```
$ git log --format="%h %s" -3 experiment/defect-demo
2fb62d6 fix: restore strict overlap comparison in booking DAO
5844eec experiment: introduce deliberate off-by-one defect in overlap rule
da225fb Merge branch 'feature/ci-release-pipeline' into develop

$ git show 5844eec --stat --format="%h %s"
5844eec experiment: introduce deliberate off-by-one defect in overlap rule

 app/dao/booking_dao.py | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)

$ git diff 5844eec~1 5844eec -- app/dao/booking_dao.py
diff --git a/app/dao/booking_dao.py b/app/dao/booking_dao.py
index 4c2c743..71a85aa 100644
--- a/app/dao/booking_dao.py
+++ b/app/dao/booking_dao.py
@@ -7,7 +7,7 @@ from app.models import Booking, BookingStatus
 # Touching stays do NOT overlap: check-out on the 5th and a new check-in on
 # the 5th is fine. Only PENDING/CONFIRMED bookings block a room.
 _OVERLAP_SQL = """
-    check_in < :check_out
+    check_in <= :check_out
     AND check_out > :check_in
     AND status IN ('PENDING', 'CONFIRMED')
 """

$ git branch --contains 5844eec
  experiment/defect-demo
  experiment/defect-demo-2
```

Rendered from real command output captured 2026-10-20 12:57 UTC in CICD_FOR_HOTEL_ROOM_BOOKING | raw text: defect-branch-history.txt

Figure 10.3 - The experiment branch history, the one-character diff, and the branches that contain the defect (rendered from real output, 2026-10-20).

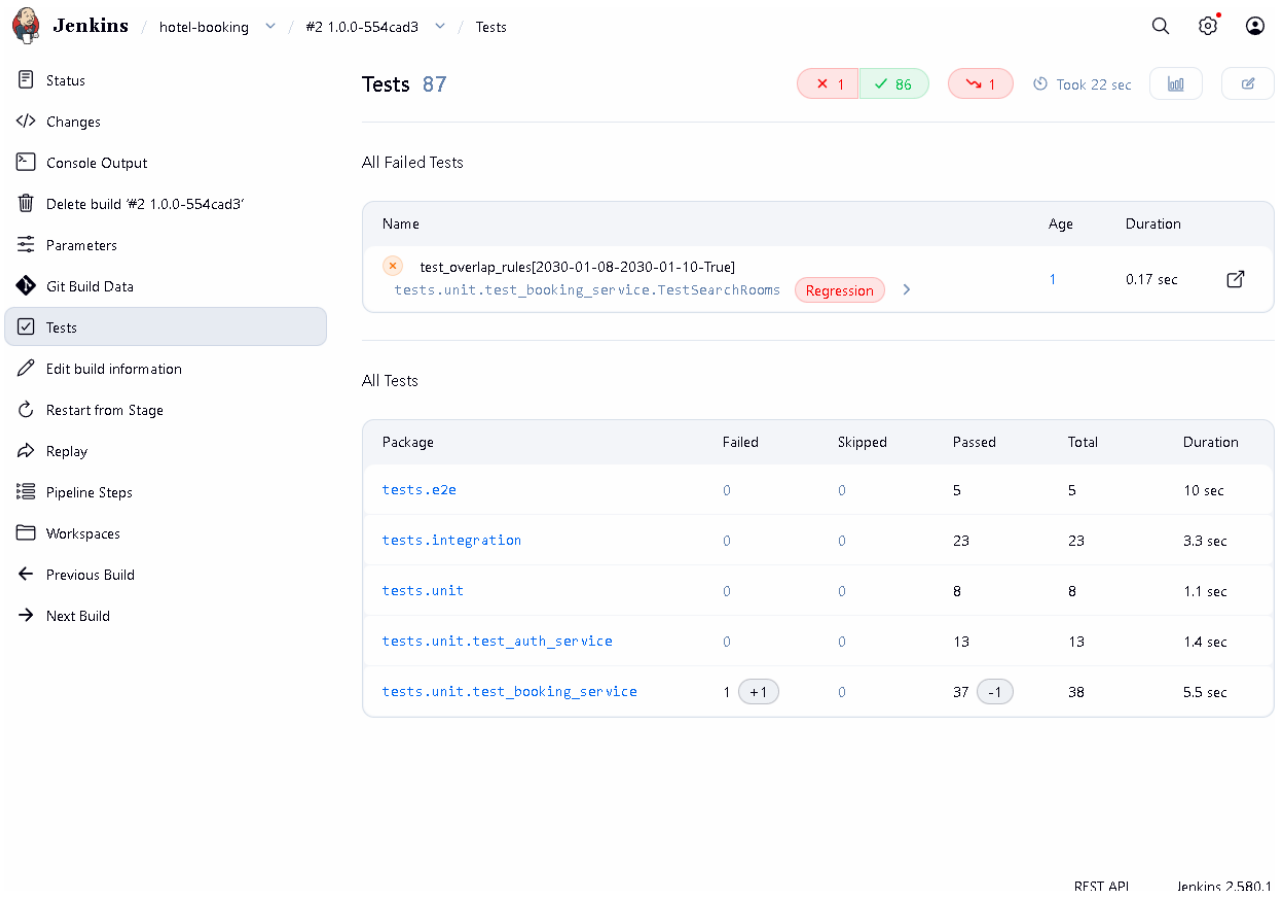


Figure 10.4 - Build #2's Tests page: 1 failed (`test_overlap_rules[...-True]`, marked Regression). Also notice `tests.e2e 5` - the stale report described above.

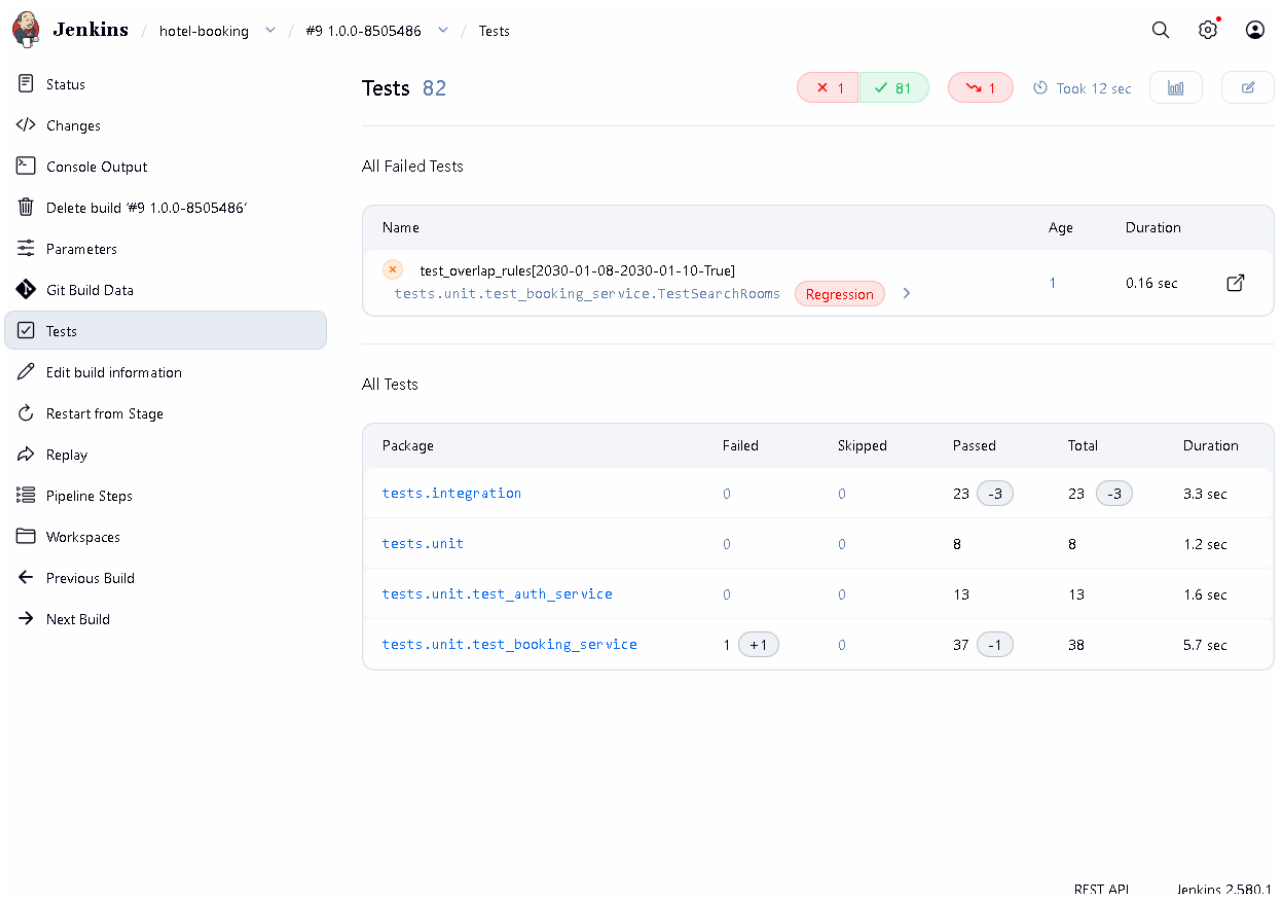


Figure 10.5 - Build #9 after the report fix: 82 tests, 1 failed, and no `tests.e2e` entry. **Proves:** the current build's report cannot reuse an older build's results.

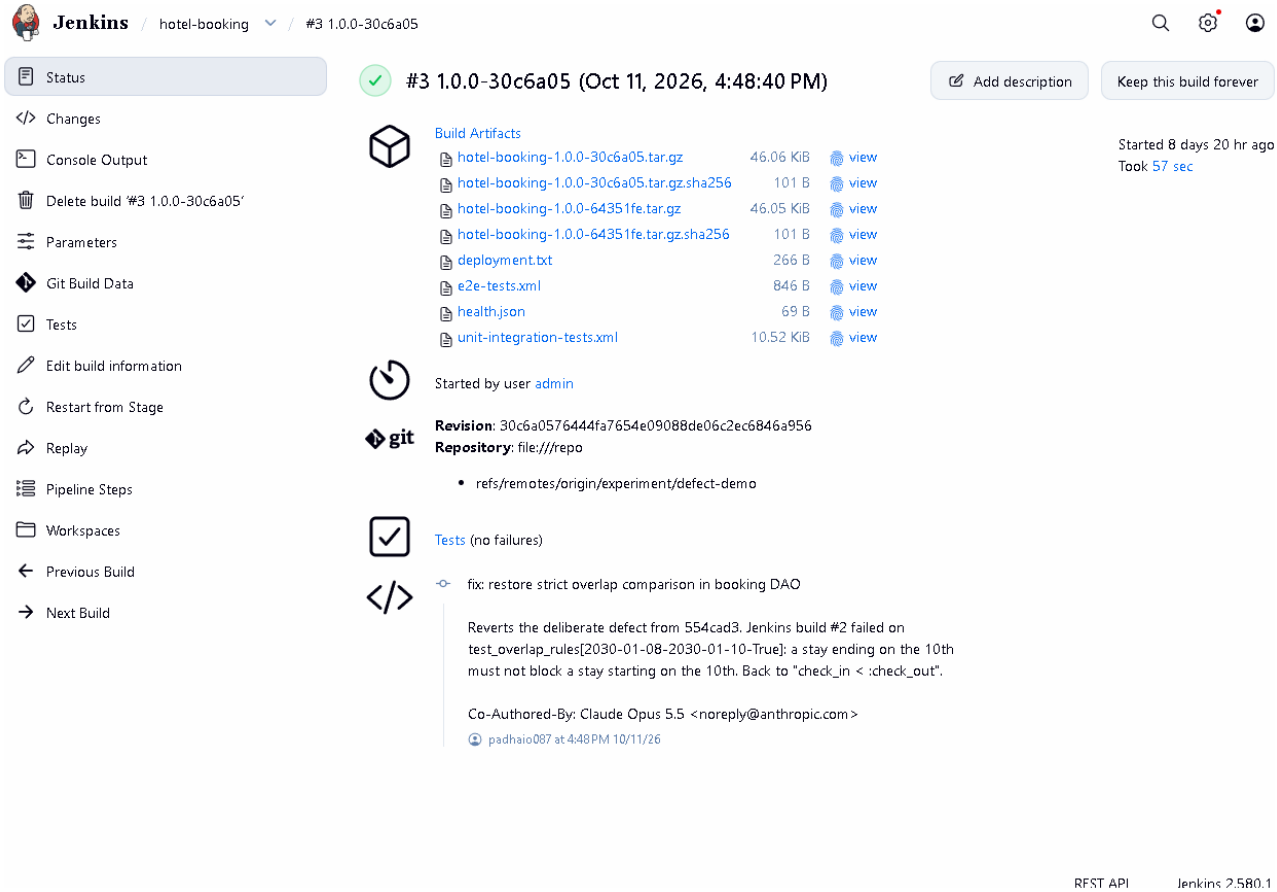


Figure 10.6 - Build #3 `1.0.0-30c6a05`: SUCCESS after the corrective commit.



Figure 10.7 - Build #3 console excerpt (rendered from the saved log): tests passed, deployed, health check OK.

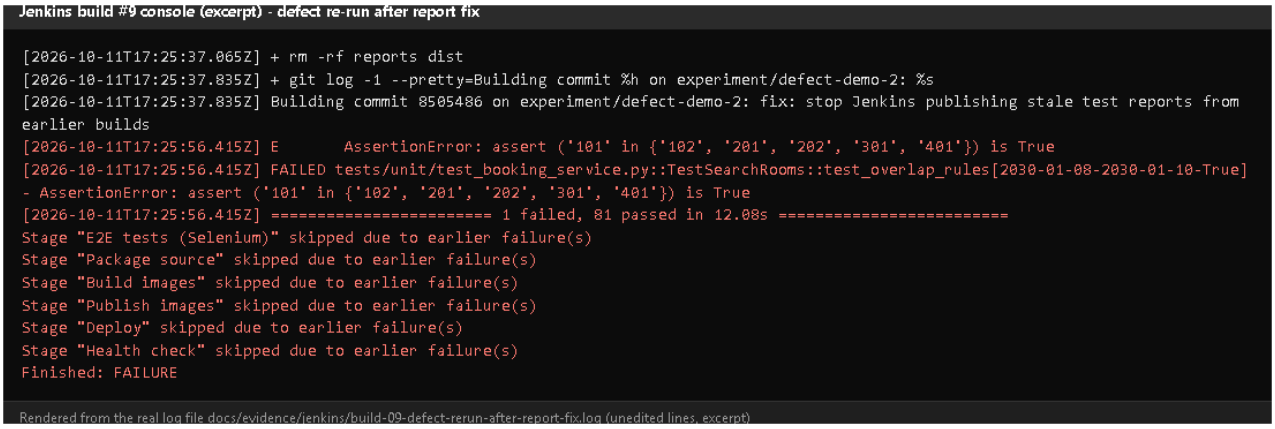


Figure 10.8 - Build #9 console excerpt: `+ rm -rf reports dist`, the same failing test, later stages skipped.

E. How to demonstrate this task to the professor (about 5 minutes)

Option A - show the recorded experiment (safe, 2 minutes).

69. Jenkins -> *hotel-booking*: point at red builds #2 and #9 in the stage view.
70. Open **#2 -> Console Output**, Ctrl+F `skipped due` - *Say:* "The test failed, so Jenkins skipped packaging and deployment."
71. Open **#2 -> Tests** - the failed overlap test.
72. Open **#3** - green after the fix.
73. In PowerShell (project folder):*Say:* "One character, `<` to `<=`, and the tests caught it."

```
git log --oneline -3 experiment/defect-demo
git diff 5844eec~1 5844eec -- app/dao/booking_dao.py
```

Option B - repeat the experiment live (about 6 minutes, disposable branch). Warning: do this only on an `experiment/*` branch; never merge it.

```
git stash -u                      # only if you have uncommitted work (today: the UI redesign)
git checkout -b experiment/live-defect develop
```

Edit `app/dao/booking_dao.py` line 10: change `check_in < :check_out` to `check_in <= :check_out`, save.

```
git commit -am "experiment: deliberate off-by-one defect (live demo)"
```

Jenkins -> *Build with Parameters* -> `BRANCH = experiment/live-defect`, `APP_ENV = staging` -> Build. *Expected:* red at "Unit & integration tests", later stages "skipped". Check http://localhost:8081/health - the commit is unchanged (not deployed). Then fix and re-run:

```
git revert --no-edit HEAD
```

Build the same branch again - *Expected:* all green, staging shows the new commit. Clean up:

```
git checkout feature/ui-redesign  # or the branch you were on
git stash pop                     # only if you stashed
git branch -D experiment/live-defect
```

F. Expected screenshots for the final submission

59. `assets/task-10/jenkins-build-02-failed.png` - **exists, 2026-10-20**.
60. `assets/task-10/build-02-defect-console.png` - **exists** (rendered from the 2026-10-11 log).
61. `assets/task-10/defect-branch-history.png` - **exists, 2026-10-20**.
62. `assets/task-10/jenkins-build-02-failed-test.png` - **exists, 2026-10-20**.
63. `assets/task-10/jenkins-build-09-tests-no-stale-report.png` - **exists, 2026-10-20**.
64. `assets/task-10/jenkins-build-03-fixed.png` - **exists, 2026-10-20**.
65. `assets/task-10/build-03-fix-console.png` - **exists** (rendered from the 2026-10-11 log).
66. `assets/task-10/build-09-rerun-console.png` - **exists** (rendered from the 2026-10-11 log).

G. Probable viva questions and answers

67. **What is a quality gate?** A check that must pass before the pipeline continues - here all lint and tests before Deploy.
68. **How does a failed test stop deployment?** pytest exits non-zero, the `sh` step fails, the stage fails and Jenkins skips all later stages.
69. **What defect did you introduce?** `<` became `<=` in the overlap SQL, blocking back-to-back stays (check-out day = next check-in day).
70. **Which test caught it?** `TestSearchRooms::test_overlap_rules[2030-01-08-2030-01-10-True]` - a stay ending on the 10th must not block one starting on the 10th.
71. **How do you know it wasn't deployed?** The log shows Deploy skipped, and staging's `/health` still reported the previous commit `64351fe`.
72. **Why on an experiment branch?** So the defect never reaches `develop` or `main`; `git branch --contains 5844eec` lists only experiment branches.
73. **(Project-specific) What was the stale-report problem?** Build #2's report showed 5 E2E tests passing although that stage never ran - an old XML file in the reused workspace. Fixed by deleting `reports/` and `dist/` at checkout; build #9 proves it.
74. **(Project-specific) Why do the IDs in Jenkins (`554cad3`) differ from Git (`5844eec`)?** Commit messages were edited once before publishing, which changed all IDs; see `docs/evidence/git/commit-id-map.md`.
75. **Build failure vs deployment failure?** Build failure: stopped before deploying (users unaffected). Deployment failure: the new version started but is unhealthy - caught by `--wait` and the health check, fixed by rollback.
76. **Do Selenium tests also gate the deploy?** Yes - the E2E stage comes before Package/Deploy.
77. **Why publish reports even when the build fails?** `post { always }` runs on failure too, so you can see *which* test failed.

H. Common mistakes and troubleshooting

Symptom	Cause	Inspect	Fix	Confirm
Report shows tests from an earlier build	stale XML in workspace	Checkout stage in the console	ensure `rm -rf reports dist` is present (it is since the fix)	report count = tests actually run
Defect build is green	defect not committed, or wrong branch built	console "Building commit ..."	commit and set `BRANCH` correctly	red stage
Experiment merged by accident	merged into develop	`git branch --contains <defect id>`	`git revert` the merge on develop	tests pass

Symptom	Cause	Inspect	Fix	Confirm
`git checkout` refuses	uncommitted work	`git status`	`git stash -u` / `git stash pop`	branch switched

I. Quick revision notes

Five key points

69. Every build runs flake8 + 85 unit/integration + 5 Selenium tests before deploying.
70. A failing test -> stage fails -> later stages (incl. Deploy) skipped.
71. Defect `<` -> `<=` (commit `5844eec`) failed build #2; fix `2fb62d6` made build #3 green.
72. Staging kept the old version during the failed build.
73. Stale-report bug found, fixed (`rm -rf reports dist`), proven by build #9.

30-60 second explanation "Tests are the quality gate of our pipeline: Jenkins runs lint, unit, integration and Selenium tests, and only if all pass does it package and deploy. To prove it I introduced a one-character bug on an experiment branch, changing `<` to `<=` in the overlap rule. Build #2 failed on the overlap test, every later stage was skipped, and staging kept the previous version. A corrective commit made build #3 green. I also found that build #2's report reused an old Selenium result; I fixed that, and build #9 proves reports now only contain tests that really ran."

Caveat: the experiment ran on 2026-10-11; on 2026-10-20 its records were re-checked, not re-run (Option B shows how to re-run it).

Readiness checklist

- â€¢ â˜ I can open builds #2, #3, #9 and explain each.
- â€¢ â˜ I can show the one-line diff.
- â€¢ â˜ I can explain the stale-report fix.
- â€¢ â˜ I know how to repeat the experiment safely on an experiment branch.

Task 11 - Docker Image and Container Lifecycle

A. Task title and objective

Task 11: Docker Image and Container Lifecycle

Objective: package the Flask application into a Docker image that runs with a production server, as a non-root user, configured by environment variables and without secrets inside; then demonstrate the full life of a container: build, tag, run, port mapping, logs, inspection, stop, restart, removal and persistent data.

Professor's required deliverables

- â€¢ Dockerfile with a suitable Python base image, reproducible dependency install, production WSGI server, exposed and mapped port, non-root user, environment-variable configuration, no baked-in secrets, `.dockerignore`, a health check
- â€¢ Demonstration of build and tag, create and run, port mapping, logs and inspection, stop / restart / remove, persistent database storage
- â€¢ Proof that the container actually starts and serves requests

Current status: Implemented and verified. The complete lifecycle was run again on 2026-10-20 (Figures 11.1-11.2): the container became healthy, answered on the mapped port, ran as `appuser`, and an account created in one container still worked in a **new** container using the same volume.

B. Explanation for a complete beginner

An everyday picture. A shipping container holds goods in a standard box that fits every ship, train and truck. A **Docker container** holds an application plus everything it needs (Python, libraries, settings) in a standard box that runs the same on any machine with Docker - "it works on my machine" stops being a problem.

Term	Plain meaning	In this project
Dockerfile	the recipe for building an image	`Dockerfile` (app), `deploy/nginx/Dockerfile`
Image	a read-only package built from the recipe (like a class / a template)	`hotel-booking-app:1.1.0`
Container	a running instance of an image (like an object)	`hotel-demo`, `hotel-prod-app-1`
Tag	the name:version label of an image	`1.1.0`, `latest`, `1.1.0-1c2103c`
Port mapping	connect a port on your computer to a port inside the container	`-p 8095:8000`
Volume	storage managed by Docker that lives outside containers	`hotel-demo-data` / `hotel-data` mounted at `/data`
Logs	what the app prints, collected by Docker	`docker logs`
Inspect	detailed JSON description of a container/image	`docker inspect`
Health check	a command Docker runs regularly to ask "are you OK?"	`GET /health` every 10 s
Docker Compose	starts several containers together from one file	`docker-compose.yml`: nginx + app + volume

Why can data survive deleting a container? A container's own file system disappears with it. Our SQLite file lives at `/data/hotel.db`, and `/data` is a **volume**, which Docker keeps until you delete the volume itself. A new container that mounts the same volume sees the same data.

Lifecycle states: created -> running -> (stopped = exited) -> running again -> removed. Health status: starting -> healthy (or unhealthy).

C. What was actually implemented

`Dockerfile` (application image) - key lines:

Line	Instruction	Why
3	`FROM python:3.11-slim-bookworm`	small official Python image, same Python as the pipeline
7-8	`ARG APP_VERSION`, `ARG GIT_COMMIT`	release info, shown by `/health` and as image labels
12-17	`ENV APP_ENV=prod`, `DATABASE_PATH=/data/hotel.db` ...	configuration by environment variables
20-22	`LABEL org.opencontainers.image.version / revision`	`docker inspect` shows which release and commit an image came from
29	`COPY certs/ ...` + `update-ca-certificates`	optional extra root CAs (antivirus HTTPS scanning); folder normally empty
35-36	`COPY requirements.txt` then `RUN pip install --no-cache-dir ...`	pinned dependencies; copied **before** the code so this slow layer is cached
38-39	`COPY app/ ...`, `COPY wsgi.py gunicorn.conf.py`	the application
44-46	`useradd ... appuser`, `USER appuser`	not root inside the container
50	`VOLUME /data`	the database lives on a volume
52	`EXPOSE 8000`	Gunicorn's port
56-57	`HEALTHCHECK ... urllib.request.urlopen('http://127.0.0.1:8000/health')`	Docker knows when the app is really ready
59	`CMD ["gunicorn", "--config", "gunicorn.conf.py", "wsgi:app"]`	production WSGI server (2 workers)

No secrets in the image: `SECRET_KEY` is given at run time (`-e` or `--env-file`); the app refuses to start in prod without it. **`.dockerignore`** excludes `.git`, `.venv`, `tests`, `docs`, `deploy/ansible`, `deploy/jenkins`, `*.md`, `.env` and `*.db`.

`docker-compose.yml` - two services: `app` (image `${IMAGE_PREFIX}hotel-booking-app:${IMAGE_TAG}`, volume `hotel-data:/data`, no published port) and `nginx` (reverse proxy, published on `${HOST_PORT:-8080}:80`, starts only when the app is `service_healthy`); both `restart: unless-stopped`. The Nginx image (`deploy/nginx/Dockerfile`) bakes in `hotel.conf` and serves `/static/` directly.

Lifecycle demo run on 2026-10-20 (raw output: `assets/raw/docker-lifecycle.txt`):

Step	Command (shortened)	Observed
build	`docker build -t hotel-booking-app:1.1.0 --build-arg APP_VERSION=1.1.0 --build-arg GIT_COMMIT=1c2103c-dirty .`	image built
tag	`docker tag hotel-booking-app:1.1.0 hotel-booking-app:latest`	two tags, same image ID

Step	Command (shortened)	Observed
inspect image	`docker image inspect -f '...'`	labels version/revision, `user=appuser`, exposed `8000/tcp`
volume	`docker volume create hotel-demo-data`	created
run	`docker run -d --name hotel-demo -p 8095:8000 --env-file ... -v hotel-demo-data:/data hotel-booking-app:1.1.0`	`Up (healthy)`, `0.0.0.0:8095->8000/tcp`
request	`curl -s http://localhost:8095/health`	`{"commit":"1c2103c-dirty","env":"prod","status":"ok","version":"1.1.0"}`
use	register a guest over HTTP	`HTTP 302` (success)
logs	`docker logs --tail 6 hotel-demo`	Gunicorn start, health checks, `POST /register 302`
inspect	`docker inspect -f 'state=... health=... mounts=...'`	`running`, `healthy`, `hotel-demo-data->/data`
user	`docker exec hotel-demo whoami`	`appuser`
stop	`docker stop hotel-demo`	`Exited (0)`
start / restart	`docker start`, `docker restart`	healthy again
remove	`docker rm -f hotel-demo`	gone
new container	same `docker run` on the same volume	healthy
data survived	log in with the account from the first container	`HTTP 302` (success)
clean up	`docker rm -f`, `docker volume rm`	removed

(`-dirty` in the commit label means the image was built from a working tree with uncommitted changes - the UI redesign - and is honest about it.)

D. Screenshots and evidence

Docker lifecycle demo, part 1: build, tag, run, ports, logs

```
# Docker lifecycle demo - 2026-10-20 12:42 UTC - commit 1c2103c-dirty
# (SECRET_KEY is passed with --env-file and is not shown)

$ docker build -q -t hotel-booking-app:1.1.0 --build-arg APP_VERSION=1.1.0 --build-arg GIT_COMMIT=1c2103c-dirty .
sha256:4507c862db2abe44c6a443f9ef9f26d107f2543675ef9b420285864d1e94758d

$ docker tag hotel-booking-app:1.1.0 hotel-booking-app:latest

$ docker image ls hotel-booking-app
IMAGE                     ID             DISK USAGE   CONTENT SIZE   EXTRA
hotel-booking-app:1.1.0   4507c862db2a        219MB         52.5MB
hotel-booking-app:latest  4507c862db2a        219MB         52.5MB

$ docker image inspect -f labels={{json .Config.Labels}} user={{.Config.User}} exposed={{json .Config.ExposedPorts}} hotel-booking-app:1.1.0
labels={"org.opencontainers.image.revision":"1c2103c-dirty","org.opencontainers.image.title":"hotel-booking-app","org.opencontainers.image.version":"1.1.0"} user=appuser exposed={"8000/tcp":{}}

$ docker volume create hotel-demo-data
hotel-demo-data

$ docker run -d --name hotel-demo -p 8095:8000 --env-file <scratch>/demo.env -v hotel-demo-data:/data hotel-booking-app:1.1.0
de9aca8d95bcce39e874fb43c4e415c99f5f9b8025c6d600868c25c2d4f1f0f9
(health status: healthy)

$ docker ps --filter name=hotel-demo --format table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}
NAMES        IMAGE                     STATUS                   PORTS
hotel-demo   hotel-booking-app:1.1.0   Up 6 seconds (healthy)   0.0.0.0:8095->8000/tcp, [::]:8095->8000/tcp

$ docker port hotel-demo
8000/tcp -> 0.0.0.0:8095
8000/tcp -> [::]:8095

$ curl -s http://localhost:8095/health
{"commit":"1c2103c-dirty","env":"prod","status":"ok","version":"1.1.0"}

$ /d/DevOPS/CICD_FOR_HOTEL_ROOM_BOOKING/.venv/Scripts/python <scratch>/guest.py http://localhost:8095 register lifecycle@example.com
register lifecycle@example.com: HTTP 302 (success, redirected)

$ docker logs --tail 6 hotel-demo
[2026-10-20 12:42:40 +0000] [1] [INFO] Control socket listening at /home/appuser/.gunicorn/gunicorn.ctl
127.0.0.1 - - [20/Oct/2026:12:42:44 +0000] "GET /health HTTP/1.1" 200 72 "-" "Python-urllib/3.11"
172.17.0.1 - - [20/Oct/2026:12:42:45 +0000] "GET /health HTTP/1.1" 200 72 "-" "curl/8.22.0"
172.17.0.1 - - [20/Oct/2026:12:42:45 +0000] "GET /register HTTP/1.1" 200 8089 "-" "Python-urllib/3.13"
[2026-10-20 12:42:45,860] INFO in auth_routes: User registered: user_id=1
172.17.0.1 - - [20/Oct/2026:12:42:45 +0000] "POST /register HTTP/1.1" 302 199 "-" "Python-urllib/3.13"

$ docker inspect -f state={{.State.Status}} health={{.State.Health.Status}} restart={{.HostConfig.RestartPolicy.Name}} mounts={{range .Mounts}}{{.Name}}->{{.Destination}}{{end}} hotel-demo
```

Rendered from the real log file docs/task-reports/assets/raw/docker-lifecycle.txt (unedited lines, excerpt)

Figure 11.1 - Lifecycle demo, part 1: build, tag, image labels/user/port, volume, run, `docker ps` with port mapping, `/health`, registration, logs (rendered from the real run on 2026-10-20).

Docker lifecycle demo, part 2: stop, restart, remove, volume

```
state=running health=healthy restart=no mounts=hotel-demo-data->/data

$ docker exec hotel-demo whoami
appuser

$ docker stop hotel-demo
hotel-demo

$ docker ps -a --filter name=hotel-demo --format table {{.Names}}\t{{.Status}}
NAMES        STATUS
hotel-demo   Exited (0) Less than a second ago

$ docker start hotel-demo
hotel-demo
(health status: healthy)

$ docker restart hotel-demo
hotel-demo
(health status: healthy)

$ docker ps --filter name=hotel-demo --format table {{.Names}}\t{{.Status}}
NAMES        STATUS
hotel-demo   Up 6 seconds (healthy)

# Remove the container completely, then create a NEW one on the same volume:

$ docker rm -f hotel-demo
hotel-demo

$ docker ps -a --filter name=hotel-demo --format {{.Names}}

$ docker run -d --name hotel-demo -p 8095:8000 --env-file <scratch>/demo.env -v hotel-demo-data:/data hotel-booking-app:1.1.0
8eb2e569109a0e1ab817538f3b64b9a0ddd0dc0ad345476cf1a1449bafba4caf
(health status: healthy)
# The account registered in the OLD container still exists (data is in the volume):

$ /d/DevOPS/CICD_FOR_HOTEL_ROOM_BOOKING/.venv/Scripts/python <scratch>/guest.py http://localhost:8095 login lifecycle@example.com
login lifecycle@example.com: HTTP 302 (success, redirected)

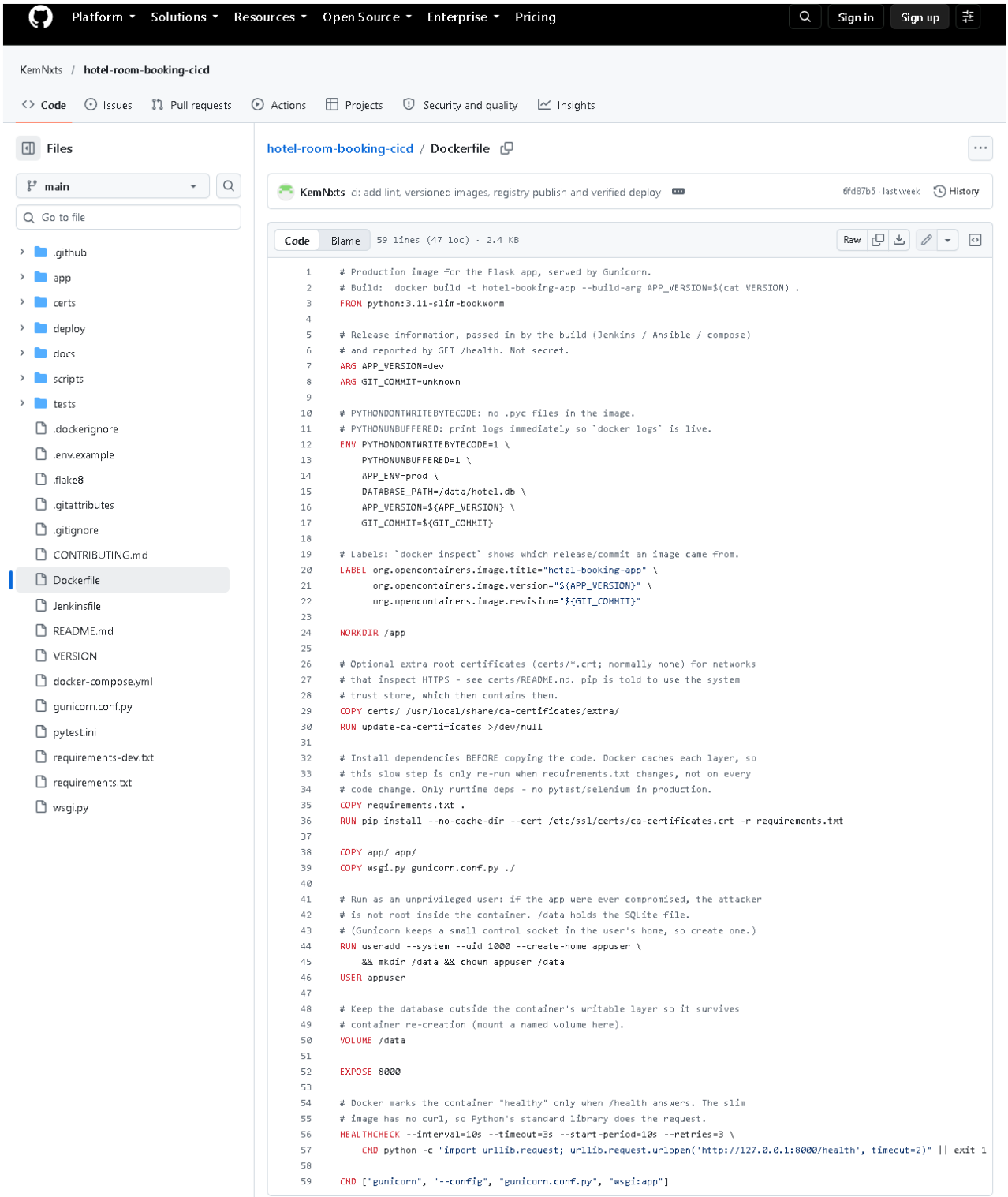
# Clean up

$ docker rm -f hotel-demo
hotel-demo

$ docker volume rm hotel-demo-data
hotel-demo-data
```

Rendered from the real log file docs/task-reports/assets/raw/docker-lifecycle.txt (unedited lines, excerpt)

Figure 11.2 - Part 2: inspect, `whoami` = appuser, stop -> Exited, start, restart, remove, a new container on the same volume, the old account still logs in, clean-up. **Proves:** every lifecycle operation, and persistence through a volume.



(part 1 of 2)

(part 2 of 2)

Figure 11.3 - `Dockerfile` on GitHub. **Notice:** slim base, dependencies before code, `USER appuser`, `VOLUME`, `EXPOSE`, `HEALTHCHECK`, Gunicorn `CMD`.

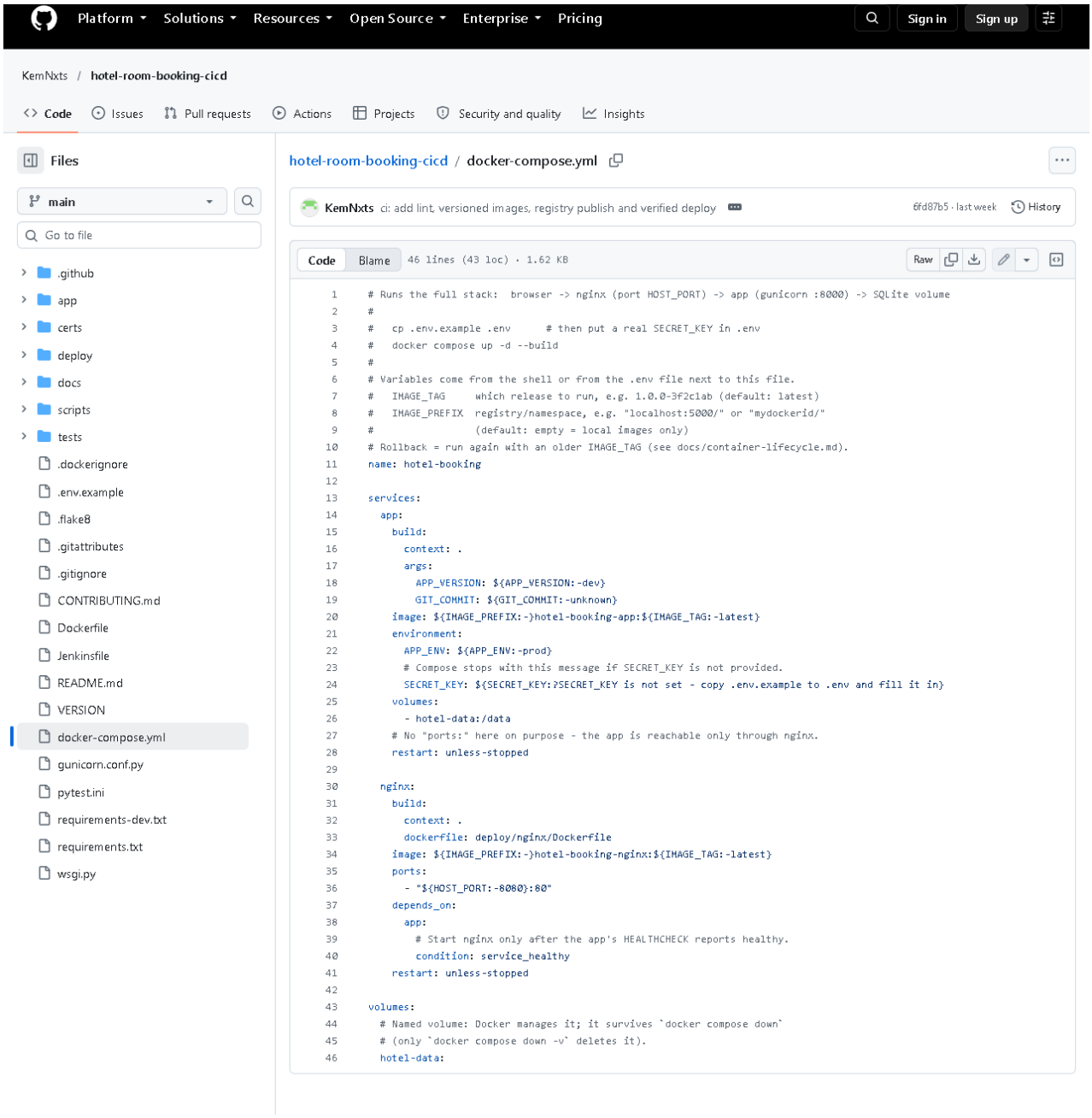


Figure 11.4 - `docker-compose.yml`: app + nginx, named volume, `service_healthy` dependency, `SECRET_KEY` required.

PowerShell - Docker images and running containers

```
$ docker image ls --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}" --filter reference=*hotel*
REPOSITORY                TAG       SIZE
hotel-ansible-lab-target  latest    222MB
hotel-booking-app         1.1.0     219MB
hotel-booking-app         latest    219MB
hotel-ansible             latest    397MB
hotel-jenkins             local     2.22GB

$ docker image ls --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}" --filter reference=localhost:5000/*
REPOSITORY                          TAG             SIZE
localhost:5000/hotel-booking-app    1.1.0-1c2103c   218MB
localhost:5000/hotel-booking-app    1.1.0-2e4c93a   218MB
localhost:5000/hotel-booking-app    1.1.0-d07929b   218MB
localhost:5000/hotel-booking-app    1.1.0-5aa909d   218MB
localhost:5000/hotel-booking-app    1.0.0-661751d   218MB
localhost:5000/hotel-booking-nginx  1.1.0-d07929b   73.6MB
localhost:5000/hotel-booking-nginx  1.0.0-661751d   73.6MB
localhost:5000/hotel-booking-nginx  1.0.0-224384f   73.6MB
localhost:5000/hotel-booking-nginx  1.1.0-2e4c93a   73.6MB
localhost:5000/hotel-booking-nginx  1.1.0-5aa909d   73.6MB
localhost:5000/hotel-booking-nginx  1.1.0-1c2103c   73.6MB
localhost:5000/hotel-booking-app    1.0.0-224384f   218MB
localhost:5000/hotel-booking-app    1.0.0-30c6a05   218MB
localhost:5000/hotel-booking-nginx  1.0.0-64351fe   73.6MB
localhost:5000/hotel-booking-nginx  1.0.0-30c6a05   73.6MB
localhost:5000/hotel-booking-app    1.0.0-64351fe   218MB

$ docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"
NAMES                       IMAGE                                          STATUS                    PORTS
hotel-ansible-lab-target-1  hotel-ansible-lab-target                       Up 12 minutes             127.0.0.1:8088->80/tcp
hotel-staging-nginx-1       localhost:5000/hotel-booking-nginx:1.1.0-1c2103c  Up 15 minutes          0.0.0.0:8081->80/tcp,
[::]:8081->80/tcp
hotel-staging-app-1         localhost:5000/hotel-booking-app:1.1.0-1c2103c    Up 15 minutes (healthy)  8000/tcp
hotel-prod-nginx-1          localhost:5000/hotel-booking-nginx:1.1.0-d07929b  Up 51 minutes          0.0.0.0:8082->80/tcp,
[::]:8082->80/tcp
hotel-prod-app-1            localhost:5000/hotel-booking-app:1.1.0-d07929b    Up 51 minutes (healthy)  8000/tcp
hotel-jenkins-jenkins-1     hotel-jenkins:local                            Up 51 minutes             127.0.0.1:8090->8080/tcp
hotel-jenkins-registry-1    registry:2                                     Up 51 minutes             127.0.0.1:5000->5000/tcp

$ docker volume ls --filter name=hotel
DRIVER    VOLUME NAME
local     hotel-ansible-lab_target-containerd
local     hotel-ansible-lab_target-docker
local     hotel-jenkins_jenkins-home
local     hotel-jenkins_registry-data
local     hotel-prod_hotel-data
local     hotel-staging_hotel-data
```

Rendered from real command output captured 2026-10-20 12:57 UTC in CICD_FOR_HOTEL_ROOM_BOOKING | raw text: docker-images-containers.txt

Figure 11.5 - `docker image ls`, `docker ps` and `docker volume ls` on 2026-10-20 (rendered from real output): versioned images, the running staging/prod/Jenkins/lab containers and the data volumes.

E. How to demonstrate this task to the professor (about 5 minutes)

Before the demo: Docker Desktop running. PowerShell in `D:\DevOPS\CICD_FOR_HOTEL_ROOM_BOOKING`. These exact commands were run on 2026-10-20.

```
# 1. build and tag the image
docker build -t hotel-booking-app:demo --build-arg APP_VERSION=1.1.0 .
docker image ls hotel-booking-app


# 2. create a volume and run a container with a port mapping
docker volume create hotel-demo-data
docker run -d --name hotel-demo -p 8095:8000 -e SECRET_KEY=demo-only-secret-123 -v hotel-demo-data:/data
hotel-booking-app:demo
docker ps --filter name=hotel-demo
```

Expected: status `Up ... (healthy)` after ~10 s and `0.0.0.0:8095->8000/tcp`. *Say:* "Port 8095 on my laptop is mapped to port 8000 inside the container."

```
# 3. it serves requests
curl.exe -s http://localhost:8095/health
```

Open http://localhost:8095/register in the browser and create an account.

```
# 4. logs and inspection
```

```
docker logs --tail 5 hotel-demo
docker inspect -f "{{.State.Health.Status}}" hotel-demo
docker exec hotel-demo whoami

# 5. stop, start, restart
docker stop hotel-demo
docker ps -a --filter name=hotel-demo
docker start hotel-demo
docker restart hotel-demo

# 6. remove the container, start a NEW one on the same volume
docker rm -f hotel-demo
docker run -d --name hotel-demo -p 8095:8000 -e SECRET_KEY=demo-only-secret-123 -v hotel-demo-data:/data
hotel-booking-app:demo
```

Log in at http://localhost:8095/login with the account from step 3. *Say:* "The container was deleted, but the database is on a volume, so the account still exists." (Use the same `SECRET_KEY` - a different one only logs existing sessions out; the account stays.)

```
# 7. clean up
docker rm -f hotel-demo
docker volume rm hotel-demo-data
docker image rm hotel-booking-app:demo
```

Notes: in PowerShell use `curl.exe` (plain `curl` is a different PowerShell command). `-e SECRET_KEY=...` here is a throw-away demo value; real deployments take it from `.env` or Jenkins credentials.

Recovery: "port is already allocated" - choose another port, e.g. `-p 8096:8000`. "Conflict. The container name is already in use" - `docker rm -f hotel-demo`. Build fails with `CERTIFICATE_VERIFY_FAILED` - antivirus HTTPS scanning; see `certs/README.md`.

F. Expected screenshots for the final submission

74. `assets/task-11/lifecycle-part1-build-run.png` - **exists, 2026-10-20** (rendered from the real run).
75. `assets/task-11/lifecycle-part2-stop-restart-volume.png` - **exists, 2026-10-20**.
76. `assets/task-11/github-file-Dockerfile.png` - **exists, 2026-10-20**.
77. `assets/task-11/github-file-docker-compose.yml.png` - **exists, 2026-10-20**.
78. `assets/task-11/docker-images-containers.png` - **exists, 2026-10-20**.
79. Earlier lifecycle evidence (2026-10-11): `docs/evidence/docker/lifecycle.txt` - **exists**.

G. Probable viva questions and answers

67. **Image vs container?** An image is the read-only package; a container is a running instance of it. Many containers can run from one image.
68. **Why copy `requirements.txt` before the code?** Docker caches each layer; dependencies are reinstalled only when `requirements.txt` changes, not on every code change.
69. **Why a non-root user?** If the app were compromised, the attacker would not be root inside the container (`whoami` shows `appuser`).
70. **How is the app configured?** Environment variables (`APP_ENV`, `SECRET_KEY`, `DATABASE_PATH`...); no secrets are in the image.
71. **What does the HEALTHCHECK do?** Every 10 s it requests `/health`; Docker marks the container healthy/unhealthy, and compose starts Nginx only when the app is healthy.
72. **What is port mapping?** `-p 8095:8000` connects port 8095 on the host to 8000 in the container.
73. **(Project-specific) How did you prove data persistence?** I created an account, deleted the container, started a new one on the same volume and logged in with that account - HTTP 302 success (2026-10-20).

74. **(Project-specific) Why Gunicorn and Nginx?** Gunicorn runs the Python app with several workers; Nginx is the only public entry point and serves CSS directly. The app port isn't even published in compose.
75. **What does `.dockerignore` do?** Keeps files like `.git`, `.venv`, tests, docs and `.env` out of the build context - smaller images, no secrets.
76. **`docker stop` vs `docker rm`?** Stop ends the process (container still exists, can start again); rm deletes the container (volume kept).
77. **What happens to data with `docker compose down -v`?** `-v` also deletes the volume - the database is lost. Plain `down` keeps it.

H. Common mistakes and troubleshooting

Symptom	Cause	Inspect	Fix	Confirm
`failed to connect to the docker API`	Docker Desktop not running	Docker Desktop window	start it, wait for "Engine running"	`docker ps` works
Container exits immediately	missing `SECRET_KEY` in prod	`docker logs hotel-demo`	pass `-e SECRET_KEY=...`	`Up (healthy)`
`port is already allocated`	port used	`docker ps`	other host port	container starts
Status stays `starting`/`unhealthy`	app error	`docker logs`, `docker inspect`	fix config; rebuild	`healthy`
Data gone after restart	no volume, or `down -v`	`docker inspect -f "{{.Mounts}}"`	always mount `-v name:/data`	data present
`curl` in PowerShell prints a table	PowerShell alias	-	use `curl.exe`	JSON shown

I. Quick revision notes

Five key points

78. Image = package, container = running instance, tag = name:version.
79. Dockerfile: slim Python, cached dependencies, non-root, `VOLUME /data`, `HEALTHCHECK`, Gunicorn.
80. Configuration by environment variables; `SECRET_KEY` only at run time.
81. Full lifecycle demonstrated on 2026-10-20, including persistence via a volume.
82. Compose runs Nginx (public) + app (private) + named volume.

30-60 second explanation "The Dockerfile builds a small Python image: dependencies first for caching, then the code, a non-root user, a volume for the SQLite database, a health check, and Gunicorn as the production server. Configuration comes from environment variables, so the image contains no secrets. I demonstrated the lifecycle: build and tag, run with a port mapping, check `/health`, read the logs, inspect, stop, start, restart and remove. Because the database is on a volume, an account created in one container still worked in a brand-new one."

Caveat: the demo image was built from a working tree with uncommitted UI changes, so its commit label ends in `-dirty`.

Readiness checklist

- â€¢ â˜ Docker Desktop is running before the viva.
- â€¢ â˜ I can run the 7-step demo without notes.
- â€¢ â˜ I can explain image vs container and why the volume keeps data.

â€¢ â˜ I remember `curl.exe` in PowerShell.

Task 12 - Jenkins-Docker Continuous Deployment

A. Task title and objective

Task 12: Jenkins-Docker Continuous Deployment

Objective: after all tests pass, have Jenkins build a Docker image, tag it with a version and the commit SHA, publish it to a registry, deploy exactly that tested image, check its health and record what was deployed.

Professor's required deliverables

80. Build the image after the mandatory tests pass
81. Tag it with a version and an identifiable commit SHA
82. Publish it to Docker Hub or an available registry
83. Deploy the tested version
84. Perform a health check
85. Record the deployed version and pipeline result

â€¢ Use Jenkins credentials securely; never deploy untested or uncommitted code

Current status: Implemented and verified (local registry). Build #11 on 2026-10-20 built, tagged `1.1.0-1c2103c`, pushed to the local registry, deployed to staging, health-checked and archived the deployment record.

Docker Hub publishing: pending external action. The pipeline supports it (`REGISTRY=docker.io/<user>` + credential `registry-creds`), but it was **never run** - there is no Docker Hub account/token. All publishing so far went to the **local registry** at `localhost:5000`.

B. Explanation for a complete beginner

An everyday picture. A bakery tests each batch, then puts the approved loaves into labelled boxes ("Batch 1.1.0, baked from recipe card #1c2103c") in a store-room. Shops collect boxes from the store-room by label. If a batch turns out bad, a shop simply takes an older box. The store-room is the **registry**, the label is the **image tag**.

Term	Meaning	In this project
Continuous Deployment (CD)	every change that passes all tests is deployed automatically	Jenkins deploys after the test stages
CI vs CD	CI = build + test every change; CD = also release/deploy it automatically	stages 1-6 are CI, 7-10 are CD
Registry	a server that stores and serves Docker images	`registry:2` container at `localhost:5000`
Docker Hub	Docker's public hosted registry on the internet	supported, not used (no account)
Local vs hosted registry	local runs on your machine (free, private, only you can reach it); hosted is on the internet (others/servers can pull)	local
Push / pull	upload an image to / download from a registry	`docker push`, `docker compose pull`
Commit SHA	the unique ID of a Git commit (short form 7 characters)	`1c2103c`
Version tag	human release number from the `VERSION` file	`1.1.0`

Why both version and SHA? `1.1.0` tells people *which release*; `1c2103c` tells you *exactly which code*. Together (`1.1.0-1c2103c`) every running container can be traced to one commit and one Jenkins build.

C. What was actually implemented

Tag calculation (Jenkinsfile, Checkout stage):

```
env.APP_VERSION  = readFile('VERSION').trim()                                    // 1.1.0
env.GIT_SHORT    = sh(script: 'git rev-parse --short HEAD', returnStdout: true).trim()  // 1c2103c
env.IMAGE_TAG    = "${env.APP_VERSION}-${env.GIT_SHORT}"                          // 1.1.0-1c2103c
env.IMAGE_PREFIX = params.REGISTRY ? "${params.REGISTRY}/" : ''                   // localhost:5000/
```

Stage by stage (build #11)

Stage	What happened
Build images	`docker build -t localhost:5000/hotel-booking-app:1.1.0-1c2103c --build-arg APP_VERSION=1.1.0 --build-arg GIT_COMMIT=1c2103c .` and the same for `hotel-booking-nginx` - **only after** both test stages passed
Publish images	`docker push` both; console shows `1.1.0-1c2103c: digest: sha256:a0386a85...` and `sha256:4ba01355...`
Deploy	with `SECRET_KEY` from the Jenkins credential: `docker compose pull` (fetch exactly the pushed images) then `docker compose up -d --no-build --wait` (no rebuild on deploy)
Health check	`/health` must contain `"version":"1.1.0"` **and** `"commit":"1c2103c"`
Record	`reports/deployment.txt` archived (Figure 12.4)

Registry login logic (Jenkinsfile lines 148-155): if `REGISTRY` starts with `localhost` no login is needed; otherwise `withCredentials([usernamePassword(credentialsId: 'registry-creds', ...)])` and `echo "$REG_PASS" | docker login -u "$REG_USER" --password-stdin ...` - the password never appears in the Jenkinsfile or the log.

Never untested or uncommitted code

- â€¢ Images are built only after lint and all 90 tests pass (stage order).
- â€¢ Jenkins checks out **commits** (`checkout scm`); uncommitted files on a laptop can never reach Jenkins.
- â€¢ Deploy uses `--no-build`: it runs the image that was pushed, not a fresh build.
- â€¢ The health check fails the build if the running commit differs.

What is deployed right now (checked 2026-10-20, Figure 12.1):

Environment	Image	/health
staging :8081	`localhost:5000/hotel-booking-app:1.1.0-1c2103c` (build #11)	`{"commit":"1c2103c","env":"staging",...,"version":"1.1.0"}`
prod :8082	`localhost:5000/hotel-booking-app:1.1.0-d07929b` (build #8, release v1.1.0)	`{"commit":"d07929b","env":"prod",...,"version":"1.1.0"}`

`d07929b` is the original ID of the v1.1.0 release commit (current ID `ce1726d`; commit messages were edited once before publishing - see `docs/evidence/git/commit-id-map.md`).

Registry contents - 8 tags of `hotel-booking-app`, one per successful build: `1.0.0-64351fe`, `1.0.0-30c6a05`, `1.0.0-224384f`, `1.0.0-661751d`, `1.1.0-5aa909d`, `1.1.0-d07929b`, `1.1.0-2e4c93a`, `1.1.0-1c2103c`. Old tags are kept, which is what makes rollback possible (Task 14).

How to switch to Docker Hub (not done): create an access token on Docker Hub; in Jenkins *Manage Jenkins -> Credentials -> Add* "Username with password", ID `registry-creds`; build with `REGISTRY=docker.io/<your-docker-id>`.

D. Screenshots and evidence

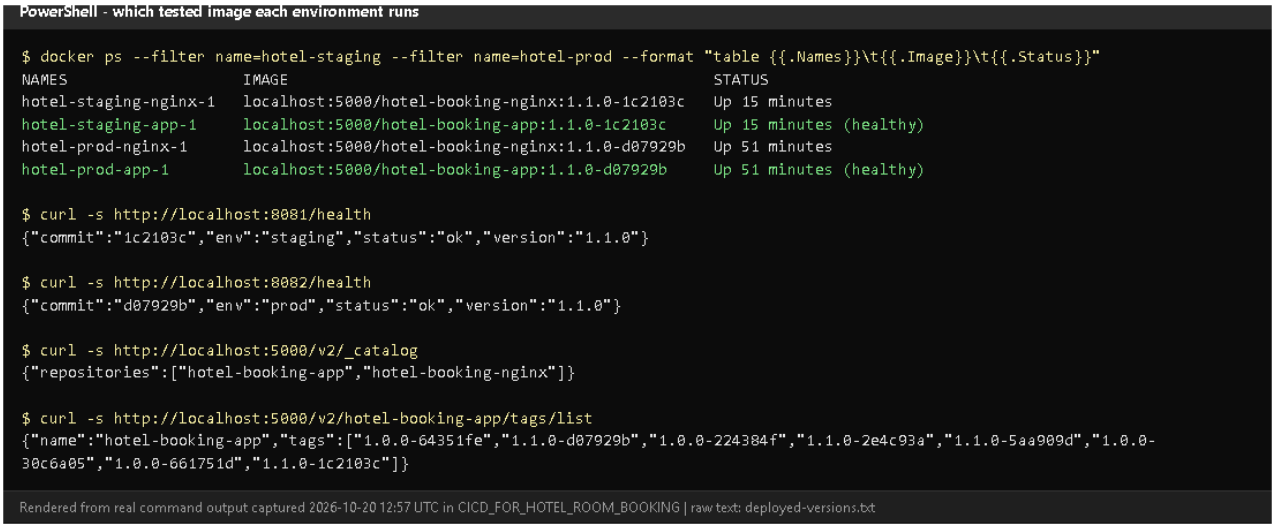


Figure 12.1 - Which image each environment runs, both `/health` answers, and the registry catalogue and tags (rendered from real output, 2026-10-20). **Proves:** images are tagged version-commit, stored in the registry and the deployed containers run them.

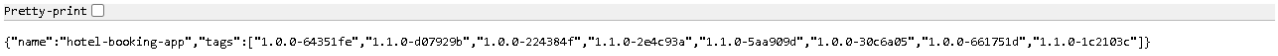


Figure 12.2 - http://localhost:5000/v2/hotel-booking-app/tags/list in the browser.



Figure 12.3 - http://localhost:8082/health: prod runs release 1.1.0 (`d07929b`).

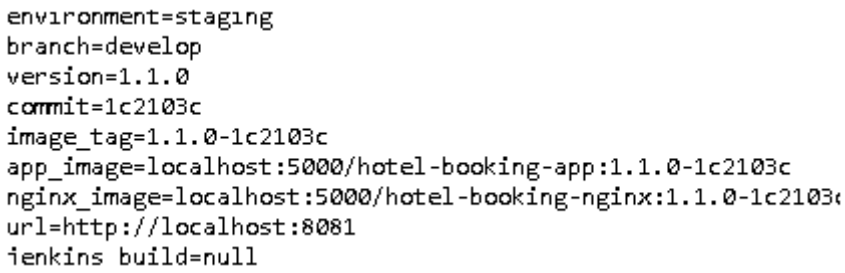


Figure 12.4 - `reports/deployment.txt` archived by build #11: environment, branch, version, commit, image names, URL.
(`jenkins_build=null`: the Jenkins URL setting is not configured - see Task 8.)

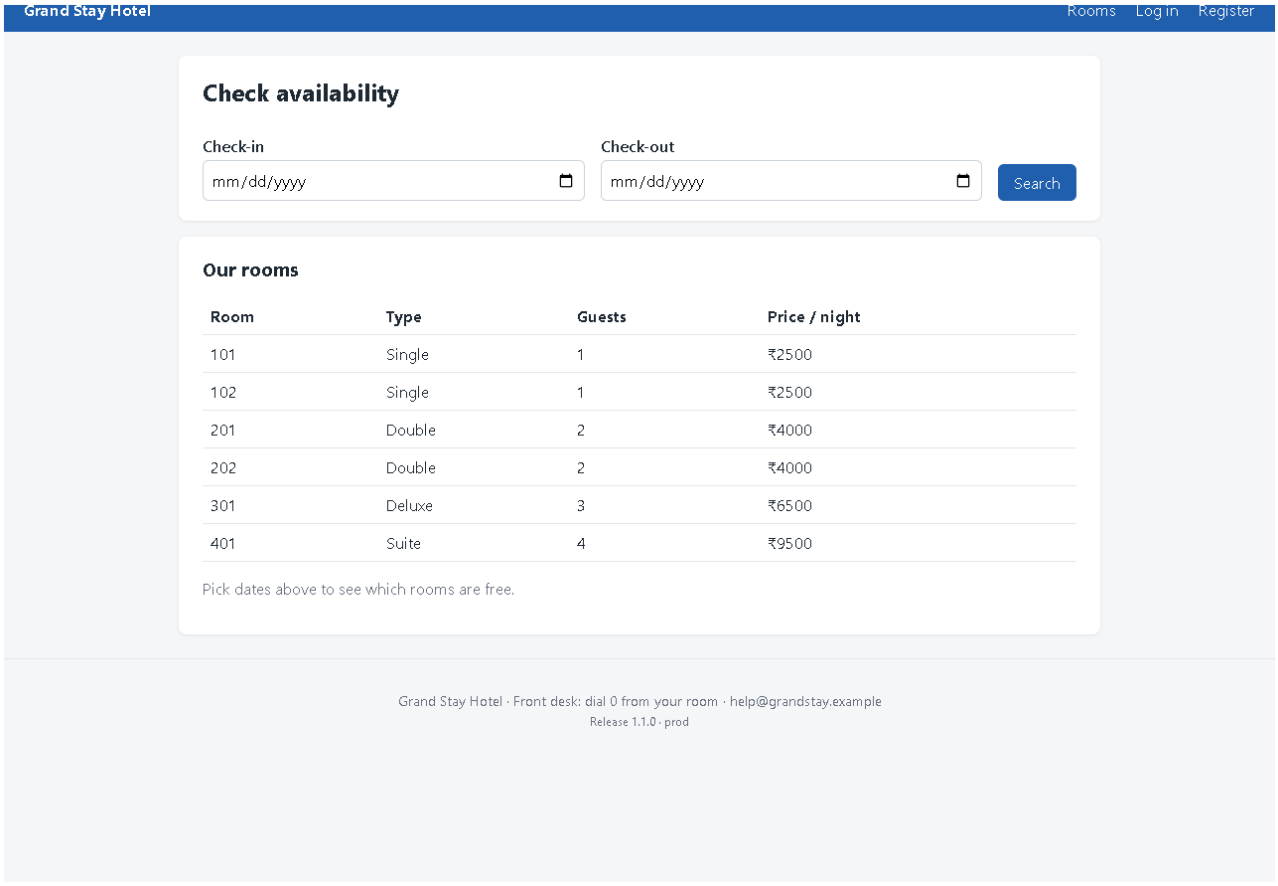


Figure 12.5 - The prod portal at http://localhost:8082 - the deployed v1.1.0 interface (the newer UI redesign is not deployed yet).

E. How to demonstrate this task to the professor (about 5 minutes)

Before the demo: Docker Desktop, Jenkins and the registry running (`docker compose ps` in `deploy\jenkins`).

78. Jenkins -> *hotel-booking* -> *Build with Parameters* -> `BRANCH = develop`, `APP_ENV = staging`, `REGISTRY = localhost:5000` -> *Build*.
79. While it runs, open the `Jenkinsfile` at the Checkout stage and explain `IMAGE_TAG = <VERSION>-<commit>`.
80. After it finishes: **Console Output**, Ctrl+F `digest:` - *Say:* "Both images were pushed to the registry."
81. Browser: http://localhost:5000/v2/hotel-booking-app/tags/list - the new tag is in the list.
82. PowerShell:*Say:* "The staging container runs the image from the registry with this exact tag, and `/health` reports the same commit."

```
docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}"
curl.exe -s http://localhost:8081/health
```

83. Open the build's **Build Artifacts -> reports/deployment.txt**.
84. *Say about Docker Hub:* "For Docker Hub I would add a `registry-creds` credential and set REGISTRY to my Docker ID; that path is implemented but not tested because I have no Docker Hub account."

Recovery: Publish stage "connection refused" - the registry container is stopped: `docker compose up -d registry` in `deploy\jenkins`. Deploy uses an old container - another stack on the same port; `docker compose -p hotel-staging down`, then build again.

F. Expected screenshots for the final submission

83. `assets/task-12/deployed-versions.png` - **exists, 2026-10-20**.

84. `assets/task-12/registry-tags.png` - **exists, 2026-10-20**.
85. `assets/task-12/prod-health.png` - **exists, 2026-10-20**.
86. `assets/task-12/jenkins-build-11-deployment-record.png` - **exists, 2026-10-20**.
87. `assets/task-12/prod-portal-v1.1.0.png` - **exists, 2026-10-20**.
88. `assets/task-12/dockerhub-repository.png` - the image on Docker Hub - **requires manual capture after publishing to Docker Hub** (pending external action).

G. Probable viva questions and answers

74. **CI vs CD?** CI builds and tests every change; CD goes further and automatically releases/deploys every change that passed.
75. **Why tag with version and SHA?** The version names the release; the SHA identifies the exact code. `1.1.0-1c2103c` links a container to one commit and one build.
76. **Registry vs Docker Hub?** A registry is any server that stores images; Docker Hub is one hosted public registry. We use our own local registry.
77. **Did you publish to Docker Hub?** No - it is implemented (`registry-creds`

    â€“ REGISTRY parameter) but not tested; the local registry was demonstrated.
78. **How do you ensure only tested code is deployed?** Images are built after all tests pass, from a committed checkout; deploy uses `--no-build` with the pushed image; the health check compares the commit.
79. **(Project-specific) What runs in prod now and how do you know?** `1.1.0-d07929b` - `docker ps` shows the image and `/health` returns `"commit":"d07929b"`.
80. **(Project-specific) Why `docker compose pull` before `up`?** To deploy exactly what is in the registry - the same thing any other server would get.
81. **Where are registry credentials kept?** In Jenkins credentials (`registry-creds`), passed with `--password-stdin`; not needed for the local registry.
82. **Why keep old images?** For rollback: deploying an older tag needs no rebuild (shown in Task 14).
83. **Local registry limitation?** Only this machine can pull from it; a separate server would need a hosted registry (Docker Hub) or network access to it.

H. Common mistakes and troubleshooting

Symptom	Cause	Inspect	Fix	Confirm
`push` fails "connection refused"	registry stopped	`docker compose ps` (deploy\jenkins)	`docker compose up -d registry`	push ok
Docker Hub push "unauthorized"	missing/wrong `registry-creds`	Jenkins credentials	add a Docker Hub access token as `registry-creds`	push ok
Health check: wrong commit	stale container	`docker ps` image tag	rebuild; ensure no other stack on the port	commit matches
Tag shows `unknown`	`GIT_COMMIT` not passed	Build images stage	keep `--build-arg GIT_COMMIT=$GIT_SHORT`	`/health` shows SHA
Registry fills the disk	every build adds a tag	`docker system df`	delete old tags you no longer need for rollback	space recovered

I. Quick revision notes

Five key points

26. After tests: build -> tag `<version>-<sha>` -> push -> pull -> deploy -> health -> record.
27. Local registry `localhost:5000` holds 8 tags; old tags enable rollback.
28. Deploy uses `--no-build`: the tested, pushed image is what runs.
29. Credentials via Jenkins (`hotel-secret-key`, `registry-creds`), never in Git.
30. Docker Hub: implemented, **not tested** (pending account).

30-60 second explanation "When all tests pass, Jenkins builds the app and Nginx images and tags them with the release version and commit, for example 1.1.0-1c2103c. It pushes them to our local Docker registry, then deploys by pulling exactly those images and starting them with Docker Compose. The health check confirms the running app reports the same version and commit, and a deployment record is archived with the build. Staging and prod run images from the registry. Docker Hub is supported by a parameter and a credential but hasn't been tested."

Caveat: only a local registry was used; Docker Hub publishing is pending.

Readiness checklist

- â€¢ â˜ I can show the registry tag list in the browser.
- â€¢ â˜ I can explain version + SHA tags.
- â€¢ â˜ I can match `docker ps` image tags with `/health`.
- â€¢ â˜ I can explain honestly why Docker Hub wasn't used.

Task 13 - Configuration Management with Ansible

A. Task title and objective

Task 13: Configuration Management with Ansible

Objective: describe in code how a server must be set up - packages, directories, configuration, permissions, services, ports, Nginx and health verification - and let Ansible bring a server to that state automatically, using variables for environment-specific values.

Professor's required deliverables

- â€¢ Ansible inventory and playbook for the selected target
- â€¢ Automated prerequisites: packages, deployment directories, application configuration, service/container setup, file permissions, network ports, Nginx configuration, application health verification
- â€¢ Variables for environment-specific configuration
- â€¢ No destructive changes; SSH/firewall settings not changed without checking
- â€¢ If no separate target exists: the safest local/virtualised setup, documented

Current status: Implemented and verified (container-based lab). On 2026-10-20 the playbook provisioned a **fresh** lab server from scratch (Docker not installed beforehand): `ok=14 changed=7 failed=0`, and `/health` answered with release 1.0.0 (Figure 13.1).

Target honesty: the "server" is a disposable **Ubuntu 24.04 Docker container with systemd and SSH** (`deploy/ansible/lab/`), not a separate virtual machine or cloud server. The real-server inventory (`deploy/ansible/inventory.ini`, `192.168.56.10`) has **not** been used.

B. Explanation for a complete beginner

An everyday picture. Instead of telling a new employee "install this, then create that folder, then copy these files...", you give them a written checklist that describes the *finished* state of the office. Anyone following it gets the same result - and if something is already done, they skip it. **Ansible** follows such a checklist on servers, over SSH.

Term	Meaning	In this project
Configuration management	describing a machine's desired state in files and applying it automatically	`deploy/ansible/`
Control node	the machine that runs Ansible	the `hotel-ansible` container (Ansible doesn't run natively on Windows)
Managed node / target	the server being configured	`hotel-server` (the lab container)
Inventory	the list of servers, in groups	`inventory.ini` (real) / `lab/inventory.ini` (lab) - group `[web]`
Playbook	a YAML file listing tasks for a group of hosts	`playbook.yml`, `rollback.yml`
Task	one desired state, using a **module**	"Install Docker engine..." using `apt`
Module	a built-in action (apt, copy, file, service, uri...)	10 built-in modules, no extra collections
Variables	values that differ per environment	`group_vars/web.yml`
become	run tasks with administrator rights (sudo)	`become: true`
Handler	a task that runs only when notified by a change	**not used** in this project (compose restarts containers itself when their image or settings change)

Term	Meaning	In this project
Idempotent	running it again changes nothing if the state is already right	see Task 14 (`changed=0`)
Agentless	nothing needs installing on the server except SSH and Python	true for Ansible

Playbook vs inventory: the inventory says *where* (which servers); the playbook says *what* (which state).

C. What was actually implemented

Files (`deploy/ansible/`)

File	Content
`inventory.ini`	`[web]` group: `hotel-server ansible_host=192.168.56.10 ansible_user=ubuntu` (edit for a real server)
`lab/inventory.ini`	the same group pointing at the lab container `target` (lab-only password, not reachable from outside Docker)
`group_vars/web.yml`	`app_dir: /opt/hotel-booking`, `app_env: prod`, `host_port: 80`, `app_version` (read from the `VERSION` file), `image_tag`, `git_commit` (from Git, `-dirty` if uncommitted changes), `secret_key` (generated once by the password lookup into the git-ignored `credentials/secret_key`)
`ansible.cfg`	inventory path, `interpreter_python = auto_silent`, `host_key_checking = False` (lab setting, commented)
`playbook.yml`	provision + deploy (14 tasks, below; the run output also shows Ansible's automatic "Gathering Facts")
`rollback.yml`	return to the previous release (Task 14)
`Dockerfile`	the control-node image (`python:3.12-slim` + `ansible-core 2.21.5` + ssh + git)
`lab/docker-compose.yml`, `lab/Dockerfile.target`	the disposable Ubuntu 24.04 target (systemd, sshd, privileged, port 8088 -> 80) and the controller

Tasks in `playbook.yml` and the deliverable each covers

Line	Task	Module	Deliverable
33	Install Docker engine and the Docker Compose plugin	`apt`	packages
42	Make sure Docker is running and starts on boot	`service`	service setup
49	Create application directories (`/opt/hotel-booking`, `.../deploy`, root 0755)	`file`	directories, permissions
60	Copy application source and Docker files	`copy`	application files
70-83	Read current `.env`, work out the running release, remember it in `previous_release`	`slurp`, `set_fact`, `copy`	release tracking (for rollback)

Line	Task	Module	Deliverable
90	Write the environment file (`SECRET_KEY`, `APP_ENV`, `HOST_PORT`, version, `IMAGE_TAG`) **mode 0600, `no_log: true`**	`copy`	application configuration, permissions, secret protection
109	Check whether images for this release and commit already exist	`command` (`docker image inspect`)	idempotency
119	Build the Docker images on the server (only if needed)	`command`	container setup
126	Start (or update) the containers and wait until healthy	`command` (`docker compose up -d --no-build --wait`)	container/service setup
134	Check the portal answers through Nginx	`uri` with retries	health verification
143	Check the expected release is the one running	`assert`	health verification
149	Show result	`debug`	-

Nginx and ports: Nginx runs as a container whose configuration is baked into the `hotel-booking-nginx` image (`deploy/nginx/hotel.conf`); it publishes `host_port` (80) on the server. **Not touched on purpose:** SSH settings and firewall rules (the playbook says so in its header). If `ufw` is active, open `host_port` yourself after checking what else is exposed.

What the first run on the fresh lab server changed (2026-10-20, `changed=7`): Install Docker; start/enable Docker; create directories; copy files; write `.env`; build images; start containers. Then `/health` = `{"commit":"224de14","env":"prod","status":"ok","version":"1.0.0"}`.

State of the server afterwards (Figure 13.5): `.env` is `600 root`, `previous_release` exists, images for 1.0.0 and 1.1.0 are present, the containers are healthy and Docker is enabled at boot.

D. Screenshots and evidence

Ansible run 1 - first provisioning of a fresh server (v1.0.0)

```
# 2026-10-20 12:44 UTC - controller repository: rel-v1.0.0 (224de14)
$ ansible-playbook -i lab/inventory.ini playbook.yml

PLAY [Provision server and deploy the Hotel Booking Portal] ******************

TASK [Gathering Facts] *******************************************************
ok: [hotel-server]

TASK [Install Docker engine and the Docker Compose plugin] *******************
changed: [hotel-server]

TASK [Make sure Docker is running and starts on boot] ************************
changed: [hotel-server]

TASK [Create application directories] ****************************************
changed: [hotel-server] => (item=/opt/hotel-booking)
changed: [hotel-server] => (item=/opt/hotel-booking/deploy)

TASK [Copy application source and Docker files] ******************************
changed: [hotel-server] => (item={'src': 'app', 'dest': '.'})
changed: [hotel-server] => (item={'src': 'wsgi.py', 'dest': '.'})
changed: [hotel-server] => (item={'src': 'gunicorn.conf.py', 'dest': '.'})
changed: [hotel-server] => (item={'src': 'requirements.txt', 'dest': '.'})
changed: [hotel-server] => (item={'src': 'VERSION', 'dest': '.'})
changed: [hotel-server] => (item={'src': 'Dockerfile', 'dest': '.'})
changed: [hotel-server] => (item={'src': '.dockerignore', 'dest': '.'})
changed: [hotel-server] => (item={'src': 'docker-compose.yml', 'dest': '.'})
changed: [hotel-server] => (item={'src': 'certs', 'dest': '.'})
changed: [hotel-server] => (item={'src': 'deploy/nginx', 'dest': 'deploy'})

TASK [Read the currently deployed configuration (if any)] ********************
ok: [hotel-server]

TASK [Work out the currently deployed release] *******************************
ok: [hotel-server]

TASK [Remember the previous release for rollback] ****************************
skipping: [hotel-server]

TASK [Write the environment file read by docker compose] *********************
changed: [hotel-server]

TASK [Check whether images for this release and commit already exist] **********
ok: [hotel-server]

TASK [Build the Docker images on the server] *********************************
changed: [hotel-server]

TASK [Start (or update) the containers and wait until healthy] ***************
changed: [hotel-server]

TASK [Check the portal answers through Nginx] ********************************
ok: [hotel-server]

TASK [Check the expected release is the one running] *************************
ok: [hotel-server]

TASK [Show result] ***********************************************************
ok: [hotel-server] => {
    "msg": "Hotel Booking Portal is up on port 80: {'commit': '224de14', 'env': 'prod', 'status': 'ok', 'version': '1.0.0'}"
}

PLAY RECAP *******************************************************************
hotel-server               : ok=14   changed=7    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
```

(part 1 of 2)

```
$ curl -s http://localhost:8088/health    # from the Windows host
{"commit":"224de14","env":"prod","status":"ok","version":"1.0.0"}
```

Rendered from the real log file docs/task-reports/assets/raw/ansible-run1-first-deploy-v1.0.0.txt (unedited lines)

(part 2 of 2)

Figure 13.1 - Run 1 on a fresh lab server, from a clean clone of tag v1.0.0 (rendered from the real run on 2026-10-20). **Notice:** each task's `changed`/`ok`, the censored `.env` task (`no_log`), the final message with `/health`, and `PLAY RECAP ... ok=14 changed=7 failed=0`. **Proves:** Ansible installed prerequisites and deployed the application automatically.

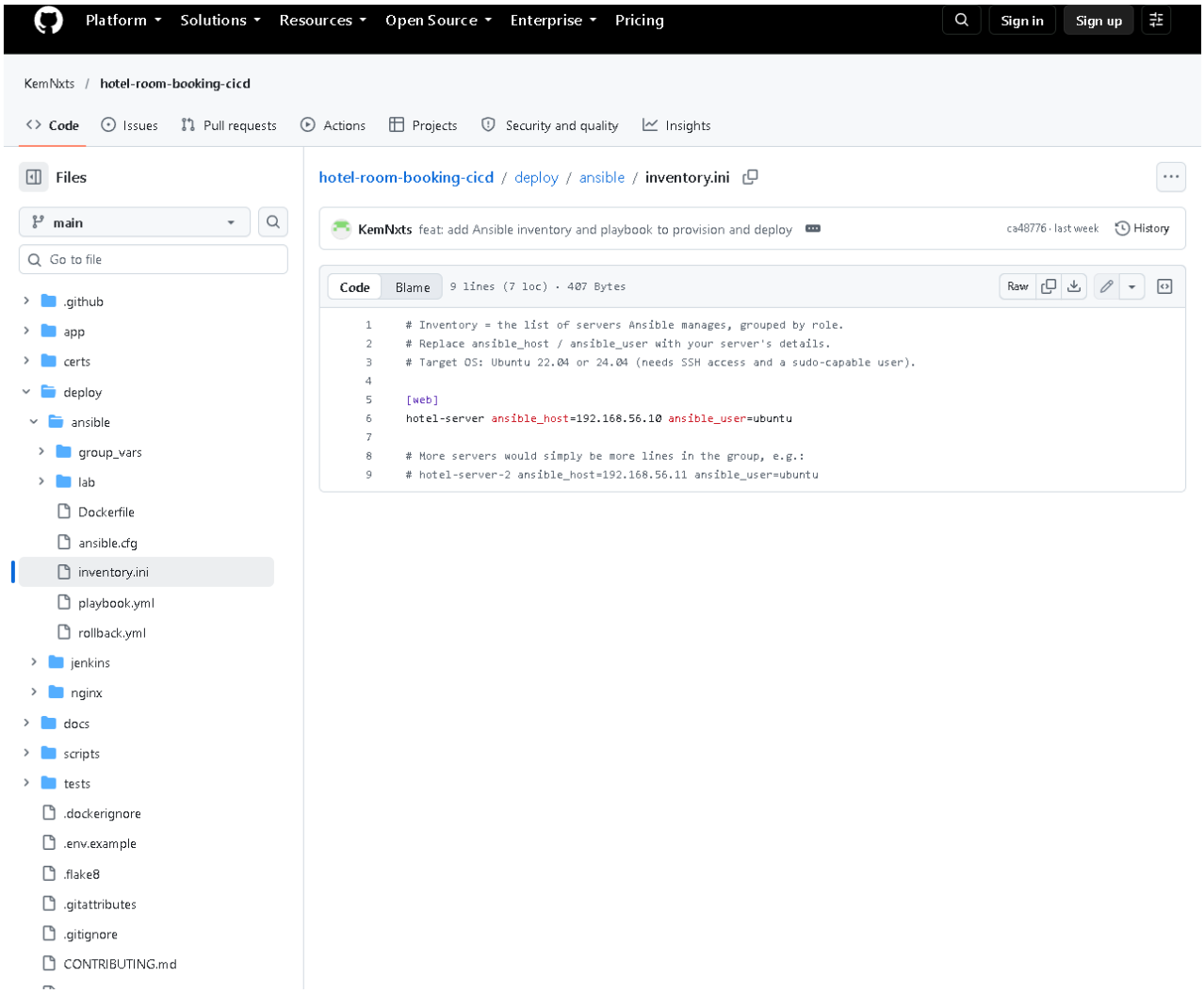


Figure 13.2 - `deploy/ansible/inventory.ini` on GitHub - group `[web]`.

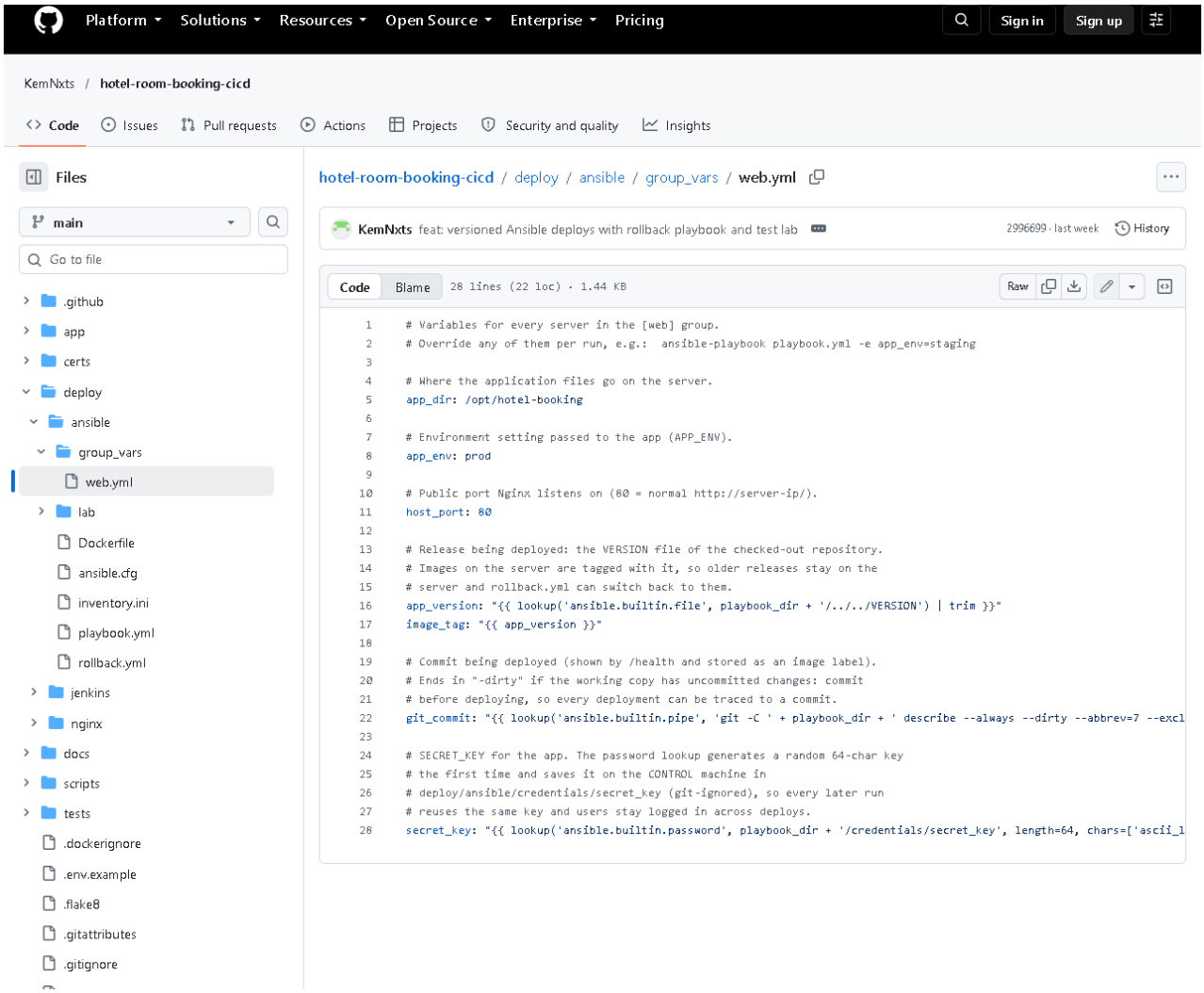
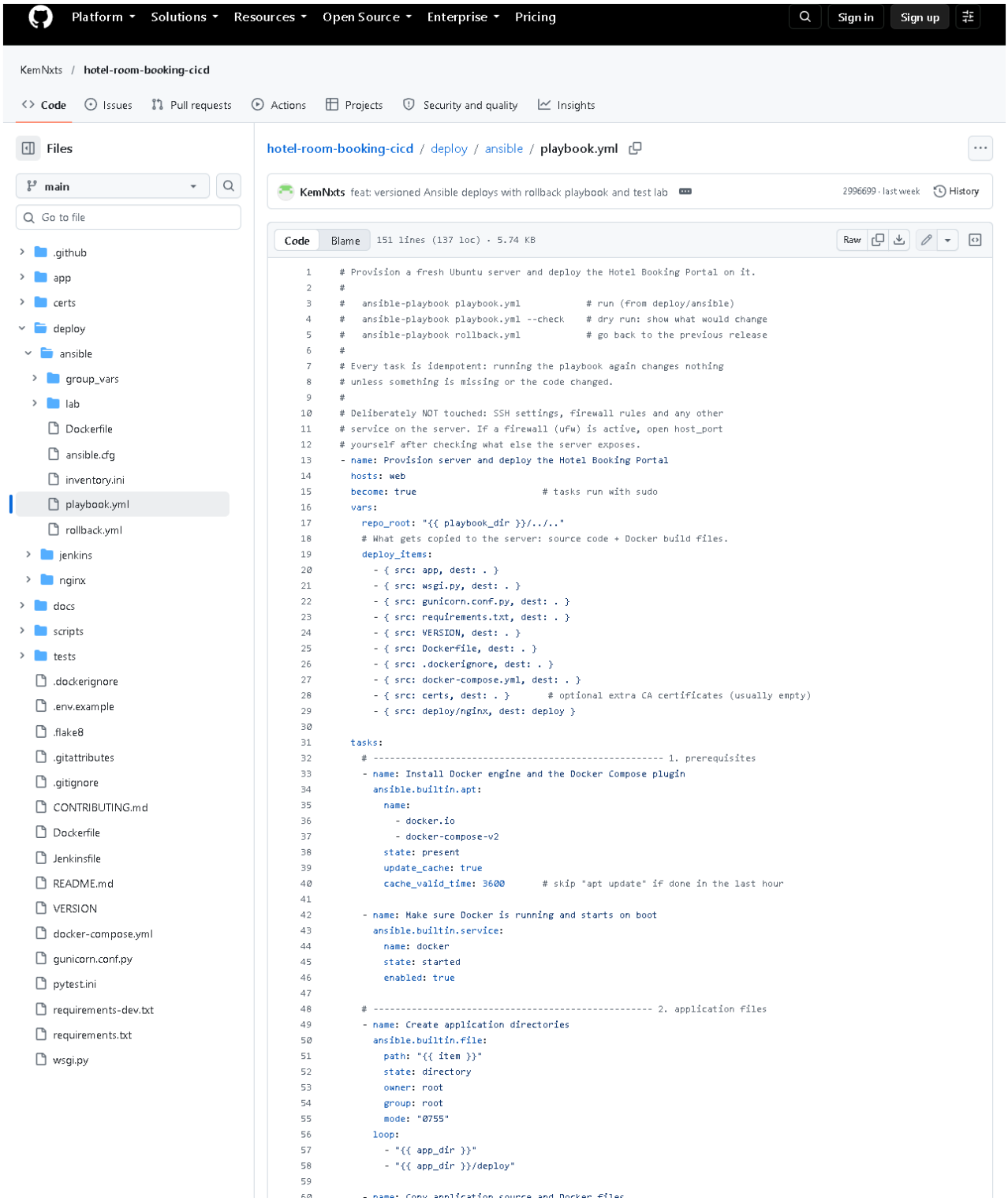


Figure 13.3 - `group_vars/web.yml`: environment-specific values, the version taken from `VERSION`, and the generated secret key.


Platform
Solutions
Resources
Open Source
Enterprise
Pricing
Sign in
Sign up
KemNxts / hotel-room-booking-cicd
Code
Issues
Pull requests
Actions
Projects
Security and quality
Insights
Files
main
Go to file
.github
app
certs
deploy
ansible
group_vars
lab
Dockerfile
ansible.cfg
inventory.ini
playbook.yml
rollback.yml
jenkins
nginx
docs
scripts
tests
.dockerignore
.env.example
.flake8
.gitattributes
.gitignore
CONTRIBUTING.md
Dockerfile
Jenkinsfile
README.md
VERSION
docker-compose.yml
gunicorn.conf.py
pytest.ini
requirements-dev.txt
requirements.txt
wsgi.py
hotel-room-booking-cicd / deploy / ansible / playbook.yml
KemNxts feat: versioned Ansible deploys with rollback playbook and test lab
2996699 · last week
History
Code
Blame
151 lines (137 loc) · 5.74 KB
Raw
Provision a fresh Ubuntu server and deploy the Hotel Booking Portal on it.
#
ansible-playbook playbook.yml   # run (from deploy/ansible)
ansible-playbook playbook.yml --check   # dry run: show what would change
ansible-playbook rollback.yml   # go back to the previous release
#
Every task is idempotent: running the playbook again changes nothing
unless something is missing or the code changed.
#
Deliberately NOT touched: SSH settings, firewall rules and any other
service on the server. If a firewall (ufw) is active, open host_port
yourself after checking what else the server exposes.
- name: Provision server and deploy the Hotel Booking Portal
  hosts: web
  become: true   # tasks run with sudo
  vars:
    repo_root: "{{ playbook_dir }}/../.."
    # What gets copied to the server: source code + Docker build files.
    deploy_items:
      - { src: app, dest: . }
      - { src: wsgi.py, dest: . }
      - { src: gunicorn.conf.py, dest: . }
      - { src: requirements.txt, dest: . }
      - { src: VERSION, dest: . }
      - { src: Dockerfile, dest: . }
      - { src: .dockerignore, dest: . }
      - { src: docker-compose.yml, dest: . }
      - { src: certs, dest: . }   # optional extra CA certificates (usually empty)
      - { src: deploy/nginx, dest: deploy }

  tasks:
    # -- 1. prerequisites
    - name: Install Docker engine and the Docker Compose plugin
      ansible.builtin.apt:
        name:
          - docker.io
          - docker-compose-v2
        state: present
        update_cache: true
        cache_valid_time: 3600   # skip "apt update" if done in the last hour

    - name: Make sure Docker is running and starts on boot
      ansible.builtin.service:
        name: docker
        state: started
        enabled: true

    # -- 2. application files
    - name: Create application directories
      ansible.builtin.file:
        path: "{{ item }}"
        state: directory
        owner: root
        group: root
        mode: "0755"
      loop:
        - "{{ app_dir }}"
        - "{{ app_dir }}/deploy"

(part 1 of 3)

```
    - name: Copy application source and Docker files
      ansible.builtin.copy:
        src: "{{ repo_root }}/{{ item.src }}"
        dest: "{{ app_dir }}/{{ item.dest }}/"
        owner: root
        group: root
        mode: preserve
      loop: "{{ deploy_items }}"

    # Remember what is running now, so rollback.yml can return to it.
    - name: Read the currently deployed configuration (if any)
      ansible.builtin.slurp:
        src: "{{ app_dir }}/.env"
      register: current_env
      failed_when: false
      no_log: true                  # the file contains SECRET_KEY

    - name: Work out the currently deployed release
      ansible.builtin.set_fact:
        current_tag: "{{ (current_env.content | b64decode | regex_search('(?m)^IMAGE_TAG=(\\S+)', '\\1') | first)
                         if current_env.content is defined else '' }}"
      no_log: true

    - name: Remember the previous release for rollback
      ansible.builtin.copy:
        dest: "{{ app_dir }}/previous_release"
        content: "{{ current_tag }}\n"
        mode: "0644"
      when: current_tag | length > 0 and current_tag != image_tag

    - name: Write the environment file read by docker compose
      ansible.builtin.copy:
        dest: "{{ app_dir }}/.env"
        content: |
          SECRET_KEY={{ secret_key }}
          APP_ENV={{ app_env }}
          HOST_PORT={{ host_port }}
          APP_VERSION={{ app_version }}
          GIT_COMMIT={{ git_commit }}
          IMAGE_TAG={{ image_tag }}
        owner: root
        group: root
        mode: "0600"                # only root can read the secret
      no_log: true                  # never print the secret in Ansible output

    # ------------------------------------------------------------ 3. deploy
    # Build only if this release + commit has not been built yet. (Even a
    # fully cached rebuild gets a new image id, which would make compose
    # restart the containers on every run - not idempotent.)
    - name: Check whether images for this release and commit already exist
      ansible.builtin.command:
        cmd: >-
          docker image inspect --format '{{ '{{' }}index .Config.Labels "org.opencontainers.image.revision"{{ '}}' }}'
          hotel-booking-app:{{ image_tag }} hotel-booking-nginx:{{ image_tag }}
      register: existing_image
      changed_when: false
      failed_when: false
      tags: [build]

    - name: Build the Docker images on the server
      ansible.builtin.command:
        cmd: docker compose build
        chdir: "{{ app_dir }}"
      when: existing_image.rc != 0 or existing_image.stdout_lines[0] != git_commit
      tags: [build]

    - name: Start (or update) the containers and wait until healthy
      ansible.builtin.command:
        cmd: docker compose up -d --no-build --wait --wait-timeout 180
        chdir: "{{ app_dir }}"
      register: up
      # compose only re-creates containers whose image or settings changed.
      changed_when: "'Started' in up.stderr or 'Recreated' in up.stderr"

    - name: Check the portal answers through Nginx
      ansible.builtin.uri:
        url: "http://127.0.0.1:{{ host_port }}/health"
```

(part 2 of 3)

```
137             return_content: true
138           register: health
139           retries: 10
140           delay: 3
141           until: health.status == 200
142
143         - name: Check the expected release is the one running
144           ansible.builtin.assert:
145             that: health.json.version == app_version
146             fail_msg: "Expected version {{ app_version }} but /health says {{ health.json.version }}"
147             quiet: true
148
149         - name: Show result
150           ansible.builtin.debug:
151             msg: "Hotel Booking Portal is up on port {{ host_port }}: {{ health.json }}"
```

(part 3 of 3)

Figure 13.4 - `playbook.yml` on GitHub: the tasks listed above.

Docker - state of the lab server after the Ansible runs

```
$ docker compose -f deploy/ansible/lab/docker-compose.yml exec -T target sh -c "stat -c '%a %U %n' /opt/hotel-booking/.env
/opt/hotel-booking/previous_release; cat /opt/hotel-booking/previous_release; ls /opt/hotel-booking; docker image ls --format
'{{.Repository}}:{{.Tag}}' | grep hotel; docker ps --format '{{.Names}} {{.Image}} {{.Status}}'; systemctl is-enabled docker"
600 root /opt/hotel-booking/.env
644 root /opt/hotel-booking/previous_release
1.1.0
Dockerfile
VERSION
app
certs
deploy
docker-compose.yml
gunicorn.conf.py
previous_release
requirements.txt
wsgi.py
hotel-booking-app:1.1.0
hotel-booking-nginx:1.1.0
hotel-booking-nginx:1.0.0
hotel-booking-app:1.0.0
hotel-booking-nginx-1 hotel-booking-nginx:1.0.0 Up 7 minutes
hotel-booking-app-1 hotel-booking-app:1.0.0 Up 7 minutes (healthy)
enabled
```

Rendered from real command output captured 2026-10-20 12:57 UTC in CICD_FOR_HOTEL_ROOM_BOOKING | raw text: lab-server-state.txt

Figure 13.5 - Inside the lab server (rendered from real output, 2026-10-20): file permissions (`600 root .env`), the files in `/opt/hotel-booking`, images, running containers, `docker` enabled.

E. How to demonstrate this task to the professor (about 5 minutes)

Before the demo: Docker Desktop running. PowerShell. These commands were run exactly like this on 2026-10-20.

```
cd D:\DevOPS\CICD_FOR_HOTEL_ROOM_BOOKING\deploy\ansible\lab

# 1. a brand-new "server" (only if you want to start from zero - this deletes the old lab)
docker compose down -v
docker compose up -d --build target
docker compose exec target sh -c "command -v docker || echo no docker yet"
```

Expected: "no docker yet". *Say:* "This is a fresh Ubuntu server with nothing installed."

```
# 2. run the playbook (takes 3-5 minutes the first time)
docker compose run --rm controller ansible-playbook -i lab/inventory.ini playbook.yml
```

Point at: the task names scrolling by, `changed` vs `ok`, the censored `.env` task, the final "Hotel Booking Portal is up" message and `PLAY RECAP ... failed=0`.

```
# 3. prove it
curl.exe -s http://localhost:8088/health
docker compose exec target stat -c "%a %U %n" /opt/hotel-booking/.env
```

Expected: `{"status":"ok",...}` and `600 root /opt/hotel-booking/.env`. Open http://localhost:8088 in the browser - the portal served from the "server".

86. Show `inventory.ini`, `group_vars/web.yml` and `playbook.yml` in VS Code and map a few tasks to the deliverables (packages, directories, config, health).
87. *Say honestly:* "The target is a container acting as a server because I had no VM; for a real server I put its IP in `inventory.ini` and run the same playbook."

For a real Ubuntu server (not done in this project): from the repository root

```
docker build -t hotel-ansible -f deploy/ansible/Dockerfile .
docker run --rm -it -v "${PWD}:/work" hotel-ansible ansible-playbook playbook.yml --ask-pass --ask-become-pass
```

Recovery: "UNREACHABLE" - the target container isn't running: `docker compose up -d target` and wait ~10 s. First run slow - it downloads Docker packages and base images. `CERTIFICATE_VERIFY_FAILED` during the image build on the server - antivirus HTTPS scanning; the `certs/` folder is copied to the server.

F. Expected screenshots for the final submission

85. `assets/task-13/ansible-run1-first-deploy-v1.0.0.png` - **exists, 2026-10-20** (rendered from the real run).
86. `assets/task-13/github-file-deploy_ansible_inventory.ini.png` - **exists, 2026-10-20**.
87. `assets/task-13/github-file-deploy_ansible_group_vars_web.yml.png` - **exists, 2026-10-20**.
88. `assets/task-13/github-file-deploy_ansible_playbook.yml.png` - **exists, 2026-10-20**.
89. `assets/task-13/lab-server-state.png` - **exists, 2026-10-20**.
90. `assets/task-13/real-vm-run.png` - a run against a real Ubuntu VM - **requires manual capture** if a VM becomes available (pending).

G. Probable viva questions and answers

89. **What is configuration management?** Describing the desired state of servers in files and applying it automatically and repeatably.
90. **Playbook vs inventory?** Inventory = which servers; playbook = which tasks/state.
91. **Why Ansible?** Agentless (only SSH + Python on the server), readable YAML, idempotent built-in modules.
92. **What does your playbook set up?** Docker + compose plugin, the docker service, `/opt/hotel-booking` directories, the app files, a root-only `.env`, the images, the containers, and a health/version check.
93. **How are environment differences handled?** Variables in `group_vars/web.yml`, overridable per run, e.g. `-e app_env=staging -e host_port=8080`.
94. **How is the secret protected?** Generated once on the control machine (git-ignored), written to `.env` with mode 0600 and `no_log: true`, so it is never printed.
95. **(Project-specific) What was your target?** A disposable Ubuntu 24.04 container with systemd and SSH, defined in `deploy/ansible/lab/` - not a separate VM.
96. **(Project-specific) What did the first run change?** 7 tasks: install Docker, enable it, create directories, copy files, write `.env`, build images, start containers.
97. **Do you use handlers?** No. Docker Compose recreates containers itself when their image or settings change, so no separate restart handler is needed.
98. **Does the playbook change the firewall or SSH?** No, deliberately - those are risky to automate without knowing the server; it is documented.
99. **Why `host_key_checking = False`?** Convenience for fresh lab machines; for real servers it should be True with known hosts.

H. Common mistakes and troubleshooting

Symptom	Cause	Inspect	Fix	Confirm
`UNREACHABLE`	target not running / wrong host	`docker compose ps` in `lab`	`docker compose up -d target`	`ok=` in recap
"Missing sudo password" (real server)	become needs a password	inventory	add `--ask-become-pass`	tasks run
Ansible ignores `ansible.cfg`	world-writable folder (Windows mount)	-	the controller image sets `ANSIBLE_CONFIG`	inventory found
Image build fails on the server	TLS interception	build task output	put the root CA in `certs/`	build ok
`assert` fails: wrong version	old containers / wrong `app_version`	`/health`	rerun; check `VERSION`	assert ok

I. Quick revision notes

Five key points

84. Inventory (where) + group_vars (values) + playbook (what) in `deploy/ansible/`.
85. 14 tasks with built-in modules: packages, service, dirs, files, `.env` 0600, build, start, health, assert.
86. Secret generated once, never printed (`no_log`), file mode 0600.
87. Fresh lab server provisioned on 2026-10-20: `changed=7`, `/health` OK.
88. Target is a container lab, not a VM; firewall/SSH deliberately untouched.

30-60 second explanation "Ansible describes the server I want: Docker installed and running, an application folder, the app files, a root-only environment file with the secret, the images built and the containers healthy. The inventory says which server, `group_vars` holds environment values, and the playbook lists the tasks using built-in modules. I ran it against a fresh Ubuntu server in a container lab: it installed everything and deployed the portal, with seven changes and no failures, and the final tasks check `/health` and the version."

Caveat: tested on a container-based lab target, not a separate VM.

Readiness checklist

- â€¢ â˜ I can start the lab and run the playbook.
- â€¢ â˜ I can map tasks to deliverables (packages, dirs, config, health).
- â€¢ â˜ I can explain inventory vs playbook vs variables.
- â€¢ â˜ I can explain the lab-vs-VM limitation.

Task 14 - Automated Provisioning and Reliability Validation (Idempotency and Rollback)

A. Task title and objective

Task 14: Automated Provisioning, Idempotency and Reliability Validation

Objective: prove that automated provisioning is safe to repeat (idempotent), that health checks confirm the application works, and that a bad release can be rolled back to a previous stable release - and that the restored release really works.

Professor's required deliverables

- â€¢ First Ansible execution
- â€¢ Second execution with evidence of idempotency
- â€¢ Application health checks
- â€¢ Recovery / rollback to a previous stable release
- â€¢ Verification that the restored release actually works
- â€¢ Commands, outcomes and limitations recorded (no unproven claims of zero downtime)

Current status: Implemented and verified (container-based lab). The whole sequence was run again on 2026-10-20 on a **fresh** lab server:

Run	What	PLAY RECAP	`/health` afterwards
1	first provisioning, release v1.0.0	`ok=14 changed=7 failed=0`	1.0.0 (`224de14`)
2	upgrade to v1.1.0	`ok=15 changed=5 failed=0`	1.1.0 (`ce1726d`)
3	**same playbook again**	`ok=13 changed=0 failed=0 skipped=2`	1.1.0
4	`rollback.yml`	`ok=11 changed=3 failed=0`	**1.0.0** (`224de14`)
5	v1.0.0's own Selenium journeys against the restored server	**5 passed**	-

Limitation: the target is a privileged Ubuntu 24.04 **container**, not a separate VM. Rollback re-creates the containers, so there is a short interruption - **zero downtime is not claimed**.

B. Explanation for a complete beginner

An everyday picture. A light switch labelled "ON": pressing it when the light is already on changes nothing. A good provisioning script works the same way - running it twice gives the same result as running it once.

Term	Meaning	In this project
Provisioning	preparing a server from nothing to ready (software, folders, config, app)	`playbook.yml` on a fresh server
Idempotency	applying the same operation again changes nothing if the state is already correct	run 3: `changed=0`
Idempotency vs repeatability	*repeatable* = gives the same end result each time; *idempotent* = additionally makes **no changes** when nothing needs changing	our playbook is both
`changed=N`	Ansible's count of tasks that actually modified the server	7 -> 5 -> **0**
Health check	asking the running app whether it works and which version it is	`GET /health` through Nginx + `assert`

Term	Meaning	In this project
Stable release	a tested, tagged version that is known to work	`v1.0.0`, `v1.1.0`
Rollback	switching back to a previous stable release	`rollback.yml`
Deployment vs rollback	deployment moves to a *new* version; rollback returns to an *older*, already-tested one, without rebuilding	-

Why `changed=0` matters: it proves the playbook *checks* before acting. Nothing was reinstalled, rewritten or restarted, so a repeat run is safe - you can run it any time to make sure the server is still correct.

C. What was actually implemented

Making the playbook idempotent

- â€¢ Built-in modules are idempotent by design: `apt state=present` does nothing if Docker is installed; `copy` does nothing if the file content is identical; `file`/`service` likewise.
- â€¢ **Special case - images:** even a fully cached `docker build` produces a new image ID, which would make compose restart containers every time. So the playbook first checks the image label (lines 109-119):

```
- name: Check whether images for this release and commit already exist
  ansible.builtin.command:
    cmd: docker image inspect --format '...org.opencontainers.image.revision...' hotel-booking-app:{{
image_tag }} hotel-booking-nginx:{{ image_tag }}
  register: existing_image
  changed_when: false          # only reads, never counts as a change
  failed_when: false

- name: Build the Docker images on the server
  ansible.builtin.command: { cmd: docker compose build, chdir: "{{ app_dir }}" }
  when: existing_image.rc != 0 or existing_image.stdout_lines[0] != git_commit   # build only if missing
```

(Excerpt simplified: the real `cmd` uses a Jinja-escaped Go template.)

- â€¢ `docker compose up` reports `changed` only if its output contains `Started` or `Recreated`.

Health checks - every run ends with `uri http://127.0.0.1:{{ host_port }}/health` (10 retries) and `assert health.json.version == app_version`.

Rollback mechanism (`deploy/ansible/rollback.yml`)

88. Each deploy tags images with the release (`hotel-booking-app:1.1.0`) and **keeps older images** on the server.
89. Before switching releases, the playbook writes the running release to `/opt/hotel-booking/previous_release`.
90. `rollback.yml` reads that file (or `-e rollback_to=1.0.0`), **refuses** if the target images are not on the server, sets `IMAGE_TAG` in `.env`, saves the release it is leaving (so running it again rolls forward), runs `docker compose up -d --no-build --wait`, then asserts `/health` shows the target version.
91. The database volume is kept, so bookings survive (no release changed the schema).

What each run changed (from the saved logs):

Run	Changed tasks
1 (7)	install Docker; start/enable Docker; create directories; copy files; write `.env`; build images; start containers
2 (5)	copy files (new code); remember previous release (1.0.0); write `.env`; build images; recreate containers
3 (0)	nothing - build skipped, previous-release skipped, everything `ok`
4 (3)	point `.env` at 1.0.0; save 1.1.0 as previous; restart containers on 1.0.0

Verification that the restored release works: after run 4, the Selenium journeys **of release v1.0.0** (from a clean clone of the tag - the current tests expect the newer UI) were run against `http://localhost:8088`: 5 of 5 passed.

Rollback without Ansible (Docker level) - also demonstrated on 2026-10-11 for prod: `IMAGE_TAG=1.0.0-661751d docker compose up -d --no-build --wait` switched prod 1.1.0 -> 1.0.0 and back (`docs/evidence/docker/rollback-prod.txt`).

D. Screenshots and evidence

Ansible run 2 - upgrade to v1.1.0

```
# 2026-10-20 12:47 UTC - controller repository: rel-v1.1.0 (ce1726d)
$ ansible-playbook -i lab/inventory.ini playbook.yml

PLAY [Provision server and deploy the Hotel Booking Portal] ********************

TASK [Gathering Facts] *********************************************************
ok: [hotel-server]

TASK [Install Docker engine and the Docker Compose plugin] *********************
ok: [hotel-server]

TASK [Make sure Docker is running and starts on boot] **************************
ok: [hotel-server]

TASK [Create application directories] ******************************************
ok: [hotel-server] => (item=/opt/hotel-booking)
ok: [hotel-server] => (item=/opt/hotel-booking/deploy)

TASK [Copy application source and Docker files] ********************************
changed: [hotel-server] => (item={'src': 'app', 'dest': '.'})
ok: [hotel-server] => (item={'src': 'wsgi.py', 'dest': '.'})
ok: [hotel-server] => (item={'src': 'gunicorn.conf.py', 'dest': '.'})
ok: [hotel-server] => (item={'src': 'requirements.txt', 'dest': '.'})
changed: [hotel-server] => (item={'src': 'VERSION', 'dest': '.'})
ok: [hotel-server] => (item={'src': 'Dockerfile', 'dest': '.'})
ok: [hotel-server] => (item={'src': '.dockerignore', 'dest': '.'})
ok: [hotel-server] => (item={'src': 'docker-compose.yml', 'dest': '.'})
ok: [hotel-server] => (item={'src': 'certs', 'dest': '.'})
ok: [hotel-server] => (item={'src': 'deploy/nginx', 'dest': 'deploy'})

TASK [Read the currently deployed configuration (if any)] **********************
ok: [hotel-server]

TASK [Work out the currently deployed release] *********************************
ok: [hotel-server]

TASK [Remember the previous release for rollback] ******************************
changed: [hotel-server]

TASK [Write the environment file read by docker compose] ***********************
changed: [hotel-server]

TASK [Check whether images for this release and commit already exist] **********
ok: [hotel-server]

TASK [Build the Docker images on the server] ***********************************
changed: [hotel-server]

TASK [Start (or update) the containers and wait until healthy] *****************
changed: [hotel-server]

TASK [Check the portal answers through Nginx] **********************************
ok: [hotel-server]

TASK [Check the expected release is the one running] ***************************
ok: [hotel-server]

TASK [Show result] *************************************************************
ok: [hotel-server] => {
    "msg": "Hotel Booking Portal is up on port 80: {'commit': 'ce1726d', 'env': 'prod', 'status': 'ok', 'version': '1.1.0'}"
}

PLAY RECAP *********************************************************************
hotel-server               : ok=15   changed=5    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

(part 1 of 2)

```
$ curl -s http://localhost:8088/health    # from the Windows host
{"commit":"ce1726d","env":"prod","status":"ok","version":"1.1.0"}
```

Rendered from the real log file docs/task-reports/assets/raw/ansible-run2-upgrade-v1.1.0.txt (unedited lines)

(part 2 of 2)

Figure 14.1 - Run 2 (upgrade to v1.1.0): `changed=5`, `/health` 1.1.0 (rendered from the real run, 2026-10-20).

Ansible run 3 - same playbook again: changed=0

```
# 2026-10-20 12:48 UTC - controller repository: rel-v1.1.0 (ce1726d)
$ ansible-playbook -i lab/inventory.ini playbook.yml

PLAY [Provision server and deploy the Hotel Booking Portal] ********************

TASK [Gathering Facts] *********************************************************
ok: [hotel-server]

TASK [Install Docker engine and the Docker Compose plugin] *********************
ok: [hotel-server]

TASK [Make sure Docker is running and starts on boot] **************************
ok: [hotel-server]

TASK [Create application directories] ******************************************
ok: [hotel-server] => (item=/opt/hotel-booking)
ok: [hotel-server] => (item=/opt/hotel-booking/deploy)

TASK [Copy application source and Docker files] ********************************
ok: [hotel-server] => (item={'src': 'app', 'dest': '.'})
ok: [hotel-server] => (item={'src': 'wsgi.py', 'dest': '.'})
ok: [hotel-server] => (item={'src': 'gunicorn.conf.py', 'dest': '.'})
ok: [hotel-server] => (item={'src': 'requirements.txt', 'dest': '.'})
ok: [hotel-server] => (item={'src': 'VERSION', 'dest': '.'})
ok: [hotel-server] => (item={'src': 'Dockerfile', 'dest': '.'})
ok: [hotel-server] => (item={'src': '.dockerignore', 'dest': '.'})
ok: [hotel-server] => (item={'src': 'docker-compose.yml', 'dest': '.'})
ok: [hotel-server] => (item={'src': 'certs', 'dest': '.'})
ok: [hotel-server] => (item={'src': 'deploy/nginx', 'dest': 'deploy'})

TASK [Read the currently deployed configuration (if any)] **********************
ok: [hotel-server]

TASK [Work out the currently deployed release] *********************************
ok: [hotel-server]

TASK [Remember the previous release for rollback] ******************************
skipping: [hotel-server]

TASK [Write the environment file read by docker compose] ***********************
ok: [hotel-server]

TASK [Check whether images for this release and commit already exist] **********
ok: [hotel-server]

TASK [Build the Docker images on the server] ***********************************
skipping: [hotel-server]

TASK [Start (or update) the containers and wait until healthy] *****************
ok: [hotel-server]

TASK [Check the portal answers through Nginx] **********************************
ok: [hotel-server]

TASK [Check the expected release is the one running] ***************************
ok: [hotel-server]

TASK [Show result] *************************************************************
ok: [hotel-server] => {
    "msg": "Hotel Booking Portal is up on port 80: {'commit': 'ce1726d', 'env': 'prod', 'status': 'ok', 'version': '1.1.0'}"
}

PLAY RECAP *********************************************************************
hotel-server               : ok=13   changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
```

(part 1 of 2)

```
$ curl -s http://localhost:8088/health    # from the Windows host
{"commit":"ce1726d","env":"prod","status":"ok","version":"1.1.0"}
```

Rendered from the real log file docs/task-reports/assets/raw/ansible-run3-repeat-idempotent.txt (unedited lines)

(part 2 of 2)

Figure 14.2 - Run 3, the same playbook again: every task `ok` or `skipping`, `PLAY RECAP ... changed=0`. **Proves:** idempotency.

Ansible run 4 - rollback.yml back to v1.0.0

```
# 2026-10-20 12:49 UTC - controller repository: rel-v1.1.0 (ce1726d)
$ ansible-playbook -i lab/inventory.ini rollback.yml

PLAY [Roll back the Hotel Booking Portal] ***********************************

TASK [Gathering Facts] ******************************************************
ok: [hotel-server]

TASK [Read the previous release recorded by the last deploy] ****************
ok: [hotel-server]

TASK [Choose the release to go back to] *************************************
ok: [hotel-server]

TASK [Read the current configuration] ***************************************
ok: [hotel-server]

TASK [Work out the release running now] *************************************
ok: [hotel-server]

TASK [Refuse to continue if the target release's images are not on the server] ***
ok: [hotel-server]

TASK [Point the deployment at the target release] ***************************
changed: [hotel-server] => (item=IMAGE_TAG)
changed: [hotel-server] => (item=APP_VERSION)

TASK [Save the release we are leaving, so it can be restored the same way] *****
changed: [hotel-server]

TASK [Restart the containers on the target release and wait until healthy] *****
changed: [hotel-server]

TASK [Check the portal answers through Nginx] *******************************
ok: [hotel-server]

TASK [Verify the restored release is the one running] ***********************
ok: [hotel-server] => {
    "changed": false,
    "msg": "Rolled back from 1.1.0 to 1.0.0: {'commit': '224de14', 'env': 'prod', 'status': 'ok', 'version': '1.0.0'}"
}

PLAY RECAP ******************************************************************
hotel-server               : ok=11   changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0


$ curl -s http://localhost:8088/health    # from the Windows host
{"commit":"224de14","env":"prod","status":"ok","version":"1.0.0"}
```

Rendered from the real log file docs/task-reports/assets/raw/ansible-run4-rollback.txt (unedited lines)

Figure 14.3 - `rollback.yml`: "Rolled back from 1.1.0 to 1.0.0", `changed=3`, `/health` 1.0.0.

Selenium journeys against the rolled-back server

```
# 2026-10-20 12:49 UTC - Selenium journeys of release v1.0.0 (its own tests, from a clean clone of tag v1.0.0) against the lab server
AFTER rollback
$ curl -s http://localhost:8088/health
{"commit":"224de14","env":"prod","status":"ok","version":"1.0.0"}

$ E2E_BASE_URL=http://localhost:8088 pytest -m e2e -v
tests/e2e/test_journeys.py::test_journey_1_register_new_guest PASSED     [ 20%]
tests/e2e/test_journeys.py::test_journey_2_login_rejects_wrong_password_then_accepts_right_one PASSED [ 40%]
tests/e2e/test_journeys.py::test_journey_3_search_request_and_confirm_booking PASSED [ 60%]
tests/e2e/test_journeys.py::test_journey_4_cancel_booking_and_track_status PASSED [ 80%]
tests/e2e/test_journeys.py::test_journey_5_booked_room_is_unavailable_to_another_guest PASSED [100%]
====================== 5 passed, 84 deselected in 25.61s ======================
```

Rendered from the real log file docs/task-reports/assets/raw/ansible-run5-e2e-after-rollback.txt (unedited lines)

Figure 14.4 - The five v1.0.0 Selenium journeys pass against the rolled-back server. **Proves:** the restored release actually works, not only `/health`.

Pretty-print

{"commit":"224de14","env":"prod","status":"ok","version":"1.0.0"}

Figure 14.5 - http://localhost:8088/health: version 1.0.0, commit `224de14`.

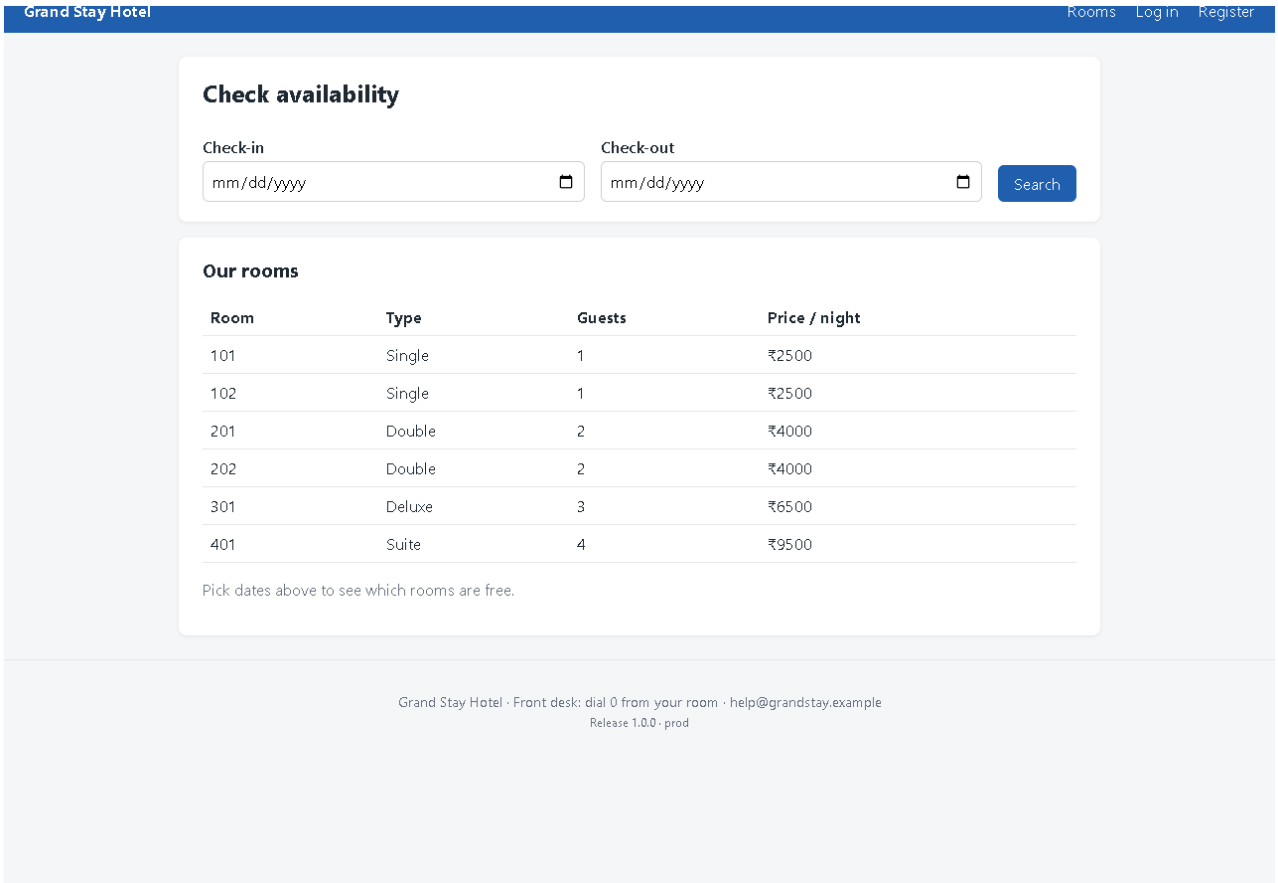
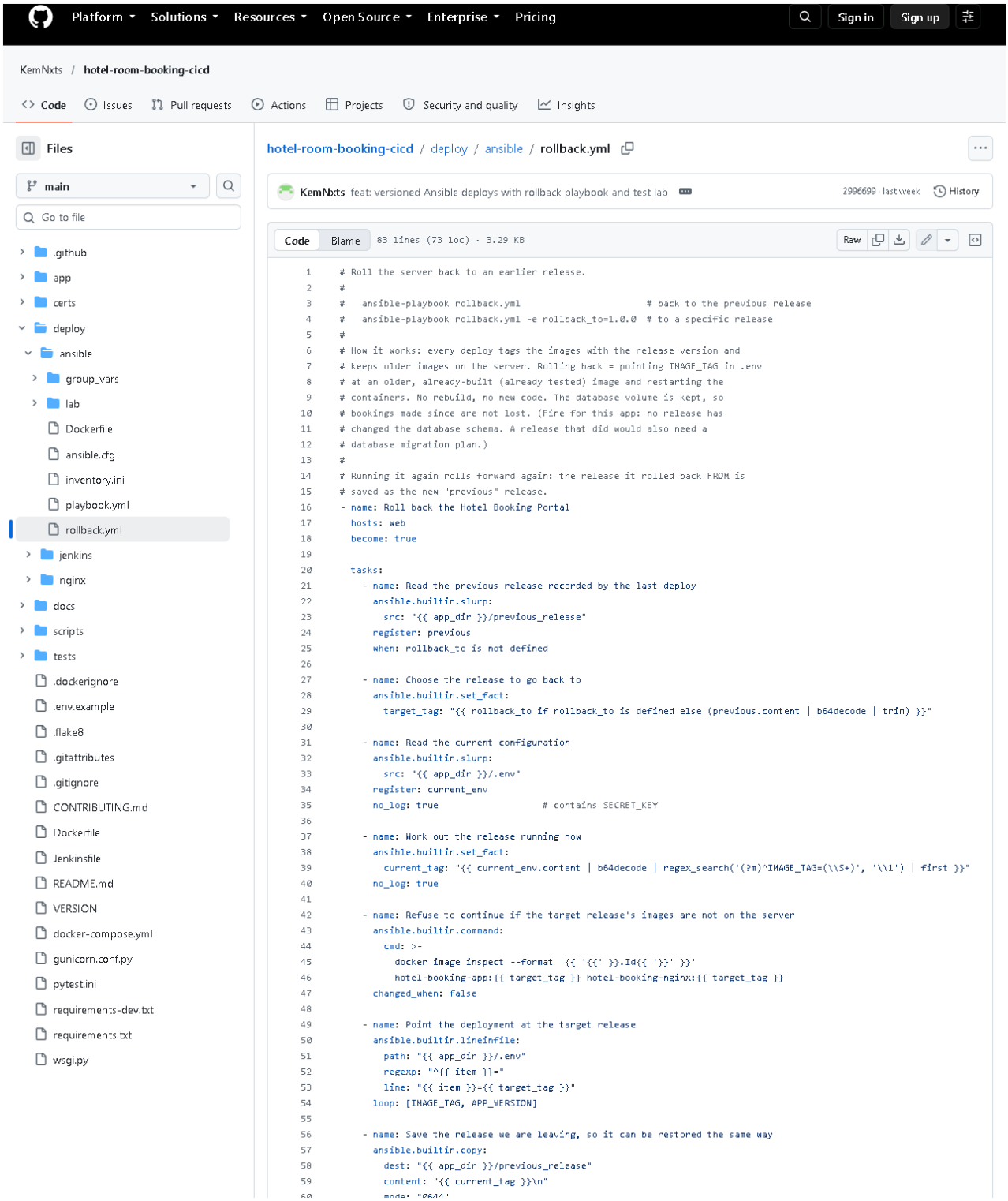


Figure 14.6 - The portal served by the lab server after the rollback (v1.0.0 interface).


Platform
Solutions
Resources
Open Source
Enterprise
Pricing
Sign in
Sign up
KemNxts / hotel-room-booking-cicd
Code
Issues
Pull requests
Actions
Projects
Security and quality
Insights
Files
main
Go to file
.github
app
certs
deploy
ansible
group_vars
lab
Dockerfile
ansible.cfg
inventory.ini
playbook.yml
rollback.yml
jenkins
nginx
docs
scripts
tests
.dockerignore
.env.example
.flake8
.gitattributes
.gitignore
CONTRIBUTING.md
Dockerfile
Jenkinsfile
README.md
VERSION
docker-compose.yml
gunicorn.conf.py
pytest.ini
requirements-dev.txt
requirements.txt
wsgi.py
hotel-room-booking-cicd / deploy / ansible / rollback.yml
KemNxts feat: versioned Ansible deploys with rollback playbook and test lab
2996699 · last week
History
Code
Blame
83 lines (73 loc) · 3.29 KB
Raw
1 # Roll the server back to an earlier release.
2 #
3 #   ansible-playbook rollback.yml   # back to the previous release
4 #   ansible-playbook rollback.yml -e rollback_to=1.0.0 # to a specific release
5 #
6 # How it works: every deploy tags the images with the release version and
7 # keeps older images on the server. Rolling back = pointing IMAGE_TAG in .env
8 # at an older, already-built (already tested) image and restarting the
9 # containers. No rebuild, no new code. The database volume is kept, so
10 # bookings made since are not lost. (Fine for this app: no release has
11 # changed the database schema. A release that did would also need a
12 # database migration plan.)
13 #
14 # Running it again rolls forward again: the release it rolled back FROM is
15 # saved as the new "previous" release.
16 - name: Roll back the Hotel Booking Portal
17   hosts: web
18   become: true
19
20   tasks:
21   - name: Read the previous release recorded by the last deploy
22   ansible.builtin.slurp:
23   src: "{{ app_dir }}/previous_release"
24   register: previous
25   when: rollback_to is not defined
26
27   - name: Choose the release to go back to
28   ansible.builtin.set_fact:
29   target_tag: "{{ rollback_to if rollback_to is defined else (previous.content | b64decode | trim) }}"
30
31   - name: Read the current configuration
32   ansible.builtin.slurp:
33   src: "{{ app_dir }}/.env"
34   register: current_env
35   no_log: true   # contains SECRET_KEY
36
37   - name: Work out the release running now
38   ansible.builtin.set_fact:
39   current_tag: "{{ current_env.content | b64decode | regex_search('(?m)^IMAGE_TAG=(\\S+)', '\\1') | first }}"
40   no_log: true
41
42   - name: Refuse to continue if the target release's images are not on the server
43   ansible.builtin.command:
44   cmd: >-
45   docker image inspect --format '{{ '{{' }}.Id{{ '}}' }}'
46   hotel-booking-app:{{ target_tag }} hotel-booking-nginx:{{ target_tag }}
47   changed_when: false
48
49   - name: Point the deployment at the target release
50   ansible.builtin.lineinfile:
51   path: "{{ app_dir }}/.env"
52   regexp: "^{{ item }}="
53   line: "{{ item }}={{ target_tag }}"
54   loop: [IMAGE_TAG, APP_VERSION]
55
56   - name: Save the release we are leaving, so it can be restored the same way
57   ansible.builtin.copy:
58   dest: "{{ app_dir }}/previous_release"
59   content: "{{ current_tag }}\n"
60   mode: "0644"

(part 1 of 2)

```
61          when: current_tag != target_tag
62
63        - name: Restart the containers on the target release and wait until healthy
64          ansible.builtin.command:
65            cmd: docker compose up -d --no-build --wait --wait-timeout 180
66            chdir: "{{ app_dir }}"
67          register: up
68          changed_when: "'Started' in up.stderr or 'Recreated' in up.stderr"
69
70        - name: Check the portal answers through Nginx
71          ansible.builtin.uri:
72            url: "http://127.0.0.1:{{ host_port }}/health"
73            return_content: true
74          register: health
75          retries: 10
76          delay: 3
77          until: health.status == 200
78
79        - name: Verify the restored release is the one running
80          ansible.builtin.assert:
81            that: health.json.version == target_tag
82            fail_msg: "Rollback failed: expected {{ target_tag }} but /health says {{ health.json.version }}"
83            success_msg: "Rolled back from {{ current_tag }} to {{ target_tag }}: {{ health.json }}"
```

(part 2 of 2)

Figure 14.7 - `deploy/ansible/rollback.yml` on GitHub.

E. How to demonstrate this task to the professor (about 6 minutes)

Before the demo: Docker Desktop running; the lab target up (`docker compose up -d target` in `deploy\ansible\lab`). These commands were run exactly like this on 2026-10-20.

```
cd D:\DevOPS\CICD_FOR_HOTEL_ROOM_BOOKING\deploy\ansible\lab

# 1. deploy (if the server is fresh this is the first provisioning)
docker compose run --rm controller ansible-playbook -i lab/inventory.ini playbook.yml

# 2. run it again - idempotency
docker compose run --rm controller ansible-playbook -i lab/inventory.ini playbook.yml
```

Point at: `PLAY RECAP ... changed=0`. *Say:* "The second run checked every task and found nothing to change - that is idempotency." (If the lab server was already up to date, even run 1 shows `changed=0` - verified on 2026-10-20. To show a first run with changes, start from a fresh server as in Task 13, step 1.)

```
# 3. simulate a new release (same code, new version number)
docker compose run --rm controller ansible-playbook -i lab/inventory.ini playbook.yml -e
app_version=1.1.1-demo
curl.exe -s http://localhost:8088/health
```

Expected: `"version":"1.1.1-demo"`. *Say:* "Pretend this release is bad."

```
# 4. roll back
docker compose run --rm controller ansible-playbook -i lab/inventory.ini rollback.yml
curl.exe -s http://localhost:8088/health
```

Expected: "Rolled back from 1.1.1-demo to 1.1.0" and `"version":"1.1.0"`. *Say:* "The previous image was still on the server, so rollback is just a switch - no rebuild."

91. **Prove the restored version works:** open http://localhost:8088, register and make a booking - or run the Selenium journeys (from the project folder):(The local tests match the interface of the working tree; for the v1.0.0 interface use the tests from that tag, as in Figure 14.4.)

```
cd D:\DevOPS\CICD_FOR_HOTEL_ROOM_BOOKING
$env:E2E_BASE_URL="http://localhost:8088"
.venv\Scripts\python -m pytest -m e2e -q
Remove-Item Env:E2E_BASE_URL
```

Recovery: `rollback.yml` fails "No such image" - that release was never deployed on this server; deploy it first or pass an existing `-e rollback_to=...`. `UNREACHABLE` - start the target container.

F. Expected screenshots for the final submission

100. `assets/task-14/ansible-run2-upgrade-v1.1.0.png` - **exists, 2026-10-20**.
101. `assets/task-14/ansible-run3-repeat-idempotent.png` - **exists, 2026-10-20**.
102. `assets/task-14/ansible-run4-rollback.png` - **exists, 2026-10-20**.
103. `assets/task-14/ansible-run5-e2e-after-rollback.png` - **exists, 2026-10-20**.
104. `assets/task-14/lab-server-health-after-rollback.png` - **exists, 2026-10-20**.
105. `assets/task-14/lab-server-after-rollback-v1.0.0.png` - **exists, 2026-10-20**.
106. `assets/task-14/github-file-deploy_ansible_rollback.yml.png` - **exists, 2026-10-20**.
107. Run 1 (first provisioning) is in Task 13: `assets/task-13/ansible-run1-first-deploy-v1.0.0.png` - **exists, 2026-10-20**.

G. Probable viva questions and answers

89. **What is idempotency?** Running the same automation again makes no changes when the system is already in the desired state. Our run 3 reported `changed=0`.
90. **Idempotency vs repeatability?** Repeatable = same end result each time; idempotent = also no unnecessary changes (nothing reinstalled or restarted).
91. **Why did run 1 change 7 things but run 3 none?** Run 1 started from an empty server; run 3 found Docker installed, files identical and images for this release and commit already present, so it skipped the build.
92. **How did you make the image build idempotent?** By checking the image's revision label first and building only if the release/commit is missing.
93. **How does rollback work?** Old images stay on the server; `rollback.yml` points `.env` to the previous tag, restarts with `--no-build` and asserts `/health` shows that version.
94. **(Project-specific) How do you know the rollback really worked?** `/health` returned 1.0.0 (`224de14`) and the five v1.0.0 Selenium journeys passed against the server.
95. **(Project-specific) What happens to bookings during rollback?** They are kept - the database is on a volume that rollback does not touch.
96. **Is there downtime?** A few seconds while containers are re-created; zero downtime would need blue/green deployment.
97. **What if the release changed the database schema?** Rollback would need a migration plan; no release here changed the schema, so it was not needed.
98. **Deployment vs rollback?** Deployment moves forward to a new version; rollback goes back to an earlier tested version without rebuilding.
99. **What are the limits of your test?** It used a container acting as a server, not a VM, and a rollback within one server.

H. Common mistakes and troubleshooting

Symptom	Cause	Inspect	Fix	Confirm
Second run shows `changed=1`	uncommitted changes since run 1 (commit label `-dirty` changed) or new code	the changed task	commit, then run twice	`changed=0`
`rollback.yml`: "No such image"	target release never deployed here	`docker compose exec target docker image ls`	deploy that release first / pick an existing one	rollback ok

Symptom	Cause	Inspect	Fix	Confirm
`/health` still shows the new version	rollback task failed	rollback output	rerun; check `previous_release`	version = old
Selenium fails after rollback	tests from a newer UI	test version	use tests from the same tag	5 passed
Lab gone after `down -v`	`-v` deletes volumes	-	`docker compose up -d --build target` + playbook	portal back

I. Quick revision notes

Five key points

31. Run 1 `changed=7`, run 2 `changed=5`, run 3 **`changed=0`** = idempotent.
32. Images are built only when the release/commit is missing (label check).
33. Each deploy remembers the previous release; images are kept.
34. `rollback.yml` switches `IMAGE_TAG`, restarts without rebuilding, asserts `/health`.
35. Restored v1.0.0 verified by `/health` **and** 5/5 Selenium journeys.

30-60 second explanation "On a fresh lab server the playbook made seven changes - installing Docker, copying the app, building images and starting containers. Upgrading to v1.1.0 made five. Running the same playbook again made zero changes, which proves it's idempotent and safe to repeat. Every deploy keeps the old images and remembers the previous release, so rollback.yml just switches the image tag and restarts. After rolling back to 1.0.0, `/health` showed 1.0.0 and that release's Selenium journeys all passed."

Caveat: container-based lab, not a VM; rollback has a short interruption.

Readiness checklist

- â€¢ â˜ I can run the playbook twice and point at `changed=0`.
- â€¢ â˜ I can simulate a release with `-e app_version=...` and roll back.
- â€¢ â˜ I can explain why run 3 skipped the build.
- â€¢ â˜ I can state the limitations honestly.

Task 15 - Final End-to-End Release, Documentation and Viva

A. Task title and objective

Task 15: Final Release, Documentation and Viva

Objective: show the complete journey from a code change to a tested, packaged, deployed, health-checked and recoverable release, and deliver the documentation and viva preparation for the whole project.

Professor's required deliverables

- â€¢ End-to-end workflow: Git commit -> Jenkins checkout/build -> automated tests -> quality gate -> Docker image -> deployment -> health check -> configuration management -> rollback/recovery
- â€¢ Final README, project report, architecture diagrams, SRS summary
- â€¢ Setup and deployment guide, Jenkins documentation, Selenium test plan and reports, Docker lifecycle commands, Ansible documentation
- â€¢ Troubleshooting guide, known limitations, future enhancements
- â€¢ Screenshots and a demo-recording checklist
- â€¢ Viva questions and answers
- â€¢ A final deliverable checklist mapped to all 15 tasks

Current status: Implemented and verified, with pending external items. Every step of the workflow was run on 2026-10-20 (Jenkins build #11, Ansible runs 1-4, Selenium after rollback). The documentation set exists (section C). Pending - outside this machine or needing your accounts/decisions:

92. GitHub issues (0) and pull requests (0)
93. Docker Hub publishing (local registry used)
94. A run against a real VM (container lab used)
95. Professor's confirmation that Python tooling replaces Maven/Tomcat
96. Recording the demo video
97. Committing and deploying the UI redesign (currently uncommitted on `feature/ui-redesign`)

B. Explanation for a complete beginner

An everyday picture. A car factory: a design change is approved (Git), the assembly line builds the car automatically (Jenkins), inspectors test it and reject faulty ones (tests = quality gate), approved cars are shipped in standard containers (Docker images in a registry), dealers prepare their showroom the same way every time (Ansible), each car gets a final check before handover (health check), and if a model has a problem, the previous model is put back in the showroom (rollback).

The whole DevOps flow in this project

Step	Tool	What happens	Evidence
1. Change	Git	commit on a feature branch, merge into `develop`, tag releases on `main`	Tasks 4-6
2. Build	Jenkins	checkout, fresh venv, pinned dependencies	build #11
3. Test	flake8, pytest, Selenium	lint + 59 unit + 26 integration + 5 browser tests	build #11: 90 passed
4. Quality gate	Jenkinsfile stage order	any failure skips everything after it	builds #2, #9
5. Package	git archive, Docker	source tarball + checksum; images `1.1.0-1c2103c`	build #11 artifacts

Step	Tool	What happens	Evidence
6. Publish	registry	images pushed to `localhost:5000`	registry tag list
7. Deploy	Docker Compose	pull + `up --wait` (staging :8081 / prod :8082)	`docker ps`
8. Verify	`/health`	version **and** commit must match	build #11 health stage
9. Configure servers	Ansible	provision a fresh server, idempotent	lab runs 1-3
10. Recover	Ansible / Compose	rollback to the previous release, verified	lab run 4 + Selenium

C. What was actually implemented

Release sequence as it really happened

92. Features merged into `develop` (Tasks 5-6); Jenkins #4 green on develop.
93. `git checkout main; git merge --no-ff develop` -> `224de14`, tag **v1.0.0**; Jenkins #5 deployed `main` to **prod**.
94. New feature (nights column) + `VERSION` 1.1.0 on develop; Jenkins #6/#7 green.
95. Merge to `main` -> `ce1726d`, tag **v1.1.0**; Jenkins #8 deployed it to prod - still running today (`1.1.0-d07929b`).
96. Rollback demonstrated with Compose (prod 1.1.0 -> 1.0.0 -> 1.1.0, 2026-10-11) and Ansible (lab, 2026-10-20).
97. 2026-10-20: Jenkins #11 rebuilt and redeployed `develop` (`1c2103c`) to staging; the lab was re-provisioned and rolled back.

Documentation delivered

Required item	Where
README	`README.md`
Project report (with troubleshooting, limitations, future work, demo checklist)	`docs/08-project-report.md`
Problem definition, agile plan, SRS + architecture/ER/use-case diagrams	`docs/01`, `docs/02`, `docs/03`
Git workflow, PR records	`docs/04-git-workflow.md`, `docs/pull-requests/`
Jenkins pipeline documentation	`docs/05-ci-cd-pipeline.md`
Selenium test plan + reports	`docs/06-test-plan.md`, `docs/evidence/tests/`
Docker lifecycle commands	`docs/container-lifecycle.md`
Ansible documentation	`docs/07-ansible.md`
UI design + before/after screenshots	`docs/09-ui-design.md`, `docs/ui-evidence/`
Deliverable checklist for all 15 tasks	`docs/15-task-checklist.md`
Viva Q&A	`docs/viva-notes.md`, `docs/task-reports/VIVA_CHEAT_SHEET.md`
Evidence (logs, screenshots)	`docs/evidence/`, `docs/task-reports/assets/`
These 15 task reports + audit	`docs/task-reports/`

Final state of all environments (2026-10-20, Figure 15.1)

Environment	URL	Version / commit
development (working tree, redesigned UI)	http://127.0.0.1:5001	`dev`
staging (Jenkins #11)	http://localhost:8081	1.1.0 / `1c2103c`
prod (Jenkins #8, release v1.1.0)	http://localhost:8082	1.1.0 / `d07929b`
Ansible lab server	http://localhost:8088	1.0.0 / `224de14` right after the rollback (Figure 15.1); 1.1.0 / `1c2103c-dirty` after the Task 14 demo commands were re-checked later the same day

Future enhancements (from `docs/08-project-report.md`): PostgreSQL for several servers; HTTPS with secure cookies; auto-expiry of PENDING bookings; a staff dashboard; e-mail confirmations; GitHub webhook + multibranch pipeline; blue/green deployment and database migrations; monitoring and alerting; image vulnerability scanning.

D. Screenshots and evidence

PowerShell - health of every running environment

```
$ curl -s http://127.0.0.1:5001/health
{
  "commit": "unknown",
  "env": "dev",
  "status": "ok",
  "version": "dev"
}

$ curl -s http://localhost:8081/health
{"commit":"1c2103c","env":"staging","status":"ok","version":"1.1.0"}

$ curl -s http://localhost:8082/health
{"commit":"d07929b","env":"prod","status":"ok","version":"1.1.0"}

$ curl -s http://localhost:8088/health
{"commit":"224de14","env":"prod","status":"ok","version":"1.0.0"}
```

Rendered from real command output captured 2026-10-20 12:57 UTC in CICD_FOR_HOTEL_ROOM_BOOKING | raw text: all-environments-health.txt

Figure 15.1 - `/health` of development, staging, prod and the lab server (rendered from real output, 2026-10-20).

Git - current branch and uncommitted work (honest state)

```
$ git branch --show-current
feature/ui-redesign

$ git status --short
 M README.md
 M app/__init__.py
 M app/static/style.css
 M app/templates/404.html
 M app/templates/base.html
 M app/templates/booking_detail.html
 M app/templates/error.html
 M app/templates/login.html
 M app/templates/my_bookings.html
 M app/templates/register.html
 M app/templates/rooms.html
 M docs/01-problem-definition.md
 M docs/06-test-plan.md
 M docs/08-project-report.md
 M tests/e2e/test_journeys.py
 M tests/integration/test_routes.py
?? app/static/app.js
?? app/static/fonts/
?? app/templates/auth_base.html
?? app/templates/partials/
?? docs/09-ui-design.md
?? docs/task-reports/
?? docs/ui-evidence/
?? scripts/capture_report_evidence.py
?? scripts/capture_ui.py

$ git log --oneline -1
1c2103c Merge branch 'docs/history-rewrite' into develop
```

Rendered from real command output captured 2026-10-20 12:57 UTC in CICD_FOR_HOTEL_ROOM_BOOKING | raw text: git-status-final.txt

Figure 15.2 - Current branch and uncommitted files (rendered from real output): the UI redesign and these reports are not yet committed.

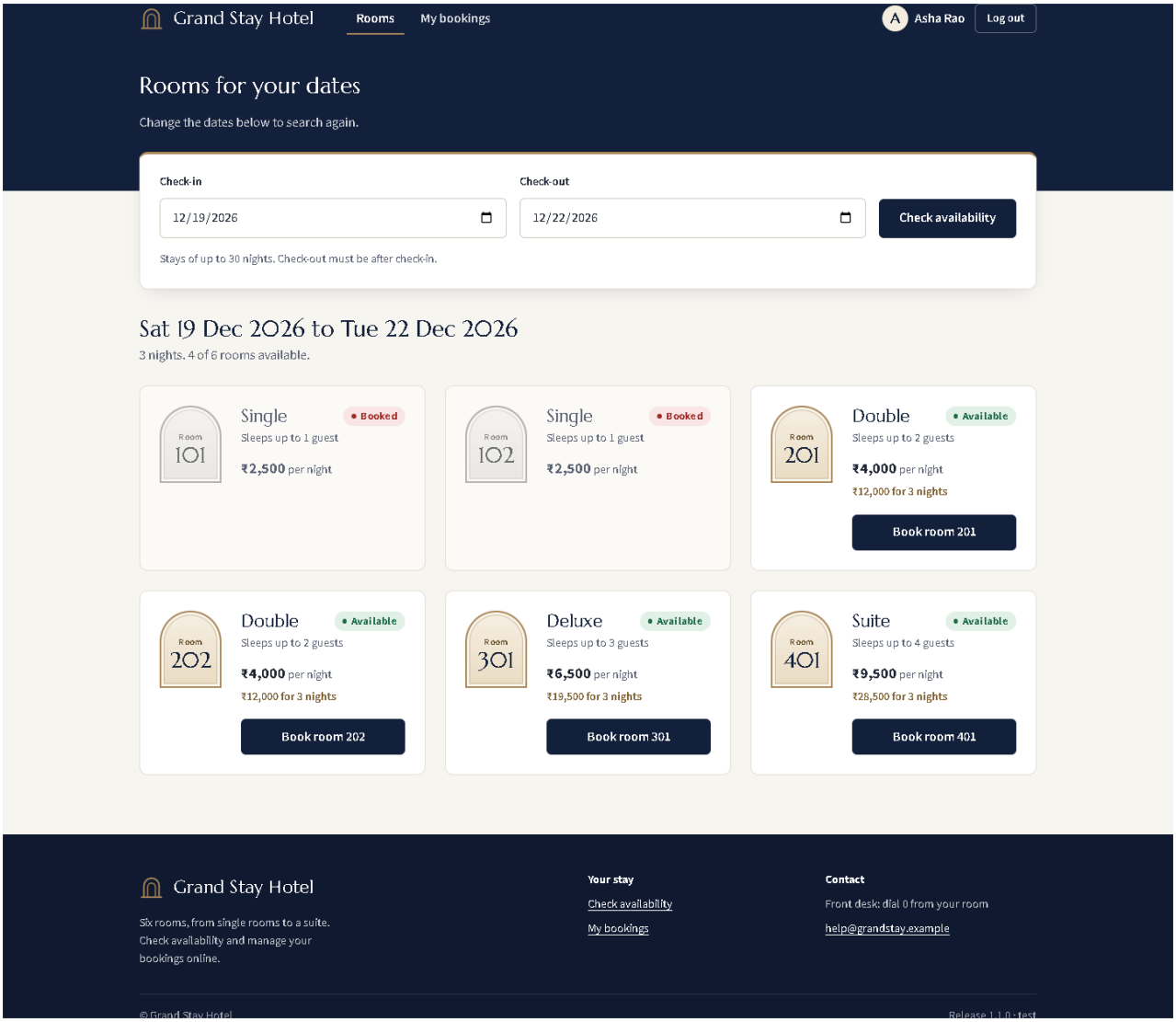
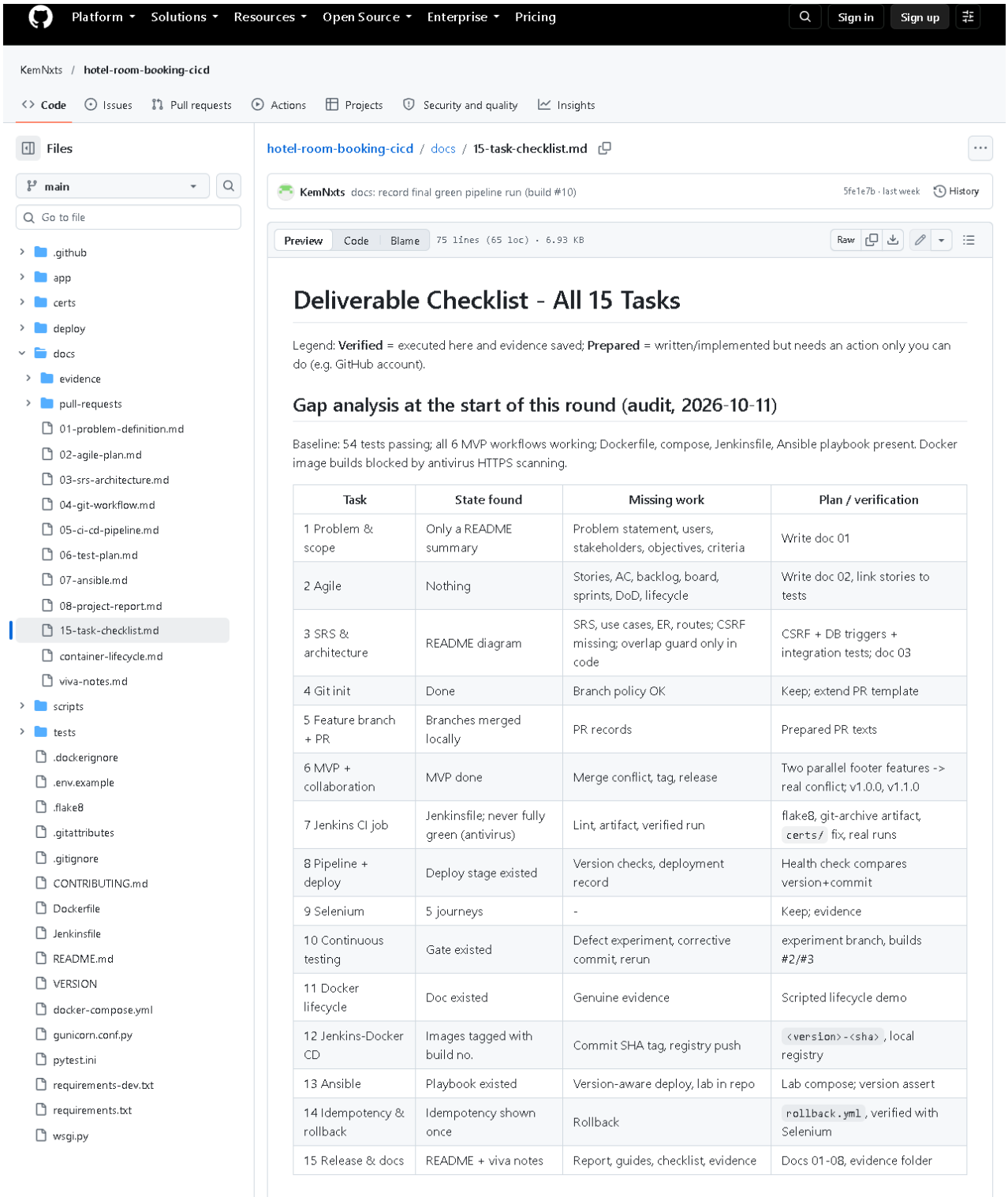


Figure 15.3 - The customer journey's central page: availability with totals for the stay (redesigned UI, working tree).



Deliverable Checklist - All 15 Tasks

Legend: **Verified** = executed here and evidence saved; **Prepared** = written/implemented but needs an action only you can do (e.g. GitHub account).

Gap analysis at the start of this round (audit, 2026-10-11)

Baseline: 54 tests passing; all 6 MVP workflows working; Dockerfile, compose, Jenkinsfile, Ansible playbook present. Docker image builds blocked by antivirus HTTPS scanning.

Task	State found	Missing work	Plan / verification
1 Problem & scope	Only a README summary	Problem statement, users, stakeholders, objectives, criteria	Write doc 01
2 Agile	Nothing	Stories, AC, backlog, board, sprints, DoD, lifecycle	Write doc 02, link stories to tests
3 SRS & architecture	README diagram	SRS, use cases, ER, routes; CSRF missing; overlap guard only in code	CSRF + DB triggers + integration tests; doc 03
4 Git init	Done	Branch policy OK	Keep; extend PR template
5 Feature branch + PR	Branches merged locally	PR records	Prepared PR texts
6 MVP + collaboration	MVP done	Merge conflict, tag, release	Two parallel footer features -> real conflict; v1.0.0, v1.1.0
7 Jenkins CI job	Jenkinsfile; never fully green (antivirus)	Lint, artifact, verified run	flake8, git-archive artifact, `certs/` fix, real runs
8 Pipeline + deploy	Deploy stage existed	Version checks, deployment record	Health check compares version+commit
9 Selenium	5 journeys	-	Keep; evidence
10 Continuous testing	Gate existed	Defect experiment, corrective commit, rerun	experiment branch, builds #2/#3
11 Docker lifecycle	Doc existed	Genuine evidence	Scripted lifecycle demo
12 Jenkins-Docker CD	Images tagged with build no.	Commit SHA tag, registry push	`<version>-<sha>`, local registry
13 Ansible	Playbook existed	Version-aware deploy, lab in repo	Lab compose; version assert
14 Idempotency & rollback	Idempotency shown once	Rollback	`rollback.yml`, verified with Selenium
15 Release & docs	README + viva notes	Report, guides, checklist, evidence	Docs 01-08, evidence folder

(part 1 of 4)

Final state

#	Deliverable	Where	Status	Evidence
1	Problem, users, stakeholders, pain points, objectives, constraints, success criteria, scope	`docs/01-problem-definition.md`	Done	-
2	User stories + AC, backlog, Kanban, sprint plan, DoD, lifecycle diagram	`docs/02-agile-plan.md`	Done	Stories mapped to test names
2	Backlog linked to GitHub issues	-	**Prepared**	Needs push to GitHub, then create issues HB-01..15
3	SRS, use-case, architecture, ER, data model, routes, setup, tech rationale	`docs/03-srs-architecture.md`	Done	-
3	6 workflows + booking rules (dates, overlap, ownership, states, concurrency)	`app/`, `app/schema.sql`	**Verified**	90 tests: `evidence/tests/local-run.txt`
4	README, .gitignore, .gitattributes, structure, templates, branch policy, baseline	repo root, `.github/`, `CONTRIBUTING.md`	Done	commit `08702db`
5	Feature branch, commits, tests, merge into develop	`feature/*` branches	**Verified**	`git log --graph`; `docs/04-git-workflow.md`
5	Pull request + review	`docs/pull-requests/`	**Prepared**	Real GitHub PRs need a push
6	All 6 workflows	app	**Verified**	Selenium 5/5, screenshots `app-*.png`
6	Second feature branch + real merge conflict + resolution	`feature/contact-footer`, `feature/release-footer`	**Verified**	`evidence/git/merge-conflict.txt`, merge `8728b62`
6	Version tags, release baseline	tags `v1.0.0`, `v1.1.0` on `main`	**Verified**	`git tag -n1`
6	Password hashing, sessions, validation, authorization, no secrets	app	**Verified**	integration tests (CSRF, cookies, 404 for others)
7	Jenkins job: checkout, Python env, deps, lint, pytest, reports, artifacts	`Jenkinsfile`, `deploy/jenkins/`	**Verified**	builds #1-#10, `evidence/jenkins/`, `jenkins-01-*.png`
7	Trigger	pollSCM (documented)	**Verified** (manual + API triggers used)	Webhook impossible without public Jenkins - documented
8	Pipeline as code with all stages, parameter, secrets outside Git, deploy, health check	`Jenkinsfile`	**Verified**	`build-01-green-staging.log`, `build-05-release-v1.0.0-prod.log`

(part 2 of 4)

9	5 Selenium journeys, stable ids, explicit waits, isolated data, screenshots on failure	`tests/e2e/`	**Verified**	`evidence/tests/`, Jenkins Tests page
10	Tests in Jenkins, reports, failure propagation, quality gate	`Jenkinsfile`	**Verified**	build #2/#9 logs: "skipped due to earlier failure(s)"
10	Deliberate defect -> failure -> corrective commit -> green rerun	`experiment/defect-demo` (554cad3 -> 30c6a05)	**Verified**	builds #2 (fail), #3 (pass)
11	Dockerfile (slim base, pinned deps, Gunicorn, non-root, env config, no secrets, HEALTHCHECK), .dockerignore	`Dockerfile`, `.dockerignore`, `deploy/nginx/`	**Verified**	`evidence/docker/lifecycle.txt`
11	Build, tag, run, port mapping, logs, inspect, stop/restart/remove, persistent data	-	**Verified**	`evidence/docker/lifecycle.txt`
12	Build after tests, tag version + SHA, publish, deploy tested version, health check, record	`Jenkinsfile`	**Verified** (local registry)	registry tags list, `deployment.txt` artifact
12	Docker Hub publishing	`REGISTRY` param + `registry-creds`	**Prepared**	Needs a Docker Hub account/token
13	Inventory, playbook, variables, packages, dirs, config, permissions, ports, Nginx, health	`deploy/ansible/`	**Verified** (lab container)	`evidence/ansible/run1-*.txt`
14	First run, idempotent second run, health checks	-	**Verified**	`run1`, `run3` (`changed=0`)
14	Rollback + restored release works	`deploy/ansible/rollback.yml`	**Verified**	`run4-rollback.txt`, `run5-e2e-after-rollback.txt` (5/5)
15	End-to-end release (commit -> Jenkins -> tests -> gate -> image -> deploy -> health -> Ansible -> rollback)	all	**Verified**	docs 05, 07; builds #4-#8
15	README, report, diagrams, SRS, guides, test plan, Docker/Ansible docs, troubleshooting, limitations, future work	`README.md`, `docs/`	Done	-
15	Screenshots + demo checklist	`docs/evidence/screenshots/`, report section 8	Done (screenshots	Demo **video** still to be recorded by you

(part 3 of 4)

	checklist	report section 8	genuine)	you
15	Viva questions and answers	`docs/viva-notes.md`	Done	-

Needs confirmation from the professor

- Whether Maven/Gradle/Ant and Tomcat are **mandatory** or examples (this Python project uses pip/pytest and Gunicorn + Nginx; see doc 03).

What only you can do

1. Push to GitHub (`docs/04-git-workflow.md` has the commands), then create issues HB-01..HB-15 and, if required, open the prepared PRs.
2. Optional: Docker Hub account + `registry-creds` credential.
3. Optional: run the Ansible playbook against a real Ubuntu VM.
4. Record the demo video (checklist in `docs/08-project-report.md`).

(part 4 of 4)

Figure 15.4 - `docs/15-task-checklist.md` on GitHub: gap analysis and the final deliverable status.

Platform Solutions Resources Open Source Enterprise Pricing Sign in Sign up

KemNxts / hotel-room-booking-cicd

Code Issues Pull requests Actions Projects Security and quality Insights

Files

main

Go to file

- .github
- app
- certs
- deploy
- docs
  - evidence
  - pull-requests
  - 01-problem-definition.md
  - 02-agile-plan.md
  - 03-srs-architecture.md
  - 04-git-workflow.md
  - 05-ci-cd-pipeline.md
  - 06-test-plan.md
  - 07-ansible.md
  - 08-project-report.md
  - 15-task-checklist.md
  - container-lifecycle.md
  - viva-notes.md
- scripts
- tests
- .dockerignore
- .env.example
- .flake8
- .gitattributes
- .gitignore
- CONTRIBUTING.md
- Dockerfile
- Jenkinsfile
- README.md
- VERSION
- docker-compose.yml
- gunicorn.conf.py
- pytest.ini
- requirements-dev.txt
- requirements.txt
- wsgi.py

hotel-room-booking-cicd / docs / 08-project-report.md

KemNxts docs: add project report, SRS, agile plan and 15-task checklist a3317da · last week History

Preview Code Blame 146 lines (123 loc) · 8.29 KB Raw

Project Report - CI/CD Pipeline for a Hotel Room Booking Portal

1. Summary

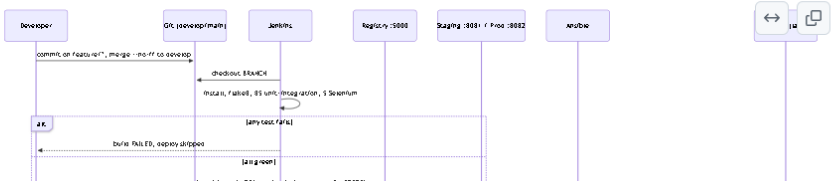
A Flask web portal lets hotel guests register, check room availability, request, confirm and cancel bookings, and track their status. Around it, a complete delivery pipeline was built and **run**: every commit is linted and tested (90 automated tests incl. 5 Selenium journeys) in Jenkins; only if everything passes is it packaged as a source archive and Docker images tagged `<version>-<commit>`, pushed to a registry, deployed with Docker Compose behind Nginx, and verified by a health check that checks the exact version. Ansible provisions a fresh Ubuntu server and deploys the same stack idempotently, and can roll back to the previous release.

Two releases were made: **v1.0.0** and **v1.1.0**, both deployed by the pipeline to "prod", and rollback was demonstrated with Docker Compose and with Ansible.

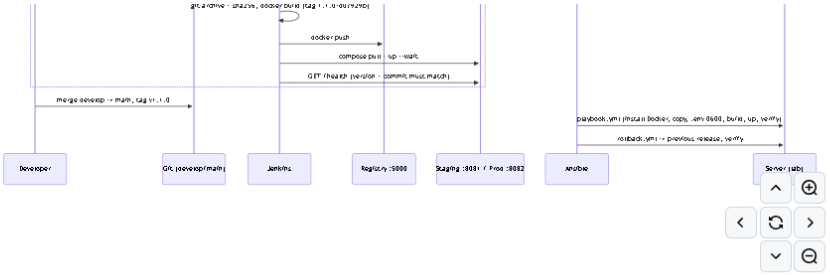
2. Document map

Topic	Document
Problem, users, objectives, success criteria (Task 1)	01-problem-definition.md
User stories, backlog, board, sprints, DoD, lifecycle (Task 2)	02-agile-plan.md
SRS, use cases, architecture, ER diagram, routes, tech choices (Task 3)	03-srs-architecture.md
Git setup, branches, PRs, merge conflict, tags (Tasks 4-6)	04-git-workflow.md
Jenkins pipeline, parameters, registry, defect experiment (Tasks 7, 8, 10, 12)	05-ci-cd-pipeline.md
Test plan, Selenium, reports (Tasks 9, 10)	06-test-plan.md
Docker image + container lifecycle + rollback (Task 11)	container-lifecycle.md
Ansible, idempotency, rollback (Tasks 13, 14)	07-ansible.md
Viva questions and answers	viva-notes.md
Deliverable checklist for all 15 tasks	15-task-checklist.md
Raw evidence (logs, screenshots, test reports)	evidence/

3. End-to-end release workflow (as executed)



(part 1 of 3)



4. Setup and deployment guide

Prerequisites: Git, Python 3.11+ (3.13 also works locally), Docker Desktop, Google Chrome (for Selenium locally).

```
# 1. Local development
python -m venv .venv; .venv\Scripts\Activate.ps1
pip install -r requirements-dev.txt
python wsgi.py                                  # http://127.0.0.1:5000
pytest                                          # 90 tests

# 2. Docker (Nginx + Gunicorn + SQLite volume)
copy .env.example .env                          # set SECRET_KEY
docker compose up -d --build                    # http://localhost:8080

# 3. Jenkins + registry
cd deploy\jenkins; copy .env.example .env       # set both values
docker compose up -d --build                    # http://localhost:8090 (admin)
# Job hotel-booking -> Build with Parameters

# 4. Ansible (lab)
cd ..\ansible\lab
docker compose up -d --build target
docker compose run --rm controller ansible-playbook -i lab/inventory.ini playbook.yml
```

Details: README.md.

5. Troubleshooting

Symptom	Cause	Fix
`SSL: CERTIFICATE_VERIFY_FAILED` during `docker build` / plugin install	Antivirus (Avast/AVG "HTTPS scanning") or a company proxy re-signs HTTPS; Linux containers don't trust its root certificate	Turn off HTTPS scanning, **or** export the root certificate to `certs/<name>.crt` (see certs/README.md) and rebuild
`failed to connect to the docker API ... dockerDesktopLinuxEngine`	Docker Desktop not running (e.g. after a reboot)	Start Docker Desktop and wait until it says "running"
`RuntimeError: SECRET_KEY must be set when APP_ENV=prod`	Started prod/staging without a real key	Put `SECRET_KEY=<random>` in `.env` / Jenkins credential
Compose: `SECRET_KEY is not set`	No `.env` next to `docker-compose.yml`	`copy .env.example .env` and edit
HTTP 400 "Your form has expired"	CSRF token missing/old (e.g. page open across a restart with a new key)	Reload the page and submit again
Selenium: `SessionNotCreatedException`	Chrome/driver mismatch or no Chrome	Install/update Chrome; Selenium Manager downloads the matching driver. In Jenkins, `CHROME_BIN` / `CHROMEDRIVER` point to Chromium
Jenkins build can't see my change	Jenkins checks out **commits** from the repo	Commit first; uncommitted changes are never built

(part 2 of 3)

Jenkins: `permission denied ... docker.sock` (Linux host)	jenkins user not in the socket's group	Set `DOCKER_GID` in `deploy/jenkins/.env` to `stat -c %g /var/run/docker.sock`
Port 8080/8081/8082/8090/5000 already in use	Another program uses the port	Change `HOST_PORT` (app) or the port mapping in the compose file
Ansible: "Missing sudo password"	Real server needs become password	Add `--ask-become-pass` (and `--ask-pass` without SSH keys)
Ansible ignores `ansible.cfg` on Windows folders	World-writable mount	The controller image sets `ANSIBLE_CONFIG` explicitly
`rollback.yml` fails "No such image"	That release was never deployed on this server	Deploy it first, or pass an existing `-e rollback_to=`

6. Known limitations

- SQLite: one writer at a time, one server only; no horizontal scaling.
- PENDING bookings never expire; there is no staff/admin UI; no payments or e-mails.
- Plain HTTP in the demo (set `SESSION_COOKIE_SECURE=1` behind HTTPS).
- Rollback re-creates containers: a few seconds' downtime (no zero-downtime claim); database changes are not rolled back.
- Local Jenkins polls the repository (no public URL for a GitHub webhook).
- Docker Hub publishing is implemented but was not run (no account); the local registry was used.
- Ansible was verified against a privileged container standing in for a server, not a real VM.
- Not pushed to GitHub yet, so issues/PRs exist as prepared documents, not GitHub objects.
- No load/performance testing.

7. Future enhancements

- PostgreSQL instead of SQLite for multiple app servers.
- HTTPS with Let's Encrypt in Nginx; Secure cookies on.
- Auto-expire PENDING bookings after N hours (scheduled job).
- Staff dashboard: all bookings, check-in/check-out.
- E-mail confirmation of bookings.
- GitHub webhook + multibranch pipeline (one job per branch, PR builds).
- Blue/green deployment for zero downtime; database migrations (Alembic).
- Monitoring: Prometheus metrics + alerting on `/health`.
- Image vulnerability scanning (e.g. Trivy) as a pipeline stage.

8. Screenshots and demo recording checklist

Genuine screenshots already captured (`python scripts/capture_screenshots.py ...`): see evidence/README.md. For the demo video, record in this order (~8 min):

- [ ] `git log --graph --oneline --all` - branches, merges, tags `v1.0.0`, `v1.1.0`.
- [ ] `git show 8728b62` - the merge-conflict resolution.
- [ ] App at http://localhost:8082: register, search, book, confirm, cancel, My bookings; point at the footer version.
- [ ] `pytest -m "not e2e"` and `pytest -m e2e` (HEADLESS=0 to show the browser).
- [ ] Jenkins: Build with Parameters (develop, staging) -> Stage View turns green; open Tests and Artifacts (`deployment.txt`).
- [ ] Jenkins build #2: red stage, failed test, "skipped due to earlier failure".
- [ ] Registry tags: http://localhost:5000/v2/hotel-booking-app/tags/list
- [ ] `docker compose ps`, `docker logs`, `/health`.
- [ ] Ansible lab: first run, second run `changed=0`, `rollback.yml`, footer shows 1.0.0.
- [ ] Close with the 15-task checklist.

(part 3 of 3)

Figure 15.5 - `docs/08-project-report.md` on GitHub: summary, document map, end-to-end sequence diagram, setup guide, troubleshooting, limitations, future work and the demo checklist.

Key end-to-end evidence in other folders: `assets/task-07/jenkins-job-stage-view.png`, `assets/task-08/jenkins-build-11-console.png`, `assets/task-12/registry-tags.png`, `assets/task-14/ansible-run3-repeat-idempotent.png`, `assets/task-14/ansible-run4-rollback.png`.

E. How to demonstrate the whole project (about 12 minutes)

Before the viva (10 minutes earlier)

108. Start **Docker Desktop**; wait for "Engine running".

109. PowerShell:*Expected:* `hotel-jenkins-jenkins-1`, `hotel-jenkins-registry-1`, `hotel-staging-*`, `hotel-prod-*` up. If Jenkins is missing: `cd deploy\jenkins; docker compose up -d; cd ..\..`

```
cd D:\DevOPS\CICD_FOR_HOTEL_ROOM_BOOKING
docker ps --format "table {{.Names}}\t{{.Status}}"
```

110. Open browser tabs: http://localhost:8090 (log in), http://localhost:8082, the GitHub repository.

The demonstration

Minute	Show	Say
0-1	`docs/01-problem-definition.md`	"The problem: double bookings and risky manual releases."
1-3	prod http://localhost:8082 - register, search, book, confirm, cancel, My bookings	"The six MVP workflows."
3-4	`git log --graph --oneline -12 develop`, `git tag -n1`	"Feature branches, merges, release tags."
4-7	Jenkins -> *Build with Parameters* (`develop`, `staging`) -> stage view	"Lint and 90 tests, then images, registry, deploy, health check."
7-8	build #2 Tests + Console (`skipped due`)	"A failing test blocks deployment."
8-9	http://localhost:5000/v2/hotel-booking-app/tags/list and `curl.exe -s http://localhost:8081/health`	"Every image is tagged version-commit; the deployed commit matches."
9-11	`cd deploy\ansible\lab`; playbook twice (`changed=0`); `-e app_version=1.1.1-demo`; `rollback.yml`; `curl.exe -s http://localhost:8088/health`	"Ansible configures servers idempotently and rolls back."
11-12	`docs/15-task-checklist.md`	"All 15 tasks with evidence; these items are still pending."

(Exact Ansible commands: Task 14 section E. If the lab target is not running: `cd deploy\ansible\lab; docker compose up -d target`.)

Demo-video checklist: record the same 12 minutes with a screen recorder (e.g. Windows *Snipping Tool -> Record*), narrating as above; keep `deploy\jenkins\.env` and any password off screen.

F. Expected screenshots for the final submission

100. `assets/task-15/all-environments-health.png` - **exists, 2026-10-20**.
101. `assets/task-15/git-status-final.png` - **exists, 2026-10-20**.
102. `assets/task-15/app-desktop-06-availability-logged-in.png` - **exists, 2026-10-20**.
103. `assets/task-15/github-doc-15-task-checklist.png` - **exists, 2026-10-20**.
104. `assets/task-15/github-doc-08-project-report.png` - **exists, 2026-10-20**.
105. Demo video file - **requires manual recording** (pending).

G. Probable viva questions and answers

36. **Walk me through your pipeline in one sentence.** Commit -> Jenkins checkout -> lint + 90 tests -> source archive and Docker images tagged version-commit -> push to registry -> deploy with Compose -> `/health` must match -> Ansible configures servers and can roll back.
37. **Where exactly is the quality gate?** In the stage order of the Jenkinsfile: Package/Build/Publish/Deploy run only if lint and all tests passed.

38. **How do you know prod runs a tested version?** It runs image `1.1.0-d07929b` from build #8, which passed all tests; `/health` reports that commit.
39. **What happens if a bad release reaches prod?** Roll back by switching to the previous image tag (Compose or `rollback.yml`); no rebuild.
40. **What role does Ansible play compared to Jenkins?** Jenkins builds, tests and deploys to its own host; Ansible prepares and deploys to (other) servers in a repeatable way.
41. **(Project-specific) What are the four stories you'd tell?** No double bookings (three layers); failed quality gate (build #2); the merge conflict (two footers, resolved keeping both); rollback (1.1.0 -> 1.0.0 verified by Selenium).
42. **(Project-specific) What is still pending?** GitHub issues/PRs, Docker Hub, a real VM, the professor's Maven/Tomcat confirmation, the demo video, and committing the UI redesign.
43. **Why Python tools instead of Maven/Tomcat?** It's a Python project; pip, pytest/flake8, Gunicorn and Nginx do those jobs. Needs the professor's approval.
44. **Biggest technical challenge?** Antivirus HTTPS scanning broke downloads in containers; solved with an optional `certs/` folder trusted by every image.
45. **What would you improve next?** HTTPS, PostgreSQL, a GitHub webhook with PR builds, blue/green deploys and monitoring.
46. **Is all your evidence genuine?** Yes - logs and screenshots come from real runs; rendered terminal images say so; pending items are labelled pending.

H. Common mistakes and troubleshooting

Symptom	Cause	Inspect	Fix	Confirm
Nothing loads	Docker Desktop not started after reboot	tray icon	start it, wait 1 min	`docker ps` lists containers
Dev server won't start on 5000	registry uses port 5000	-	`python -m flask --app wsgi run --port 5001`	page opens
Jenkins build doesn't show new code	not committed	`git status`	commit, then build the branch	console commit ID
Selenium fails against staging/prod	deployed UI older than the local tests	`/health` version	use tests of the deployed version	5 passed
Lab rollback "No such image"	release never deployed there	lab image list	deploy first	rollback ok

I. Quick revision notes

Five key points

1. Flow: Git -> Jenkins -> tests (gate) -> images -> registry -> deploy -> `/health` -> Ansible -> rollback.
2. Every step was run on 2026-10-20 (build #11, Ansible runs 1-4, Selenium 5/5).
3. Versions are traceable: `VERSION` + commit in tags, `/health` and the page footer.
4. All documentation is in `docs/` and `docs/task-reports/`.
5. Pending: issues/PRs, Docker Hub, VM, Maven approval, video, committing the redesign.

30-60 second explanation "A change starts as a commit on a feature branch. Jenkins checks it out, installs pinned dependencies and runs flake8 plus 90 tests. Only if all pass does it package the source, build Docker images tagged with version and commit, push them to our registry and deploy them with Docker Compose; the health check confirms the right commit is running. Releases are merged to main and tagged. Ansible provisions

servers the same way every time - a second run changes nothing - and rollback.yml returns to the previous release, which I verified with Selenium."

Caveat: several external steps (GitHub issues/PRs, Docker Hub, real VM) are still pending.

Readiness checklist

- â€¢ â˜ Docker Desktop started; all containers up.
- â€¢ â˜ Jenkins logged in; GitHub tab open.
- â€¢ â˜ I can do the 12-minute demo without notes.
- â€¢ â˜ I can list the pending items honestly.
- â€¢ â˜ I have read `VIVA_CHEAT_SHEET.md`.

Viva Cheat Sheet - CI/CD Pipeline for a Hotel Room Booking Portal

30-second introduction

"My project is a hotel room booking portal with a complete CI/CD pipeline. Guests register, check availability, request, confirm and cancel bookings and track their status. Every commit is checked by Jenkins with flake8 and 90 automated tests, including five Selenium browser journeys. Only if everything passes are Docker images built, tagged with the version and commit, pushed to a registry, deployed and health-checked. Ansible provisions a server the same way every time, a second run changes nothing, and one command rolls back to the previous release."

2-minute explanation

98. **Problem** - a small hotel books by phone and spreadsheet: double bookings, no self-service, risky manual releases.
99. **Application** - Python Flask with layers: controllers (HTTP), services (rules), DAOs (all SQL), SQLite. Six workflows. Passwords hashed, CSRF tokens, owner checks.
100. **Git** - `main` for releases, `develop` for integration, one `feature/*` branch per change, `--no-ff` merges, tags `v1.0.0` and `v1.1.0`, published on GitHub.
101. **Tests** - 59 unit, 26 integration, 5 Selenium; isolated temporary database; screenshots on failure.
102. **Jenkins** - the `Jenkinsfile` has 10 stages: checkout, install, static checks, unit & integration, Selenium, package source, build images, publish, deploy, health check. Parameter `APP_ENV` = staging (8081) or prod (8082). Secrets from Jenkins credentials.
103. **Docker** - slim Python image, non-root, volume for the database, HEALTHCHECK, Gunicorn behind Nginx; images tagged `<version>-<commit>` in a local registry.
104. **Ansible** - inventory, variables and a playbook that installs Docker, copies the app, writes a root-only `.env`, builds, starts and checks `/health`; idempotent; `rollback.yml`.
105. **Evidence** - Jenkins builds #1-#11, Ansible runs with `changed=7/5/0`, rollback verified by Selenium.

The workflow in order

```
commit (feature branch) -> merge into develop -> Jenkins: checkout -> pip install (pinned)
-> flake8 -> pytest unit+integration -> Selenium -> [any failure: STOP, nothing deployed]
-> git archive + sha256 -> docker build (tag 1.1.0-<sha>) -> docker push (registry)
-> docker compose pull + up --wait -> /health must match version + commit -> deployment record
-> release: merge develop into main + tag vX.Y.Z -> deploy prod
-> Ansible: provision/deploy servers (idempotent) -> rollback.yml if needed
```

Tools and their roles

Tool	Role here
Git / GitHub	history, branches, tags / hosted copy (public)
Flask, SQLite	web app, database
pip + `requirements*.txt`	pinned dependencies (the Python equivalent of Maven's dependencies)
flake8, compileall	static checks
pytest	unit + integration tests, JUnit XML reports
Selenium WebDriver	browser journeys (headless Chrome)
Jenkins + `Jenkinsfile`	CI/CD pipeline as code
Docker, Docker Compose	images, containers, volume, health checks

Tool	Role here
`registry:2`	local image registry (Docker Hub supported, not used)
Gunicorn, Nginx	production WSGI server, reverse proxy (instead of Tomcat)
Ansible	configuration management, provisioning, rollback

The four implementation stories

1. No double bookings (three layers). The service checks availability; the DAO checks and inserts inside `BEGIN IMMEDIATE` (a write-locked transaction), so simultaneous requests queue; and SQLite triggers refuse any overlapping active booking even if the code had a bug. Rule: existing `check_in < new check_out AND check_out > new check_in`. A test fires 10 threads at once - exactly one booking is created.

2. A failed quality gate prevented deployment. On `experiment/defect-demo` I changed `<` to `<=` in the overlap SQL (commit `5844eec`). Jenkins build #2 failed on `test_overlap_rules[2030-01-08-2030-01-10-True]`; Package, Build images, Publish, Deploy and Health check were skipped; staging kept the old version. The fix `2fb62d6` made build #3 green. Bonus: build #2's report also showed 5 old Selenium results from a reused workspace - fixed with `rm -rf reports dist`, proven by build #9.

3. Merge conflict. `feature/contact-footer` and `feature/release-footer` both started at `da225fb` and added a footer in the same place. The second merge conflicted in `base.html`, `style.css` and `test_routes.py`. I kept both features, ran flake8 and all tests (89 passed) and committed merge `9e766ac`. It can be replayed live in a throw-away clone.

4. Rollback. Every deploy tags images with the release and keeps old ones; the playbook stores the previous release. `rollback.yml` switches `IMAGE_TAG`, restarts with `--no-build` and asserts `/health`. On a fresh lab server: v1.0.0 -> v1.1.0 -> repeat run `changed=0` -> rollback to 1.0.0 -> v1.0.0's own Selenium journeys 5/5 passed.

Common examiner questions

Question	Short answer
CI vs CD?	CI = build + test every change; CD = also deploy automatically after tests pass
Git vs GitHub?	local history tool vs hosted service with issues/PRs
Branch vs tag?	moving pointer vs fixed label on one commit (releases)
Image vs container?	read-only package vs running instance
Registry vs Docker Hub?	any image store vs one hosted public registry; we use a local one
Playbook vs inventory?	what to do vs which servers
Idempotency vs repeatability?	no changes when already correct vs same end result each time
Unit vs integration vs Selenium?	one rule / real HTTP routes in-process / full browser journey
Build failure vs deployment failure?	stopped before deploy (users unaffected) vs new version unhealthy (rollback)
Deployment vs rollback?	forward to a new version vs back to an older tested image, no rebuild
What is a quality gate?	tests must pass before Deploy can run (stage order)
How is the secret handled?	Jenkins credential / Ansible-generated, `.env` 0600, `no_log`, never in Git
Why `/health`?	Docker, Jenkins and Ansible use it to check the app and its version/commit
Why Nginx + Gunicorn?	Gunicorn runs Python; Nginx is the public entry point and serves static files

Limitations - honest answers

If asked	Answer
"Show me your GitHub issues / pull requests."	"There are none yet - GitHub shows 0. The backlog is in `docs/02-agile-plan.md`, merges were done locally with `--no-ff` and pushed, and review records are in `docs/pull-requests/`. Creating the issues and PRs is my next step."
"Did you publish to Docker Hub?"	"No. The pipeline supports it with a `registry-creds` credential and the REGISTRY parameter, but I tested only the local registry."
"Did you deploy to a real server?"	"Ansible was run against a disposable Ubuntu container with systemd and SSH, not a VM. The same playbook targets a real server by editing `inventory.ini`."
"Where is Maven / Tomcat?"	"It's a Python project: pip, flake8 and pytest do the dependency/compile/test jobs; Gunicorn + Nginx serve it. I need your confirmation that this equivalent is acceptable."
"Why polling and not a webhook?"	"Jenkins runs on localhost, which GitHub can't reach."
"Why do commit IDs differ between logs and GitHub?"	"Commit messages were edited once before publishing; the map is `docs/evidence/git/commit-id-map.md`."
"Is the new design deployed?"	"Not yet - the redesign is on `feature/ui-redesign`, uncommitted. Prod runs v1.1.0."
"Zero downtime?"	"No - rollback re-creates containers, a few seconds' interruption."

Commands for the practical (PowerShell, from `D:\DevOPS\CICD_FOR_HOTEL_ROOM_BOOKING`)

```
# 0. before anything: start Docker Desktop, then
docker ps --format "table {{.Names}}\t{{.Status}}"

# app locally (port 5000 is used by the registry)
.venv\Scripts\python -m flask --app wsgi run --port 5001

# tests
.venv\Scripts\python -m flake8 -j 1 app tests scripts wsgi.py gunicorn.conf.py
.venv\Scripts\python -m pytest -q -m "not e2e"
$env:HEADLESS="0"; .venv\Scripts\python -m pytest -m e2e -v; Remove-Item Env:HEADLESS

# git
git log --graph --oneline -12 develop
git tag -n1
git show --stat 9e766ac

# deployed versions and registry
curl.exe -s http://localhost:8081/health
curl.exe -s http://localhost:8082/health
curl.exe -s http://localhost:5000/v2/hotel-booking-app/tags/list

# Jenkins (if not running)
cd deploy\jenkins; docker compose up -d; cd ..\..
#   -> http://localhost:8090, job hotel-booking, Build with Parameters

# Ansible lab
cd deploy\ansible\lab
docker compose up -d target
docker compose run --rm controller ansible-playbook -i lab/inventory.ini playbook.yml
docker compose run --rm controller ansible-playbook -i lab/inventory.ini playbook.yml -e
app_version=1.1.1-demo
docker compose run --rm controller ansible-playbook -i lab/inventory.ini rollback.yml
curl.exe -s http://localhost:8088/health
```

Screenshot inventory

Generated from the figure references in the 15 task reports on 2026-10-20. Every file listed as *verified* exists on disk. "Browser screenshot" = captured from the running system or GitHub with headless Chrome; "rendered from real output" = a real command or log rendered as an image by `scripts/capture_report_evidence.py` (the raw text is kept next to it or in `assets/raw/`).

86 referenced screenshots, 0 missing references, 9 still to be captured manually.

Task	File	Evidence purpose	Captured	Status
1	`assets/task-01/github-doc-01-problem-definition.png`	`docs/01-problem-definition.md` as published on the public GitHub repository (captured 2026-10-20).	2026-10-20	verified (browser screenshot)
2	`assets/task-02/github-doc-02-agile-plan.png`	`docs/02-agile-plan.md` on GitHub (captured 2026-10-20).	2026-10-20	verified (browser screenshot)
2	`assets/task-02/github-issues-empty.png`	The repository's Issues page (all states) shows no issues (captured 2026-10-20).	2026-10-20	verified (browser screenshot)
3	`assets/task-03/github-doc-03-srs-architecture.png`	`docs/03-srs-architecture.md` on GitHub with diagrams rendered (captured 2026-10-20).	2026-10-20	verified (browser screenshot)
3	`assets/task-03/flask-routes.png`	`python -m flask --app wsgi routes` run on 2026-10-20 (rendered from real output).	2026-10-20	verified (rendered from real command/log output)
3	`assets/task-03/project-structure.png`	Files under `app/` and `tests/` (from `git ls-files`, rendered from real output).	2026-10-20	verified (rendered from real command/log output)
3	`assets/task-03/github-file-app_schema.sql.png`	`app/schema.sql` on GitHub.	2026-10-20	verified (browser screenshot)
3	`assets/task-03/app-desktop-01-home.png`	Figure 3.5 - The running application	2026-10-20	verified (browser screenshot)
4	`assets/task-04/git-branches-tags-remote.png`	`git branch -a`, `git tag -n1`, `git remote -v` (rendered from real output, 2026-10-20).	2026-10-20	verified (rendered from real command/log output)

Task	File	Evidence purpose	Captured	Status
4	`assets/task-04/git-main-history.png`	Main-branch history and the skeleton commit `541dad5` with its files (rendered from real output).	2026-10-20	verified (rendered from real command/log output)
4	`assets/task-04/gitignore-and-templates.png`	`.github` templates, `.gitattributes` and `.gitignore` (rendered from real output).	2026-10-20	verified (rendered from real command/log output)
4	`assets/task-04/github-repository-home.png`	The public GitHub repository home page (captured 2026-10-20).	2026-10-20	verified (browser screenshot)
4	`assets/task-04/github-branches.png`	All branches on GitHub.	2026-10-20	verified (browser screenshot)
4	`assets/task-04/github-templates-folder.png`	`.github/` on GitHub: `ISSUE_TEMPLATE/` and `pull_request_template.md`.	2026-10-20	verified (browser screenshot)
5	`assets/task-05/feature-branch-history.png`	`git log --graph` of `feature/security-hardening` and `git show --stat 716b910` (rendered from real output, 2026-10-20).	2026-10-20	verified (rendered from real command/log output)
5	`assets/task-05/all-feature-merges.png`	`git log --merges --first-parent develop` (rendered from real output).	2026-10-20	verified (rendered from real command/log output)
5	`assets/task-05/github-compare-merged-into-develop.png`	GitHub compare `develop...feature/nights-column` (captured 2026-10-20).	2026-10-20	verified (browser screenshot)
5	`assets/task-05/github-pull-requests-empty.png`	The repository's Pull requests page (all states): none.	2026-10-20	verified (browser screenshot)
5	`assets/task-05/github-compare-feature-changes.png`	GitHub compare from the branch's starting point `a2bb052` to `feature/nights-column`.	2026-10-20	verified (browser screenshot)
6	`assets/task-06/app-desktop-03-register.png`	Workflow 1: registration form with labelled fields and the password rule.	2026-10-20	verified (browser screenshot)

Task	File	Evidence purpose	Captured	Status
6	`assets/task-06/app-desktop-05-login-error.png`	Workflow 1: login with wrong credentials shows a neutral error (does not reveal which part was wrong).	2026-10-20	verified (browser screenshot)
6	`assets/task-06/app-desktop-06-availability-logged-in.png`	Workflow 2: availability for the chosen dates - booked rooms marked "Booked", free rooms with the total for the stay.	2026-10-20	verified (browser screenshot)
6	`assets/task-06/app-desktop-08-booking-requested.png`	Workflow 3: the new booking is PENDING.	2026-10-20	verified (browser screenshot)
6	`assets/task-06/app-desktop-09-booking-confirmed.png`	Workflow 4: CONFIRMED, progress shows both steps done.	2026-10-20	verified (browser screenshot)
6	`assets/task-06/app-desktop-10-cancel-dialog.png`	Workflow 5: cancelling opens a confirmation dialog first.	2026-10-20	verified (browser screenshot)
6	`assets/task-06/app-desktop-11-booking-cancelled.png`	Workflow 5: status CANCELLED, room released.	2026-10-20	verified (browser screenshot)
6	`assets/task-06/app-desktop-12-my-bookings.png`	Workflow 6: status tracking for all of the guest's bookings (PENDING, CANCELLED, CONFIRMED).	2026-10-20	verified (browser screenshot)
6	`assets/task-06/app-mobile-12-my-bookings.png`	The same page at phone width (375 px).	2026-10-20	verified (browser screenshot)
6	`assets/task-06/merge-conflict-output.png`	The conflict as recorded on 2026-10-11 (rendered from `docs/evidence/git/merge-conflict.txt`; original commit IDs).	2026-10-20	verified (rendered from real command/log output)
6	`assets/task-06/merge-conflict-resolution.png`	`git log --graph` and `git show --stat 9e766ac` today (rendered from real output).	2026-10-20	verified (rendered from real command/log output)
6	`assets/task-06/release-tags.png`	`git tag -n1` and the annotated tag object of `v1.1.0` (rendered from real output).	2026-10-20	verified (rendered from real command/log output)

Task	File	Evidence purpose	Captured	Status
6	`assets/task-06/github-release-tags.png`	`v1.0.0` and `v1.1.0` on GitHub.	2026-10-20	verified (browser screenshot)
6	`assets/task-06/github-doc-04-git-workflow.png`	`docs/04-git-workflow.md` on GitHub: branching model, merges, the conflict story and the release procedure.	2026-10-20	verified (browser screenshot)
7	`assets/task-07/jenkins-job-stage-view.png`	Job `hotel-booking`: build history #1-#11, test-result trend, last artifacts and the stage view (captured 2026-10-20).	2026-10-20	verified (browser screenshot)
7	`assets/task-07/jenkins-job-configuration.png`	Job configuration: the three parameters, "Do not allow concurrent builds", Poll SCM `H/5	2026-10-20	verified (browser screenshot)
7	`assets/task-07/jenkins-build-11-summary.png`	Build #11 (`1.1.0-1c2103c`), started 2026-10-20: result SUCCESS.	2026-10-20	verified (browser screenshot)
7	`assets/task-07/jenkins-build-11-tests.png`	Test report of build #11: 90 tests, 0 failures, grouped by package (unit, integration, e2e).	2026-10-20	verified (browser screenshot)
7	`assets/task-07/jenkins-build-11-artifacts.png`	Archived artifacts of build #11: source tarball + `.sha256`, test XML, `health.json`, `deployment.txt`.	2026-10-20	verified (browser screenshot)
7	`assets/task-07/jenkins-files.png`	`deploy/jenkins/plugins.txt` and the running Jenkins + registry containers (rendered from real output, 2026-10-20).	2026-10-20	verified (rendered from real command/log output)
8	`assets/task-08/github-file-Jenkinsfile.png`	`Jenkinsfile` on GitHub (captured 2026-10-20).	2026-10-20	verified (browser screenshot)
8	`assets/task-08/jenkinsfile-stages.png`	Line numbers of every stage and parameter (`findstr` on the Jenkinsfile, rendered from real output).	2026-10-20	verified (rendered from real command/log output)
8	`assets/task-08/jenkins-build-with-parameters.png`	The	2026-10-20	verified (browser screenshot)

Task	File	Evidence purpose	Captured	Status
8	`assets/task-08/jenkins-build-11-console.png`	Build #11 console excerpt (rendered from the saved console log): each stage starting, 85 + 5 tests passed, the archive checksum, image pushes with digests, `/health` and "Deployed 1.1.0-1c2103c to staging".	2026-10-20	verified (rendered from real command/log output)
8	`assets/task-08/staging-health-after-build-11.png`	http://localhost:8081/health in a browser after build #11: commit `1c2103c`, env `staging`, version `1.1.0`.	2026-10-20	verified (browser screenshot)
8	`assets/task-08/jenkins-credentials-masked.png`		2026-10-20	verified (browser screenshot)
9	`assets/task-09/selenium-run.png`	`pytest -m e2e -v` on 2026-10-20 (rendered from real output): 5 journeys PASSED.	2026-10-20	verified (rendered from real command/log output)
9	`assets/task-09/full-test-suite.png`	flake8 (no findings), then unit 59, integration 26 and e2e 5 passed (rendered from real output, 2026-10-20).	2026-10-20	verified (rendered from real command/log output)
9	`assets/task-09/github-file-tests_e2e_test_journeys.py.png`	Figure 9.3 - Journey source on GitHub	2026-10-20	verified (browser screenshot)
9	`assets/task-09/selenium-failure-demo-output.png`	A temporary, deliberately failing test run on 2026-10-20 (rendered from real output): "failure screenshot saved: reports\screenshots...png", then `1 failed`; the folder contains a `.png` and an `.html`.	2026-10-20	verified (rendered from real command/log output)
9	`assets/task-09/selenium-failure-screenshot-saved-by-hook.png`	The actual image saved by the hook: what the browser showed at the moment of failure.	2026-10-20	verified (browser screenshot)
9	`assets/task-09/app-desktop-07-my-bookings-empty.png`	The empty "My bookings" state (redesigned UI) - one of the pages the journeys navigate.	2026-10-20	verified (browser screenshot)
10	`assets/task-10/jenkins-build-02-failed.png`	Build #2 `1.0.0-554cad3` in Jenkins (checked 2026-10-20): result FAILURE.	2026-10-20	verified (browser screenshot)
10	`assets/task-10/build-02-defect-console.png`	Console excerpt of build #2 (rendered from the saved log `docs/evidence/jenkins/build-02-defect-fails.log`).	2026-10-20	verified (rendered from real command/log output)

Task	File	Evidence purpose	Captured	Status
10	`assets/task-10/defect-branch-history.png`	The experiment branch history, the one-character diff, and the branches that contain the defect (rendered from real output, 2026-10-20).	2026-10-20	verified (rendered from real command/log output)
10	`assets/task-10/jenkins-build-02-failed-test.png`	Build #2's Tests page: 1 failed (`test_overlap_rules[...-True]`, marked Regression). Also notice `tests.e2e 5` - the stale report described above.	2026-10-20	verified (browser screenshot)
10	`assets/task-10/jenkins-build-09-tests-no-stale-report.png`	Build #9 after the report fix: 82 tests, 1 failed, and no `tests.e2e` entry.	2026-10-20	verified (browser screenshot)
10	`assets/task-10/jenkins-build-03-fixed.png`	Build #3 `1.0.0-30c6a05`: SUCCESS after the corrective commit.	2026-10-20	verified (browser screenshot)
10	`assets/task-10/build-03-fix-console.png`	Build #3 console excerpt (rendered from the saved log): tests passed, deployed, health check OK.	2026-10-20	verified (rendered from real command/log output)
10	`assets/task-10/build-09-rerun-console.png`	Build #9 console excerpt: `+ rm -rf reports dist`, the same failing test, later stages skipped.	2026-10-20	verified (rendered from real command/log output)
11	`assets/task-11/lifecycle-part1-build-run.png`	Lifecycle demo, part 1: build, tag, image labels/user/port, volume, run, `docker ps` with port mapping, `/health`, registration, logs (rendered from the real run on 2026-10-20).	2026-10-20	verified (rendered from real command/log output)
11	`assets/task-11/lifecycle-part2-stop-restart-volume.png`	Part 2: inspect, `whoami` = appuser, stop -> Exited, start, restart, remove, a new container on the same volume, the old account still logs in, clean-up.	2026-10-20	verified (rendered from real command/log output)
11	`assets/task-11/github-file-Dockerfile.png`	`Dockerfile` on GitHub.	2026-10-20	verified (browser screenshot)
11	`assets/task-11/github-file-docker-compose.yml.png`	`docker-compose.yml`: app + nginx, named volume, `service_healthy` dependency, `SECRET_KEY` required.	2026-10-20	verified (browser screenshot)
11	`assets/task-11/docker-images-containers.png`	`docker image ls`, `docker ps` and `docker volume ls` on 2026-10-20 (rendered from real output): versioned images, the running staging/prod/Jenkins/lab containers and the data volumes.	2026-10-20	verified (rendered from real command/log output)

Task	File	Evidence purpose	Captured	Status
12	`assets/task-12/deployed-versions.png`	Which image each environment runs, both `/health` answers, and the registry catalogue and tags (rendered from real output, 2026-10-20).	2026-10-20	verified (rendered from real command/log output)
12	`assets/task-12/registry-tags.png`	http://localhost:5000/v2/hotel-booking-app/tags/list in the browser.	2026-10-20	verified (browser screenshot)
12	`assets/task-12/prod-health.png`	http://localhost:8082/health: prod runs release 1.1.0 (`d07929b`).	2026-10-20	verified (browser screenshot)
12	`assets/task-12/jenkins-build-11-deployment-record.png`	`reports/deployment.txt` archived by build #11: environment, branch, version, commit, image names, URL.	2026-10-20	verified (browser screenshot)
12	`assets/task-12/prod-portal-v1.1.0.png`	The prod portal at http://localhost:8082 - the deployed v1.1.0 interface (the newer UI redesign is not deployed yet).	2026-10-20	verified (browser screenshot)
13	`assets/task-13/ansible-run1-first-deploy-v1.0.0.png`	Run 1 on a fresh lab server, from a clean clone of tag v1.0.0 (rendered from the real run on 2026-10-20).	2026-10-20	verified (rendered from real command/log output)
13	`assets/task-13/github-file-deploy_ansible_inventory.ini.png`	`deploy/ansible/inventory.ini` on GitHub - group `[web]`.	2026-10-20	verified (browser screenshot)
13	`assets/task-13/github-file-deploy_ansible_group_vars_web.yml.png`	`group_vars/web.yml`: environment-specific values, the version taken from `VERSION`, and the generated secret key.	2026-10-20	verified (browser screenshot)
13	`assets/task-13/github-file-deploy_ansible_playbook.yml.png`	`playbook.yml` on GitHub: the tasks listed above.	2026-10-20	verified (browser screenshot)
13	`assets/task-13/lab-server-state.png`	Inside the lab server (rendered from real output, 2026-10-20): file permissions (`600 root .env`), the files in `/opt/hotel-booking`, images, running containers, `docker` enabled.	2026-10-20	verified (rendered from real command/log output)
14	`assets/task-14/ansible-run2-upgrade-v1.1.0.png`	Run 2 (upgrade to v1.1.0): `changed=5`, `/health` 1.1.0 (rendered from the real run, 2026-10-20).	2026-10-20	verified (rendered from real command/log output)

Task	File	Evidence purpose	Captured	Status
14	`assets/task-14/ansible-run3-repeat-idempotent.png`	Run 3, the same playbook again: every task `ok` or `skipping`, `PLAY RECAP ... changed=0`.	2026-10-20	verified (rendered from real command/log output)
14	`assets/task-14/ansible-run4-rollback.png`	`rollback.yml`: "Rolled back from 1.1.0 to 1.0.0", `changed=3`, `/health` 1.0.0.	2026-10-20	verified (rendered from real command/log output)
14	`assets/task-14/ansible-run5-e2e-after-rollback.png`	The five v1.0.0 Selenium journeys pass against the rolled-back server.	2026-10-20	verified (rendered from real command/log output)
14	`assets/task-14/lab-server-health-after-rollback.png`	http://localhost:8088/health: version 1.0.0, commit `224de14`.	2026-10-20	verified (browser screenshot)
14	`assets/task-14/lab-server-after-rollback-v1.0.0.png`	The portal served by the lab server after the rollback (v1.0.0 interface).	2026-10-20	verified (browser screenshot)
14	`assets/task-14/github-file-deploy_ansible_rollback.yml.png`	`deploy/ansible/rollback.yml` on GitHub.	2026-10-20	verified (browser screenshot)
15	`assets/task-15/all-environments-health.png`	`/health` of development, staging, prod and the lab server (rendered from real output, 2026-10-20).	2026-10-20	verified (rendered from real command/log output)
15	`assets/task-15/git-status-final.png`	Current branch and uncommitted files (rendered from real output): the UI redesign and these reports are not yet committed.	2026-10-20	verified (rendered from real command/log output)
15	`assets/task-15/app-desktop-06-availability-logged-in.png`	The customer journey's central page: availability with totals for the stay (redesigned UI, working tree).	2026-10-20	verified (browser screenshot)
15	`assets/task-15/github-doc-15-task-checklist.png`	`docs/15-task-checklist.md` on GitHub: gap analysis and the final deliverable status.	2026-10-20	verified (browser screenshot)
15	`assets/task-15/github-doc-08-project-report.png`	`docs/08-project-report.md` on GitHub: summary, document map, end-to-end sequence diagram, setup guide, troubleshooting, limitations, future work and the demo checklist.	2026-10-20	verified (browser screenshot)

Awaiting manual capture (external actions)

Task	File	Purpose	Status
2	`assets/task-02/github-issues-created.png`	Issues HB-01..HB-15 on GitHub	awaiting manual capture
4	`assets/task-04/github-branch-protection.png`	Branch protection settings (only if configured)	awaiting manual capture
5	`assets/task-05/github-pr-open.png`	A real pull request	awaiting manual capture
5	`assets/task-05/github-pr-review-comment.png`	A review comment	awaiting manual capture
5	`assets/task-05/github-pr-merged.png`	The merged pull request	awaiting manual capture
9	`(video/photo)`	Visible browser run with HEADLESS=0 (live demo)	awaiting manual capture
12	`assets/task-12/dockerhub-repository.png`	Image published on Docker Hub	awaiting manual capture
13	`assets/task-13/real-vm-run.png`	Playbook against a real Ubuntu VM	awaiting manual capture
15	`(video)`	Demo recording	awaiting manual capture